book 12

design patterns, concurrency, and distributed systems

an engineering handbook

Raeliosol

version 5.0, compiled 10 2026

Contents

1. pattern discipline 5
   1.1. the eight execution models 5
   1.2. satisfaction is structural 6
   1.3. accept interfaces, return concrete types 14
   1.4. embedding is forwarding, not inheritance 18
   1.5. what each language changes about the shapes 33
   1.6. across the eight languages 34
2. creational patterns 35
   2.1. constructors are functions 35
   2.2. functional options 35
   2.3. builder, when assembly can fail midway 44
   2.4. lazy and once 53
   2.5. prototype as an explicit clone 58
   2.6. flyweight through unique 65
   2.7. across the eight languages 71
3. structural patterns 72
   3.1. adapter 72
   3.2. decorator as middleware 76
   3.3. facade 84
   3.4. composite 90
   3.5. proxy 99
   3.6. across the eight languages 108
4. behavioral patterns 110
   4.1. strategy 110
   4.2. observer with channels 118
   4.3. command with undo 129
   4.4. iterator, push style 136
   4.5. state as a transition table 145
   4.6. visitor as a type switch 153
   4.7. across the eight languages 162
5. functional alternatives 163
   5.1. the trio, generic 163
   5.2. composition and partial application 168
   5.3. the loop variable, fixed 175
   5.4. memoization 181
   5.5. across the eight languages 193
6. concurrency fundamentals and the memory model 194
   6.1. the model in one rule 194
   6.2. happens-before, the doors 194
   6.3. the wait edge 196
   6.4. the rendezvous edge 203
   6.5. atomics, the word-sized escape hatch 211
   6.6. close and capacity 217
   6.7. broken idioms, named 226
   6.8. across the eight languages 226
7. sync primitives in depth 228
   7.1. mutex and rwmutex 228
   7.2. once, the lazy family 236

7.3. cond, waiting for a predicate ... 243
7.4. sync.map ... 251
7.5. atomics, beyond the counter ... 257
7.6. waitgroup and the error group ... 263
7.7. across the eight languages ... 272
8. concurrency patterns ... 274
8.1. cancellation first ... 274
8.2. generator and stage ... 275
8.3. fan-in, merge ... 284
8.4. worker pool ... 289
8.5. bounded parallel map, ordered ... 295
8.6. racing sources, first wins ... 304
8.7. the leak test ... 311
8.8. across the eight languages ... 315
9. hazards and detectors ... 317
9.1. deadlock, and the cycle rule ... 317
9.2. the guard, hangs become failures ... 324
9.3. starvation, and the rwmutex decision ... 328
9.4. the detectors ... 336
9.5. synctest, deterministic time ... 341
9.6. across the eight languages ... 356
10. distributed fundamentals ... 357
10.1. the fallacies, as constraints ... 357
10.2. why wall clocks are not clocks ... 357
10.3. lamport clocks ... 358
10.4. vector clocks ... 363
10.5. quorums ... 373
10.6. cap, as an engineering decision ... 376
10.7. across the eight languages ... 376
11. replication and consensus: raft ... 378
11.1. the state machine approach ... 378
11.2. terms and leader election ... 378
11.3. the election restriction ... 383
11.4. log replication and the consistency check ... 386
11.5. commitment ... 394
11.6. the state, and what the capstone keeps ... 398
11.7. across the eight languages ... 398
12. partitioning and consistent hashing ... 400
12.1. the naive baseline and its cost ... 400
12.2. consistent hashing with virtual nodes ... 400
12.3. the bug the probe caught ... 415
12.4. rendezvous, highest random weight ... 420
12.5. jump consistent hash ... 427
12.6. across the eight languages ... 430
13. messaging semantics and idempotency ... 431
13.1. the three semantics ... 431
13.2. the outbox, and a crash in the middle ... 440
13.3. per-key ordering, lanes ... 449
13.4. poison messages ... 459
13.5. across the eight languages ... 460
14. observability ... 461

14.1. structured logging, the capture handler 461
14.2. the counter registry, and the one-line mount 470
14.3. traces as context propagation 479
14.4. across the eight languages 488
15. resilience 489
15.1. retry with exponential backoff and full jitter 489
15.2. the circuit breaker 497
15.3. rate limiting with a token bucket 507
15.4. bulkhead and fallback 515
15.5. across the eight languages 524
16. capstone: raft replicated configuration service 526
16.1. the shape 526
16.2. elections 527
16.3. replication and commitment 529
16.4. the service 534
16.5. what the tests prove 535
17. appendices: topic matrix and sources 538
17.1. topic coverage 538
17.2. pinned sources 540

1. pattern discipline

The classic pattern vocabulary was written for languages with classes, inheritance, and virtual dispatch. Go has none of the three, and neither do five of the eight languages asking the question beside it in this book. The first engineering decision is not which pattern to apply but which patterns survive translation at all: some collapse into a function, some into a struct literal or a plain table, and the ones that remain take a different shape in each language's grain.

This book asks in eight voices. The go lane stays the frozen reference: one module, `patternsbook` on go 1.27, living at `patterns-concurrency-distributed/samples/`, its listings and tests untouched while seven sibling trees grew around it, and every shared fixture in those trees cross-verified against its implementation. The siblings carry the same samples, one concept per file, each in its own spelling:

tree	shape	verify leg
`samples-c/`	one standalone `main` per file, libc plus C23 `<threads.h>` and `<stdatomic.h>`	`verify-c`
`samples-rust/`	one standalone `main` per file, std only, edition 2024 on the pinned rustc 1.99	`verify-rust`
`samples/`	one go module, go 1.27, the frozen reference lane	`verify-go`, race leg included
`samples-java/`	one public class per file, checks printed as `ok N name` on the pinned jdk 27	`verify-java`
`samples-cs/`	one net solution, sources beside xunit test projects	`verify-csharp`
`samples-js/`	pure esm modules, one test file per chapter	`verify-ts`
`samples-py/`	script-style modules, each ending in `ok N name`	`verify-py`
`samples-lua/`	files returning named test rows to a shared `run.lua`	`verify-lua`

Every leg also runs chapter scoped, the form the dry runs in this book assume:

1. C: `-SampleRoot books/patterns-concurrency-distributed/samples-c/src -Chapter ChNN`
2. rust: `-SampleRoot books/patterns-concurrency-distributed/samples-rust/src -Chapter ChNN` under `run-rust-samples`
3. go: `go test ./chNN`
4. java: `-SampleRoot books/patterns-concurrency-distributed/samples-java/src -Chapter ChNN` under `run-java-samples`
5. C#: `dotnet test` filtering the solution to `FullyQualifiedName~ChNN`
6. javascript: `node --test test/chNN*.test.mjs`
7. python: the matching `-SampleRoot books/patterns-concurrency-distributed/samples-py/src -Chapter ChNN`
8. lua: the runner always reads the whole suite, each chapter's rows kept in order

The caption contract for everything that follows: every featured section shows all eight listings, labeled with their language, in corpus order c, rust, go, java, c#, javascript, python, lua. Go is one voice among eight, the reference the others are checked against, no longer the book's default. The chapter 16 raft capstone and the chapter 17 appendices stay go only, single-language capstones on purpose.

The discipline this chapter sets for the next four: a pattern is a name attached to a recurring tradeoff, never a folder of boilerplate. When the language makes the tradeoff disappear, the pattern goes with it, and saying so is more useful than imitating java.

1.1. the eight execution models

Most of the patterns ahead are concurrency patterns, so the map worth pinning once is the execution model under each tree. C spawns real operating system threads through C23 `<threads.h>` and shares state through `<stdatomic.h>`: the kernel preempts, the scheduler lives outside the program, and every claim about interleaving has to survive that. Rust spawns the same real threads through `std::thread` and moves half the sharing question into the type

system, `Send` and `Sync` gating what may cross a thread boundary and ownership transfer replacing hidden aliasing. Go multiplexes goroutines onto a pool, book 4, ch 9 (go 1.27, runtime) opens the scheduler they ride, and the culture hands data through native channels. Java rides virtual threads, cheap threads the jvm multiplexes onto a carrier pool, with `ExecutorService` for pools and `VarHandle` fences for shared state, the primitives the concurrency chapters ahead lean on. C# multiplexes tasks onto a pool and reaches for `System.Threading.Channels` beside go's native ones. Javascript runs one thread per realm: the event loop is the only scheduler, every `await` is a queue entry, and true parallelism means `worker_threads` exchanging messages, book 7, ch 6 (javascript es2026 and typescript 7, async javascript) tours that loop. Python runs threads under the GIL and does its serious concurrency in asyncio, where the interpreter interleaves coroutines at await points, book 8, ch 12 (python 3.14, how cpython runs your code) is that machine. Lua has no scheduler at all: coroutines suspend by hand, so every lua lane in this book is a scripted coroutine under a deterministic driver, book 9, ch 7 (lua 5.5, coroutines) builds on that model.

The testing discipline follows from the map, and it is the same in all eight trees. Where the lesson is what happens when operations interleave, the test drives the interleaving itself: scripted state-machine walks, coroutine schedulers, async queue pumps, a patched clock. Where the lesson needs real threads, the test asserts invariants, exact totals, multiset equality, bounds, never timings and never free order. And one boundary stated once here and again in chapter 9: race detection in this corpus is `go test -race`, the only detector any verify leg runs. This platform's C toolchain ships no thread sanitizer, the wall the icpc book documents for its own C toolbox, so the C tree earns its concurrency claims by construction, every observed interleaving either scripted or invariant checked.

1.2. satisfaction is structural

A Go interface is a method set, and a type satisfies it by having the methods, with no declaration binding the two. Effective Go states the rule plainly: a type implements an interface just by implementing its methods. The consequence for patterns is large: adapters and decorators can be retrofitted onto types their author never knew about, which is why half of the structural and behavioral patterns in chapter 3 and 4 need nothing but a new type wrapping an old one.

The dry run: the prefix logger records exactly `app: hello`, the null object emits nothing and satisfies anyway, and a value without the method fails the shape check, at build time in go, C, rust, C#, and java, at run time in javascript, python, and lua. The literal is pinned by the seven new trees, the frozen go test asserts the fresh-empty start and the Discard null object, both against the same implementation.

```c
// the whole interface: one method. the void self pointer plays the
// receiver, so one vtable type fits every satisfier
typedef struct {
  void (*log)(void *self, const char *msg);
} logger;

typedef struct {
  const char *prefix;
  char lines[8][64];
  int count;
} prefix_logger;

static void prefix_log(void *self, const char *msg) {
  prefix_logger *p = self;
  if (p->count < 8)
    snprintf(p->lines[p->count], 64, "%s: %s", p->prefix, msg);
  p->count++;
}

// the null object satisfies the interface with zero behavior
static void discard_log(void *self, const char *msg) {
  (void)self;
  (void)msg;
}

// the compile-time proof the go lane spells with blank assignments: each
// initializer demands a method of the interface's exact shape, so these
// tables only build while the types keep satisfying logger
static const logger prefix_iface = {prefix_log};
static const logger discard_iface = {discard_log};

// one slot per satisfier, iterated uniformly: structural satisfaction
// means the caller never knows which concrete type answered
typedef struct {
  logger vt;
  void *self;
} logger_slot;
```

listing 1: C, the interface is a vtable, the proof a static initializer that stops building when a slot goes missing (source: patterns-concurrency-distributed/samples-c/src/Ch01/satisfaction.c, lines 19-55)

```rust
trait Logger {
    fn log(&mut self, msg: &str);
}

struct PrefixLogger {
    prefix: &'static str,
    lines: Vec<String>,
}

impl Logger for PrefixLogger {
    fn log(&mut self, msg: &str) {
        self.lines.push(format!("{}: {}", self.prefix, msg));
    }
}

struct Discard;

impl Logger for Discard {
    fn log(&mut self, _msg: &str) {}
}

const _: fn(&mut PrefixLogger, &str) = Logger::log;
const _: fn(&mut Discard, &str) = Logger::log;
```

listing 2: Rust, satisfaction is the impl block, the proof a const function item the compiler checks and the binary never carries (source: patterns-concurrency-distributed/samples-rust/src/Ch01/satisfaction.rs, lines 13-35)

```go
// Logger is the one-method interface this chapter keeps returning to.
type Logger interface {
  Log(msg string)
}

// PrefixLogger prepends a fixed prefix and records what it emitted,
// so tests observe the formatting without touching io.
type PrefixLogger struct {
  Prefix string
  Lines  []string
}

func (p *PrefixLogger) Log(msg string) {
  p.Lines = append(p.Lines, p.Prefix+": "+msg)
}

// Discard is the null object: it satisfies Logger by doing nothing.
type Discard struct{}

func (Discard) Log(string) {}
```

listing 3: Go, one method, two implementors, one of them the null object (source: patterns-concurrency-distributed/samples/ch01/discipline.go, lines 5-24)

```go
// compile-time proof of satisfaction. The declaration fails to build
// when a type stops satisfying the interface, and _ keeps the check
// out of the binary.
var (
  _ Logger = (*PrefixLogger)(nil)
  _ Logger = Discard{}
)
```

listing 4: Go, blank identifier assertions fail the build when satisfaction breaks (source: patterns-concurrency-distributed/samples/ch01/discipline.go, lines 26-32)

```java
  // the whole interface: one method
  interface Logger {
    void log(String msg);
  }

  static final class PrefixLogger implements Logger {
    final String prefix;
    final List<String> lines = new ArrayList<>();

    PrefixLogger(String prefix) {
      this.prefix = prefix;
    }

    public void log(String msg) {
      lines.add(prefix + ": " + msg);
    }
  }

  // the null object satisfies the interface with zero behavior
  static final class Discard implements Logger {
    public void log(String msg) {}
  }
```

listing 5: Java, satisfaction is nominal, the implements clause is the proof, checked where it is written (source: patterns-concurrency-distributed/samples-java/src/Ch01/Satisfaction.java, lines 18-39)

```
// Logger is the one-method interface this chapter keeps returning to.
// C# satisfaction is nominal: the class declares the interface, and the
// compiler enforces the method set at the declaration site. Go's
// structural check needs no declaration, C#'s proof is the declaration.
public interface Logger
{
    void Log(string msg);
}

// PrefixLogger prepends a fixed prefix and records what it emitted,
// so tests observe the formatting without touching the console.
public sealed class PrefixLogger(string prefix) : Logger
{
    public string Prefix { get; } = prefix;
    public List<string> Lines { get; } = [];

    public void Log(string msg) => Lines.Add(Prefix + ": " + msg);
}

// Discard is the null object: it satisfies Logger by doing nothing.
public sealed class Discard : Logger
{
    public void Log(string msg)
    {
    }
}
```

listing 6: C#, satisfaction is nominal, the declaration is the whole proof (source: patterns-concurrency-distributed/samples-cs/src/Ch01/Satisfaction.cs, lines 3-28)

```
// ch01: interface satisfaction without declarations. js has no
// compile-time method-set check, so the proof a type satisfies the
// logger shape is a runtime predicate any caller can run.

// structural check: does the value carry the one method consumers
// call? nothing else is involved, no inheritance, no declaration.
export function isLogger(x) {
  return x != null && typeof x.log === "function";
}

// PrefixLogger prepends a fixed prefix and records what it emitted,
// so tests observe the formatting without touching io.
export class PrefixLogger {
  constructor(prefix) {
    this.prefix = prefix;
    this.lines = [];
  }

  log(msg) {
    this.lines.push(this.prefix + ": " + msg);
  }
}

// Discard is the null object: it satisfies the logger shape with
// zero behavior.
export class Discard {
  log() {}
}
```

listing 7: JavaScript, no declarations anywhere, the proof is a predicate any caller can run (source: patterns-concurrency-distributed/samples-js/src/ch01-satisfaction.mjs, lines 1-28)

```python
@runtime_checkable
class Logger(Protocol):
    def log(self, msg: str) -> None: ...


class PrefixLogger:
    # records what it emitted, so the checks observe formatting
    # without touching io
    def __init__(self, prefix):
        self.prefix = prefix
        self.lines = []

    def log(self, msg):
        self.lines.append(f"{self.prefix}: {msg}")


class Discard:
    # the null object: satisfies Logger by doing nothing
    def log(self, msg):
        pass


# the runtime_checkable isinstance is the module-level analog of
# go's `var _ Logger = ...`: it fails loudly the moment a type loses
# the method. it checks method presence only, the signatures are the
# type checker's job.
check(isinstance(PrefixLogger("app"), Logger), "the method set satisfies the protocol")
check(isinstance(Discard(), Logger), "the null object satisfies it the same way")
check(not isinstance(object(), Logger), "a bare object does not")
```

listing 8: Python, a runtime-checkable Protocol moves the proof to check time (source: patterns-concurrency-distributed/samples-py/src/Ch01/satisfaction.py, lines 16-45)

```lua
-- the protocol is a caller. Run accepts anything with a Log method
local function Run(l, msg)
  l:Log(msg)
end

-- PrefixLogger prepends a fixed prefix and records what it emitted,
-- so the tests observe the formatting without touching io
local PrefixLogger = {}
PrefixLogger.__index = PrefixLogger

function PrefixLogger.new(prefix)
  return setmetatable({ prefix = prefix, lines = {} }, PrefixLogger)
end

function PrefixLogger:Log(msg)
  self.lines[#self.lines + 1] = self.prefix .. ": " .. msg
end

-- the null object: it satisfies the protocol with zero behavior
local Discard = {}
Discard.__index = Discard

function Discard.new()
  return setmetatable({ logged = 0 }, Discard)
end

function Discard:Log() end
```

listing 9: Lua, the protocol is a caller, any table carrying the method fits (source: patterns-concurrency-distributed/samples-lua/ch01_satisfaction.lua, lines 6-32)

Each language spends the proof differently. Go writes it as dead code, `var _ Logger = (*PrefixLogger)(nil)` never runs, allocates nothing, and turns a would-be runtime surprise into a compile error. C spends a static initializer: `prefix_iface` only builds while `prefix_log` matches the slot's signature, the same contract enforced by the linker's type checker instead of the compiler's. Java spends nothing, `implements Logger` on the class header is checked where it is written, and the file's `quiet instanceof Logger` check is the runtime witness go spells as dead code. C# spends nothing the same way, `class PrefixLogger : Logger` at the declaration, the two nominal trees where the classic interface shape is native. Javascript has no build-time method-set check, so its honest proof is the `isLogger` predicate, runnable by any caller and by tests. Python's `runtime_checkable` Protocol moves go's compile error to an `isinstance` in the checks, method presence only, signatures stay the type checker's job. Rust's satisfaction is nominal, the `impl` block is the declaration, and its dead-code proof is go's blank assignment translated, `const _: fn(&mut PrefixLogger, &str) = Logger::log`, a function item coerced at build time and dropped before linking. Lua has no check to move: the lookup happens at the call, one lookup too late to save you, so the row table pins that a bare table throws.

`Discard` is the null object pattern in five lines: it satisfies the interface with zero behavior, so callers never need a nil check. Stdlib reached the same shape with `io.Discard` and `http.NoBody`.

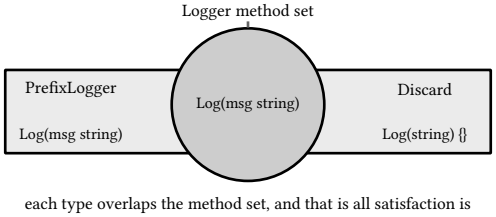


figure 1: the overlap is the implements relation, no declaration edge exists

1.3. accept interfaces, return concrete types

A Go function that takes an interface and returns a concrete struct gives callers looseness at the boundary and precision at the result. The sample's decorator does exactly that, and the shape translates with the wrapping mechanism swapped per language: a struct holding the next vtable in C, a class holding the next interface in C# and java, an object holding a duck-typed inner in the dynamic three.

The dry run: one call carrying an embedded newline lands at the inner logger as `svc: line one line two` and the wrapper's counter reads exactly 1, numbers that pin in all eight lanes. Stacking two sanitizers composes with only the innermost recording, a row the rust, C#, java, python, and lua lanes pin.

```c
// the decorator: it holds the next logger by its interface value and
// counts one rewrite per call
typedef struct {
  logger next;
  void *next_self;
  int sanitized;
} sanitizing_logger;

static void replace_newlines(char *buf) {
  for (char *c = buf; *c; c++)
    if (*c == '\n')
      *c = ' ';
}

static void sanitizing_log(void *self, const char *msg) {
  sanitizing_logger *s = self;
  char clean[128];
  snprintf(clean, sizeof clean, "%s", msg);
  replace_newlines(clean);
  s->sanitized++;
  s->next.log(s->next_self, clean);
}

// the decorator satisfies logger itself, so decorations stack
static const logger sanitizing_iface = {sanitizing_log};

static sanitizing_logger new_sanitizing(logger next, void *target) {
  sanitizing_logger s = {.next = next, .next_self = target};
  return s;
}
```

listing 10: C, the decorator holds the next interface value, the wrap is a function plus its state struct (source: patterns-concurrency-distributed/samples-c/src/Ch01/decorator.c, lines 44-73)

```rust
struct SanitizingLogger<L: Logger> {
    next: L,
    sanitized: usize,
}

impl<L: Logger> SanitizingLogger<L> {
    fn new(next: L) -> Self {
        SanitizingLogger {
            next,
            sanitized: 0,
        }
    }
}

impl<L: Logger> Logger for SanitizingLogger<L> {
    fn log(&mut self, msg: &str) {
        let clean: String = msg
            .chars()
            .map(|c| if c == '\n' { ' ' } else { c })
            .collect();
        self.sanitized += 1;
        self.next.log(&clean);
    }
}
```

listing 11: Rust, the decorator is generic over the logger it owns by value, no trait object anywhere (source: patterns-concurrency-distributed/samples-rust/src/Ch01/decorator.rs, lines 34-57)

```go
// SanitizingLogger decorates any Logger and strips newlines so a
// hostile message cannot forge extra log lines. Accept an interface,
// return a concrete type.
type SanitizingLogger struct {
  Next     Logger
  sanitized int
}

func NewSanitizing(next Logger) *SanitizingLogger { return &SanitizingLogger{Next: next} }

func (s *SanitizingLogger) Log(msg string) {
  s.sanitized++
  s.Next.Log(strings.ReplaceAll(msg, "\n", " "))
}

func (s *SanitizingLogger) SanitizedCount() int { return s.sanitized }
```

listing 12: Go, decorator number one: sanitize before forwarding (source: patterns-concurrency-distributed/samples/ch01/discipline.go, lines 34-49)

```
  // the decorator: it holds the next logger by its interface value and
  // counts one rewrite per call. it satisfies Logger itself, so
  // decorations stack
  static final class SanitizingLogger implements Logger {
    final Logger next;
    int sanitized;

    SanitizingLogger(Logger next) {
      this.next = next;
    }

    public void log(String msg) {
      sanitized++;
      next.log(msg.replace("\n", " "));
    }

    int sanitizedCount() {
      return sanitized;
    }
  }
```

listing 13: Java, the constructor takes the interface, the returned class keeps its counter (source: patterns-concurrency-distributed/samples-java/src/Ch01/Decorator.java, lines 40-59)

```
namespace PatternsBook.Ch01;

// SanitizingLogger decorates any Logger and strips newlines so a
// hostile message cannot forge extra log lines. Accept the interface,
// hand back a concrete type: the constructor takes Logger, the result
// is a SanitizingLogger with its full API, no casts needed.
public sealed class SanitizingLogger(Logger next) : Logger
{
    private int _sanitized;

    public Logger Next { get; } = next;

    public void Log(string msg)
    {
        _sanitized++;
        Next.Log(msg.Replace("\n", " "));
    }

    public int SanitizedCount() => _sanitized;
}
```

listing 14: C#, the constructor takes the interface, the result is the concrete class (source: patterns-concurrency-distributed/samples-cs/src/Ch01/Decorator.cs, lines 1-20)

```javascript
// ch01: the decorator. SanitizingLogger wraps any logger and strips
// newlines so a hostile message cannot forge extra log lines. accept
// the duck-typed shape, return a concrete class.

export class SanitizingLogger {
  constructor(next) {
    this.next = next;
    this.sanitized = 0;
  }

  log(msg) {
    this.sanitized++;
    this.next.log(msg.replaceAll("\n", " "));
  }

  sanitizedCount() {
    return this.sanitized;
  }
}
```

listing 15: JavaScript, the wrapper is a class over any object with a log method (source: patterns-concurrency-distributed/samples-js/src/ch01-decorator.mjs, lines 1-19)

```python
class Discard:
    def log(self, msg):
        pass


class SanitizingLogger:
    # strips newlines so a hostile message cannot forge extra log
    # lines. next is any logger, the wrapper satisfies the same
    # protocol, and callers keep the wrapper's own api
    def __init__(self, nxt):
        self.nxt = nxt
        self.sanitized = 0

    def log(self, msg):
        self.sanitized += 1
        self.nxt.log(msg.replace("\n", " "))

    def sanitized_count(self):
        return self.sanitized
```

listing 16: Python, the null object rides underneath, the wrapper stays inspectable (source: patterns-concurrency-distributed/samples-py/src/Ch01/decorator.py, lines 30-48)

```lua
local Sanitizing = {}
Sanitizing.__index = Sanitizing

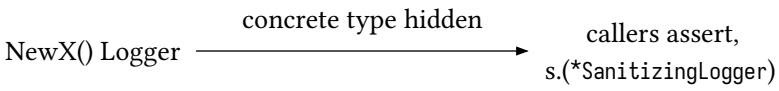
-- next is any logger; the return value is a concrete object whose
-- whole api the caller keeps
local function NewSanitizing(next)
  return setmetatable({ next = next, sanitized = 0 }, Sanitizing)
end

function Sanitizing:Log(msg)
  self.sanitized = self.sanitized + 1
  self.next:Log((msg:gsub("\n", " ")))
end

function Sanitizing:SanitizedCount()
  return self.sanitized
end
```

listing 17: Lua, Sanitizing is a metatable, decoration stacks through next (source: patterns-concurrency-distributed/samples-lua/ch01_decorator.lua, lines 26-42)

`NewSanitizing` accepts any `Logger` and hands back a `*SanitizingLogger` with its own extra methods. The reverse shape, a function returning an interface, hides the concrete type and forces assertions on every caller that wants more. C keeps the distinction cheap by returning the concrete struct by value, rust the same way one generic deeper, `SanitizingLogger<L: Logger>` hands back a type whose parameter already fixed the inner logger at build time, C# and java by returning the class, and in javascript, python, and lua the returned object already carries everything it has, so the go row mostly evaporates and the samples say so instead of imitating a cast. There is a place for returning interfaces, error values and empty-interface style apis being the honest ones, but it is the exception.



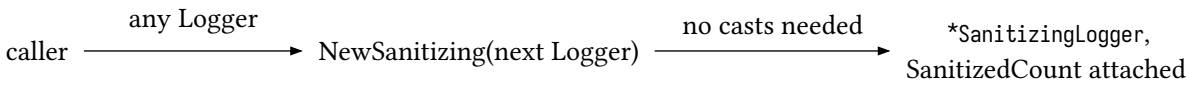


figure 2: any logger in, the concrete sanitizer out, the reverse hides the type

1.4. embedding is forwarding, not inheritance

Struct embedding looks like subclassing and behaves like composition. The methods of the embedded type are promoted, available on the outer type without forwarding stubs, but the receiver stays the inner type. Effective Go's phrasing is exact: when an embedded method is invoked, the receiver is the inner type, not the outer one.

The dry run: `d.Name()` resolves to the outer override in every language. `d.Describe()` prints `i am base` in go, C, and rust, and `i am derived` in C#, java, javascript, python, and lua. That split is the whole section, and each lane's name counter proves which call actually ran.

```c
typedef struct {
  int name_calls;
} base;

static const char *base_name(base *b) {
  b->name_calls++;
  return "base";
}

static void base_describe(base *b, char *out, int sz) {
  snprintf(out, sz, "i am %s", base_name(b));
}

// derived embeds base and overrides name. what go spells as promotion,
// c spells as the compiler letting derived reach d.base fields directly
typedef struct {
  base base;
  const char *override;
} derived;

static const char *derived_name(derived *d) { return d->override; }

// the promoted describe: a one line forward into base's code, exactly
// what go generates for the embedded method set
static void derived_describe(derived *d, char *out, int sz) {
  base_describe(&d->base, out, sz);
}
```

listing 18: C, derived embeds base as a member, the forward into base's code is written out by hand (source: patterns-concurrency-distributed/samples-c/src/Ch01/embedding.c, lines 19-45)

```rust
struct Base {
    name_calls: usize,
}

impl Base {
    fn name(&mut self) -> &'static str {
        self.name_calls += 1;
        "base"
    }

    fn describe(&mut self) -> String {
        format!("i am {}", self.name())
    }

    fn call_count(&self) -> usize {
        self.name_calls
    }
}

struct Derived {
    base: Base,
    outer_name: &'static str,
}

impl Derived {
    fn name(&self) -> &'static str {
        self.outer_name
    }

    fn describe(&mut self) -> String {
        self.base.describe()
    }
}

impl Deref for Derived {
    type Target = Base;

    fn deref(&self) -> &Base {
        &self.base
    }
}
```

listing 19: Rust, derived holds base by value, describe forwards into it, and Deref promotes base methods with the inner receiver (source: patterns-concurrency-distributed/samples-rust/src/Ch01/embedding.rs, lines 15-55)

```go
// Base and Derived demonstrate the embedding trap. Derived promotes
// Name and Describe from Base, but Describe's body calls Base.Name
// even when the outer type declares its own: promotion is forwarding,
// not virtual dispatch.
type Base struct {
  nameCalls int
}

func (b *Base) Name() string { b.nameCalls++; return "base" }

func (b *Base) Describe() string { return "i am " + b.Name() }

type Derived struct {
  Base
  Override string
}

func (d *Derived) Name() string { return d.Override }
```

listing 20: Go, derived overrides name, but describe never notices (source: patterns-concurrency-distributed/samples/ch01/discipline.go, lines 51-68)

```
  static class Base {
    int nameCalls;

    String name() {
      nameCalls++;
      return "base";
    }

    String describe() {
      return "i am " + name();
    }
  }

  static class Derived extends Base {
    final String override;

    Derived(String override) {
      this.override = override;
    }

    @Override
    String name() {
      return override;
    }
  }

  // Holder reproduces go's embedding exactly: it holds a base in a
  // field and forwards describe into base's closed code, so the outer
  // name override can never reach the call inside
  static final class Holder {
    final Base base = new Base();
    final String override;

    Holder(String override) {
      this.override = override;
    }

    String name() {
      return override;
    }

    String describe() {
      return base.describe();
    }
  }
```

listing 21: Java, describe dispatches name virtually, and the Holder class forwards into a held base to rebuild go's trap on purpose (source: patterns-concurrency-distributed/samples-java/src/Ch01/Embedding.java, lines 19-63)

```cs
namespace PatternsBook.Ch01;

// Base and Derived demonstrate the contrast with go's embedding trap.
// Describe is Base's code, but Name is virtual: the call dispatches to
// the outermost override, so Derived.Name is visible from inside.
public class Base
{
    public int NameCalls { get; protected set; }

    public virtual string Name()
    {
        NameCalls++;
        return "base";
    }

    public string Describe() => "i am " + Name();
}

public class Derived(string @override) : Base
{
    public string Override { get; } = @override;
    public int DerivedNameCalls { get; private set; }

    public override string Name()
    {
        DerivedNameCalls++;
        return Override;
    }
}

// Hidden reproduces go's embedding exactly: `new` hides rather than
// overrides, so Name through a Hidden reference is a separate method
// that Base's own code can never see. Describe still runs Base.Name.
public class Hidden(string @override) : Base
{
    public string Override { get; } = @override;

    public new string Name() => Override;
}
```

listing 22: C#, Name is virtual so Describe reaches the override, and new-hiding reproduces go's trap (source: patterns-concurrency-distributed/samples-cs/src/Ch01/Embedding.cs, lines 1-39)

```javascript
// ch01: the embedding trap, js edition. a class chain dispatches
// methods virtually: describe is base's code, but the name it calls
// resolves through the receiver, so the derived override always wins.
// go's embedding is forwarding and the base method wins instead; that
// contrast is the whole section.

export class Base {
  constructor() {
    this.baseNameCalls = 0;
  }

  name() {
    this.baseNameCalls++;
    return "base";
  }

  describe() {
    return "i am " + this.name();
  }
}

export class Derived extends Base {
  constructor(override) {
    super();
    this.override = override;
  }

  name() {
    return this.override;
  }
}
```

listing 23: JavaScript, the prototype chain dispatches through the receiver, the override always wins (source: patterns-concurrency-distributed/samples-js/src/ch01-embedding.mjs, lines 1-31)

```python
class Base:
    def __init__(self):
        self.name_calls = 0

    def name(self):
        self.name_calls += 1
        return "base"

    def describe(self):
        return "i am " + self.name()


class Derived(Base):
    # inherits describe, overrides name
    def __init__(self, override):
        super().__init__()
        self.override = override

    def name(self):
        return self.override
```

listing 24: Python, self always consults the instance's class first, the go trap has no analog (source: patterns-concurrency-distributed/samples-py/src/Ch01/embedding.py, lines 15-34)

```lua
local Base = {}
Base.__index = Base

function Base.new()
  return setmetatable({ nameCalls = 0 }, Base)
end

function Base:Name()
  self.nameCalls = self.nameCalls + 1
  return "base"
end

function Base:Describe()
  return "i am " .. self:Name()
end

-- Derived reuses Base's methods through the class chain but carries
-- its own Name, which shadows the promoted one
local Derived = {}
Derived.__index = Derived
setmetatable(Derived, { __index = Base })

function Derived.new(override)
  return setmetatable({ override = override, nameCalls = 0 }, Derived)
end

function Derived:Name()
  self.nameCalls = self.nameCalls + 1
  return self.override
end
```

listing 25: Lua, the class chain shares base's methods, lookup stays receiver-driven (source: patterns-concurrency-distributed/samples-lua/ch01_embedding.lua, lines 8-37)

C matches go by construction: embedding a base struct as the first member and forwarding `derived_describe` into `base_describe` is exactly what go's promotion generates, and neither runtime re-dispatches through the outer type. Rust joins them by ownership: `Derived` holds `Base` as a plain field, `describe` is a one line forward into `self.base.describe()`, and `Deref` is the promotion itself, `d.call_count()` compiles only because the borrow derefs to the inner value. The five class-dispatch languages agree with each other instead: `Describe` is one virtual call away from the override. C#'s file carries both behaviors in one language, virtual `Name` for the contrast and the `new`-hiding `Hidden` class for go's exact trap, which makes it the one place C# reads simpler than go, and java needs the keyword for neither half: `Derived extends Base` gets the virtual dispatch for free while `Holder` forwards `describe` into a held base against a fixed receiver, the trap rebuilt by hand.

The test side is half the fixture, and each tree answers with its own test artifact: the frozen go test file, the xunit and node test files, and the check regions of the C, rust, java, python, and lua modules.

```c
int main(void) {
  derived d = {.override = "derived"};
  char out[64];

  // promotion: name resolves to the outermost method
  CHECK(strcmp(derived_name(&d), "derived") == 0);

  // the trap: describe is base's code, its name call is base's
  derived_describe(&d, out, sizeof out);
  CHECK(strcmp(out, "i am base") == 0);
  CHECK(d.base.name_calls == 1);

  // reaching the embedded value directly behaves identically
  base_describe(&d.base, out, sizeof out);
  CHECK(strcmp(out, "i am base") == 0);
  CHECK(d.base.name_calls == 2);

  // the fix is delegation the caller can reach: build describe from the
  // outer name instead of forwarding into base's closed code
  snprintf(out, sizeof out, "i am %s", derived_name(&d));
  CHECK(strcmp(out, "i am derived") == 0);

  printf("ok %d embedding\n", n);
  return 0;
```

listing 26: C, the check region: promotion reads the outer name, describe stays base, the counter catches it (source: patterns-concurrency-distributed/samples-c/src/Ch01/embedding.c, lines 47-70)

```rust
fn main() {
    let mut d = Derived {
        base: Base { name_calls: 0 },
        outer_name: "derived",
    };

    ok(
        d.name() == "derived",
        "name resolves to the outermost method, no promotion needed",
    );

    let out = d.describe();
    ok(
        out == "i am base",
        "the forward runs base's code, so its name call is base's",
    );
    ok(
        d.base.name_calls == 1,
        "the counter proves the inner name ran",
    );

    let direct = d.base.describe();
    ok(
        direct == "i am base",
        "reaching the embedded value directly behaves identically",
    );
    ok(d.base.name_calls == 2, "the second describe calls base's name again");

    ok(
        format!("i am {}", d.name()) == "i am derived",
        "the fix is delegation the caller can reach, built from the outer name",
    );

    ok(
        d.call_count() == 2,
        "deref promotion reaches base methods with the inner receiver",
    );
}
```

listing 27: Rust, the check region: the outer name wins, the forward runs base's code, deref still reaches the inner counter (source: patterns-concurrency-distributed/samples-rust/src/Ch01/embedding.rs, lines 57-94)

```
}

func TestEmbeddingIsNotInheritance(t *testing.T) {
  d := &Derived{Override: "derived"}

  // promotion: Name resolves to Derived.Name, the outermost method.
  if d.Name() ≠ "derived" {
    t.Fatalf("d.Name() = %q", d.Name())
  }

  // the trap: Describe is Base's code, so the Name it calls is
  // Base.Name, never the outer override.
  if got := d.Describe(); got ≠ "i am base" {
    t.Fatalf("d.Describe() = %q, forwarding must not see the override", got)
  }
  if d.nameCalls ≠ 1 {
    t.Fatalf("base.Name called %d times, want 1", d.nameCalls)
  }
```

listing 28: Go, the outer method wins by name, the inner body sees only the inner receiver (source: patterns-concurrency-distributed/samples/ch01/discipline_test.go, lines 27-44)

```
  public static void main(String[] args) {
    Derived d = new Derived("derived");
    ok(d.name().equals("derived"), "the override wins on the instance");
    ok(d.describe().equals("i am derived"), "describe dispatches name virtually");
    ok(d.nameCalls == 0, "base.name never ran, dispatch skipped it");

    Holder h = new Holder("derived");
    ok(h.describe().equals("i am base"), "forwarding into the held base prints the base name");
    ok(h.base.nameCalls == 1, "the counter proves which name ran");

    Cooperating c = new Cooperating("derived");
    ok(c.name().equals("derived"), "the override still wins directly");
    ok(c.describe().equals("i am base"), "but describe can pin the base explicitly");
    ok(c.nameCalls == 1, "the super call is the only base.name hit");
  }
```

listing 29: Java, the check region: dispatch skips base.name, the Holder counter catches the forward, super pins the base (source: patterns-concurrency-distributed/samples-java/src/Ch01/Embedding.java, lines 85-99)

```
using PatternsBook.Ch01;

namespace PatternsBook.Tests.Ch01;

public class EmbeddingTests
{
    [Fact]
    public void VirtualDispatchSeesTheOverride()
    {
        var d = new Derived("derived");

        // dispatch: Name resolves to Derived.Name, the outermost method
        Assert.Equal("derived", d.Name());

        // and Describe, though written in Base, calls the override:
        // this is the virtual-dispatch contrast with go's forwarding
        Assert.Equal("i am derived", d.Describe());
        Assert.Equal(0, d.NameCalls);
        Assert.Equal(2, d.DerivedNameCalls);
    }

    [Fact]
    public void HidingReproducesTheForwardingTrap()
    {
        var h = new Hidden("derived");

        // static dispatch: through the Hidden reference the new method
        Assert.Equal("derived", h.Name());

        // but Base's code keeps calling Base.Name: forwarding, not
        // dispatch, exactly like go's embedded method promotion
        Assert.Equal("i am base", h.Describe());
        Assert.Equal(1, h.NameCalls);
    }
```

listing 30: C#, xunit pins virtual dispatch seeing the override and hiding reproducing forwarding (source: patterns-concurrency-distributed/samples-cs/tests/Ch01/EmbeddingTests.cs, lines 1-34)

```
describe("ch01 embedding", () => {
  it("promotion analog: name resolves to the outer override", () => {
    const d = new Derived("derived");
    assert.equal(d.name(), "derived");
  });

  it("the contrast: describe is virtual, so the override wins", () => {
    const d = new Derived("derived");
    // go forwards describe to base.name and prints "i am base";
    // js dispatches through the receiver and prints the override.
    assert.equal(d.describe(), "i am derived");
    // the override intercepted every call: base.name never ran.
    assert.equal(d.baseNameCalls, 0);
  });

  it("a plain base still describes itself", () => {
    const b = new Base();
    assert.equal(b.describe(), "i am base");
    assert.equal(b.baseNameCalls, 1);
  });
});
```

listing 31: JavaScript, node:test pins the contrast, the override intercepts every call (source: patterns-concurrency-distributed/samples-js/test/ch01.test.mjs, lines 48-68)

```
d = Derived("derived")
check(d.name() == "derived", "the override wins on the instance")
check(d.describe() == "i am derived", "describe dispatches self.name virtually")
check(d.name_calls == 0, "base.name never ran, dispatch skipped it")

# the delegation a caller can still reach: call the base
# implementation explicitly through the class
check(Base.name(d) == "base", "the base method itself is reachable")
check(d.name_calls == 1, "and it counts when called that way")


# the go fix mapped to python: an override that wants the base
# behavior asks for it with super instead of hoping for forwarding
class Cooperating(Base):
    def __init__(self, override):
        super().__init__()
        self.override = override

    def name(self):
        return self.override

    def describe(self):
        return "i am " + super().name()


c = Cooperating("derived")
check(c.name() == "derived", "the override still wins directly")
check(c.describe() == "i am base", "but describe can pin the base explicitly")
check(c.name_calls == 1, "the super call is the only base.name hit")


# and inheritance alone gives the base behavior to a subclass that
# overrides nothing
class Plain(Base):
    pass


p = Plain()
check(p.describe() == "i am base", "an untouched subclass forwards fine")
check(p.name_calls == 1, "through the one base.name call")
```

listing 32: Python, the script region: dispatch skips base.name, super() pins it back on purpose (source: patterns-concurrency-distributed/samples-py/src/Ch01/embedding.py, lines 36-76)

```
return {
  {name = "the outer override wins both lookups", fn = function()
    local d = Derived.new("derived")
    T.eq(d:Name(), "derived")
    T.eq(d:Describe(), "i am derived")
    T.eq(d.nameCalls, 2)
  end},

  {name = "dispatch reaches the override: base's counter is untouched", fn = function()
    local d = Derived.new("derived")
    T.eq(d:Describe(), "i am derived")
    T.eq(d.nameCalls, 1)
    local b = Base.new()
    T.eq(b:Describe(), "i am base")
    T.eq(b.nameCalls, 1)
  end},

  {name = "forwarding against a fixed receiver reproduces the go behavior", fn = function()
    local h = Holder.new("derived")
    T.eq(h:Name(), "derived")
    T.eq(h:Describe(), "i am base")
    T.eq(h.base.nameCalls, 1)
  end},

  {name = "promotion is reuse without declaration", fn = function()
    T.eq(rawget(Derived, "Describe"), nil)
    T.eq(rawget(Derived, "Name") ~= nil, true)
    T.eq(Derived.new("x"):Describe(), "i am x")
  end},
}
```

listing 33: Lua, the row table: dispatch reaches the override, a fixed-receiver forward reproduces go (source: patterns-concurrency-distributed/samples-lua/ch01_embedding.lua, lines 60-89)

`d.Name()` resolves to the outer method, ordinary method-set rules. But `d.Describe()` runs `Base`'s code, and inside that code `b.Name()` is `Base.Name`, so the override is invisible and the counter proves the inner method ran. In a virtual dispatch language `Describe` would print `i am derived`. Template method, the gang of four pattern that depends on exactly that late binding, does not translate: the go shape is to pass the varying piece in, as a function value or an interface field, which chapter 5 does with closures.

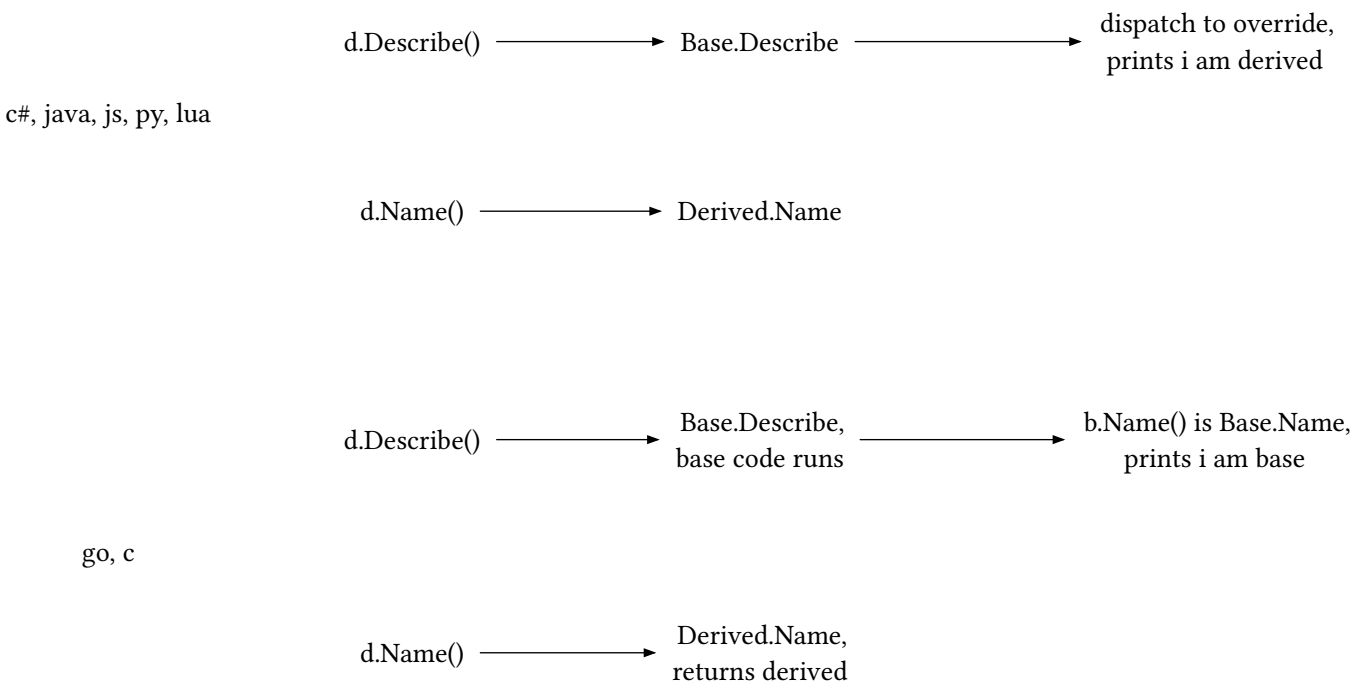


figure 3: go and c forward, the five class-dispatch languages reach the override

When composition needs the outer type, the fix is explicit delegation, a named field plus forwarding methods, or an interface field the inner code consults. Both are shown by contrast in the decorator above: `SanitizingLogger` holds `Next Logger` and forwards through it, so the chain is traversed by value, not by implicit receiver. Python's `Cooperating` class in the check region shows the same fix, `super().name()` when base behavior is what the override wants, and java's `Cooperating` spells the same pin `super.name()`.

1.5. what each language changes about the shapes

Two go facts keep the reference lane's pattern code honest this year. Methods may now declare their own type parameters, so a generic helper can be a method on a concrete type, while interface methods still cannot be generic, so a strategy or visitor interface cannot demand type-parameterized behavior from its implementors. Struct literal keys accept field selectors, which trims some builder-shaped ceremony around nested literals, covered with the creational chapter.

The wider question is what each tree changes about every shape, and the answer is one line per language, the cards this book keeps filling in:

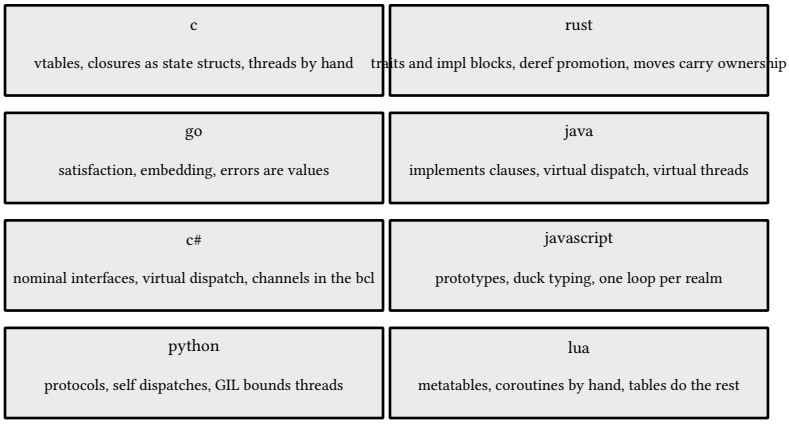


figure 4: eight rule cards, one language each, the idiom summary every chapter returns to

The table this book keeps repeating, set once here in its go spelling:

NOTE: translation table

The diagram below is the table. Two cells carry less than their rows deserve: singleton is package scope, or `sync.Once` when lazy, chapter 7, and adapter is a one method wrapper that needs no inheritance because satisfaction is structural. Every row has a per-language spelling, the map this chapter opened with.

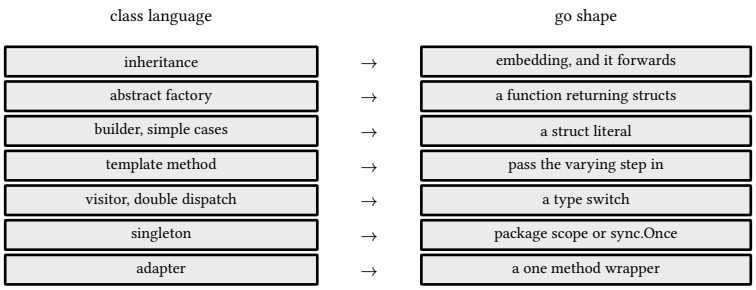


figure 5: the translation table, class patterns to go shapes

1.6. across the eight languages

The build sizes count non-comment source lines, the go column the chapter's one frozen file:

language	build SLOC	dependency	boundary note
c	171	libc	vtable of function pointers with void self, embedding a one-line forward
rust	224	std only, no crates	impl blocks satisfy traits, const fn items prove it, Deref is embedding by hand
go	38	stdlib	satisfaction needs no declaration, the promoted Describe forwards into base
java	181	jdk 27 stdlib	the implements clause is the whole proof, go's forwarding trap must be rebuilt by hand as the Holder class
c#	55	bcl	the declaration is the proof, virtual Name dispatches to the outermost override
javascript	49	node stdlib	duck typing proven by a runtime predicate, typeof x.log the whole check
python	127	stdlib only	runtime_checkable isinstance standing in for go's compile-time assertion
lua	163	lib.lua harness	metatable objects satisfy by carrying the method, checked one lookup too late

sources: go.dev/doc/effective_go for satisfaction, embedding, and receiver semantics, go.dev/doc/go1.27 for generic methods and literal selector keys, accessed 2026-09-08. Verified by the eight chapter legs: 3 Ch01 C programs with 16 embedded checks, 3 Ch01 rust programs in `samples-rust/src` with 18 embedded checks under `run-rust-samples`, `go test` at 4 tests in `patternsbook/ch01`, the java runner's 22 Ch01 checks under `run-java-samples`, 9 xunit facts, node's 8 cases in `test/ch01.test.mjs`, 3 python modules with 25 embedded checks, and the lua runner's 12 ch01 rows.

2. creational patterns

Creation gravitates toward the plain in every tree this book runs. Go builds a struct literal with named fields, C an aggregate initializer, rust a struct expression that names every field, java a constructor over plain fields, C# an object initializer, javascript and lua a literal object or table, python a call with keyword arguments, and each form is constructor, builder, and configuration file in one. The creational patterns that earn their keep are the ones solving a problem the plain form cannot express, validating before the value exists, assembling over several fallible steps, deferring work to first use, or collapsing duplicates of unbounded input.

2.1. constructors are functions

The `New` prefix convention is the whole factory story for most go types. A function, unlike a constructor in a class language, can refuse, and every tree refuses in its own grain, a `(T, error)` pair in go, a reason string beside an untouched out struct in C, a `Result` carrying the reason in rust, a thrown exception in java, C#, javascript, and python, nil with a message in lua. Nothing stops a type from having several:

```
func NewServer(name string, opts ...Option) (*Server, error)
func NewTLSProxy(name string, cert tls.Certificate) (*Server, error)
```

fragment, illustrative, not executed

The error return is the part that carries the weight. Class languages throw from inside a failed constructor. In go the invalid value never escapes, in C the out parameter is never written, the C options lane pins a sentinel struct surviving a rejected build untouched, and every caller handles the failure at the same place it asked for the object.

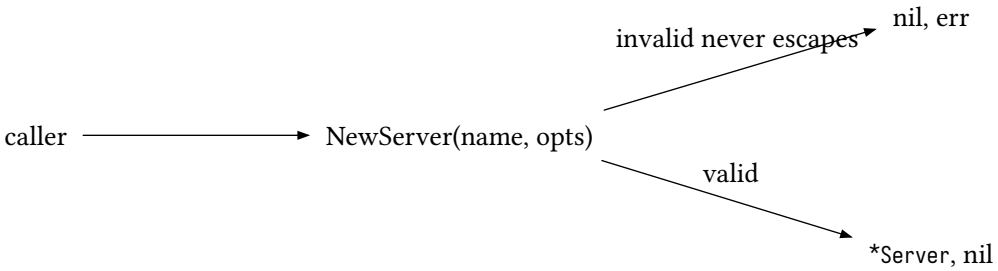


figure 6: a constructor returns the failure instead of throwing from inside

2.2. functional options

When a constructor grows past a handful of optional knobs, positional flags become unreadable and a nil-heavy config struct becomes a second api. Functional options keep named construction with per-option validation, one function value per knob that mutates the target or refuses:

The dry run: the fixture builds one server with `WithTimeout(1s)` and `WithReadOnly()` and reads timeout 1s beside the untouched default of 3 retries and read only true. The rejected lanes are a negative timeout and 11 retries, each failing construction with its named reason wrapped inside the server's name. The go lane's frozen test pins its own rejection constant, 99 against the same range check, both verified against the same implementation.

1. Defaults live in one place, timeout 5 seconds and retries 3, and each option overrides exactly one field.
2. A rejected option fails the whole build and no partial server escapes, while zero retries is an override, the boundary the range check draws.

```c
// the option type: apply inspects the captured argument, mutates the
// half-built server, and returns NULL or a reason that outlives the call
typedef struct {
  const char *(*apply)(const void *arg, server *s, char *err, int errsz);
  const void *arg;
} option;

static const char *apply_timeout(const void *arg, server *s, char *err,
                                 int errsz) {
  long ms = *(const long *)arg;
  if (ms ≤ 0) {
    snprintf(err, errsz, "timeout must be positive, got %ld", ms);
    return err;
  }
  s->timeout_ms = ms;
  return NULL;
}

static const char *apply_retries(const void *arg, server *s, char *err,
                                 int errsz) {
  int count = *(const int *)arg;
  if (count < 0 || count > 10) {
    snprintf(err, errsz, "retries out of range: %d", count);
    return err;
  }
  s->retries = count;
  return NULL;
}

static const char *apply_read_only(const void *arg, server *s, char *err,
                                   int errsz) {
  (void)arg;
  (void)err;
  (void)errsz;
  s->read_only = true;
  return NULL;
}

static option with_timeout(const long *ms) {
  return (option){apply_timeout, ms};
}

static option with_retries(const int *count) {
  return (option){apply_retries, count};
}

static option with_read_only(void) { return (option){apply_read_only, NULL}; }
```

listing 34: C, no closures, an option is an apply function pointer beside its captured argument (source: patterns-concurrency-distributed/samples-c/src/Ch02/options.c, lines 27-73)

```rust
type ServerOption = Box<dyn Fn(&mut Server) -> Result<(), String>>;

fn with_timeout(ms: i64) -> ServerOption {
    Box::new(move |s| {
        if ms ≤ 0 {
            Err(format!("timeout must be positive, got {ms}"))
        } else {
            s.timeout_ms = ms;
            Ok(())
        }
    })
}

fn with_retries(count: i32) -> ServerOption {
    Box::new(move |s| {
        if !(0..=10).contains(&count) {
            Err(format!("retries out of range: {count}"))
        } else {
            s.retries = count;
            Ok(())
        }
    })
}

fn with_read_only() -> ServerOption {
    Box::new(|s| {
        s.read_only = true;
        Ok(())
    })
}

fn new_server(name: &str, opts: Vec<ServerOption>) -> Result<Server, String> {
    let mut s = Server {
        name: name.to_string(),
        timeout_ms: 5000,
        retries: 3,
        read_only: false,
    };
    for opt in &opts {
        if let Err(reason) = opt(&mut s) {
            return Err(format!("server \"{name}\": {reason}"));
        }
    }
    Ok(s)
}
```

listing 35: Rust, an option is a boxed closure returning Result, the fold wraps refusal in the server name (source: patterns-concurrency-distributed/samples-rust/src/Ch02/options.rs, lines 21-65)

```
type Option func(*Server) error

func WithTimeout(d time.Duration) Option {
  return func(s *Server) error {
    if d <= 0 {
      return fmt.Errorf("timeout must be positive, got %s", d)
    }
    s.Timeout = d
    return nil
  }
}
```

listing 36: Go, an option is a closure over one field (source: patterns-concurrency-distributed/samples/ch02/creational.go, lines 20-30)

```
func NewServer(name string, opts ...Option) (*Server, error) {
  s := &Server{Name: name, Timeout: 5 * time.Second, Retries: 3}
  for _, opt := range opts {
    if err := opt(s); err ≠ nil {
      return nil, fmt.Errorf("server %q: %w", name, err)
    }
  }
  return s, nil
}

// Request uses a builder because assembly is multi step and can fail
```

listing 37: Go, defaults in one place, options applied in order (source: patterns-concurrency-distributed/samples/ch02/creational.go, lines 46-56)

```
  // the option type: inspect the captured argument, mutate the
  // half-built server, return null when applied or the rejection
  // reason
  interface Option {
    String apply(Server s);
  }

  static Option withTimeout(long ms) {
    return s -> {
      if (ms <= 0)
        return "timeout must be positive, got " + ms;
      s.timeoutMs = ms;
      return null;
    };
  }

  static Option withRetries(int count) {
    return s -> {
      if (count < 0 || count > 10)
        return "retries out of range: " + count;
      s.retries = count;
      return null;
    };
  }

  static Option withReadOnly() {
    return s -> {
      s.readOnly = true;
      return null;
    };
  }

  // defaults here, overrides one field at a time, first failure aborts
  // the build with the server name wrapped around the option's reason
  static Server newServer(String name, Option... opts) {
    Server s = new Server(name);
    s.timeoutMs = 5000;
    s.retries = 3;
    s.readOnly = false;
    for (Option o : opts) {
      String reason = o.apply(s);
      if (reason != null)
        throw new ServerError("server \"" + name + "\": " + reason);
    }
    return s;
  }
```

listing 38: Java, an option is a lambda over the half-built server, null when applied or its rejection reason (source: patterns-concurrency-distributed/samples-java/src/Ch02/Options.java, lines 38-83)

```
// an option inspects or mutates the server under construction and
// returns null on success or the rejection reason
public delegate string? Option(Server s);

public static class ServerFactory
{
    public static Option WithTimeout(TimeSpan d) => s =>
    {
        if (d <= TimeSpan.Zero)
        {
            return "timeout must be positive, got " + d;
        }
        s.Timeout = d;
        return null;
    };

    public static Option WithRetries(int n) => s =>
    {
        if (n < 0 || n > 10)
        {
            return "retries out of range: " + n;
        }
        s.Retries = n;
        return null;
    };

    public static Option WithReadOnly() => s =>
    {
        s.ReadOnly = true;
        return null;
    };

    public static (Server? server, string? error) NewServer(string name, params Option[] opts)
    {
        var s = new Server { Name = name, Timeout = TimeSpan.FromSeconds(5), Retries = 3 };
        foreach (var opt in opts)
        {
            var err = opt(s);
            if (err is not null)
            {
                return (null, "server \"" + name + "\": " + err);
            }
        }
        return (s, null);
    }
}
```

listing 39: C#, the delegate list mirrors go one to one, each lambda capturing its argument (source: patterns-concurrency-distributed/samples-cs/src/Ch02/Options.cs, lines 13-58)

```
export function withTimeout(ms) {
  return (s) => {
    if (ms ≤ 0) return new Error(`timeout must be positive, got ${ms}`);
    s.timeout = ms;
    return null;
  };
}

export function withRetries(n) {
  return (s) => {
    if (n < 0 || n > 10) return new Error(`retries out of range: ${n}`);
    s.retries = n;
    return null;
  };
}

export function withReadOnly() {
  return (s) => {
    s.readOnly = true;
    return null;
  };
}

export function newServer(name, ...opts) {
  const s = { name, timeout: 5000, retries: 3, readOnly: false };
  for (const opt of opts) {
    const err = opt(s);
    if (err) throw new Error(`server "${name}": ${err.message}`);
  }
  return s;
}
```

listing 40: JavaScript, arrow closures over one object, the constructor throws on the first refusal (source: patterns-concurrency-distributed/samples-js/src/ch02-options.mjs, lines 5-35)

```
def with_timeout(seconds):
    def opt(s):
        if seconds ≤ 0:
            raise ServerError(f"timeout must be positive, got {seconds}s")
        s.timeout = seconds

    return opt


def with_retries(count):
    def opt(s):
        if count < 0 or count > 10:
            raise ServerError(f"retries out of range: {count}")
        s.retries = count

    return opt


def with_read_only():
    def opt(s):
        s.read_only = True

    return opt


def new_server(name, *opts):
    s = Server(name)
    for opt in opts:
        try:
            opt(s)
        except ServerError as e:
            raise ServerError(f'server "{name}": {e}') from e
    return s
```

listing 41: Python, closures validate and raise, new_server wraps the failure with the server name (source: patterns-concurrency-distributed/samples-py/src/Ch02/options.py, lines 26-58)

```lua
-- Server(name, opts): the constructor every option mutates
local function Server(name, opts)
  local s = { name = name, timeout = 5000, retries = 3, readonly = false }
  for _, opt in ipairs(opts) do
    local err = opt(s)
    if err then
      return nil, ("server %q: %s"):format(name, err)
    end
  end
  return s, nil
end

local function WithTimeout(ms)
  return function(s)
    if ms ≤ 0 then
      return ("timeout must be positive, got %d"):format(ms)
    end
    s.timeout = ms
    return nil
  end
end

local function WithRetries(n)
  return function(s)
    if n < 0 or n > 10 then
      return ("retries out of range: %d"):format(n)
    end
    s.retries = n
    return nil
  end
end

local function WithReadOnly()
  return function(s)
    s.readonly = true
    return nil
  end
end
```

listing 42: Lua, each option returns nil or a reason, Server folds them and wraps the first refusal (source: patterns-concurrency-distributed/samples-lua/ch02_options.lua, lines 6-43)

Each `With` function returns a closure that mutates the target and reports why it cannot, and `NewServer` seeds the defaults and folds the options in, so partial application is impossible to get wrong. C has no closures, so its option is an apply function pointer beside a pointer to the caller-owned argument, the captured state a closure would hide written out by hand. Rust has closures but one static type per function, so heterogeneous options meet in a `Box<dyn Fn(&mut Server) → Result<(), String>>`, one allocation and one virtual call per knob buying the single signature the fold needs. C# mirrors go one to one because a lambda over the parameter is the same closure, and java is one lane over: its `Option` is a single-method interface, so the lambda is the option, returning null when applied or the reason when refusing. The dynamic three already own cheap named construction, keyword arguments and object literals, so what the closure list buys them is the refusal row, one checked rejection per knob. The alternatives each fail differently, and knowing how is the tradeoff: config structs accept anything and validate later, setters allow partially configured intermediates, and subtypes of a fluent builder duplicate the validation. Options cost one closure per call site, which is the price of validation at creation.

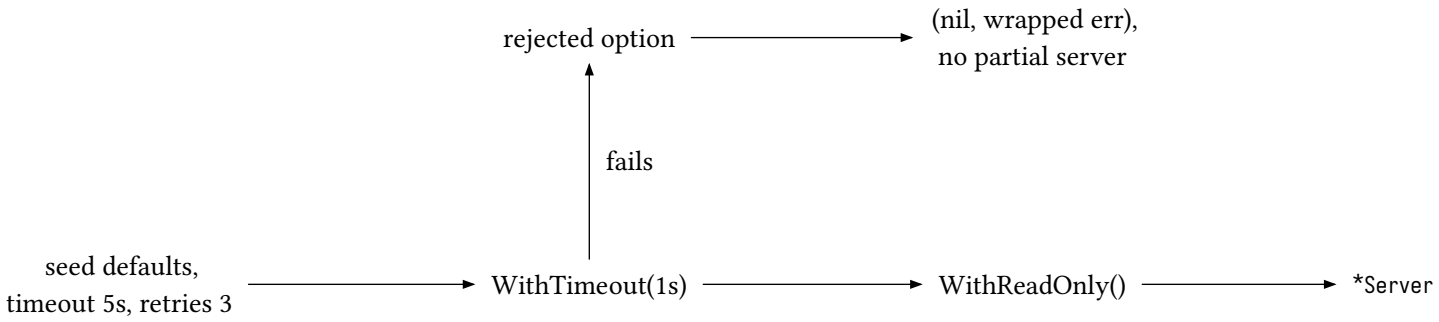


figure 7: options fold over seeded defaults, one failure aborts the build

2.3. builder, when assembly can fail midway

The builder earns its lines only when construction is a genuine sequence with validation best expressed at the end. The sample collects issues as it goes and joins them at `Build`, and each tree joins with its own multi-error spelling:

The dry run: the fixture walks `Method("")` then `URL("ftp://x")` into one build that fails with a single error naming both the empty method and the refused url, while a good build lands its method and url with the unset header and body fields left at zero. The both-names reading is pinned by the seven new trees, the frozen go test pins that the join exists and that the url reason rides it, both verified against the same implementation.

1. One error carries every issue, so a caller learns all its mistakes in one round trip instead of fixing them one at a time.
2. The steps keep chaining after a recorded issue, header calls still ride along, and build stays the single validation point.

```c
static request_builder *req_method(request_builder *b, const char *m) {
  if (m[0] == '\0')
    return req_issue(b, "method cannot be empty", m);
  snprintf(b->r.method, sizeof b->r.method, "%s", m);
  return b;
}

static request_builder *req_url(request_builder *b, const char *u) {
  size_t len = strlen(u);
  if (len < 8 ||
      (strncmp(u, "http://", 7) != 0 && strncmp(u, "https://", 8) != 0))
    return req_issue(b, "url must be http(s): \"%s\"", u);
  snprintf(b->r.url, sizeof b->r.url, "%s", u);
  return b;
}

static request_builder *req_header(request_builder *b, const char *k,
                                   const char *v) {
  if (b->r.n_headers < MAX_HEADERS) {
    snprintf(b->r.header_key[b->r.n_headers], 24, "%s", k);
    snprintf(b->r.header_val[b->r.n_headers], 24, "%s", v);
    b->r.n_headers++;
  }
  return b;
}

// build validates once: every recorded issue joins into one error, and a
// struct that never got its required fields reports them here
static const char *req_build(request_builder *b, request *out, char *err,
                             int errsz) {
  if (b->n_issues > 0) {
    int off = 0;
    for (int i = 0; i < b->n_issues; i++)
      off +=
          snprintf(err + off, errsz - off, "%s%s", i ? "; " : "", b->issues[i]);
    return err;
  }
  if (b->r.method[0] == '\0' || b->r.url[0] == '\0') {
    snprintf(err, errsz, "method and url are required");
    return err;
  }
  *out = b->r;
  return NULL;
}
```

listing 43: C, issues collect in a fixed array, build joins them into one message (source: patterns-concurrency-distributed/samples-c/src/Ch02/builder.c, lines 48-91)

```
    fn issue(self, complaint: String) -> Self {
        let mut b = self;
        b.issues.push(complaint);
        b
    }

    fn method(self, m: &str) -> Self {
        if m.is_empty() {
            return self.issue("method cannot be empty".to_string());
        }
        let mut b = self;
        b.method = Some(m.to_string());
        b
    }

    fn url(self, u: &str) -> Self {
        if u.len() < 8 || !(u.starts_with("http://") || u.starts_with("https://")) {
            return self.issue(format!("url must be http(s): \"{u}\""));
        }
        let mut b = self;
        b.url = Some(u.to_string());
        b
    }

    fn header(self, k: &str, v: &str) -> Self {
        let mut b = self;
        b.headers.push((k.to_string(), v.to_string()));
        b
    }

    fn build(self) -> Result<Request, String> {
        if !self.issues.is_empty() {
            return Err(self.issues.join("; "));
        }
        match (self.method, self.url) {
            (Some(method), Some(url)) => Ok(Request {
                method,
                url,
                headers: self.headers,
                body: String::new(),
            }),
            _ => Err("method and url are required".to_string()),
        }
    }
}
```

listing 44: Rust, each step takes self by value and hands it back, build joins the issue vec with a semicolon (source: patterns-concurrency-distributed/samples-rust/src/Ch02/builder.rs, lines 37-81)

```
func (b *RequestBuilder) Header(k, v string) *RequestBuilder {
  if b.r.Header == nil {
    b.r.Header = map[string]string{}
  }
  b.r.Header[k] = v
  return b
}

func (b *RequestBuilder) Build() (Request, error) {
  if len(b.issues) > 0 {
    return Request{}, errors.Join(b.issues...)
  }
  if b.r.Method == "" || b.r.URL == "" {
    return Request{}, errors.New("method and url are required")
  }
  return b.r, nil
}

// ParseOnce is lazy creation via sync.OnceValue: the parse happens on
// first call and never again, and the function is safe for
```

listing 45: Go, collect issues, validate once at build (source: patterns-concurrency-distributed/samples/ch02/creational.go, lines 90-109)

```
  static final class RequestBuilder {
    final Request r = new Request();
    final List<String> issues = new ArrayList<>();

    RequestBuilder method(String m) {
      if (m.isEmpty())
        issues.add("method cannot be empty");
      else
        r.method = m;
      return this;
    }

    RequestBuilder url(String u) {
      if (u.length() < 8 || !(u.startsWith("http://") || u.startsWith("https://")))
        issues.add("url must be http(s): \"" + u + "\"");
      else
        r.url = u;
      return this;
    }

    RequestBuilder header(String key, String val) {
      r.headers.add(new String[] {key, val});
      return this;
    }

    Request build() {
      if (!issues.isEmpty())
        throw new BuildError(String.join("; ", issues));
      if (r.method.isEmpty() || r.url.isEmpty())
        throw new BuildError("method and url are required");
      return r;
    }
  }
```

listing 46: Java, the fluent chain collects issues, build joins them with String.join and throws BuildError (source: patterns-concurrency-distributed/samples-java/src/Ch02/Builder.java, lines 34-66)

```
    public RequestBuilder SetMethod(string m)
    {
        if (m.Length == 0)
        {
            _issues.Add("method cannot be empty");
            return this;
        }
        _r.Method = m;
        return this;
    }

    public RequestBuilder SetUrl(string u)
    {
        bool ok = u.Length >= 8 && (u.StartsWith("http://", StringComparison.Ordinal)
            || u.StartsWith("https://", StringComparison.Ordinal));
        if (!ok)
        {
            _issues.Add("url must be http(s): \"" + u + "\"");
            return this;
        }
        _r.Url = u;
        return this;
    }

    public RequestBuilder SetHeader(string k, string v)
    {
        _r.Header ??= new Dictionary<string, string>();
        _r.Header[k] = v;
        return this;
    }

    public (Request? request, string? error) Build()
    {
        if (_issues.Count > 0)
        {
            return (null, string.Join("\n", _issues));
        }
        if (_r.Method.Length == 0 || _r.Url.Length == 0)
        {
            return (null, "method and url are required");
        }
        return (_r, null);
    }
```

listing 47: C#, the fluent chain records strings, build joins them and returns the tuple (source: patterns-concurrency-distributed/samples-cs/src/Ch02/Builder.cs, lines 21-63)

```
export class RequestBuilder {
  constructor() {
    this.req = { method: "", url: "", header: null, body: "" };
    this.issues = [];
  }

  method(m) {
    if (m === "") {
      this.issues.push(new Error("method cannot be empty"));
      return this;
    }
    this.req.method = m;
    return this;
  }

  url(u) {
    if (u.length < 8 || (u.slice(0, 7) !== "http://" && u.slice(0, 8) !== "https://")) {
      this.issues.push(new Error(`url must be http(s): "${u}"`));
      return this;
    }
    this.req.url = u;
    return this;
  }

  header(k, v) {
    if (this.req.header === null) this.req.header = new Map();
    this.req.header.set(k, v);
    return this;
  }

  build() {
    if (this.issues.length > 0) {
      throw new Error(this.issues.map((e) => e.message).join("\n"));
    }
    if (this.req.method === "" || this.req.url === "") {
      throw new Error("method and url are required");
    }
    return this.req;
  }
}
```

listing 48: JavaScript, each step records its Error, build joins the messages and throws (source: patterns-concurrency-distributed/samples-js/src/ch02-builder.mjs, lines 5-44)

```
class RequestBuilder:
    def __init__(self):
        self.r = Request()
        self.issues = []

    def method(self, m):
        if not m:
            self.issues.append("method cannot be empty")
            return self
        self.r.method = m
        return self

    def url(self, u):
        ok = u.startswith("http://") or u.startswith("https://")
        if len(u) < 8 or not ok:
            self.issues.append(f"url must be http(s): {u!r}")
            return self
        self.r.url = u
        return self

    def header(self, k, v):
        self.r.header[k] = v
        return self

    def build(self):
        if self.issues:
            raise BuildError(self.issues)
        if not self.r.method or not self.r.url:
            raise BuildError(["method and url are required"])
        return self.r
```

listing 49: Python, BuildError carries the issue list, build is the gate that raises it (source: patterns-concurrency-distributed/samples-py/src/Ch02/builder.py, lines 29-58)

```lua
function Builder:Method(m)
  if m == "" then
    self.issues[#self.issues + 1] = "method cannot be empty"
    return self
  end
  self.r.method = m
  return self
end

function Builder:URL(u)
  if #u < 8 or (u:sub(1, 7) ~= "http://" and u:sub(1, 8) ~= "https://") then
    self.issues[#self.issues + 1] = ("url must be http(s): %q"):format(u)
    return self
  end
  self.r.url = u
  return self
end

function Builder:Header(k, v)
  if self.r.header == nil then
    self.r.header = {}
  end
  self.r.header[k] = v
  return self
end

function Builder:Build()
  if #self.issues > 0 then
    return nil, table.concat(self.issues, "\n")
  end
  if self.r.method == nil or self.r.url == nil then
    return nil, "method and url are required"
  end
  return self.r, nil
end
```

listing 50: Lua, issues collect in a table, Build concatenates them after the colon call chain (source: patterns-concurrency-distributed/samples-lua/ch02_builder.lua, lines 13-47)

`errors.Join` from go 1.20 bundles every complaint into one error. The siblings spell the same join in their own error idioms, `String.join` inside a thrown `BuildError` in java, `string.Join` over the issue list in C#, `table.concat` in lua, a `BuildError` carrying every issue in python, a `Vec<String>` folded into one `Err` string with `join("; ")` in rust, arrays of strings in C and message arrays joined with a newline in javascript. Rust's chain also consumes itself, every step takes `self` by value and returns it, so a half-built builder cannot be branched to a second use the way a shared mutable one can. What never varies is the round trip: all the mistakes, one error, one fix cycle.

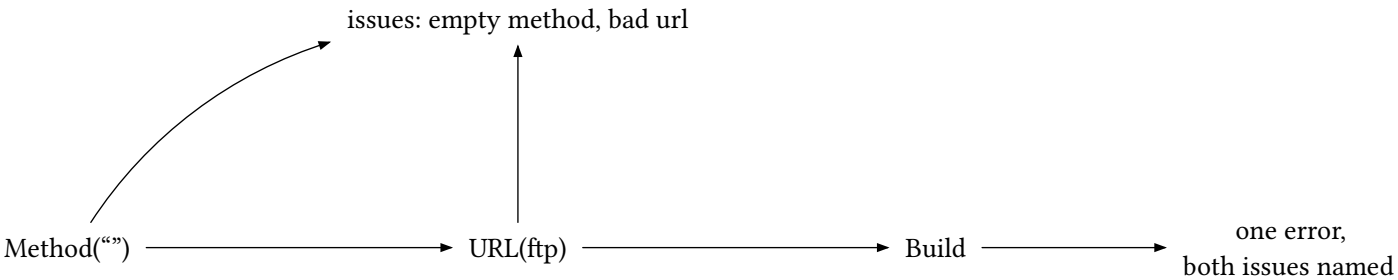


figure 8: issues accumulate across the steps, build joins them into one round trip

2.4. lazy and once

Lazy creation is `sync.OnceValue` since go 1.21, and it replaced a page of double checked locking boilerplate with one call. Every tree has a native once or builds one from a lock: C's `call_once` over a `once_flag` is the standard's own, the same C23 threads.h discipline book 1, ch 15 (c23: the language, the machine, and the cloud, threads and synchronization) teaches, rust's `OnceLock` takes the initializer inline as `get_or_init`, java ships no `OnceValue`, so its `Once<T>` is one synchronized method over a done flag, C# wraps `Lazy<T>` in `ExecutionAndPublication` mode, javascript's memo closure needs no lock because one realm runs one thread, python writes the lock and done flag by hand, and lua drives an idle, running, done state machine whose overlapping callers park until the value exists:

The dry run: the parse runs on the first call and never again, and every later caller reads the same payload, alpha 0, beta 1, gamma 2. The concurrent half is pinned by the threaded lanes and lua's scripted overlap, 8 callers in C, 8 scoped threads in rust, 8 more behind a `CyclicBarrier` in java, 20 racing tasks in C#, 20 behind a barrier in python, 20 overlapping coroutines in lua, all at exactly one parse. The frozen go test pins the sequential once-only property, the concurrent walks and their counting are pinned by the six new trees.

1. Calls 2 through N replay the identical value, identity pinned where the language exposes object identity.
2. Lua keeps the failure on stage, a naive flag that parses once per overlapping caller, and C# pins the once caching its exception across 3 calls.

```c
static struct {
  once_flag flag;
  kv map[3];
  int calls;
} box = {ONCE_FLAG_INIT, {{0}}, 0};

static void parse(void) {
  box.calls++;
  const char *words[3] = {"alpha", "beta", "gamma"};
  for (int i = 0; i < 3; i++) {
    snprintf(box.map[i].key, sizeof box.map[i].key, "%s", words[i]);
    box.map[i].value = i;
  }
}

// once_value: the payload is built under the flag, every caller reads it
static const kv *parse_once(void) {
  call_once(&box.flag, parse);
  return box.map;
}
```

listing 51: C, call_once over a once_flag, the payload built under the flag, every caller reads it (source: patterns-concurrency-distributed/samples-c/src/Ch02/lazy.c, lines 27-46)

```rust
static MAP: OnceLock<[Kv; 3]> = OnceLock::new();
static CALLS: AtomicUsize = AtomicUsize::new(0);
const NCALLERS: usize = 8;

fn parse() -> [Kv; 3] {
    CALLS.fetch_add(1, Ordering::SeqCst);
    [
        Kv {
            key: "alpha",
            value: 0,
        },
        Kv {
            key: "beta",
            value: 1,
        },
        Kv {
            key: "gamma",
            value: 2,
        },
    ]
}

fn parse_once() -> &'static [Kv; 3] {
    MAP.get_or_init(parse)
}

fn map_get(map: &[Kv; 3], key: &str) -> Option<i32> {
    map.iter().find(|kv| kv.key == key).map(|kv| kv.value)
}
```

listing 52: Rust, OnceLock::get_or_init runs the parse once, the static hands every caller the same reference (source: patterns-concurrency-distributed/samples-rust/src/Ch02/lazy.rs, lines 22-50)

```go
var ParseOnce = sync.OnceValue(func() map[string]int {
  callsToParse++
  out := map[string]int{}
  for i, word := range []string{"alpha", "beta", "gamma"} {
    out[word] = i
  }
  return out
})
```

listing 53: Go, parse happens on first call, never again, concurrent callers safe (source: patterns-concurrency-distributed/samples/ch02/creational.go, lines 111-119)

```
  static final class Once<T> {
    private final Supplier<T> fn;
    private boolean done;
    private T value;

    Once(Supplier<T> fn) {
      this.fn = fn;
    }

    synchronized T call() {
      if (!done) {
        value = fn.get();
        done = true;
      }
      return value;
    }
  }

  static int parseCalls;

  static Map<String, Integer> parseWords() {
    parseCalls++;
    Map<String, Integer> m = new LinkedHashMap<>();
    String[] words = {"alpha", "beta", "gamma"};
    for (int i = 0; i < words.length; i++)
      m.put(words[i], i);
    return m;
  }

  static final Once<Map<String, Integer>> parseOnce = new Once<>(Lazy::parseWords);
```

listing 54: Java, no OnceValue in the jdk, a generic Once whose synchronized call holds a done flag and the value (source: patterns-concurrency-distributed/samples-java/src/Ch02/Lazy.java, lines 20-50)

```
// OnceValue mirrors sync.OnceValue: the wrapped function runs at most
// once, every caller waits for or reuses the same result.
public static class Once
{
    public static Func<T> OnceValue<T>(Func<T> f)
    {
        var lazy = new Lazy<T>(f, LazyThreadSafetyMode.ExecutionAndPublication);
        return () => lazy.Value;
    }
}

// ParseOnce is lazy creation: the parse happens on the first Value()
// call and never again, and the class is safe for concurrent callers.
// The call counter makes "exactly once" observable.
public sealed class ParseOnce
{
    private int _calls;
    private readonly Func<IReadOnlyDictionary<string, int>> _once;

    public ParseOnce()
    {
        _once = Once.OnceValue(() =>
        {
            Interlocked.Increment(ref _calls);
            var map = new Dictionary<string, int>();
            string[] words = ["alpha", "beta", "gamma"];
            for (int i = 0; i < words.Length; i++)
            {
                map[words[i]] = i;
            }
            return map;
        });
    }

    public IReadOnlyDictionary<string, int> Value() => _once();

    public int Calls => Volatile.Read(ref _calls);
}
```

listing 55: C#, Lazy in ExecutionAndPublication mode is the bcl once, the counter makes it observable (source: patterns-concurrency-distributed/samples-cs/src/Ch02/Lazy.cs, lines 3-40)

```
let callsToParse = 0;

function parse() {
  callsToParse++;
  return new Map([
    ["alpha", 0],
    ["beta", 1],
    ["gamma", 2],
  ]);
}

export function onceValue(fn) {
  let done = false;
  let value;
  return () => {
    if (!done) {
      value = fn();
      done = true;
    }
    return value;
  };
}

export const parseOnce = onceValue(parse);

export function parseCalls() {
  return callsToParse;
}
```

listing 56: JavaScript, the memo closure, one thread means a done flag is the whole lock (source: patterns-concurrency-distributed/samples-js/src/ch02-lazy.mjs, lines 5-32)

```
class Once:
    def __init__(self, fn):
        self.fn = fn
        self.lock = threading.Lock()
        self.done = False
        self.value = None

    def call(self):
        with self.lock:
            if not self.done:
                self.value = self.fn()
                self.done = True
            return self.value


def parse_words():
    global word_parse_calls
    word_parse_calls += 1
    return {w: i for i, w in enumerate(["alpha", "beta", "gamma"])}


word_parse_calls = 0
parse_once = Once(parse_words)
```

listing 57: Python, a lock plus done flag, the once python writes by hand (source: patterns-concurrency-distributed/samples-py/src/Ch02/lazy.py, lines 16-38)

```lua
-- the fix: idle -> running -> done. a caller that meets the running
-- state parks as a waiter and wakes holding the value
local function Once(S)
  local state, value = "idle", nil
  local calls = 0
  local waiters = {}
  local fn = function()
    if state == "done" then
      return value
    end
    if state == "running" then
      waiters[#waiters + 1] = coroutine.running()
      coroutine.yield("park")
      return value
    end
    state = "running"
    calls = calls + 1
    value = parse()
    state = "done"
    for i = 1, #waiters do
      S.ready[#S.ready + 1] = waiters[i]
    end
    waiters = {}
    return value
  end
  return fn, function() return calls end
end
```

listing 58: Lua, idle, running, done, overlapping callers park and wake holding the value (source: patterns-concurrency-distributed/samples-lua/ch02_lazy.lua, lines 61-87)

The eight spellings differ in who owns the hard part: go and C# delegate it to the runtime, `OnceValue` and `Lazy<T>`, C carries `call_once` out of threads.h, rust hands the closure to `OnceLock` and the payload out as `&'static`, java pays a monitor it writes itself, one `synchronized` on the call, python pays a lock it writes itself, javascript gets correctness free from the single thread, and lua's scheduler turns once into a state machine. The replay contract is identical everywhere, one evaluation, every caller the same value, or in go and C# the same panic. Chapter 7 opens the whole family, `OnceFunc`, `OnceValue`, `OnceValues`, but as a creational tool this is the singleton go actually endorses: not one global instance of a type, but one evaluation of a function, scoped to whatever var holds it.

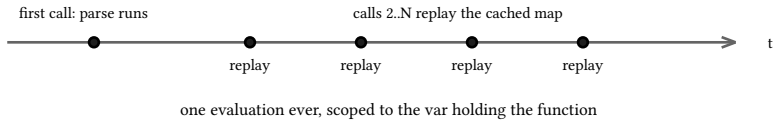


figure 9: one parse on the first call, every later caller replays the value

2.5. prototype as an explicit clone

There is no clone protocol in go, no `MemberwiseClone`, no copy constructor, and rust is the one sibling that ships one, its `Clone` trait derived when fields copy and written by hand when they reference. The prototype pattern survives elsewhere as a hand written method, and the only interesting part is depth:

The dry run: a server clone renamed to core with retries pushed to 9 never leaks back to the original holding edge and 1, and a pool clone that replaces its first backend and grows the array leaves the original's 2 entries a and b intact. The value-only clone is plain assignment or its per-language twin, the reference field is the trap every lane re-copies by hand.

1. Scalar fields copy with the struct and reference fields share their storage, so `Clone` must re-copy each of those or hand out aliased state.
2. Java, python, and lua keep the villain on stage, a shallow copy whose writes do leak through the shared list or array.

```c
// every field is a value: the copy is the clone
static server server_clone(const server *s) { return *s; }

typedef struct {
  char name[16];
  char **backends;
  int n;
} pool;

static char *xstrdup(const char *s) {
  size_t z = strlen(s) + 1;
  char *p = malloc(z);
  if (p)
    memcpy(p, s, z);
  return p;
}

// the reference field is the trap: the struct assignment copies the
// pointer, so the clone re-copies the array and every string in it
static int pool_clone(const pool *p, pool *out) {
  *out = *p;
  out->backends = malloc(sizeof(char *) * (size_t)(p->n ? p->n : 1));
  if (!out->backends)
    return 0;
  for (int i = 0; i < p->n; i++)
    out->backends[i] = xstrdup(p->backends[i]);
  return 1;
}
```

listing 59: C, the value struct clones by assignment, the pointer array by hand (source: patterns-concurrency-distributed/samples-c/src/Ch02/prototype.c, lines 26-53)

```rust
#[derive(Clone, PartialEq)]
struct ServerConfig {
    name: String,
    retries: i32,
    timeout_ms: i64,
}

struct Pool {
    name: String,
    backends: Vec<String>,
}

impl Clone for Pool {
    fn clone(&self) -> Self {
        Pool {
            name: self.name.clone(),
            backends: self.backends.iter().cloned().collect(),
        }
    }
}
```

listing 60: Rust, derive Clone for the value struct, the pool's manual impl re-copies its Vec (source: patterns-concurrency-distributed/samples-rust/src/Ch02/prototype.rs, lines 15-34)

```go
// Pool is the prototype with reference state: a plain copy would
// share the backends slice, so Clone re-copies it by hand.
type Pool struct {
  Name     string
  Backends []string
}

func (p *Pool) Clone() *Pool {
  if p == nil {
    return nil
  }
  c := *p
  c.Backends = append([]string(nil), p.Backends...)
  return &c
}
```

listing 61: Go, copy the struct, then fix every reference field by hand (source: patterns-concurrency-distributed/samples/ch02/creational.go, lines 158-172)

```
  static final class Server {
    String name;
    int retries;
    long timeoutMs;

    // every field is a value: the copy is the clone
    Server copy() {
      Server c = new Server();
      c.name = name;
      c.retries = retries;
      c.timeoutMs = timeoutMs;
      return c;
    }
  }

  static final class Pool {
    String name;
    List<String> backends = new ArrayList<>();

    // the reference field is the trap: a field-by-field copy would
    // alias the same list, so the clone re-copies it
    Pool copy() {
      Pool c = new Pool();
      c.name = name;
      c.backends.addAll(backends);
      return c;
    }
  }
```

listing 62: Java, the value class copies field by field, the pool re-copies its list or the two alias it (source: patterns-concurrency-distributed/samples-java/src/Ch02/Prototype.java, lines 19-46)

```
// Prototype as C# writes it: an explicit deep copy method, no magic.
// The shallow copy of a Server has only value fields, so field-by-field
// copy is already deep.
public static class Prototype
{
    public static Server? Clone(Server? s)
    {
        if (s is null)
        {
            return null;
        }
        return new Server
        {
            Name = s.Name,
            Timeout = s.Timeout,
            Retries = s.Retries,
            ReadOnly = s.ReadOnly,
        };
    }
}

// Pool is the prototype with reference state: a plain field copy would
// share the backends list, so Clone re-copies it by hand.
public sealed class Pool
{
    public required string Name { get; set; }
    public required List<string> Backends { get; init; }

    public Pool? Clone()
    {
        return new Pool
        {
            Name = Name,
            Backends = [.. Backends],
        };
    }
}
```

listing 63: C#, hand written clones, the list field re-copied into a fresh one (source: patterns-concurrency-distributed/samples-cs/src/Ch02/Prototype.cs, lines 3-39)

```
export class Server {
  constructor(name, timeout, retries, readOnly) {
    this.name = name;
    this.timeout = timeout;
    this.retries = retries;
    this.readOnly = readOnly;
  }

  clone() {
    return new Server(this.name, this.timeout, this.retries, this.readOnly);
  }
}

export class Pool {
  constructor(name, backends) {
    this.name = name;
    this.backends = backends;
  }

  clone() {
    return new Pool(this.name, this.backends.slice());
  }
}
```

listing 64: JavaScript, the clone re-creates the object, slice copies the array one level deep (source: patterns-concurrency-distributed/samples-js/src/ch02-prototype.mjs, lines 6-28)

```
class Server:
    def __init__(self, name, retries=3):
        self.name = name
        self.retries = retries
        self.timeout = 5.0
        self.read_only = False

    def clone(self):
        c = Server(self.name, self.retries)
        c.timeout = self.timeout
        c.read_only = self.read_only
        return c


class Pool:
    def __init__(self, name, backends):
        self.name = name
        self.backends = backends

    def clone(self):
        return Pool(self.name, list(self.backends))
```

listing 65: Python, the clone method copies scalars and re-copies the list (source: patterns-concurrency-distributed/samples-py/src/Ch02/prototype.py, lines 15-35)

```lua
function Server:Clone()
  local c = {}
  for k, v in pairs(self) do c[k] = v end
  return setmetatable(c, getmetatable(self))
end

local Pool = {}
Pool.__index = Pool

function Pool.new(name, backends)
  return setmetatable({ name = name, backends = backends }, Pool)
end

-- Clone re-copies the reference field: a plain field copy would hand
-- both objects the same backends array
function Pool:Clone()
  local c = {}
  for k, v in pairs(self) do c[k] = v end
  c.backends = {}
  for i, b in ipairs(self.backends) do c.backends[i] = b end
  return setmetatable(c, getmetatable(self))
end

-- the trap itself, kept as a named villain: the shallow copy
function Pool:ShallowClone()
  local c = {}
  for k, v in pairs(self) do c[k] = v end
  return setmetatable(c, getmetatable(self))
end
```

listing 66: Lua, the field loop copies, backends gets its own array, ShallowClone is the villain (source: patterns-concurrency-distributed/samples-lua/ch02_prototype.lua, lines 17-45)

A struct assignment copies the scalar fields and shares the reference ones, slices, maps, pointers, and the other class trees meet the same rule one level down: a field-by-field java copy would alias the same `List`, javascript's spread and slice copy exactly one level, python's assignment shares every mutable field, and lua's tables are references. Rust makes the leak unrepresentable: cloning a `Vec<String>` is a deep copy by definition, so the villain the other lanes stage cannot be written without reaching for a shared handle, which is exactly what the sample's `Rc` lane demonstrates instead. For value shaped types with no reference fields, plain assignment is the clone and a method is ceremony, which is why the C lane's server clone is `return *s`.

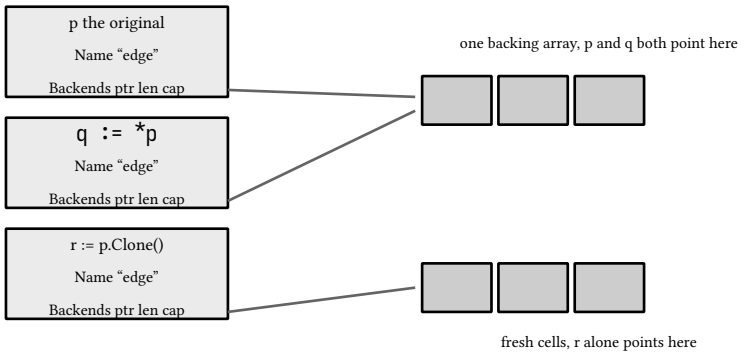


figure 10: assignment shares the backing cells, clone must allocate fresh ones

2.6. flyweight through unique

The flyweight, many handles to few shared instances, has a stdlib implementation since go 1.24: the `unique` package interns comparable values behind handles that are cheap to compare and safe as map keys. Every runtime already canonicalizes something, and each lane picks its own native door, C keeps a canonical pointer per word, java keeps the first-seen reference in a map so == is the handle, C# reaches for `string.Intern`, javascript leans on the engine's own interning with the map value as the handle, python calls `sys.intern`, and lua's VM interns strings already, so the sample demonstrates the property through a value keyed table of handle objects:

The dry run: 100 adds of one long string and 1 of another leave 2 distinct entries with counts 100 and 1, and handle equality mirrors value equality everywhere.

1. The 100 adds collapse onto one handle, and the per-word counts read 100 and 1.
2. Equal values share a handle and different values never do, the equality the map keys rely on.

```c
typedef struct {
  char *canon;
  int count;
} entry;

typedef struct {
  entry e[TABLE];
  int n;
} intern;

static char *xstrdup(const char *s) {
  size_t z = strlen(s) + 1;
  char *p = malloc(z);
  if (p)
    memcpy(p, s, z);
  return p;
}

// add returns the canonical pointer: same word, same address, and the
// per-word count says how many adds collapsed into it
static const char *intern_add(intern *t, const char *word) {
  for (int i = 0; i < t->n; i++)
    if (strcmp(t->e[i].canon, word) == 0) {
      t->e[i].count++;
      return t->e[i].canon;
    }
  if (t->n ≥ TABLE)
    return NULL;
  t->e[t->n].canon = xstrdup(word);
  t->e[t->n].count = 1;
  return t->e[t->n++].canon;
}
```

listing 67: C, the canonical pointer is the handle, one allocation per distinct word (source: patterns-concurrency-distributed/samples-c/src/Ch02/flyweight.c, lines 21-52)

```rust
struct Intern {
    entries: Vec<(Rc<str>, u32)>,
}

impl Intern {
    fn new() -> Self {
        Intern {
            entries: Vec::new(),
        }
    }

    fn add(&mut self, word: &str) -> Option<Rc<str>> {
        for (canon, count) in &mut self.entries {
            if &**canon == word {
                *count += 1;
                return Some(Rc::clone(canon));
            }
        }
        if self.entries.len() ≥ 8 {
            return None;
        }
        let canon: Rc<str> = Rc::from(word);
        self.entries.push((Rc::clone(&canon), 1));
        Some(canon)
    }

    fn count(&self, word: &str) -> u32 {
        self.entries
            .iter()
            .find(|(canon, _)| &**canon == word)
            .map(|(_, count)| *count)
            .unwrap_or(0)
    }

    fn len(&self) -> usize {
        self.entries.len()
    }
}
```

listing 68: Rust, Rc<str> is the handle, the table hands every equal word the same allocation (source: patterns-concurrency-distributed/samples-rust/src/Ch02/flyweight.rs, lines 15-52)

```go
type Intern struct {
  seen map[unique.Handle[string]]int
}

func NewIntern() *Intern { return &Intern{seen: map[unique.Handle[string]]int{}} }

func (in *Intern) Add(word string) {
  h := unique.Make(word)
  in.seen[h]++
}

func (in *Intern) Distinct() int { return len(in.seen) }

func (in *Intern) Count(word string) int {
  return in.seen[unique.Make(word)]
}
```

listing 69: Go, interning strings through globally unique handles (source: patterns-concurrency-distributed/samples/ch02/creational.go, lines 137-152)

```java
  static final class Intern {
    final Map<String, String> canon = new HashMap<>();
    final Map<String, Integer> counts = new HashMap<>();

    // add returns the canonical instance: same word, same reference,
    // and the per-word count says how many adds collapsed into it
    String add(String word) {
      String c = canon.get(word);
      if (c == null) {
        c = word;
        canon.put(word, c);
      }
      counts.merge(c, 1, Integer::sum);
      return c;
    }

    int distinct() {
      return canon.size();
    }

    int count(String word) {
      return counts.getOrDefault(word, 0);
    }
  }
```

listing 70: Java, add returns the first-seen reference, so equal words compare true under == (source: patterns-concurrency-distributed/samples-java/src/Ch02/Flyweight.java, lines 18-41)

```cs
// Intern dedupes long-lived strings through the runtime's intern pool,
// the flyweight: equal strings collapse to one canonical reference
// behind a comparable handle. string.Intern is the C# home of the
// pattern, ReferenceEquals is the handle equality.
public sealed class Intern
{
    private readonly Dictionary<string, int> _seen = [];

    public void Add(string word)
    {
        string canonical = string.Intern(word);
        _seen[canonical] = _seen.TryGetValue(canonical, out int n) ? n + 1 : 1;
    }

    public int Distinct() => _seen.Count;

    public int Count(string word) =>
        _seen.TryGetValue(string.Intern(word), out int n) ? n : 0;

    public static bool HandleEqual(string a, string b) =>
        ReferenceEquals(string.Intern(a), string.Intern(b));
}
```

listing 71: C#, string.Intern makes the handle, ReferenceEquals is its equality (source: patterns-concurrency-distributed/samples-cs/src/Ch02/Flyweight.cs, lines 3-24)

```js
export class Intern {
  constructor() {
    this.seen = new Map();
  }

  add(word) {
    this.seen.set(word, (this.seen.get(word) ?? 0) + 1);
  }

  distinct() {
    return this.seen.size;
  }

  count(word) {
    return this.seen.get(word) ?? 0;
  }
}

// handle equality mirrors value equality: there is nothing lighter
// to compare in js, the string is already the handle.
export function handleEqual(a, b) {
  return a === b;
}
```

listing 72: JavaScript, the string is already the handle, the map counts per value (source: patterns-concurrency-distributed/samples-js/src/ch02-flyweight.mjs, lines 5-27)

```
class Intern:
    def __init__(self):
        self.counts = {}

    def add(self, word):
        key = sys.intern(word)
        self.counts[key] = self.counts.get(key, 0) + 1

    def distinct(self):
        return len(self.counts)

    def count(self, word):
        return self.counts.get(sys.intern(word), 0)


def plugin_word():
    # join at runtime builds a fresh string object per call, so the
    # collapse below is interning at work, not the compiler folding
    # a constant
    return "".join(["plugin-"] * 8) + "loaded"
```

listing 73: Python, sys.intern returns the canonical object, dict identity does the rest (source: patterns-concurrency-distributed/samples-py/src/Ch02/flyweight.py, lines 16-35)

```lua
local Intern = {}
Intern.__index = Intern

function Intern.new()
  return setmetatable({ seen = {} }, Intern)
end

-- Canonical returns the one handle object for the key, creating it
-- on first sight. every later caller of an equal key shares it
function Intern:Canonical(word)
  local h = self.seen[word]
  if h == nil then
    h = { count = 0 }
    self.seen[word] = h
  end
  return h
end

function Intern:Add(word)
  local h = self:Canonical(word)
  h.count = h.count + 1
end

function Intern:Distinct()
  local n = 0
  for _ in pairs(self.seen) do n = n + 1 end
  return n
end

function Intern:Count(word)
  local h = self.seen[word]
  if h == nil then return 0 end
  return h.count
end
```

listing 74: Lua, a value keyed table hands every equal word one handle object (source: patterns-concurrency-distributed/samples-lua/ch02_flyweight.lua, lines 6-39)

`unique.Make` returns the same `Handle[string]` for equal strings, and the handle comparison is pointer shaped, not string shaped: C compares canonical pointers, rust runs `Rc::ptr_eq` over the shared `Rc<str>` allocation, java == over the first-seen references, C# `ReferenceEquals` over interned strings, python the `is` operator over interned objects, javascript value equality because the string already is the handle, lua identity of the one handle object per word. The cost model matters: `Make` with a fresh value interns it forever, since the runtime cannot know when the last handle dies, so this is for long lived duplicates, plugin names, cache keys, tag sets, not for unbounded user input. The C lane owns its table and frees it, the one spelling where the flyweight's lifetime stays the programmer's. Rust's `Rc` counts its holders, so the handle dies with its last user and lifetime rides the value instead of the table. Chapter 14's metrics dedup uses the same trick for label values.

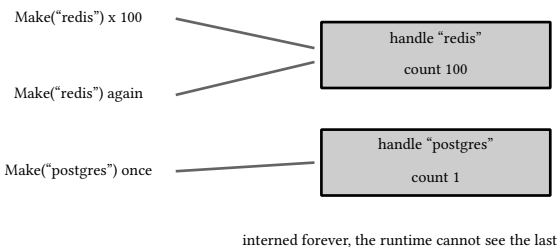


figure 11: equal values converge on one handle, the map holds the distinct few

NOTE: when a pattern is a literal

Before reaching for any of the above, check the zero value. A go struct usable as `&Server{Name: "edge"}` with the rest zero is a design goal, and the sync types are built for it: `Mutex` works unzeroed, `bytes.Buffer` works unzeroed, `http.Client` works unzeroed. The siblings have their own free starts, field defaults in C#, undefined until assigned in javascript, class attributes in python, the empty table in lua, the opt-in `Default` trait in rust, but go is the one that turns the stance into a stated rule. The best creational pattern is often a constructor that does nothing but document that.

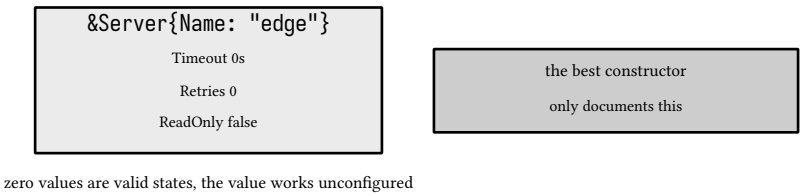


figure 12: the zero value already works, the literal finishes it

2.7. across the eight languages

The build sizes count non-comment source lines, the go column the chapter's one frozen file:

language	build SLOC	dependency	boundary note
c	444	libc plus threads.h	call_once over once_flag, clones by assignment, references re-copied by hand
rust	454	std only, no crates	boxed-closure options refusing with Result, OnceLock lazies, Clone as the one protocol, `Rc<str>` handles
go	132	stdlib	Option closures, OnceValue lazy, unique.Make hands the flyweight its handle
java	354	jdk 27 stdlib	Option lambdas returning null or the reason, a hand-written synchronized Once, first-seen references as handles
c#	178	bcl	delegates returning string?, Lazy at ExecutionAndPublication, Intern handles
javascript	128	node stdlib	options mutate and return error or null, the string itself the handle
python	264	stdlib only	a Lock under the lazy parse, Barrier(20) exposing the flag race, sys.intern
lua	399	lib.lua harness	value-keyed handle table, pool clones re-copy the array so mutations never leak

sources: go.dev/pkg/sync for the `Once` family, go.dev/pkg/unique for handle semantics and the interning cost note, go.dev/pkg/errors for `Join`, go.dev/doc/effective_go composite literals, accessed 2026-09-08. Verified by the eight chapter legs: 5 Ch02 C programs with 64 embedded checks, 5 Ch02 rust programs in `samples-rust/src` with 36 embedded checks under `run-rust-samples`, `go test` at 7 tests in `patternsbook/ch02`, 5 Ch02 java programs with 35 checks under `run-java-samples`, 16 xunit facts, node's 12 cases in `test/ch02.test.mjs`, 5 python modules with 40 embedded checks, and the lua runner's 21 ch02 rows.

3. structural patterns

Structural patterns wire values together, and the wrapper mechanism is each tree's own. Go's structural interfaces make a wrapper type declaring the same method set an adapter, decorator, or proxy before any framework shows up, C folds the same shape into a struct holding the next function table, rust and java declare the interface once and implement it on the wrapper side alone, C# declares the interface once and implements it on both sides, and the dynamic three wrap duck typed objects without asking anyone's permission. The wiring is the cheap half. The interesting engineering content is what each wrapper owes its caller, error transparency, ordering, and access control, which is what the tests in this chapter pin.

3.1. adapter

The adapter exists to satisfy an interface a type cannot be edited to satisfy, third party code, generated code, or a stable legacy api:

The dry run: the fixture is 72 fahrenheit reading as the exact double 22.22222222222222, byte for byte across the seven sibling lanes, computed as `(f - 32) * 5 / 9` with no epsilon anywhere. The go lane's frozen test pins its own input, 212 boiling to 100 through a 1e-9 tolerance on its own constant, an anchor the sibling trees pin beside the shared literal, both verified against the same conversion.

1. The computed quotient equals the printed literal as the same ieee double, the exactness the whole section stands on.
2. The legacy side is untouched, its field and accessor unchanged, and the adapter is the only new code either side sees.

```c
typedef struct {
  double f;
} legacy_sensor;

static double legacy_read_f(const legacy_sensor *l) { return l->f; }

// the modern interface this file targets: read celsius
typedef struct {
  legacy_sensor sensor;
} celsius_adapter;

static double celsius_read_c(const celsius_adapter *c) {
  return (legacy_read_f(&c->sensor) - 32) * 5 / 9;
}
```

listing 75: C, the adapter embeds the legacy struct and converts through its accessor (source: patterns-concurrency-distributed/samples-c/src/Ch03/adapter.c, lines 18-31)

```rust
trait CelsiusSensor {
    fn read_c(&self) -> f64;
}

struct CelsiusAdapter {
    sensor: LegacySensor,
}

impl CelsiusSensor for CelsiusAdapter {
    fn read_c(&self) -> f64 {
        (self.sensor.read_f() - 32.0) * 5.0 / 9.0
    }
}

fn report(sensor: &dyn CelsiusSensor) -> f64 {
    sensor.read_c()
}
```

listing 76: Rust, the adapter implements the trait, the legacy struct never learns it, the boundary takes dyn (source: patterns-concurrency-distributed/samples-rust/src/Ch03/adapter.rs, lines 23-39)

```go
type CelsiusReader interface {
  ReadC() (float64, error)
}

type LegacySensor struct{ F float64 }

func (l *LegacySensor) ReadF() (float64, error) { return l.F, nil }

type CelsiusAdapter struct{ Sensor LegacySensor }

func (c CelsiusAdapter) ReadC() (float64, error) {
  f, err := c.Sensor.ReadF()
  if err ≠ nil {
    return 0, err
  }
  return (f - 32) * 5 / 9, nil
}
```

listing 77: Go, fahrenheit sensor behind a celsius interface (source: patterns-concurrency-distributed/samples/ch03/structural.go, lines 14-31)

```
  // the modern interface this file targets: read celsius
  interface CelsiusReader {
    double readC();
  }

  // legacy code: no celsius method, no implements clause
  static class LegacySensor {
    private final double f;

    LegacySensor(double f) {
      this.f = f;
    }

    double readF() {
      return f;
    }
  }

  static final class CelsiusAdapter implements CelsiusReader {
    private final LegacySensor sensor;

    CelsiusAdapter(LegacySensor sensor) {
      this.sensor = sensor;
    }

    public double readC() {
      return (sensor.readF() - 32) * 5 / 9;
    }
  }
```

listing 78: Java, nominal satisfaction, the adapter implements the new interface, the legacy class never learns it (source: patterns-concurrency-distributed/samples-java/src/Ch03/Adapter.java, lines 19-47)

```
public interface CelsiusReader
{
    double ReadC();
}

public sealed class LegacySensor
{
    public double F { get; init; }

    public double ReadF() => F;
}

public sealed class CelsiusAdapter(LegacySensor sensor) : CelsiusReader
{
    private readonly LegacySensor _sensor = sensor;

    public double ReadC()
    {
        double f = _sensor.ReadF();
        return (f - 32) * 5 / 9;
    }
}
```

listing 79: C#, nominal satisfaction, the adapter declares the interface the legacy type never will (source: patterns-concurrency-distributed/samples-cs/src/Ch03/Adapter.cs, lines 6-27)

```
export class LegacySensor {
  constructor(f) {
    this.f = f;
  }

  readF() {
    return this.f;
  }
}

export class CelsiusAdapter {
  constructor(sensor) {
    this.sensor = sensor;
  }

  readC() {
    return (this.sensor.readF() - 32) * 5 / 9;
  }
}
```

listing 80: JavaScript, duck typing, anything with a readC method is a celsius reader (source: patterns-concurrency-distributed/samples-js/src/ch03-adapter.mjs, lines 6-24)

```
@runtime_checkable
class CelsiusReader(Protocol):
    def read_c(self) -> float: ...


class LegacySensor:
    def __init__(self, f):
        self.f = f

    def read_f(self):
        return self.f


class CelsiusAdapter:
    def __init__(self, sensor):
        self.sensor = sensor

    def read_c(self):
        f = self.sensor.read_f()
        return (f - 32) * 5 / 9
```

listing 81: Python, a runtime checkable Protocol the adapter satisfies and the legacy type does not (source: patterns-concurrency-distributed/samples-py/src/Ch03/adapter.py, lines 16-35)

```lua
local LegacySensor = {}
LegacySensor.__index = LegacySensor

function LegacySensor.new(f)
  return setmetatable({ f = f }, LegacySensor)
end

function LegacySensor:ReadF()
  return self.f, nil
end

local CelsiusAdapter = {}
CelsiusAdapter.__index = CelsiusAdapter

function CelsiusAdapter.new(sensor)
  return setmetatable({ sensor = sensor }, CelsiusAdapter)
end

function CelsiusAdapter:ReadC()
  local f, err = self.sensor:ReadF()
  if err then
    return 0, err
  end
  return (f - 32) * 5 / 9, nil
end
```

listing 82: Lua, the adapter carries ReadC, the legacy metatable never learns it (source: patterns-concurrency-distributed/samples-lua/ch03_adapter.lua, lines 6-30)

Because satisfaction is structural, `CelsiusAdapter` needs no inheritance and `LegacySensor` needs no changes. The adapter owns the conversion and its error, and the caller sees one interface. In a class language this pattern needs an interface declaration on both sides plus an adapter class, in go it is 8 lines, which is why adapters in go code are usually anonymous local types or function values, seldom named citizens. C is the same economy with the function table folded into the struct, rust pays the one trait declaration and hands the adapter over as `&dyn CelsiusSensor`, the legacy field still reached by its own accessor, java pays the one interface declaration and proves the legacy side stays outside with an `instanceof` check in the checks, C# pays the one interface declaration and gets the compiler checking both sides, and the dynamic three never needed permission at all: python's Protocol checks at run time, javascript's predicate is any caller's, lua's lookup happens at the call.

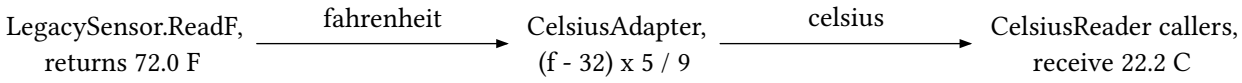


figure 13: the adapter owns the conversion and its error, neither side changes

3.2. decorator as middleware

The most run decorator in the ecosystem is `net/http` middleware, because `http.Handler` is a one method interface and a decorator of it is just a function:

The dry run: one allowed request and one denied request walk the chain, and the counters read hits 2, denials 1, with the inner handler executing exactly once and seeing the bearer header. The denied request never reaches it.

1. Ordering is behavior: with the log outermost, a denied request is still counted as a hit, and the token layer's 401 is the response.
2. The fold runs outside in, the first middleware in the list sees the request first. C, rust, and java pin the swapped order, a denial the log never sees, and lua pins the fold order as one, two, three, inner.

```c
// a handler: one method plus its receiver, any satisfier fits the field
typedef struct {
  void (*serve)(void *self, response *res, const request *req);
  void *self;
} handler;

// a middleware: handler in, wrapped handler out, with capture state
typedef struct {
  handler (*wrap)(void *state, handler next);
  void *state;
} middleware;

struct access_log {
  long *hits;
  handler next;
};

static void access_log_serve(void *self, response *res, const request *req) {
  struct access_log *m = self;
  m->hits[0]++;
  m->next.serve(m->next.self, res, req);
}

static handler access_log_wrap(void *state, handler next) {
  struct access_log *m = state;
  m->next = next;
  handler h = {access_log_serve, state};
  return h;
}

static middleware with_access_log(struct access_log *state, long *hits) {
  state->hits = hits;
  middleware m = {access_log_wrap, state};
  return m;
}
```

listing 83: C, a handler is a function pointer plus receiver, a middleware wraps one into another, the state struct is the closure go hides (source: patterns-concurrency-distributed/samples-c/src/Ch03/middleware.c, lines 29-63)

```
type Handler = Box<dyn Fn(&Request) -> Response>;
type Middleware = Box<dyn Fn(Handler) -> Handler>;

fn with_access_log(hits: Rc<Cell<i64>>) -> Middleware {
    Box::new(move |next: Handler| {
        let hits = Rc::clone(&hits);
        Box::new(move |req: &Request| {
            hits.set(hits.get() + 1);
            next(req)
        })
    })
}

fn with_token(token: &'static str, denials: Rc<Cell<i64>>) -> Middleware {
    Box::new(move |next: Handler| {
        let denials = Rc::clone(&denials);
        Box::new(move |req: &Request| {
            let want = format!("Bearer {token}");
            if req.authorization ≠ want {
                denials.set(denials.get() + 1);
                return Response {
                    code: 401,
                    body: "denied\n".to_string(),
                };
            }
            next(req)
        })
    })
}

fn app(saw_auth: Rc<RefCell<String>>, execs: Rc<Cell<i64>>) -> Handler {
    Box::new(move |req: &Request| {
        *saw_auth.borrow_mut() = req.authorization.clone();
        execs.set(execs.get() + 1);
        Response {
            code: 200,
            body: String::new(),
        }
    })
}

fn chain(inner: Handler, mws: Vec<Middleware>) -> Handler {
    mws.into_iter()
        .rev()
        .fold(inner, |handler, middleware| middleware(handler))
}
```

listing 84: Rust, middleware as boxed closures over boxed handlers, the counters riding Rc<Cell>, the fold over rev (source: patterns-concurrency-distributed/samples-rust/src/Ch03/middleware.rs, lines 25-70)

```
type Middleware func(http.Handler) http.Handler

func Chain(h http.Handler, mws ...Middleware) http.Handler {
  for i := len(mws) - 1; i ≥ 0; i-- {
    h = mws[i](h)
  }
  return h
}

func WithAccessLog(hits *atomic.Int64) Middleware {
  return func(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
      hits.Add(1)
      next.ServeHTTP(w, r)
    })
  }
}

func WithToken(token string, denials *atomic.Int64) Middleware {
  return func(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
      if r.Header.Get("authorization") ≠ "Bearer "+token {
        denials.Add(1)
        http.Error(w, "denied", http.StatusUnauthorized)
        return
      }
      next.ServeHTTP(w, r)
    })
  }
}

// Facade. Three steps with three error surfaces collapse into one
```

listing 85: Go, middleware chain: log outermost, token inside, inner handler last (source: patterns-concurrency-distributed/samples/ch03/structural.go, lines 35-66)

```
// a handler: one method, any satisfier fits the chain
interface Handler {
  Response handle(Request req);
}

// fold outside in: the last wrap call is the innermost layer
static Handler chain(Handler inner, List<UnaryOperator<Handler>> mws) {
  Handler h = inner;
  for (int i = mws.size() - 1; i ≥ 0; i--)
    h = mws.get(i).apply(h);
  return h;
}

static UnaryOperator<Handler> withAccessLog(long[] hits) {
  return next ->
      req -> {
        hits[0]++;
        return next.handle(req);
      };
}

static UnaryOperator<Handler> withToken(String token, long[] denials) {
  return next ->
      req -> {
        if (!req.authorization().equals("Bearer " + token)) {
          denials[0]++;
          return new Response(401, "denied");
        }
        return next.handle(req);
      };
}
```

listing 86: Java, a middleware is a UnaryOperator of Handler, chain folds from the end of the list (source: patterns-concurrency-distributed/samples-java/src/Ch03/Middleware.java, lines 23-53)

```
public sealed class Ctx
{
    public required string Path { get; init; }
    public string? Authorization { get; init; }
    public int Status { get; set; } = 200;
    public string? SawAuth { get; set; }
    public int InnerRuns { get; set; }
}

public delegate void Handler(Ctx ctx);

public delegate Handler Middleware(Handler next);

public static class MiddlewareChain
{
    public static Handler Chain(Handler h, params Middleware[] mws)
    {
        for (int i = mws.Length - 1; i ≥ 0; i--)
        {
            h = mws[i](h);
        }
        return h;
    }

    public static Middleware WithAccessLog(Counter hits) ⇒ next ⇒ ctx ⇒
    {
        hits.Add(1);
        next(ctx);
    };

    public static Middleware WithToken(string token, Counter denials) ⇒ next ⇒ ctx ⇒
    {
        if (ctx.Authorization ≠ "Bearer " + token)
        {
            denials.Add(1);
            ctx.Status = 401;
            return;
        }
        next(ctx);
    };
}
```

listing 87: C#, delegates all the way down, the chain folds middlewares into one handler (source: patterns-concurrency-distributed/samples-cs/src/Ch03/Middleware.cs, lines 6-46)

```
// chain(inner, a, b) builds a(b(inner)): a runs first.
export function chain(handler, ...middlewares) {
  let h = handler;
  for (let i = middlewares.length - 1; i ≥ 0; i--) {
    h = middlewares[i](h);
  }
  return h;
}

export function withAccessLog(hits) {
  return (next) => (req) => {
    hits.n++;
    return next(req);
  };
}

export function withToken(token, denials) {
  return (next) => (req) => {
    if (req.headers.authorization =/= "Bearer " + token) {
      denials.n++;
      return { status: 401, body: "denied\n" };
    }
    return next(req);
  };
}
```

listing 88: JavaScript, next => req => arrows, the shape its frameworks made famous (source: patterns-concurrency-distributed/samples-js/src/ch03-middleware.mjs, lines 6-30)

```
def chain(handler, *middlewares):
    for mw in reversed(middlewares):
        handler = mw(handler)
    return handler


def with_access_log(hits):
    def middleware(nxt):
        def handler(request):
            hits.append(1)
            return nxt(request)

        return handler

    return middleware


def with_token(token, denials):
    def middleware(nxt):
        def handler(request):
            if request.authorization ≠ "Bearer " + token:
                denials.append(1)
                return Response(401, "denied")
            return nxt(request)

        return handler

    return middleware
```

listing 89: Python, closures nested three deep, the chain folds with reversed (source: patterns-concurrency-distributed/samples-py/src/Ch03/middleware.py, lines 25-52)

```lua
-- handler: function(request) -> response
local function Chain(h, mws)
  for i = #mws, 1, -1 do
    h = mws[i](h)
  end
  return h
end

local function WithAccessLog(hits)
  return function(next)
    return function(req)
      hits.n = hits.n + 1
      return next(req)
    end
  end
end

local function WithToken(token, denials)
  return function(next)
    return function(req)
      local auth = ""
      if type(req.headers) == "table" then
        auth = req.headers["authorization"] or ""
      end
      if auth ~= "Bearer " .. token then
        denials.n = denials.n + 1
        return { status = 401, body = "denied" }
      end
      return next(req)
    end
  end
end
```

listing 90: Lua, closures returning closures, Chain counts down to fold outside in (source: patterns-concurrency-distributed/samples-lua/ch03_middleware.lua, lines 7-38)

`Chain` folds the list outside in, so the first middleware in the slice sees the request first, and every language above writes the same fold with its own function-value grain: C writes the closure by hand as a state struct beside the function pointer, rust stacks boxed closures whose hit and denial counters ride `Rc<Cell<i64>>`, shared mutable state the borrow checker only allows behind a cell, java stacks lambdas through `UnaryOperator<Handler>`, C# composes delegates, javascript stacks arrow closures, python nests three closures, lua returns closures from closures. `http.HandlerFunc` is the adapter from function to interface in the same stroke, the stdlib's own demonstration of chapter 1. This shape generalizes past http: any single method interface, `io.Reader`, `CelsiusReader`, a project's `Store`, can grow decorators the same way, and `Chain` is the whole framework.

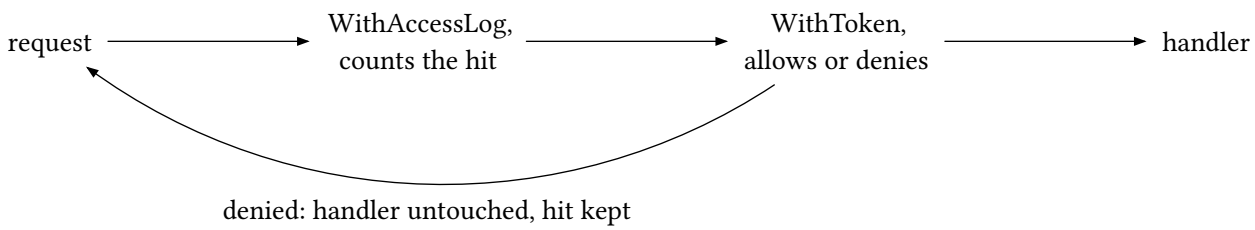


figure 14: the chain folds outside in, a denial is still a counted hit

3.3. facade

A facade collapses a multi step subsystem into one call without hiding the steps' failures:

The dry run: each failing step surfaces with its step name, the fetch wrap carrying the url, parse and validate naming themselves, and the happy path returning the parsed items a, b, c. The frozen go test pins the fetch wrap, the parse and validate step names are pinned by the seven new trees, both against the same implementation.

1. One call, three injected steps, fetch, parse, validate, each a strategy object from chapter 4 in miniature.
2. Every failure is attributable to its step, the url riding the fetch wrap and the cause readable inside it.

```c
typedef struct {
  fetcher fetch;
  parser parse;
  validator validate;
} ingest;

// load wraps each failure with its step name, fetch also carries the url
static const char *ingest_load(const ingest *g, const char *url,
                               char items[][12], int *nitems, char *err,
                               int errsz) {
  char raw[128];
  const char *cause = g->fetch(url, raw, sizeof raw);
  if (cause) {
    snprintf(err, errsz, "fetch %s: %s", url, cause);
    return err;
  }
  cause = g->parse(raw, items, nitems, 16);
  if (cause) {
    snprintf(err, errsz, "parse: %s", cause);
    return err;
  }
  cause = g->validate(items, *nitems);
  if (cause) {
    snprintf(err, errsz, "validate: %s", cause);
    return err;
  }
  return NULL;
}
```

listing 91: C, step function pointers in a struct, load wraps each failure with its step name (source: patterns-concurrency-distributed/samples-c/src/Ch03/facade.c, lines 61-88)

```rust
struct Ingest {
    fetch: Fetcher,
    parse: Parser,
    validate: Validator,
}

impl Ingest {
    fn load(&self, url: &str) -> Result<Vec<String>, String> {
        let raw = (self.fetch)(url).map_err(|cause| format!("fetch {url}: {cause}"))?;
        let items = (self.parse)(&raw).map_err(|cause| format!("parse: {cause}"))?;
        (self.validate)(&items).map_err(|cause| format!("validate: {cause}"))?;
        Ok(items)
    }
}
```

listing 92: Rust, plain fn pointers as injected fields, map_err prefixes each step's failure at the question mark (source: patterns-concurrency-distributed/samples-rust/src/Ch03/facade.rs, lines 41-54)

```
type Fetcher func(url string) (string, error)
type Parser func(raw string) ([]string, error)
type Validator func(items []string) error

type Ingest struct {
  Fetch    Fetcher
  Parse    Parser
  Validate Validator
}

func (g Ingest) Load(url string) ([]string, error) {
  raw, err := g.Fetch(url)
  if err ≠ nil {
    return nil, fmt.Errorf("fetch %s: %w", url, err)
  }
  items, err := g.Parse(raw)
  if err ≠ nil {
    return nil, fmt.Errorf("parse: %w", err)
  }
  if err := g.Validate(items); err ≠ nil {
    return nil, fmt.Errorf("validate: %w", err)
  }
  return items, nil
}

// Composite. Leaf and container satisfy the same one method, and the
```

listing 93: Go, three injected steps, one wrapped error surface (source: patterns-concurrency-distributed/samples/ch03/structural.go, lines 68-93)

```
  // the three step signatures the facade stitches together
  interface Fetcher {
    String fetch(String url);
  }

  interface Parser {
    List<String> parse(String raw);
  }

  interface Validator {
    List<String> validate(List<String> items);
  }

  static final class Ingest {
    private final Fetcher fetch;
    private final Parser parse;
    private final Validator validate;

    Ingest(Fetcher fetch, Parser parse, Validator validate) {
      this.fetch = fetch;
      this.parse = parse;
      this.validate = validate;
    }

    // load wraps each failure with its step name, fetch also carries
    // the url
    List<String> load(String url) {
      String raw;
      try {
        raw = fetch.fetch(url);
      } catch (StepError e) {
        throw new StepError("fetch " + url + ": " + e.getMessage());
      }
      List<String> items;
      try {
        items = parse.parse(raw);
      } catch (StepError e) {
        throw new StepError("parse: " + e.getMessage());
      }
      try {
        return validate.validate(items);
      } catch (StepError e) {
        throw new StepError("validate: " + e.getMessage());
      }
    }
  }
```

listing 94: Java, three single-method interfaces injected, load rethrows each failure prefixed with its step (source: patterns-concurrency-distributed/samples-java/src/Ch03/Facade.java, lines 27-72)

```
// Facade. Three steps with three error surfaces collapse into one
// call with one error, each failure wrapped with its step name.
public delegate (string raw, string? error) Fetcher(string url);

public delegate (string[] items, string? error) Parser(string raw);

public delegate string? Validator(string[] items);

public sealed class Ingest(Fetcher fetch, Parser parse, Validator validate)
{
    public (string[] items, string? error) Load(string url)
    {
        var (raw, err) = fetch(url);
        if (err is not null)
        {
            return ([], "fetch " + url + ": " + err);
        }
        var (items, perr) = parse(raw);
        if (perr is not null)
        {
            return ([], "parse: " + perr);
        }
        var verr = validate(items);
        if (verr is not null)
        {
            return ([], "validate: " + verr);
        }
        return (items, null);
    }
}
```

listing 95: C#, three delegates injected, Load prefixes each failure with its step (source: patterns-concurrency-distributed/samples-cs/src/Ch03/Facade.cs, lines 3-32)

```
export class Ingest {
  constructor(fetch, parse, validate) {
    this.fetch = fetch;
    this.parse = parse;
    this.validate = validate;
  }

  load(url) {
    let raw;
    try {
      raw = this.fetch(url);
    } catch (e) {
      throw new Error(`fetch ${url}: ${e.message}`, { cause: e });
    }
    let items;
    try {
      items = this.parse(raw);
    } catch (e) {
      throw new Error(`parse: ${e.message}`, { cause: e });
    }
    try {
      this.validate(items);
    } catch (e) {
      throw new Error(`validate: ${e.message}`, { cause: e });
    }
    return items;
  }
}
```

listing 96: JavaScript, each try wraps a new Error with cause, the step name in the message (source: patterns-concurrency-distributed/samples-js/src/ch03-facade.mjs, lines 4-31)

```
class Ingest:
    def __init__(self, fetch, parse, validate):
        self.fetch = fetch
        self.parse = parse
        self.validate = validate

    def load(self, url):
        try:
            raw = self.fetch(url)
        except StepError as e:
            raise StepError(f"fetch {url}: {e}") from e
        try:
            items = self.parse(raw)
        except StepError as e:
            raise StepError(f"parse: {e}") from e
        try:
            return self.validate(items)
        except StepError as e:
            raise StepError(f"validate: {e}") from e
```

listing 97: Python, raise from chains the cause, the step name in the new message (source: patterns-concurrency-distributed/samples-py/src/Ch03/facade.py, lines 35-53)

```lua
local Ingest = {}
Ingest.__index = Ingest

function Ingest.new(fetch, parse, validate)
  return setmetatable({ fetch = fetch, parse = parse, validate = validate }, Ingest)
end

function Ingest:Load(url)
  local raw, err = self.fetch(url)
  if err then
    return nil, "fetch " .. url .. ": " .. err
  end
  local items, perr = self.parse(raw)
  if perr then
    return nil, "parse: " .. perr
  end
  local verr = self.validate(items)
  if verr then
    return nil, "validate: " .. verr
  end
  return items, nil
end
```

listing 98: Lua, nil plus a step prefixed message, the cause concatenated (source: patterns-concurrency-distributed/samples-lua/ch03_facade.lua, lines 5-26)

The three function fields are injected at construction. `Load` adds the only logic the caller actually wants, sequence plus error wrapping with `%w`, the wrapping chain book 4, ch 7 (go 1.27, error handling) grounds, so a failure is attributable to its step and matchable downstream. The siblings wrap in their own idiom: C returns the prefixed reason through an error buffer, rust threads `Result` through `map_err`, the prefix composed in the closure at each `?`, java catches the `StepError` and rethrows a new one carrying the prefix, C# composes the string, javascript chains `Error` with `cause`, python raises `from` the caught step, lua concatenates onto `nil`. What every lane keeps is attribution, the caller learning which step failed without learning the subsystem's internals.

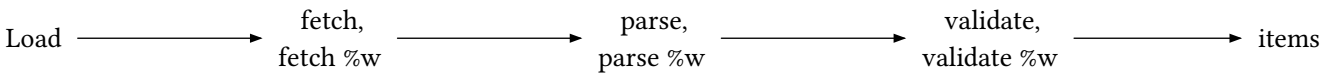


figure 15: one call, three injected steps, every failure wrapped with its step

3.4. composite

The composite pattern is recursive satisfaction: leaf and container implement the same interface, and the container aggregates children by asking each one:

The dry run: the leaves measure hi 2, ok 2, and wide! 5, the row sums its children to 4, the stack maxes to 5, and containers nest to any depth. The go lane's frozen test pins its own widget counts, save and cancel composing to 10 and 12, both verified against the same recursion.

1. Leaf and container answer the same one question, and nesting is free, a row inside a stack inside a row keeps summing and maxing.
2. Java, python, and lua pin the empty container as width 0, an absence every lane treats as a value.

```c
// the interface: one width method, self dispatching through the slot
typedef struct widget {
  int (*width)(const struct widget *w);
} widget;

typedef struct {
  widget w;
  const char *text;
} label;

static int label_width(const widget *w) {
  const label *l = (const label *)w;
  return (int)strlen(l->text);
}

static label label_of(const char *text) {
  label l = {{label_width}, text};
  return l;
}

typedef struct {
  widget w;
  const widget *kids[MAX_KIDS];
  int n;
} row;

static int row_width(const widget *w) {
  const row *r = (const row *)w;
  int total = 0;
  for (int i = 0; i < r->n; i++)
    total += r->kids[i]->width(r->kids[i]);
  return total;
}

typedef struct {
  widget w;
  const widget *kids[MAX_KIDS];
  int n;
} stack;

static int stack_width(const widget *w) {
  const stack *s = (const stack *)w;
  int widest = 0;
  for (int i = 0; i < s->n; i++) {
    int w = s->kids[i]->width(s->kids[i]);
    if (w > widest)
      widest = w;
  }
  return widest;
}
```

listing 99: C, one width slot in the struct, row and stack dispatch through it recursively (source: patterns-concurrency-distributed/samples-c/src/Ch03/composite.c, lines 20-69)

```rust
trait Widget {
    fn width(&self) -> usize;
}

struct Label {
    text: String,
}

impl Widget for Label {
    fn width(&self) -> usize {
        self.text.len()
    }
}

struct Row {
    kids: Vec<Rc<dyn Widget>>,
}

impl Widget for Row {
    fn width(&self) -> usize {
        self.kids.iter().map(|w| w.width()).sum()
    }
}

struct Stack {
    kids: Vec<Rc<dyn Widget>>,
}

impl Widget for Stack {
    fn width(&self) -> usize {
        self.kids.iter().map(|w| w.width()).max().unwrap_or(0)
    }
}

fn label(text: &str) -> Rc<dyn Widget> {
    Rc::new(Label {
        text: text.to_string(),
    })
}
```

listing 100: Rust, the trait on leaf and container alike, children as `Rc<dyn Widget>`, nesting shared nodes (source: patterns-concurrency-distributed/samples-rust/src/Ch03/composite.rs, lines 15-53)

```
type Widget interface {
  Width() int
}

type Label struct{ Text string }

func (l Label) Width() int { return len(l.Text) }

type Row struct{ Children []Widget }

func (r Row) Width() int {
  total := 0
  for _, c := range r.Children {
    total += c.Width()
  }
  return total
}

type Stack struct{ Children []Widget }

func (s Stack) Width() int {
  widest := 0
  for _, c := range s.Children {
    if w := c.Width(); w > widest {
      widest = w
    }
  }
  return widest
}
```

listing 101: Go, label is a leaf, row sums, stack maxes, nesting is free (source: patterns-concurrency-distributed/samples/ch03/structural.go, lines 95-123)

```
// the interface: one width method
interface Widget {
  int width();
}

record Label(String text) implements Widget {
  public int width() {
    return text.length();
  }
}

static final class Row implements Widget {
  final List<Widget> kids;

  Row(Widget... kids) {
    this.kids = List.of(kids);
  }

  public int width() {
    int total = 0;
    for (Widget k : kids)
      total += k.width();
    return total;
  }
}

static final class Stack implements Widget {
  final List<Widget> kids;

  Stack(Widget... kids) {
    this.kids = List.of(kids);
  }

  public int width() {
    int widest = 0;
    for (Widget k : kids)
      widest = Math.max(widest, k.width());
    return widest;
  }
}
```

listing 102: Java, a record Label and two container classes, one width method over them all (source: patterns-concurrency-distributed/samples-java/src/Ch03/Composite.java, lines 19-58)

```
public interface Widget
{
    int Width();
}

public sealed class Label(string text) : Widget
{
    public string Text { get; } = text;

    public int Width() => Text.Length;
}

public sealed class Row(params Widget[] children) : Widget
{
    public IReadOnlyList<Widget> Children { get; } = children;

    public int Width()
    {
        int total = 0;
        foreach (var c in Children)
        {
            total += c.Width();
        }
        return total;
    }
}

public sealed class Stack(params Widget[] children) : Widget
{
    public IReadOnlyList<Widget> Children { get; } = children;

    public int Width()
    {
        int widest = 0;
        foreach (var c in Children)
        {
            int w = c.Width();
            if (w > widest)
            {
                widest = w;
            }
        }
        return widest;
    }
}
```

listing 103: C#, the interface on leaf and container alike, children as IReadOnlyList (source: patterns-concurrency-distributed/samples-cs/src/Ch03/Composite.cs, lines 5-49)

```
export class Label {
  constructor(text) {
    this.text = text;
  }

  width() {
    return this.text.length;
  }
}

export class Row {
  constructor(children) {
    this.children = children;
  }

  width() {
    let total = 0;
    for (const c of this.children) total += c.width();
    return total;
  }
}

export class Stack {
  constructor(children) {
    this.children = children;
  }

  width() {
    let widest = 0;
    for (const c of this.children) {
      if (c.width() > widest) widest = c.width();
    }
    return widest;
  }
}
```

listing 104: JavaScript, classes with the one method, the consumer never asks which kind (source: patterns-concurrency-distributed/samples-js/src/ch03-composite.mjs, lines 4-38)

```
class Label:
    def __init__(self, text):
        self.text = text

    def width(self):
        return len(self.text)


class Row:
    def __init__(self, children):
        self.children = children

    def width(self):
        return sum(child.width() for child in self.children)


class Stack:
    def __init__(self, children):
        self.children = children

    def width(self):
        return max((child.width() for child in self.children), default=0)
```

listing 105: Python, sum and max do the aggregation, the default handles the empty stack (source: patterns-concurrency-distributed/samples-py/src/Ch03/composite.py, lines 14-35)

```lua
local function Label(text)
  return { kind = "label", text = text }
end

local function Row(...)
  return { kind = "row", children = { ... } }
end

local function Stack(...)
  return { kind = "stack", children = { ... } }
end

local function Width(w)
  if w.kind == "label" then
    return #w.text
  end
  if w.kind == "row" then
    local total = 0
    for _, c in ipairs(w.children) do
      total = total + Width(c)
    end
    return total
  end
  if w.kind == "stack" then
    local widest = 0
    for _, c in ipairs(w.children) do
      local cw = Width(c)
      if cw > widest then
        widest = cw
      end
    end
    return widest
  end
  error("unknown widget kind " .. tostring(w.kind))
end
```

listing 106: Lua, tagged tables and one recursive Width function (source: patterns-concurrency-distributed/samples-lua/ch03_composite.lua, lines 6-40)

`Row` and `Stack` hold `[]Widget`, which can contain further rows and stacks, and the recursion bottoms out at `Label` with no type switches anywhere. The eight spellings differ only in how the one method is found, a function pointer slot in the C struct, an interface method in go, java, and C#, a trait method behind `Rc<dyn Widget>` in rust, where one node can sit in two containers because the pointer count allows the sharing, a method on the class in the dynamic three, with lua collapsing the whole pattern into one recursive function over tagged tables. Tree shaped problems, layout, ASTs, file systems, config merges, all take this shape, and the alternative in class languages, a visitor with double dispatch, is chapter 4′s type switch instead.

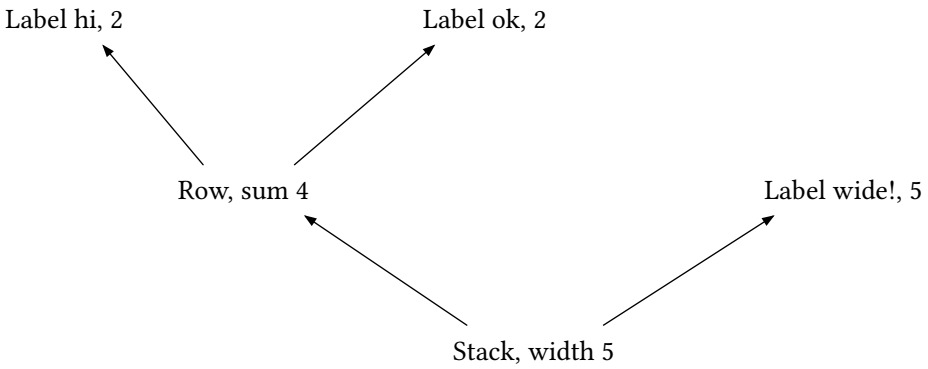


figure 16: leaves and containers share one method, widths compose recursively

3.5. proxy

A proxy stands in front of a real object and controls access or adds caching, and the distinction from decorator is intent: a decorator adds behavior the caller asked for, a proxy interposes policy the caller cannot bypass:

The dry run: a forbidden key fails with the sentinel and the real store's call counter stays at 0, and a permitted key fetched twice costs exactly 1 real call while the cache holds 1 entry.

1. Policy first, cache second: the guard runs before any lookup, so an unpermitted key never reaches the store and its error wraps the sentinel with the key.
2. The permitted key pays the real store once, the second read is the cache. C, rust, java, C#, python, and lua also pin a permitted key the store lacks surfacing the store's own error uncached.

```
typedef struct {
  store real;
  const char *allowed[PAIRS];
  int n_allowed;
  mtx_t mu;
  kv cache[PAIRS];
  int n_cached;
} store_proxy;

static bool proxy_permits(const store_proxy *p, const char *key) {
  for (int i = 0; i < p->n_allowed; i++)
    if (strcmp(p->allowed[i], key) == 0)
      return true;
  return false;
}

static const char *proxy_get(void *self, const char *key, char *out,
                             int outsz) {
  store_proxy *p = self;
  if (!proxy_permits(p, key)) {
    static char forbidden[64];
    snprintf(forbidden, sizeof forbidden,
             "\"%s\": key not permitted through proxy", key);
    return forbidden;
  }
  mtx_lock(&p->mu);
  for (int i = 0; i < p->n_cached; i++)
    if (strcmp(p->cache[i].key, key) == 0) {
      snprintf(out, outsz, "%s", p->cache[i].val);
      mtx_unlock(&p->mu);
      return NULL;
    }
  mtx_unlock(&p->mu);

  char tmp[24];
  const char *err = p->real.get(p->real.self, key, tmp, sizeof tmp);
  if (err)
    return err;

  mtx_lock(&p->mu);
  if (p->n_cached < PAIRS) {
    snprintf(p->cache[p->n_cached].key, 16, "%s", key);
    snprintf(p->cache[p->n_cached].val, 24, "%s", tmp);
    p->n_cached++;
  }
  mtx_unlock(&p->mu);
  snprintf(out, outsz, "%s", tmp);
  return NULL;
}
```

listing 107: C, the allow list gates, the cache rides a mutex, only permitted misses pay the store (source: patterns-concurrency-distributed/samples-c/src/Ch03/proxy.c, lines 53-101)

```rust
struct StoreProxy {
    real: Arc<dyn Store>,
    allowed: Vec<&'static str>,
    cache: Mutex<Vec<(String, String)>>,
}

impl StoreProxy {
    fn permits(&self, key: &str) -> bool {
        self.allowed.iter().any(|k| *k == key)
    }

    fn get(&self, key: &str) -> Result<String, String> {
        if !self.permits(key) {
            return Err(format!("\"{key}\": key not permitted through proxy"));
        }
        {
            let cache = self.cache.lock().unwrap();
            for (k, v) in cache.iter() {
                if k == key {
                    return Ok(v.clone());
                }
            }
        }
        let value = self.real.get(key)?;
        let mut cache = self.cache.lock().unwrap();
        cache.push((key.to_string(), value.clone()));
        Ok(value)
    }

    fn cached_keys(&self) -> usize {
        self.cache.lock().unwrap().len()
    }
}
```

listing 108: Rust, the allow list gates, the cache rides a Mutex, the miss question-marks the store's own error through (source: patterns-concurrency-distributed/samples-rust/src/Ch03/proxy.rs, lines 39-71)

```
type Store interface {
  Get(key string) (string, error)
}

type RealStore struct {
  Values map[string]string
  Calls  int
}

func (r *RealStore) Get(key string) (string, error) {
  r.Calls++
  v, ok := r.Values[key]
  if !ok {
    return "", fmt.Errorf("no value for %q", key)
  }
  return v, nil
}

type StoreProxy struct {
  Real    *RealStore
  Allowed map[string]bool

  mu    sync.Mutex
  cache map[string]string
}

func NewStoreProxy(real *RealStore, allowed map[string]bool) *StoreProxy {
  return &StoreProxy{Real: real, Allowed: allowed, cache: map[string]string{}}
}

var ErrForbidden = errors.New("key not permitted through proxy")

func (p *StoreProxy) Get(key string) (string, error) {
  if !p.Allowed[key] {
    return "", fmt.Errorf("%q: %w", key, ErrForbidden)
  }
  p.mu.Lock()
  v, cached := p.cache[key]
  p.mu.Unlock()
  if cached {
    return v, nil
  }
  v, err := p.Real.Get(key)
  if err != nil {
    return "", err
  }
  p.mu.Lock()
  p.cache[key] = v
  p.mu.Unlock()
  return v, nil
}
```

listing 109: Go, permission gate in front, memoization behind (source: patterns-concurrency-distributed/samples/ch03/structural.go, lines 127-177)

```
  interface Store {
    String get(String key);
  }

  static final class RealStore implements Store {
    private final Map<String, String> rows;
    int calls;

    RealStore(Map<String, String> rows) {
      this.rows = rows;
    }

    public String get(String key) {
      calls++;
      String v = rows.get(key);
      if (v == null)
        throw new NotFound("no value for \"" + key + "\"");
      return v;
    }
  }

  static final class StoreProxy implements Store {
    private final Store real;
    private final Set<String> allowed;
    private final Map<String, String> cache = new HashMap<>();

    StoreProxy(Store real, Set<String> allowed) {
      this.real = real;
      this.allowed = allowed;
    }

    public String get(String key) {
      // the guard refuses before the real store is ever reached
      if (!allowed.contains(key))
        throw new Forbidden("\"" + key + "\": key not permitted through proxy");
      synchronized (cache) {
        String hit = cache.get(key);
        if (hit != null)
          return hit;
      }
      // NotFound propagates uncached
      String value = real.get(key);
      synchronized (cache) {
        cache.put(key, value);
      }
      return value;
    }
```

listing 110: Java, Forbidden and NotFound keep C's literal messages, the guard runs before the synchronized cache (source: patterns-concurrency-distributed/samples-java/src/Ch03/Proxy.java, lines 35-81)

```
public sealed class StoreProxy(RealStore real, IReadOnlySet<string> allowed) : Store
{
    public const string ErrForbidden = "key not permitted through proxy";

    private readonly object _gate = new();
    private readonly Dictionary<string, string> _cache = [];

    public (string value, string? error) Get(string key)
    {
        if (!allowed.Contains(key))
        {
            return ("", "\"" + key + "\": " + ErrForbidden);
        }
        lock (_gate)
        {
            if (_cache.TryGetValue(key, out string? cached))
            {
                return (cached, null);
            }
        }
        var (v, err) = real.Get(key);
        if (err is not null)
        {
            return ("", err);
        }
        lock (_gate)
        {
            _cache[key] = v;
        }
        return (v, null);
    }

    public int CachedKeys()
    {
        lock (_gate)
        {
            return _cache.Count;
        }
    }
}
```

listing 111: C#, the forbidden constant, the lock object, the cache dictionary beside it (source: patterns-concurrency-distributed/samples-cs/src/Ch03/Proxy.cs, lines 27-66)

```
export const errForbidden = new Error("key not permitted through proxy");

export class RealStore {
  constructor(values) {
    this.values = values;
    this.calls = 0;
  }

  get(key) {
    this.calls++;
    if (!(key in this.values)) {
      throw new Error(`no value for "${key}"`);
    }
    return this.values[key];
  }
}

export class StoreProxy {
  constructor(real, allowed) {
    this.real = real;
    this.allowed = allowed;
    this.cache = new Map();
  }

  get(key) {
    if (!this.allowed.has(key)) {
      throw new Error(`"${key}": ${errForbidden.message}`, { cause: errForbidden });
    }
    if (this.cache.has(key)) {
      return this.cache.get(key);
    }
    const v = this.real.get(key);
    this.cache.set(key, v);
    return v;
  }

  cachedKeys() {
    return this.cache.size;
  }
}
```

listing 112: JavaScript, the sentinel as a shared Error instance carried as cause (source: patterns-concurrency-distributed/samples-js/src/ch03-proxy.mjs, lines 5-44)

```
class Forbidden(Exception):
    pass


class NotFound(Exception):
    pass


class RealStore:
    def __init__(self, values):
        self.values = values
        self.calls = 0

    def get(self, key):
        self.calls += 1
        if key not in self.values:
            raise NotFound(f"no value for {key!r}")
        return self.values[key]


class StoreProxy:
    def __init__(self, real, allowed):
        self.real = real
        self.allowed = allowed
        self.lock = threading.Lock()
        self.cache = {}

    def get(self, key):
        if key not in self.allowed:
            raise Forbidden(f"{key!r}: key not permitted through proxy")
        with self.lock:
            if key in self.cache:
                return self.cache[key]
        v = self.real.get(key)
        with self.lock:
            self.cache[key] = v
        return v

    def cached_keys(self):
        with self.lock:
            return len(self.cache)
```

listing 113: Python, Forbidden and NotFound types, a lock around the cache (source: patterns-concurrency-distributed/samples-py/src/Ch03/proxy.py, lines 16-56)

```lua
local RealStore = {}
RealStore.__index = RealStore

function RealStore.new(values)
  return setmetatable({ values = values, calls = 0 }, RealStore)
end

function RealStore:Get(key)
  self.calls = self.calls + 1
  local v = self.values[key]
  if v == nil then
    return nil, ("no value for %q"):format(key)
  end
  return v, nil
end

local StoreProxy = {}
StoreProxy.__index = StoreProxy

function StoreProxy.new(real, allowed)
  return setmetatable({ real = real, allowed = allowed, cache = {} }, StoreProxy)
end

function StoreProxy:Get(key)
  if not self.allowed[key] then
    return nil, ("%q: key not permitted through proxy"):format(key)
  end
  local v = self.cache[key]
  if v ~= nil then
    return v, nil
  end
  local rv, err = self.real:Get(key)
  if err then
    return nil, err
  end
  self.cache[key] = rv
  return rv, nil
end

function StoreProxy:CachedKeys()
  local n = 0
  for _ in pairs(self.cache) do n = n + 1 end
  return n
end
```

listing 114: Lua, the guard consults the allow table, the cache is a plain table (source: patterns-concurrency-distributed/samples-lua/ch03_proxy.lua, lines 6-49)

`ErrForbidden` is a sentinel, wrapped with the key so callers can match it with `errors.Is` while still reading which key failed. The siblings match in their own idiom, the exception types in java, C#, and python, the shared `Error` instance through `cause` in javascript, the message text in C, rust, and lua. The memo map is guarded by a mutex because proxies at api boundaries see concurrent callers, the locking rules of chapter 7 applied where they matter, rust locking the lookup and the store windows separately so the wrapped call runs outside the hold, and the single-threaded lanes need no lock at all. The test pins the contract from the outside: a forbidden key never reaches the real store, the underlying call counter stays at zero, and a permitted key fetched twice costs one underlying call, a bill rust's four threaded readers over the warmed cache keep at one.

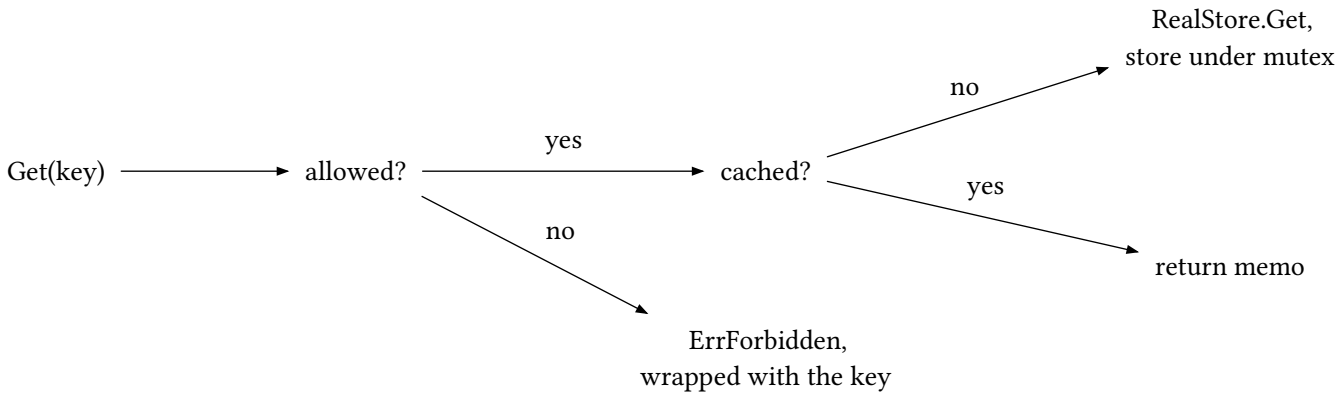


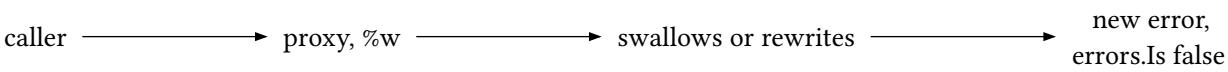
figure 17: proxy get: policy first, cache second, real store last

The cache invalidation problem is deliberately absent, the proxy caches forever for the sample's lifetime, and that boundary is the tradeoff a real deployment pays to name: ttl, explicit invalidation, or versioned keys, each of which turns this proxy into a subsystem of its own.

PITFALL: wrapper transparency is a contract

Every wrapper in this chapter forwards errors with `%w` and preserves the wrapped type's semantics. The moment a decorator swallows an error or a proxy rewrites a value it did not fetch itself, the caller's `errors.Is` and `errors.As` chains break silently. Wrapping code is trusted code. The sibling idioms carry the same duty, the rethrown exception type in java, `InnerException` in C#, `cause` in javascript, `raise from` in python, the question mark forwarding verbatim in rust, the concatenated message in C and lua.

broken



faithful

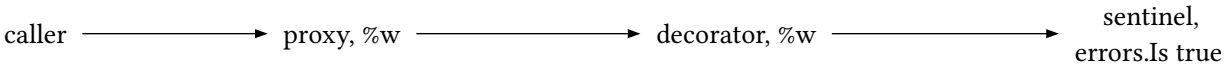


figure 18: wrapping with %w at every hop keeps the errors.is chain intact

3.6. across the eight languages

The build sizes count non-comment source lines, the go column the chapter's one frozen file:

language	build SLOC	dependency	boundary note
c	454	libc plus threads.h	composite widths through vtable slots, only the proxy cache takes a mutex
rust	450	std only, no crates	boxed-closure middleware with `Rc<Cell>` counters, `Rc<dyn Widget>` trees sharing nodes, the proxy cache under a Mutex
go	142	stdlib	Middleware as func(http.Handler) http.Handler, sentinels wrapped for errors.Is

language	build SLOC	dependency	boundary note
java	359	jdk 27 stdlib	UnaryOperator folds the middleware chain, Forbidden and NotFound keep C's literal messages
c#	184	bcl	leaf and Row share Widget, forbidden and missing surface as exception types
javascript	136	node stdlib	chain folds a(b(inner)), the sentinel a shared Error carried as cause
python	247	stdlib only	typed exceptions matched at the boundary, the proxy cache under a lock
lua	380	lib.lua harness	the allow table gates, plain-table cache, errors matched by message text

sources: go.dev/pkg/net/http for `HandlerFunc` and middleware conventions, go.dev/pkg/errors for sentinel wrapping with `Is`, go.dev/doc/effective_go embedding for why none of these wrappers need inheritance, accessed 2026-09-08. Verified by the eight chapter legs: 5 Ch03 C programs with 31 embedded checks, 5 Ch03 rust programs in `samples-rust/src` with 31 embedded checks under `run-rust-samples`, `go test` at 5 tests in `patternsbook/ch03`, 5 Ch03 java programs with 33 checks under `run-java-samples`, 14 xunit facts, node's 8 cases in `test/ch03.test.mjs`, 5 python modules with 36 embedded checks, and the lua runner's 21 ch03 rows.

4. behavioral patterns

Behavioral patterns distribute responsibility between objects. In go the first draft of every one of them is a function value, because a function is an object that carries behavior, compares against nil, and slots into fields and parameters. The chapter builds the six that survive translation, strategy, observer, command, iterator, state, and visitor, and each section runs one shared fixture through c, rust, go, java, c#, javascript, python, and lua, the dry-run block stating the contract before the eight listings.

4.1. strategy

A strategy is a named function type with one implementation per algorithm, and selection becomes a lookup:

The dry run: the fixture is aaabbc packing to the byte and count pairs (97,3)(98,2)(99,1), the empty input encoding to nothing, a single a to (97,1), and the 257 run splitting at the count byte's 255 cap into (122,2)(113,255)(113,2). The null strategy passes its input through untouched and an unknown name fails selection. The go lane's frozen test pins its own string, aaabbb to two pairs, both verified against the same implementation.

1. aaabbc walks 3 runs, and each pair carries the byte value beside its count.
2. A run longer than 255 cannot fit its count byte, so zz followed by 257 q's splits into 2, 255, and 2, the cap every lane pins.
3. Selection hands back the function value itself, and the unknown name is an error in each language's own error idiom.

```c
typedef int (*compression)(const unsigned char *in, int len, unsigned char *out,
                           int outsz);

static int rle(const unsigned char *in, int len, unsigned char *out,
               int outsz) {
  int o = 0;
  int i = 0;
  while (i < len) {
    unsigned char b = in[i];
    int j = i + 1;
    while (j < len && in[j] == b && j - i < 255)
      j++;
    if (o + 2 > outsz)
      return -1;
    out[o++] = b;
    out[o++] = (unsigned char)(j - i);
    i = j;
  }
  return o;
}

static int null_pass(const unsigned char *in, int len, unsigned char *out,
                     int outsz) {
  if (len > outsz)
    return -1;
  memcpy(out, in, (size_t)len);
  return len;
}

typedef struct {
  const char *name;
  compression fn;
} named;

static compression by_name(const char *name) {
  static const named table[] = {{"rle", rle}, {"null", null_pass}};
  for (int i = 0; i < 2; i++)
    if (strcmp(table[i].name, name) == 0)
      return table[i].fn;
  return NULL;
}
```

listing 115: C, a function pointer typedef, rle capping runs at 255, selection from a name table (source: patterns-concurrency-distributed/samples-c/src/Ch04/strategy.c, lines 21-61)

```rust
trait Compression: Sync {
    fn compress(&self, input: &[u8], out: &mut [u8]) -> isize;
}

struct Rle;

impl Compression for Rle {
    fn compress(&self, input: &[u8], out: &mut [u8]) -> isize {
        let mut o = 0usize;
        let mut i = 0usize;
        while i < input.len() {
            let b = input[i];
            let mut j = i + 1;
            while j < input.len() && input[j] == b && j - i < 255 {
                j += 1;
            }
            if o + 2 > out.len() {
                return -1;
            }
            out[o] = b;
            out[o + 1] = (j - i) as u8;
            o += 2;
            i = j;
        }
        o as isize
    }
}

struct NullPass;

impl Compression for NullPass {
    fn compress(&self, input: &[u8], out: &mut [u8]) -> isize {
        if input.len() > out.len() {
            return -1;
        }
        out[..input.len()].copy_from_slice(input);
        input.len() as isize
    }
}

fn by_name(name: &str) -> Option<&'static dyn Compression> {
    static TABLE: [(&str, &dyn Compression); 2] = [("rle", &Rle), ("null", &NullPass)];
    TABLE.iter().find(|(n, _)| *n == name).map(|(_, c)| *c)
}
```

listing 116: Rust, strategies are zero-sized types behind a trait, selection a static table borrowed as a trait object (source: patterns-concurrency-distributed/samples-rust/src/Ch04/strategy.rs, lines 13-56)

```
type Compression func(data []byte) ([]byte, error)

func Rle(data []byte) ([]byte, error) {
  var out []byte
  for i := 0; i < len(data); {
    b, j := data[i], i+1
    for j < len(data) && data[j] == b && j-i < 255 {
      j++
    }
    out = append(out, b, byte(j-i))
    i = j
  }
  return out, nil
}

func Null(data []byte) ([]byte, error) { return data, nil }

func CompressionByName(name string) (Compression, error) {
  switch name {
  case "rle":
    return Rle, nil
  case "null":
    return Null, nil
  }
  return nil, fmt.Errorf("unknown compression %q", name)
}

// Observer. Subscribers get buffered channels; Notify is
// non-blocking and drops on a full buffer, and Close releases
```

listing 117: Go, compression as a function type, rle and null as values (source: patterns-concurrency-distributed/samples/ch04/behavioral.go, lines 12-40)

```
interface Compression {
  byte[] compress(byte[] data);
}

// (byte, run) pairs with the run capped at 255, the widest one byte
// can hold
static final Compression RLE =
    data -> {
      ByteArrayOutputStream out = new ByteArrayOutputStream();
      int i = 0;
      while (i < data.length) {
        byte b = data[i];
        int j = i + 1;
        while (j < data.length && data[j] == b && j - i < 255)
          j++;
        out.write(b);
        out.write(j - i);
        i = j;
      }
      return out.toByteArray();
    };

// the null strategy never shrinks
static final Compression NULL_PASS = data -> data;

// selection rejects the unknown with a null return
static Compression byName(String name) {
  return switch (name) {
    case "rle" -> RLE;
    case "null" -> NULL_PASS;
    default -> null;
  };
}
```

listing 118: Java, the strategy is a lambda over a one-method interface, selection answers null for the unknown name (source: patterns-concurrency-distributed/samples-java/src/Ch04/Strategy.java, lines 19-51)

```
public delegate byte[] Compression(byte[] data);

public static class Compress
{
    // Rle caps every run at 255 because the count lives in one byte.
    public static byte[] Rle(byte[] data)
    {
        var outBytes = new List<byte>();
        int i = 0;
        while (i < data.Length)
        {
            byte b = data[i];
            int j = i + 1;
            while (j < data.Length && data[j] == b && j - i < 255)
            {
                j++;
            }
            outBytes.Add(b);
            outBytes.Add((byte)(j - i));
            i = j;
        }
        return [.. outBytes];
    }

    public static byte[] Null(byte[] data) => data;

    public static (Compression? alg, string? error) ByName(string name)
    {
        switch (name)
        {
            case "rle": return (Rle, null);
            case "null": return (Null, null);
        }
        return (null, "unknown compression \"" + name + "\"");
    }
```

listing 119: C#, a delegate type, rle and null as static values, the error riding a tuple (source: patterns-concurrency-distributed/samples-cs/src/Ch04/Strategy.cs, lines 4-38)

```
export function rle(data) {
  const out = [];
  let i = 0;
  while (i < data.length) {
    const b = data[i];
    let j = i + 1;
    while (j < data.length && data[j] === b && j - i < 255) {
      j++;
    }
    out.push(b, j - i);
    i = j;
  }
  return out;
}

export function nullStrategy(data) {
  return [...data];
}

export function compressionByName(name) {
  switch (name) {
    case "rle":
      return rle;
    case "null":
      return nullStrategy;
  }
  throw new Error(`unknown compression "${name}"`);
}
```

listing 120: JavaScript, plain functions as values, selection throws on the unknown name (source: patterns-concurrency-distributed/samples-js/src/ch04-strategy.mjs, lines 6-33)

```
def rle(data):
    # (byte, run) pairs with the run capped at 255, the widest one
    # byte can hold
    out = []
    i = 0
    while i < len(data):
        b = data[i]
        j = i + 1
        while j < len(data) and data[j] == b and j - i < 255:
            j += 1
        out.append((b, j - i))
        i = j
    return out


def null(data):
    return list(data)


class UnknownCompression(Exception):
    pass


def compression_by_name(name):
    if name == "rle":
        return rle
    if name == "null":
        return null
    raise UnknownCompression(f"unknown compression {name!r}")
```

listing 121: Python, first class functions, selection raises UnknownCompression (source: patterns-concurrency-distributed/samples-py/src/Ch04/strategy.py, lines 13-41)

```lua
local function Rle(data)
  local out = {}
  local i = 1
  while i ≤ #data do
    local b = data:byte(i)
    local j = i + 1
    while j ≤ #data and data:byte(j) == b and j - i < 255 do
      j = j + 1
    end
    out[#out + 1] = b
    out[#out + 1] = j - i
    i = j
  end
  return out
end

local function Null(data)
  return { data:byte(1, -1) }
end

local byName = { rle = Rle, null = Null }

local function CompressionByName(name)
  local f = byName[name]
  if not f then
    return nil, ("unknown compression %q"):format(name)
  end
  return f, nil
end
```

listing 122: Lua, function values in a table, the nil plus message refusal (source: patterns-concurrency-distributed/samples-lua/ch04_strategy.lua, lines 6-34)

Function types give the pattern for free: `Compression` is documented, comparable to nil, and the caller passes `Rle` like any value. The sibling trees say the same thing in their own spelling. C writes a function pointer typedef and answers the unknown name with NULL, rust's strategies are zero-sized types handed out as `&'static dyn Compression` from a static table, `None` the unknown name, java spells the strategy as a lambda against a one-method interface and answers the unknown name with null, its signed `byte` forcing `& 0xff` on the count before the pairs can print, C# uses a delegate with the refusal riding a tuple, javascript and python pass functions natively and signal refusal by throwing and raising, and lua looks the name up in a table and hands back nil with a message. When strategies need configuration or state, they become closures, chapter 5's territory.

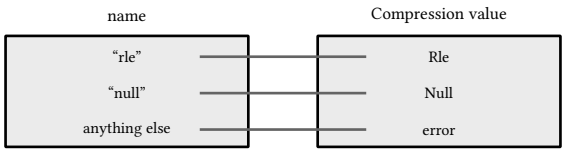


figure 19: selection is data, a lookup from name to function value

4.2. observer with channels

Book 4 toured the channel syntaxbook 4, ch 6 (go 1.27, concurrency), and the observer pattern is where channels earn a salary. Subscribers receive buffered channels, the hub's `Notify` is non-blocking with a `select` default, and `Close` releases everyone:

NOTE: the base layer

> Three channel facts this dry run stands on, rules chapter 6 lays in full: a buffered channel of capacity C holds C values before a further send must wait, a send on a full channel inside a `select` with a default takes the default and drops the event instead of blocking, and a closed channel keeps its buffered values, drain first, then the closed read.

The dry run: the fixture is a fast subscriber with buffer 2 and a slow one with buffer 1. Two notifies deliver 2 then 1, the fast subscriber taking both events while the slow keeps its one, the third notify lands nowhere, and after `Close` the slow subscriber still drains its buffered event before reading the channel as closed.

1. The first notify reaches both subscribers and the second fills the fast buffer while the slow one drops it, delivered counts 2 then 1.
2. The third notify is dropped everywhere because both buffers are full, and the producer never waits.
3. Close marks every channel closed without erasing buffers, so the lagging subscriber drains its one event, then reads closed.

```c
static int hub_subscribe(hub *h, int cap) {
  mtx_lock(&h->mu);
  int id = -1;
  if (!h->closed && h->n < MAXSUBS) {
    id = h->n++;
    memset(&h->subs[id], 0, sizeof(subscriber));
    h->subs[id].cap = cap;
  }
  mtx_unlock(&h->mu);
  return id;
}

static void hub_cancel(hub *h, int id) {
  mtx_lock(&h->mu);
  if (id ≥ 0 && id < h->n)
    h->subs[id].closed = true;
  mtx_unlock(&h->mu);
}
```

listing 123: C, subscribe under the hub mutex, cancel marks one subscriber closed (source: patterns-concurrency-distributed/samples-c/src/Ch04/observer.c, lines 77-94)

```rust
impl Hub {
    fn new() -> Self {
        Hub {
            inner: Mutex::new(HubInner {
                subs: Vec::new(),
                closed: false,
            }),
        }
    }

    fn subscribe(&self, cap: usize) -> Option<usize> {
        let mut h = self.inner.lock().unwrap();
        if h.closed {
            return None;
        }
        let id = h.subs.len();
        h.subs.push(Subscriber {
            buf: Vec::new(),
            cap,
            closed: false,
        });
        Some(id)
    }
```

listing 124: Rust, subscribe under the hub mutex, an int id per subscriber, closed hubs hand back none (source: patterns-concurrency-distributed/samples-rust/src/Ch04/observer.rs, lines 31-53)

```go
type Event struct {
  Kind string
  Body string
}

type Hub struct {
  mu     sync.Mutex
  next   int
  subs   map[int]chan Event
  closed bool
}

func NewHub() *Hub { return &Hub{subs: map[int]chan Event{}} }

func (h *Hub) Subscribe(buffer int) (<-chan Event, func()) {
  h.mu.Lock()
  defer h.mu.Unlock()
  if h.closed {
    ch := make(chan Event)
    close(ch)
    return ch, func() {}
  }
  id := h.next
  h.next++
  ch := make(chan Event, buffer)
  h.subs[id] = ch
  return ch, func() { h.remove(id) }
```

listing 125: Go, hub: subscribe returns a channel and its cancel function (source: patterns-concurrency-distributed/samples/ch04/behavioral.go, lines 42-68)

```
  static final class Subscriber {
    final ArrayDeque<String> buf = new ArrayDeque<>();
    final int cap;
    boolean closed;

    Subscriber(int cap) {
      this.cap = cap;
    }

    boolean send(String kind) {
      if (closed || buf.size() ≥ cap)
        return false;
      buf.add(kind);
      return true;
    }
  }

  static final class Hub {
    final List<Subscriber> subs = new ArrayList<>();
    boolean closed;

    synchronized int subscribe(int cap) {
      if (closed || subs.size() ≥ 4)
        return -1;
      subs.add(new Subscriber(cap));
      return subs.size() - 1;
    }
```

listing 126: Java, subscribe hands back an int id over C's api, each subscriber a bounded ArrayDeque (source: patterns-concurrency-distributed/samples-java/src/Ch04/Observer.java, lines 18-44)

```
    public (ChannelReader<Event> reader, Action cancel) Subscribe(int buffer)
    {
        lock (_gate)
        {
            if (_closed)
            {
                var dead = Channel.CreateBounded<Event>(1);
                dead.Writer.Complete();
                return (dead.Reader, () => { });
            }
            int id = _next++;
            var ch = Channel.CreateBounded<Event>(new BoundedChannelOptions(buffer)
            {
                SingleReader = true,
            });
            _subs[id] = ch;
            return (ch.Reader, () => Remove(id));
        }
    }

    private void Remove(int id)
    {
        lock (_gate)
        {
            if (_subs.Remove(id, out var ch))
            {
                ch.Writer.Complete();
            }
        }
    }
```

listing 127: C#, Subscribe returns a bounded reader and its cancel, Remove completes the writer (source: patterns-concurrency-distributed/samples-cs/src/Ch04/Observer.cs, lines 17-46)

```
  subscribe(capacity) {
    const q = { events: [], capacity, open: true, cancel: null };
    if (this.#closed) {
      return q;
    }
    const id = this.#next++;
    this.#subs.set(id, q);
    q.cancel = () => this.#remove(id);
    return q;
  }

  #remove(id) {
    const q = this.#subs.get(id);
    if (q) {
      this.#subs.delete(id);
      q.open = false;
    }
  }
```

listing 128: JavaScript, subscribe returns a queue record and its cancel closure (source: patterns-concurrency-distributed/samples-js/src/ch04-observer.mjs, lines 12-29)

```
    def subscribe(self, cap):
        if self.closed:
            dead = Subscription(0)
            dead.closed = True
            return dead
        s = Subscription(cap)
        self.subs[self.next_id] = s
        self.next_id += 1
        return s

    def notify(self, kind):
        if self.closed:
            return 0
        delivered = 0
        for s in self.subs.values():
            if s.send(kind):
                delivered += 1
        return delivered

    def unsubscribe(self, s):
        s.closed = True
        for k, sub in list(self.subs.items()):
            if sub is s:
                del self.subs[k]
```

listing 129: Python, subscribe hands back a Subscription, unsubscribe releases it (source: patterns-concurrency-distributed/samples-py/src/Ch04/observer.py, lines 45-69)

```
function Hub:Subscribe(buffer)
  local s = Sub.new(buffer)
  if self.closed then
    s.closed = true
  else
    self.subs[#self.subs + 1] = s
  end
  local id = #self.subs
  local canceled = false
  -- unsubscribe: mark the sub closed so Notify skips it and its
  -- buffered values still drain, exactly like closing the channel
  return s, function()
    if not canceled and self.subs[id] == s then
      canceled = true
      s.closed = true
      self.subs[id] = nil
    end
  end
end
```

listing 130: Lua, Subscribe returns the sub and a double-call-safe cancel (source: patterns-concurrency-distributed/samples-lua/ch04_observer.lua, lines 33-51)

Registration is the quieter half of the pattern, and the notify half carries the two contract points the tests pin:

```c
// try-send: a full buffer drops rather than blocking the producer
static int hub_notify(hub *h, const char *kind) {
  mtx_lock(&h->mu);
  int delivered = 0;
  if (!h->closed) {
    for (int i = 0; i < h->n; i++) {
      subscriber *s = &h->subs[i];
      if (s->closed)
        continue;
      if (s->count < s->cap) {
        snprintf(s->buf[s->count].kind, 8, "%s", kind);
        s->count++;
        delivered++;
      }
    }
  }
  mtx_unlock(&h->mu);
  return delivered;
}
```

listing 131: C, try-send into each bounded buffer, a full one drops (source: patterns-concurrency-distributed/samples-c/src/Ch04/observer.c, lines 46-64)

```
    fn notify(&self, kind: &str) -> usize {
        let mut h = self.inner.lock().unwrap();
        let mut delivered = 0;
        if !h.closed {
            for s in &mut h.subs {
                if s.closed {
                    continue;
                }
                if s.buf.len() < s.cap {
                    s.buf.push(kind.to_string());
                    delivered += 1;
                }
            }
        }
        delivered
    }

    fn cancel(&self, id: usize) {
        let mut h = self.inner.lock().unwrap();
        if let Some(s) = h.subs.get_mut(id) {
            s.closed = true;
        }
    }

    fn close(&self) {
        let mut h = self.inner.lock().unwrap();
        h.closed = true;
        for s in &mut h.subs {
            s.closed = true;
        }
    }

    fn receive(&self, id: usize) -> Option<String> {
        let mut h = self.inner.lock().unwrap();
        match h.subs.get_mut(id) {
            Some(s) if !s.buf.is_empty() => Some(s.buf.remove(0)),
            _ => None,
        }
    }
```

listing 132: Rust, notify counts deliveries into bounded buffers, close marks every sub, receive drains what was sent (source: patterns-concurrency-distributed/samples-rust/src/Ch04/observer.rs, lines 55-93)

```go
func (h *Hub) Notify(e Event) (delivered int) {
  h.mu.Lock()
  defer h.mu.Unlock()
  if h.closed {
    return 0
  }
  for _, ch := range h.subs {
    select {
    case ch <- e:
      delivered++
    default: // full buffer: drop rather than block the producer
    }
  }
  return delivered
}

func (h *Hub) Close() {
  h.mu.Lock()
  defer h.mu.Unlock()
  h.closed = true
  for id, ch := range h.subs {
    delete(h.subs, id)
    close(ch)
  }
}
```

listing 133: Go, non-blocking fan-out, then close on shutdown (source: patterns-concurrency-distributed/samples/ch04/behavioral.go, lines 80-104)

```java
    // try-send to every live subscriber: a full buffer drops rather
    // than blocking the producer
    synchronized int notify(String kind) {
      int delivered = 0;
      if (!closed)
        for (Subscriber s : subs)
          if (!s.closed && s.send(kind))
            delivered++;
      return delivered;
    }

    synchronized void cancel(int id) {
      if (id ≥ 0 && id < subs.size())
        subs.get(id).closed = true;
    }

    synchronized void close() {
      closed = true;
      for (Subscriber s : subs)
        s.closed = true;
    }

    // receive one buffered event, or null once drained
    synchronized String receive(int id) {
      return subs.get(id).buf.poll();
    }
```

listing 134: Java, notify try-sends to live subscribers, close marks them closed, receive still drains the buffer (source: patterns-concurrency-distributed/samples-java/src/Ch04/Observer.java, lines 46-71)

```
    public int Notify(Event e)
    {
        lock (_gate)
        {
            if (_closed)
            {
                return 0;
            }
            int delivered = 0;
            foreach (var ch in _subs.Values)
            {
                // full buffer: drop rather than block the producer
                if (ch.Writer.TryWrite(e))
                {
                    delivered++;
                }
            }
            return delivered;
        }
    }

    public void Close()
    {
        lock (_gate)
        {
            _closed = true;
            foreach (var ch in _subs.Values)
            {
                ch.Writer.Complete();
            }
            _subs.Clear();
        }
    }
```

listing 135: C#, TryWrite is the drop, Complete is the close (source: patterns-concurrency-distributed/samples-cs/src/Ch04/Observer.cs, lines 48-80)

```
notify(e) {
  if (this.#closed) return 0;
  let delivered = 0;
  for (const q of this.#subs.values()) {
    if (q.events.length < q.capacity) {
      q.events.push(e);
      delivered++;
    }
  }
  return delivered;
}

close() {
  this.#closed = true;
  for (const q of this.#subs.values()) q.open = false;
  this.#subs.clear();
}
```

listing 136: JavaScript, notify walks the queues, close flips every one (source: patterns-concurrency-distributed/samples-js/src/ch04-observer.mjs, lines 31-47)

```
    def notify(self, kind):
        if self.closed:
            return 0
        delivered = 0
        for s in self.subs.values():
            if s.send(kind):
                delivered += 1
        return delivered

    def unsubscribe(self, s):
        s.closed = True
        for k, sub in list(self.subs.items()):
            if sub is s:
                del self.subs[k]

    def close(self):
        self.closed = True
        for s in self.subs.values():
            s.closed = True
```

listing 137: Python, notify counts deliveries, close releases without erasing (source: patterns-concurrency-distributed/samples-py/src/Ch04/observer.py, lines 55-73)

```
function Hub:Notify(e)
  if self.closed then
    return 0
  end
  local delivered = 0
  for _, s in ipairs(self.subs) do
    if not s.closed and #s.buf < s.cap then
      s.buf[#s.buf + 1] = e
      delivered = delivered + 1
    end
  end
  return delivered
end

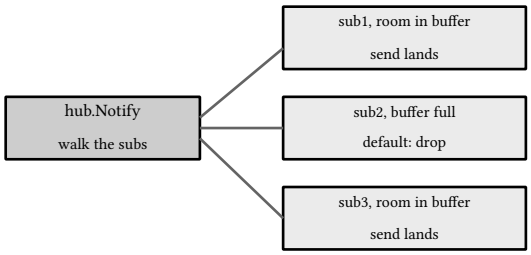
function Hub:Close()
  self.closed = true
  for _, s in ipairs(self.subs) do
    s.closed = true
  end
end
```

listing 138: Lua, Notify skips closed subs, Close marks them all (source: patterns-concurrency-distributed/samples-lua/ch04_observer.lua, lines 55-74)

Dropping is deliberate: a slow subscriber with a full buffer loses the event instead of blocking the producer, the drop policy of last resort, and the alternatives, infinite buffers, blocking sends, one goroutine per subscriber, are chapter 8's fan-in fan-out discussion. And `close` does not erase buffered values: a subscriber that lags behind shutdown still drains what it was sent, which the test asserts by draining `slow` after `Close` and finding its one buffered event.

The cancel function returned from `Subscribe` is the registration lifecycle made explicit, the caller owns unsubscription and double cancel is safe because `remove` checks the map first. The sibling trees build the channel by hand where they must: C keeps a fixed array of bounded subscriber buffers under one mutex, rust is the same shape as `Mutex` over a Vec of bounded subscribers with int ids, plus a producer on another thread filling the buffer, java rides

the same int-id api with a synchronized Hub over one ArrayDeque per subscriber, C# hands each subscriber a real bounded Channel where `TryWrite` stands in for the select default, javascript pushes onto plain arrays inside a Map whose insertion order fixes delivery order, python's Subscription answers takes with a value, a closed marker, or a BLOCKED sentinel naming where a goroutine would park, and lua keeps one table per subscriber with its own bounded buffer.



close releases everyone, a lagging subscriber still drains what it buffered

figure 20: notify is a non-blocking fan-out, slow subscribers drop, close preserves buffers

4.3. command with undo

A command object captures an operation and its inverse. In go the operation is a closure and the inverse is captured alongside:

The dry run: the fixture walk sets one key to 7, then to 9, and undo restores 7, undo again removes the key, and undo on the spent history reports false. Interleaved keys replay newest first. The go lane's frozen test walks its own constants, retries 3 then 5 beside mode 1, against the same implementation.

1. Set captures the previous value and whether the key existed, so the undo of an overwrite is a restore and the undo of an insert is a delete.
2. Undo pops the history last in first out and replays the inverse it finds there.
3. The history is a stack of captures, one per command, not a diff per key, which the interleaved walk pins.

```
static void session_set(session *s, const char *key, int v) {
  int i = slot_of(s, key);
  if (i < 0) {
    for (i = 0; i < SLOTS; i++)
      if (!s->used[i])
        break;
    snprintf(s->key[i], 16, "%s", key);
    s->used[i] = true;
    s->val[i] = v;
    if (s->n_undo < UNDO) {
      s->undo[s->n_undo] = (undo_record){{0}, 0, false};
      snprintf(s->undo[s->n_undo].key, 16, "%s", key);
      s->n_undo++;
    }
    return;
  }
  if (s->n_undo < UNDO) {
    s->undo[s->n_undo].prev = s->val[i];
    s->undo[s->n_undo].existed = true;
    snprintf(s->undo[s->n_undo].key, 16, "%s", key);
    s->n_undo++;
  }
  s->val[i] = v;
}

// pop the newest record and run its inverse: restore, or delete when the
// command was the insert
static bool session_undo(session *s) {
  if (s->n_undo == 0)
    return false;
  undo_record r = s->undo[--s->n_undo];
  int i = slot_of(s, r.key);
  if (i < 0)
    return true;
  if (r.existed)
    s->val[i] = r.prev;
  else
    s->used[i] = false;
  return true;
}

static bool session_get(session *s, const char *key, int *out) {
  int i = slot_of(s, key);
  if (i < 0)
    return false;
  *out = s->val[i];
  return true;
}
```

listing 139: C, explicit undo records in a fixed array, undo restores or deletes (source: patterns-concurrency-distributed/samples-c/src/Ch04/command.c, lines 43-90)

```
impl Session {
    fn new() -> Self {
        Session {
            slots: HashMap::new(),
            undo: Vec::new(),
        }
    }

    fn set(&mut self, key: &str, v: i32) {
        match self.slots.get(key) {
            Some(&prev) => {
                self.undo.push(UndoRecord {
                    key: key.to_string(),
                    prev,
                    existed: true,
                });
                self.slots.insert(key.to_string(), v);
            }
            None => {
                self.slots.insert(key.to_string(), v);
                self.undo.push(UndoRecord {
                    key: key.to_string(),
                    prev: 0,
                    existed: false,
                });
            }
        }
    }

    fn undo(&mut self) -> bool {
        let Some(record) = self.undo.pop() else {
            return false;
        };
        if record.existed {
            self.slots.insert(record.key, record.prev);
        } else {
            self.slots.remove(&record.key);
        }
        true
    }
```

listing 140: Rust, set pushes a key, prev, existed record, undo pops and restores or removes (source: patterns-concurrency-distributed/samples-rust/src/Ch04/command.rs, lines 26-65)

```go
type Session struct {
  values map[string]int
  undo   []func()
}

func NewSession() *Session { return &Session{values: map[string]int{}} }

func (s *Session) Set(key string, v int) {
  prev, existed := s.values[key]
  s.values[key] = v
  s.undo = append(s.undo, func() {
    if existed {
      s.values[key] = prev
    } else {
      delete(s.values, key)
    }
  })
}

func (s *Session) Undo() bool {
  if len(s.undo) == 0 {
    return false
  }
  s.undo[len(s.undo)-1]()
  s.undo = s.undo[:len(s.undo)-1]
  return true
}

func (s *Session) Get(key string) (int, bool) { v, ok := s.values[key]; return v, ok }
```

listing 141: Go, set records the previous value, undo pops and replays it (source: patterns-concurrency-distributed/samples/ch04/behavioral.go, lines 107-136)

```
  record Undo(String key, Integer prev, boolean existed) {}

  static final class Session {
    final Map<String, Integer> values = new HashMap<>();
    final ArrayDeque<Undo> undo = new ArrayDeque<>();

    void set(String key, int v) {
      undo.push(new Undo(key, values.get(key), values.containsKey(key)));
      values.put(key, v);
    }

    // pop the newest record and run its inverse: restore, or remove
    // when the command was the insert
    boolean undoOne() {
      Undo r = undo.poll();
      if (r == null)
        return false;
      if (r.existed())
        values.put(r.key(), r.prev());
      else
        values.remove(r.key());
      return true;
    }

    Integer get(String key) {
      return values.get(key);
    }
  }
```

listing 142: Java, each set pushes an Undo record, undo replays restore or remove newest first (source: patterns-concurrency-distributed/samples-java/src/Ch04/Command.java, lines 18-45)

```
public sealed class Session
{
    private readonly Dictionary<string, int> _values = [];
    private readonly List<Action> _undo = [];

    public void Set(string key, int v)
    {
        bool existed = _values.TryGetValue(key, out int prev);
        _values[key] = v;
        _undo.Add(() =>
        {
            if (existed)
            {
                _values[key] = prev!;
            }
            else
            {
                _values.Remove(key);
            }
        });
    }

    public bool Undo()
    {
        if (_undo.Count == 0)
        {
            return false;
        }
        _undo[^1]();
        _undo.RemoveAt(_undo.Count - 1);
        return true;
    }
```

listing 143: C#, each Set pushes a restoring closure, Undo pops and runs it (source: patterns-concurrency-distributed/samples-cs/src/Ch04/Command.cs, lines 5-36)

```
export class Session {
  #values = new Map();
  #undo = [];

  set(key, v) {
    const existed = this.#values.has(key);
    const prev = this.#values.get(key);
    this.#values.set(key, v);
    this.#undo.push(() => {
      if (existed) {
        this.#values.set(key, prev);
      } else {
        this.#values.delete(key);
      }
    });
  }

  undo() {
    const step = this.#undo.pop();
    if (step === undefined) return false;
    step();
    return true;
  }

  get(key) {
    return [this.#values.get(key), this.#values.has(key)];
  }
```

listing 144: JavaScript, the undo stack holds closures over existed and prev (source: patterns-concurrency-distributed/samples-js/src/ch04-command.mjs, lines 5-31)

```
class Session:
    def __init__(self):
        self.values = {}
        self.undo = []

    def set(self, key, v):
        existed = key in self.values
        prev = self.values.get(key)
        self.values[key] = v
        self.undo.append((key, prev, existed))

    def undo_one(self):
        if not self.undo:
            return False
        key, prev, existed = self.undo.pop()
        if existed:
            self.values[key] = prev
        else:
            del self.values[key]
        return True

    def get(self, key):
        return self.values.get(key)
```

listing 145: Python, the history is a list of key, prev, existed tuples (source: patterns-concurrency-distributed/samples-py/src/Ch04/command.py, lines 14-36)

```lua
function Session.new()
  return setmetatable({ values = {}, undo = {} }, Session)
end

function Session:Set(key, v)
  local prev = self.values[key]
  local existed = self.values[key] ~= nil
  self.values[key] = v
  self.undo[#self.undo + 1] = function()
    if existed then
      self.values[key] = prev
    else
      self.values[key] = nil
    end
  end
end

function Session:Undo()
  local n = #self.undo
  if n == 0 then
    return false
  end
  local top = self.undo[n]
  self.undo[n] = nil
  top()
  return true
end

function Session:Get(key)
  return self.values[key]
end
```

listing 146: Lua, Set's closure captures prev and existed for Undo (source: patterns-concurrency-distributed/samples-lua/ch04_command.lua, lines 9-39)

The `Set` method closes over `prev` and `existed`, so the undo of an insert is a delete and the undo of an overwrite is a restore. The test walks all three cases. Seven of the eight trees capture that inverse as a closure, a tuple, or a record, rust as a plain `UndoRecord` struct of key, prev, and existed, and C stores explicit undo records in a fixed array because it has no closures to capture anything, the same trade its strategy listing made. This is also the seed of the raft capstone's restart story: a write ahead log is a command history, replay is redo, and chapter 11 returns to the equivalence as idempotent re-delivery, chapter 16 as replay on restart.

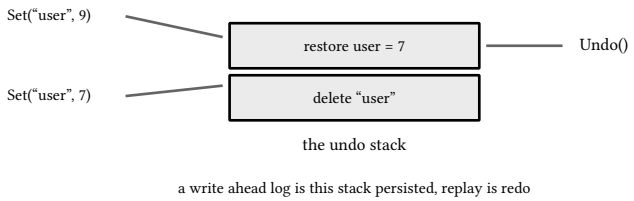


figure 21: set captures the inverse, undo pops the stack and replays it

4.4. iterator, push style

Go 1.23 grew iterators into the language: `range` accepts functions of type `func(yield func(V) bool)`, and the `iter` package names them. The pattern is a function that calls `yield` for each element and stops the moment `yield` returns false:

The dry run: the fixture is the pangram the quick brown fox jumps over the lazy dog. Filtering words longer than 3 keeps quick, brown, jumps, over, lazy, sorting by length replays over, lazy, quick, brown, jumps with ties keeping source order, and early exit takes exactly the first two, quick and brown. The source observes the stop after offering exactly 3 words, the, quick, and brown. The go lane's frozen test pins the filtered set, the stable sort, and the first two, the offered-count stop probe is the seven new trees' lane.

1. The consumer's break makes yield return false, and the false verdict unwinds Filter into the source, which stops where it stands after 3 offers, not 9.
2. Sorted materializes once, a full drain of all 9 words before the first replay.
3. The stability pin: over and lazy tie at 4 letters and keep source order, so the adapter never scrambles its input's tie order.

```c
static void words_run(void *self, yield_fn yield, void *yctx) {
  words_state *ws = self;
  const char *p = ws->text;
  while (*p) {
    while (*p == ' ')
      p++;
    if (!*p)
      break;
    char word[WORDLEN];
    int len = 0;
    while (*p && *p != ' ' && len < WORDLEN - 1)
      word[len++] = *p++;
    while (*p && *p != ' ')
      p++;
    word[len] = '\0';
    ws->offered++;
    if (!yield(yctx, word))
      return;
  }
}

static seq words_seq(words_state *st, const char *text) {
  st->text = text;
  st->offered = 0;
  seq s = {words_run, st};
  return s;
}
```

listing 147: C, the source drives a yield callback, a false verdict stops it (source: patterns-concurrency-distributed/samples-c/src/Ch04/iterator.c, lines 35-61)

```rust
struct Words {
    text: &'static str,
    offered: usize,
}

impl Seq for Words {
    fn run(&mut self, yield_: YieldFn) {
        for word in self.text.split(' ').filter(|w| !w.is_empty()) {
            self.offered += 1;
            if !yield_(word) {
                return;
            }
        }
    }
}

struct Filter<S: Seq> {
    inner: S,
    keep: fn(&str) -> bool,
}

impl<S: Seq> Seq for Filter<S> {
    fn run(&mut self, yield_: YieldFn) {
        let keep = self.keep;
        self.inner.run(&mut |w| if keep(w) {
            yield_(w)
        } else {
            true
        });
    }
}

struct Sorted<S: Seq> {
    inner: S,
    items: Vec<String>,
}

impl<S: Seq> Seq for Sorted<S> {
    fn run(&mut self, yield_: YieldFn) {
        self.items.clear();
        self.inner.run(&mut |w| {
            self.items.push(w.to_string());
            true
        });
        self.items.sort_by_key(|w| w.len());
        for item in &self.items {
            if !yield_(item.as_str()) {
                return;
            }
        }
    }
}
```

listing 148: Rust, no yield either, a run trait over a yield callback, Filter and Sorted as generic adapters over it (source: patterns-concurrency-distributed/samples-rust/src/Ch04/iterator.rs, lines 19-70)

```
func Words(text string) iter.Seq[string] {
  return func(yield func(string) bool) {
    for _, w := range strings.Fields(text) {
      if !yield(w) {
        return
      }
    }
  }
}

// Filter adapts one iterator into another, a decorator for
// sequences.
func Filter[V any](seq iter.Seq[V], keep func(V) bool) iter.Seq[V] {
  return func(yield func(V) bool) {
    seq(func(v V) bool {
      if keep(v) {
        return yield(v)
      }
      return true
    })
  }
}

// Sorted adapts an iterator by materializing it once. Stable, so
// equal elements keep their source order.
func Sorted[V any](seq iter.Seq[V], less func(a, b V) int) iter.Seq[V] {
  return func(yield func(V) bool) {
    all := []V{}
    seq(func(v V) bool { all = append(all, v); return true })
    slices.SortStableFunc(all, less)
    for _, v := range all {
      if !yield(v) {
        return
      }
    }
  }
}

// State. A turnstile as a transition table: state, event, next
```

listing 149: Go, words as a sequence, filter and sorted as sequence adapters (source: patterns-concurrency-distributed/samples/ch04/behavioral.go, lines 139-177)

```java
// false stops the source, true keeps it going
interface Seq {
  void run(Predicate<String> yield);
}

// the source: offers one word at a time, counts what it offered
static final class Words implements Seq {
  private final String text;
  int offered;

  Words(String text) {
    this.text = text;
  }

  public void run(Predicate<String> yield) {
    for (String w : text.split(" ")) {
      offered++;
      if (!yield.test(w))
        return;
    }
  }
}

// filter: a decorator for sequences, it forwards the verdict verbatim
static Seq filtered(Seq inner, Predicate<String> keep) {
  return yield -> inner.run(w -> !keep.test(w) || yield.test(w));
}

// sorted: drain the source once, stable sort by length, replay
// ordered. List.sort is stable, so equal lengths keep source order,
// the guarantee the go lane pins with SortStableFunc
static Seq sorted(Seq inner) {
  return yield -> {
    List<String> all = new ArrayList<>();
    inner.run(
        w -> {
          all.add(w);
          return true;
        });
    all.sort(Comparator.comparingInt(String::length));
    for (String w : all)
      if (!yield.test(w))
        return;
  };
}
```

listing 150: Java, no yield keyword, Seq pushes into a Predicate whose false verdict stops the source, List.sort is stable (source: patterns-concurrency-distributed/samples-java/src/Ch04/Iterator.java, lines 20-64)

```
    public static IEnumerable<string> Words(string text)
    {
        var word = new List<char>();
        foreach (char c in text)
        {
            if (c == ' ' || c == '\t' || c == '\n' || c == '\r')
            {
                if (word.Count > 0)
                {
                    yield return new string([.. word]);
                    word.Clear();
                }
            }
            else
            {
                word.Add(c);
            }
        }
        if (word.Count > 0)
        {
            yield return new string([.. word]);
        }
    }

    // Filter adapts one sequence into another, a decorator for
    // sequences, still lazy end to end.
    public static IEnumerable<T> Filter<T>(IEnumerable<T> seq, Func<T, bool> keep)
    {
        foreach (var v in seq)
        {
            if (keep(v))
            {
                yield return v;
            }
        }
    }
```

listing 151: C#, yield return words, Filter stays lazy, Sorted materializes below (source: patterns-concurrency-distributed/samples-cs/src/Ch04/Iterator.cs, lines 8-43)

```
export function* words(text, probe = null) {
  try {
    for (const w of text.split(" ").filter((s) => s.length > 0)) {
      if (probe) probe.yielded++;
      yield w;
    }
  } finally {
    if (probe) probe.closed = true;
  }
}

export function* filter(seq, keep) {
  for (const v of seq) {
    if (keep(v)) yield v;
  }
}

export function* sorted(seq, cmp) {
  const all = [...seq];
  all.sort(cmp);
  yield* all;
}
```

listing 152: JavaScript, generator functions, the finally block observes the stop (source: patterns-concurrency-distributed/samples-js/src/ch04-iterator.mjs, lines 8-29)

```
def words(text, trace=None):
    try:
        for w in text.split():
            if trace is not None:
                trace["scanned"] += 1
            yield w
    finally:
        if trace is not None:
            trace["stopped"] = True


def filtered(seq, keep):
    try:
        for v in seq:
            if keep(v):
                yield v
    finally:
        close = getattr(seq, "close", None)
        if close is not None:
            close()


def ordered_by(seq, key):
    # an adapter that must materialize: it drains the source once,
    # sorts stably, then replays
    everything = list(seq)
    close = getattr(seq, "close", None)
    if close is not None:
        close()
    yield from sorted(everything, key=key)
```

listing 153: Python, generators, close runs the finally, ordered_by drains once (source: patterns-concurrency-distributed/samples-py/src/Ch04/iterator.py, lines 14-43)

```lua
local function Words(text)
  return function(yield)
    for w in text:gmatch("%S+") do
      if not yield(w) then
        return
      end
    end
  end
end

-- an adapter over a sequence, a decorator for sequences: values that
-- fail the predicate never reach the consumer, and the consumer's
-- stop still propagates to the source
local function Filter(seq, keep)
  return function(yield)
    seq(function(v)
      if keep(v) then
        return yield(v)
      end
      return true
    end)
  end
end

-- table.sort is not stable, so the sort runs over positions and
-- breaks ties by source order
local function Sorted(seq, less)
  return function(yield)
    local all = {}
    seq(function(v)
      all[#all + 1] = v
      return true
    end)
    local idx = {}
    for i = 1, #all do idx[i] = i end
    table.sort(idx, function(a, b)
      if less(all[a], all[b]) then return true end
      if less(all[b], all[a]) then return false end
      return a < b
    end)
    for _, i in ipairs(idx) do
      if not yield(all[i]) then
        return
      end
    end
  end
end
```

listing 154: Lua, a sequence is a function over a yield callback, sort stability by index tiebreak (source: patterns-concurrency-distributed/samples-lua/ch04_iterator.lua, lines 7-53)

`Filter` and `Sorted` are decorators for sequences, chapter 3's idea with the language doing the wrapping. `Sorted` materializes once and uses `SortStableFunc`, because a plain unstable sort reorders equal elements and an adapter that scrambles its input's tie order is a surprise factory, the sample's own test caught exactly that before the fix. Early exit works through the chain: a `break` in the caller's range loop makes `yield` return false, `Filter`'s inner walk sees it and unwinds, and the test takes the first two words and stops.

The sibling trees split by what the language gives them. C# and python grow push iterators natively as `yield return` and generators, the interpreter pausing the function at each yieldbook 8, ch 4 (python 3.14, builtin collections, iteration, and generators). Javascript generators do the same and add the honest probe: closing the generator runs its `finally` block, so the source's stop observation is a language guarantee. Java has no yield at all, so its `Seq` is an interface that pushes into a `Predicate<String>`, the boolean verdict the generator's channel, and `List.sort` is stable, so the sorted adapter needs no index tiebreak. Rust joins java in having no yield, its `Seq` a trait whose `run` takes the yield as a `&mut dyn FnMut(&str) → bool`, and its adapters are generic structs, `Filter<S: Seq>` and `Sorted<S: Seq>` each a distinct monomorphized type, the composition free at run time and one concrete type per chain at build time. The file closes by pinning the same walk in the stdlib's pull form, `split_whitespace().filter().take(2)`.

Lua's push sequence is a function that feeds a yield callback, and `coroutine.wrap` gives the same iteration shape natively over cooperative threadsbook 9, ch 7 (lua 5.5, coroutines). C has neither closures nor generators, so the yield callback carries a context pointer and the consumer's verdict is its return value, and the sorted adapter breaks ties by source index because the platform sort is unstable.

For pull-style iteration, `iter.Pull` converts a sequence into `next, stop` functions for loops that are not shaped like for statements, consuming from two sequences in lockstep being the canonical case.

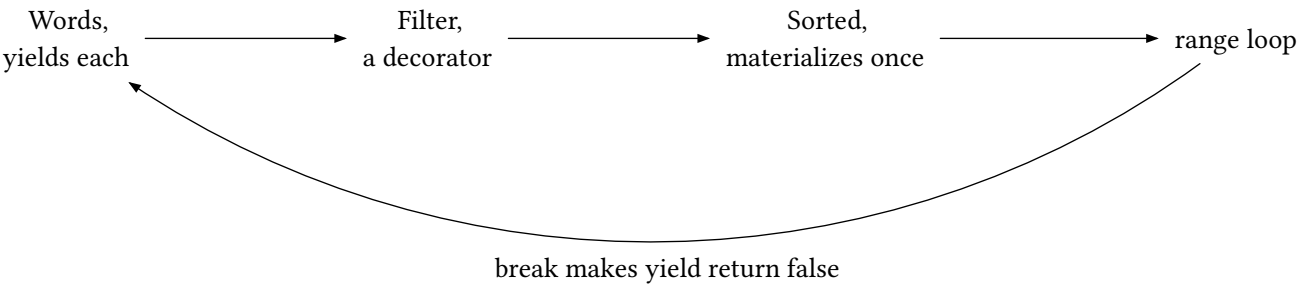


figure 22: adapters wrap sequences, early exit unwinds the whole chain

4.5. state as a transition table

A state machine is a map keyed by state and event, holding the next state and an action, and firing an event is one lookup:

The dry run: the fixture walk fires coin, push, coin, coin, push, kick from the locked gate, and the outcomes pin row by row as (unlocked, thanks), (locked, click), (unlocked, thanks), (unlocked, thanks), (locked, click), and (rejected, locked). The log records the 5 accepted actions and skips the kick. The go lane's frozen test walks its own script, kick, push, coin, push, with the log alarm, thanks, click, against the same table.

1. The table holds 4 real transitions, 2 states by 2 events, and the whole machine reads at a glance.
2. The unknown event is rejected without moving the state and without logging, so rejection costs nothing.
3. The action log doubles as the observable output, which makes the walk a pinned sequence instead of a print.

```
// rows: [state][event]. locked+push alarms, unlocked+push clicks, both
// coin rows thank, kick is rejected from every state
static const transition table[NSTATES][NEVENTS] = {
    [LOCKED] = {[EV_COIN] = {UNLOCKED, 0},
                [EV_PUSH] = {LOCKED, 2},
                [EV_KICK] = {LOCKED, -1}},
    [UNLOCKED] = {[EV_COIN] = {UNLOCKED, 0},
                  [EV_PUSH] = {LOCKED, 1},
                  [EV_KICK] = {LOCKED, -1}},
};

typedef struct {
  state current;
  int log[12];
  int n_log;
} turnstile;

static void turnstile_init(turnstile *t) {
  t->current = LOCKED;
  t->n_log = 0;
}

// fire returns the resulting state and records the action, or rejects
static bool turnstile_fire(turnstile *t, event ev, state *out,
                           const char **action) {
  const transition *row = &table[t->current][ev];
  if (ev < 0 || ev ≥ NEVENTS || row->action < 0) {
    *out = t->current;
    *action = "rejected";
    return false;
  }
  t->current = row->next;
  if (t->n_log < 12)
    t->log[t->n_log++] = row->action;
  *out = row->next;
  *action = action_name[row->action];
  return true;
}
```

listing 155: C, the table as a 2d array of transitions, fire is one row lookup (source: patterns-concurrency-distributed/samples-c/src/Ch04/state.c, lines 30-67)

```
struct Turnstile {
    current: State,
    log: Vec<Action>,
}

impl Turnstile {
    fn new() -> Self {
        Turnstile {
            current: State::Locked,
            log: Vec::new(),
        }
    }

    fn fire(&mut self, ev: Event) -> (bool, State, &'static str) {
        let row = &TABLE[self.current as usize][ev as usize];
        match row.action {
            None => (false, self.current, "rejected"),
            Some(action) => {
                self.current = row.next;
                self.log.push(action);
                (true, row.next, ACTION_NAME[action as usize])
            }
        }
    }
}

fn event_by_name(name: &str) -> Event {
    match name {
        "coin" => Event::Coin,
        "push" => Event::Push,
        _ => Event::Kick,
    }
}
```

listing 156: Rust, the const table indexed by enum casts, a None action is the rejection that never logs (source: patterns-concurrency-distributed/samples-rust/src/Ch04/state.rs, lines 72-104)

```
type State string

const (
  Locked   State = "locked"
  Unlocked State = "unlocked"
)

type Action string

const (
  Thanks Action = "thanks"
  Click  Action = "click"
  Alarm  Action = "alarm"
)

type Transition struct {
  Next  State
  Action Action
}

type Turnstile struct {
  state State
  table map[State]map[string]Transition
  Log   []Action
}

func NewTurnstile() *Turnstile {
  return &Turnstile{
    state: Locked,
    table: map[State]map[string]Transition{
      Locked: {
        "coin": {Unlocked, Thanks},
        "push": {Locked, Alarm},
      },
      Unlocked: {
        "coin": {Unlocked, Thanks},
        "push": {Locked, Click},
      },
    },
  }
}

func (t *Turnstile) Fire(event string) (State, Action, bool) {
  row, ok := t.table[t.state][event]
  if !ok {
    return t.state, "", false
  }
  t.state = row.Next
  t.Log = append(t.Log, row.Action)
  return row.Next, row.Action, true
}

// Visitor as a type switch. The variants are plain structs; the
```

listing 157: Go, turnstile: two states, two events, four transitions, all visible at once (source: patterns-concurrency-distributed/samples/ch04/behavioral.go, lines 179-231)

```
enum St {
  LOCKED,
  UNLOCKED
}

enum Ev {
  COIN,
  PUSH,
  KICK
}

record Transition(St next, String action) {}

record Outcome(St state, String action, boolean moved) {}

// rows: [state][event]. locked+push alarms, unlocked+push clicks,
// both coin rows thank, kick is rejected from every state, a null
// row is the rejection
static final Transition[][] TABLE = {
    {new Transition(St.UNLOCKED, "thanks"), new Transition(St.LOCKED, "alarm"), null},
    {new Transition(St.UNLOCKED, "thanks"), new Transition(St.LOCKED, "click"), null},
};

static final class Turnstile {
  St current = St.LOCKED;
  final List<String> log = new ArrayList<>();

  Outcome fire(Ev ev) {
    Transition row = TABLE[current.ordinal()][ev.ordinal()];
    if (row == null)
      return new Outcome(current, "rejected", false);
    current = row.next();
    log.add(row.action());
    return new Outcome(row.next(), row.action(), true);
  }
}
```

listing 158: Java, enums index a Transition table, a null row is the rejection, fire is one lookup (source: patterns-concurrency-distributed/samples-java/src/Ch04/State.java, lines 19-54)

```
public sealed class Turnstile
{
    private string _state = TurnstileStates.Locked;
    private readonly Dictionary<string, Dictionary<string, Transition>> _table;
    public List<string> Log { get; } = [];

    public Turnstile()
    {
        _table = new Dictionary<string, Dictionary<string, Transition>>
        {
            [TurnstileStates.Locked] = new()
            {
                ["coin"] = new(TurnstileStates.Unlocked, Actions.Thanks),
                ["push"] = new(TurnstileStates.Locked, Actions.Alarm),
            },
            [TurnstileStates.Unlocked] = new()
            {
                ["coin"] = new(TurnstileStates.Unlocked, Actions.Thanks),
                ["push"] = new(TurnstileStates.Locked, Actions.Click),
            },
        };
    }

    public (string state, string action, bool ok) Fire(string evt)
    {
        if (_table.TryGetValue(_state, out var row) && row.TryGetValue(evt, out var t))
        {
            _state = t.Next;
            Log.Add(t.Action);
            return (t.Next, t.Action, true);
        }
        return (_state, "", false);
    }
}
```

listing 159: C#, nested dictionaries, Fire moves and logs (source: patterns-concurrency-distributed/samples-cs/src/Ch04/State.cs, lines 20-53)

```
export class Turnstile {
  #state = LOCKED;
  #table = new Map([
    [`${LOCKED}|coin`, { next: UNLOCKED, action: "thanks" }],
    [`${LOCKED}|push`, { next: LOCKED, action: "alarm" }],
    [`${UNLOCKED}|coin`, { next: UNLOCKED, action: "thanks" }],
    [`${UNLOCKED}|push`, { next: LOCKED, action: "click" }],
  ]);
  log = [];

  fire(event) {
    const row = this.#table.get(`${this.#state}|${event}`);
    if (row === undefined) {
      return { state: this.#state, action: "", accepted: false };
    }
    this.#state = row.next;
    this.log.push(row.action);
    return { state: row.next, action: row.action, accepted: true };
  }
}
```

listing 160: JavaScript, a Map keyed by the state and event pair (source: patterns-concurrency-distributed/samples-js/src/ch04-state.mjs, lines 8-27)

```
TABLE = {
    ("locked", "coin"): ("unlocked", "thanks"),
    ("locked", "push"): ("locked", "alarm"),
    ("unlocked", "coin"): ("unlocked", "thanks"),
    ("unlocked", "push"): ("locked", "click"),
}


class Turnstile:
    def __init__(self):
        self.state = "locked"
        self.log = []

    def fire(self, event):
        row = TABLE.get((self.state, event))
        if row is None:
            return self.state, None, False
        self.state = row[0]
        self.log.append(row[1])
        return row[0], row[1], True
```

listing 161: Python, a dict keyed by state and event tuples (source: patterns-concurrency-distributed/samples-py/src/Ch04/state.py, lines 14-33)

```lua
local STANDARD = {
  locked = {
    coin = { "unlocked", "thanks" },
    push = { "locked", "alarm" },
  },
  unlocked = {
    coin = { "unlocked", "thanks" },
    push = { "locked", "click" },
  },
}

-- a gate with no latch: pushing while unlocked changes nothing
local FREEWALK = {
  locked = {
    coin = { "unlocked", "thanks" },
    push = { "locked", "alarm" },
  },
  unlocked = {
    coin = { "unlocked", "thanks" },
    push = { "unlocked", "thanks" },
  },
}

local function Fire(t, event)
  local row = t.table[t.state][event]
  if not row then
    return t.state, "rejected", false
  end
  t.state = row[1]
  t.log[#t.log + 1] = row[2]
  return row[1], row[2], true
end
```

listing 162: Lua, nested tables, and a swapped table is a different machine (source: patterns-concurrency-distributed/samples-lua/ch04_state.lua, lines 11-42)

The table makes the whole machine inspectable at a glance, which is the entire argument for it over a switch per state. Unknown events are rejected without a transition, and the log of actions doubles as the observable output. The spelling of the key tracks the language: a 2d array with a rejected marker in C, a const 2d array of struct transitions indexed by enum casts in rust where a `None` action is the rejection, enums indexing a `Transition[][]` in java where a null row is the rejection, nested dictionaries in C#, a template string key in javascript, a tuple key in python, nested tables in lua, where handing the constructor a different table yields a different machine with the same Fire. This table returns in the capstone as the follower, candidate, leader spine of raft, with timers instead of pushes.

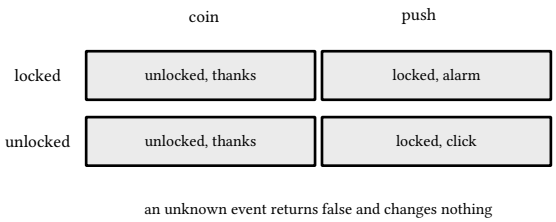


figure 23: the whole machine in one table, every transition visible at once

4.6. visitor as a type switch

The visitor pattern's double dispatch exists to compensate for languages that dispatch on one receiver. Go's type switch does the job directly:

The dry run: the fixture is Area on a circle of radius 2 landing 12.566370614359172 with the literal 3.141592653589793, a 3 by 2 rect landing 6, and the describe strings circle r=2.0 and rect 3 x 2. The go lane's frozen test pins its own circle of radius 1 against the same literal pi.

1. The variants are one closed set and the visitors are functions over it: Area and Describe in go, Perimeter a third visitor in the lua file.
2. Adding a visitor costs one function, adding a variant touches every visitor, the trade stated as a count.
3. The unknown variant answers zero and unknown instead of guessing.

```c
typedef enum { SHAPE_CIRCLE, SHAPE_RECT } shape_tag;

typedef struct {
  shape_tag tag;
  union {
    struct {
      double r;
    } circle;
    struct {
      double w, h;
    } rect;
  } as;
} shape;

static shape circle(double r) {
  shape s = {.tag = SHAPE_CIRCLE};
  s.as.circle.r = r;
  return s;
}

static shape rect(double w, double h) {
  shape s = {.tag = SHAPE_RECT};
  s.as.rect.w = w;
  s.as.rect.h = h;
  return s;
}

static double area(const shape *s) {
  switch (s->tag) {
  case SHAPE_CIRCLE:
    return 3.141592653589793 * s->as.circle.r * s->as.circle.r;
  case SHAPE_RECT:
    return s->as.rect.w * s->as.rect.h;
  }
  return 0;
}

static void describe(const shape *s, char *out, int sz) {
  switch (s->tag) {
  case SHAPE_CIRCLE:
    snprintf(out, sz, "circle r=%.1f", s->as.circle.r);
    return;
  case SHAPE_RECT:
    snprintf(out, sz, "rect %g x %g", s->as.rect.w, s->as.rect.h);
    return;
  }
  snprintf(out, sz, "%s", "unknown");
}
```

listing 163: C, a tagged union and a switch, adding a variant adds a case (source: patterns-concurrency-distributed/samples-c/src/Ch04/visitor.c, lines 18-65)

```
enum Shape {
    Circle { r: f64 },
    Rect { w: f64, h: f64 },
}

fn circle(r: f64) -> Shape {
    Shape::Circle { r }
}

fn rect(w: f64, h: f64) -> Shape {
    Shape::Rect { w, h }
}

fn area(s: &Shape) -> f64 {
    match s {
        Shape::Circle { r } => PI * r * r,
        Shape::Rect { w, h } => w * h,
    }
}

fn describe(s: &Shape) -> String {
    match s {
        Shape::Circle { r } => format!("circle r={r:.1}"),
        Shape::Rect { w, h } => format!("rect {w} x {h}"),
    }
}
```

listing 164: Rust, a real enum is the closed set, match arms the visitor, the compiler keeps every match exhaustive (source: patterns-concurrency-distributed/samples-rust/src/Ch04/visitor.rs, lines 15-40)

```
type Shape interface{ isShape() }

type Circle struct{ R float64 }
type Rect struct{ W, H float64 }

func (Circle) isShape() {}
func (Rect) isShape()   {}

func Area(s Shape) float64 {
  switch v := s.(type) {
  case Circle:
    return 3.141592653589793 * v.R * v.R
  case Rect:
    return v.W * v.H
  }
  return 0
}

func Describe(s Shape) string {
  switch v := s.(type) {
  case Circle:
    return fmt.Sprintf("circle r=%.1f", v.R)
  case Rect:
    return fmt.Sprintf("rect %v x %v", v.W, v.H)
  }
  return "unknown"
```

listing 165: Go, variants as plain structs, visitors as functions that switch (source: patterns-concurrency-distributed/samples/ch04/behavioral.go, lines 233-258)

```
sealed interface Shape permits Circle, Rect {}

record Circle(double r) implements Shape {}

record Rect(double w, double h) implements Shape {}

static final double PI = 3.141592653589793;

static double area(Shape s) {
  return switch (s) {
    case Circle c -> PI * c.r() * c.r();
    case Rect r -> r.w() * r.h();
  };
}

// c's %g prints whole doubles bare: 3, not 3.0
static String whole(double v) {
  long iv = (long) v;
  return v == iv ? Long.toString(iv) : Double.toString(v);
}

static String describe(Shape s) {
  return switch (s) {
    case Circle c -> String.format("circle r=%.1f", c.r());
    case Rect r -> "rect " + whole(r.w()) + " x " + whole(r.h());
  };
}
```

listing 166: Java, a sealed interface over record variants, the exhaustive switch enforced at compile time (source: patterns-concurrency-distributed/samples-java/src/Ch04/Visitor.java, lines 19-45)

```
public abstract record Shape;

public sealed record Circle(double R) : Shape;

public sealed record Rect(double W, double H) : Shape;

public static class Shapes
{
    public static double Area(Shape s)
    {
        switch (s)
        {
            case Circle c:
                return 3.141592653589793 * c.R * c.R;
            case Rect r:
                return r.W * r.H;
        }
        return 0;
    }

    public static string Describe(Shape s)
    {
        switch (s)
        {
            case Circle c:
                return "circle r=" + c.R.ToString("F1");
            case Rect r:
                return "rect " + r.W + " x " + r.H;
        }
        return "unknown";
    }
}
```

listing 167: C#, records as the closed set, pattern matching as the switch (source: patterns-concurrency-distributed/samples-cs/src/Ch04/Visitor.cs, lines 5-36)

```
export function circle(r) {
  return { kind: "circle", r };
}

export function rect(w, h) {
  return { kind: "rect", w, h };
}

export function area(s) {
  switch (s.kind) {
    case "circle":
      return 3.141592653589793 * s.r * s.r;
    case "rect":
      return s.w * s.h;
  }
  return 0;
}

export function describeShape(s) {
  switch (s.kind) {
    case "circle":
      return `circle r=${s.r.toFixed(1)}`;
    case "rect":
      return `rect ${s.w} x ${s.h}`;
  }
  return "unknown";
}
```

listing 168: JavaScript, kind-tagged objects, switch on the tag (source: patterns-concurrency-distributed/samples-js/src/ch04-visitor.mjs, lines 5-31)

```
class Circle:
    def __init__(self, r):
        self.r = r


class Rect:
    def __init__(self, w, h):
        self.w = w
        self.h = h


def area(s):
    match s:
        case Circle(r=r):
            return 3.141592653589793 * r * r
        case Rect(w=w, h=h):
            return w * h
    return 0.0


def describe(s):
    match s:
        case Circle(r=r):
            return f"circle r={r:.1f}"
        case Rect(w=w, h=h):
            return f"rect {w} x {h}"
    return "unknown"
```

listing 169: Python, match over class patterns (source: patterns-concurrency-distributed/samples-py/src/Ch04/visitor.py, lines 13-39)

```lua
local function Circle(r)
  return { kind = "circle", r = r }
end

local function Rect(w, h)
  return { kind = "rect", w = w, h = h }
end

local function Area(s)
  if s.kind == "circle" then
    return PI * s.r * s.r
  end
  if s.kind == "rect" then
    return s.w * s.h
  end
  return 0
end

local function Describe(s)
  if s.kind == "circle" then
    return ("circle r=%.1f"):format(s.r)
  end
  if s.kind == "rect" then
    return ("rect %d x %d"):format(s.w, s.h)
  end
  return "unknown"
end
```

listing 170: Lua, tag fields and if chains, a second visitor costs one function (source: patterns-concurrency-distributed/samples-lua/ch04_visitor.lua, lines 8-34)

`Area` and `Describe` are two visitors over the same variant set, each a few lines instead of a visitor interface plus accept methods on every variant. The cost is honest: adding a variant compiles nothing until someone runs it, while adding a method to an interface fails every implementor at build time. The marker method `isShape` at least keeps foreign types out of the union, a closed set by construction. The closed set itself wears a different costume per tree: a tagged union in C, a switch over a tag field in javascript, if chains on a tag in lua, records under pattern matching in C#, and python's `match` binding class patterns directly, the structural side of its dataclass storybook 8, ch 9 (python 3.14, dataclasses, enums, and pattern matching). Java and rust are the two trees that turn the cost into a compile error: java's sealed interface permits exactly `Circle` and `Rect`, the pattern switch over it exhaustive, and rust's `enum Shape` is the closed set by construction, every `match` over it exhaustive, so in both a new variant refuses to build until every visitor grows its case, the rule the C comment can only state.

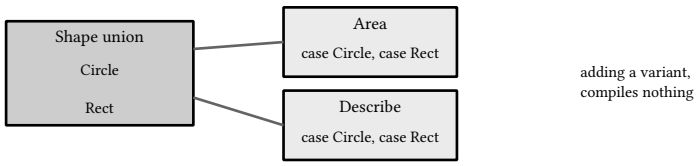


figure 24: one closed union, visitors are switches over it

NOTE: the default is a function

Mediator is dependency injection via parameters, chain of responsibility is a slice of handlers tried in order, template method is a parameter the caller fills, and interpreter is a tree of the composite from chapter 3 with an eval method. When a behavioral pattern feels heavy in go, the missing ingredient is usually a function value, not another interface.

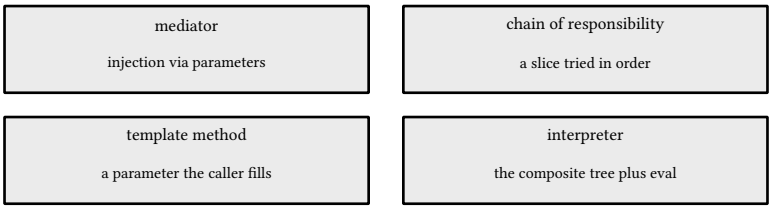


figure 25: four patterns, four shapes of passing a function

4.7. across the eight languages

The build sizes count non-comment source lines, the go column the chapter's one frozen file:

language	build SLOC	dependency	boundary note
c	618	libc plus threads.h	fixed subscriber arrays under one mutex, undo records instead of closures, the iterator a yield callback with a context pointer
rust	678	std only, no crates	static trait-object strategy table, generic Seq adapters over a yield callback, enum-indexed transition table, exhaustive match visitors
go	211	stdlib	frozen reference lane, iter.Seq adapters, SortStableFunc, buffered channel subscribers
java	375	jdk 27 stdlib	Predicate verdicts for push iteration, sealed visitors the compiler keeps exhaustive, enums indexing the transition table
c#	285	bcl	Channels subscribers with TryWrite drops, yield return iterators, record pattern matching visitors
javascript	154	node stdlib	generators whose finally observes the stop, Map insertion order fixing delivery order
python	311	stdlib only	generator close runs finally, match visitors, a tuple keyed transition table
lua	521	lib.lua harness	push sequences as functions over yield callbacks, index tiebreak for sort stability, swapped tables make new machines

sources: go.dev/pkg/iter for `Seq`, `Seq2`, and `Pull` semantics, go.dev/ref/spec range over function, go.dev/pkg/slices for `SortStableFunc` stability, go.dev/pkg/net/http for the middleware as chain of responsibility precedent, accessed 2026-09-08. Verified by the eight chapter legs: 6 Ch04 C programs with 47 embedded checks, 6 Ch04 rust programs in `samples-rust/src` with 49 embedded checks under `run-rust-samples`, `go test -race` at 6 tests in `patternsbook/ch04`, 6 Ch04 java programs with 46 checks under `run-java-samples`, 17 C# facts over `PatternsBook.slnx`, 11 `node --test` cases, 52 Python checks across 6 files, and 28 Lua rows under `run.lua`.

5. functional alternatives

Go is not a functional language and does not try to be: no curried syntax, no monadic chaining, no tail call guarantee. But functions are values, generics arrived in 1.18, and a surprising share of the classic pattern vocabulary reduces to passing and returning functions. The same reduction runs through all eight trees, and the per-language spelling is the chapter's real content: the trio is one generic away in go and C#, std iterator adapters behind plain functions in rust, three plain loops in C, hand-rolled walks over int arrays in java, an array method in javascript, a comprehension in python, a table walk in lua. The chapter builds that toolkit and, just as carefully, marks the line where the functional imitation should stop.

5.1. the trio, generic

The trio is where generics earn their keep, one type parameter saying what six handwritten loops would repeat:

The dry run: the fixture is 1 through 6. Map doubling lands 2 4 6 8 10 12, Filter keeping the evens lands 2 4 6, Reduce summing from 0 lands 21, and all eight lanes assert these exact values. Map of empty stays a real empty container, never nil or null, and Reduce of empty hands back its init, pinned at 9.

1. Map preserves positions, Filter shrinks, Reduce collapses to one value carrying the init through.
2. The empty contracts: map of empty is an allocated zero-length slice in go, a real buffer in C, an empty Vec in rust, a real zero-length array in java, a fresh array in javascript, a list in python, a table in lua.
3. Reduce of empty never calls the fold, so the init comes back untouched, the identity every lane pins at 9.

```c
static int *map_ints(const int *s, int len, int (*f)(int)) {
  // allocate at least one slot so the empty walk stays non-null
  int *out = malloc(sizeof(int) * (size_t)(len ? len : 1));
  if (!out)
    return NULL;
  for (int i = 0; i < len; i++)
    out[i] = f(s[i]);
  return out;
}

static int *filter_ints(const int *s, int len, bool (*keep)(int), int *outlen) {
  int cap = len ? len : 1;
  int *out = malloc(sizeof(int) * (size_t)cap);
  if (!out)
    return NULL;
  int n_out = 0;
  for (int i = 0; i < len; i++)
    if (keep(s[i]))
      out[n_out++] = s[i];
  *outlen = n_out;
  return out;
}

static long reduce_ints(const int *s, int len, long init,
                        long (*f)(long, int)) {
  long acc = init;
  for (int i = 0; i < len; i++)
    acc = f(acc, s[i]);
  return acc;
}
```

listing 171: C, the trio over int arrays, a function pointer per element, map of empty still returns a real buffer (source: patterns-concurrency-distributed/samples-c/src/Ch05/trio.c, lines 20-49)

```rust
fn map_ints(s: &[i32], f: impl Fn(i32) -> i32) -> Vec<i32> {
    s.iter().map(|&v| f(v)).collect()
}

fn filter_ints(s: &[i32], keep: impl Fn(i32) -> bool) -> Vec<i32> {
    s.iter().copied().filter(|&v| keep(v)).collect()
}

fn reduce_ints(s: &[i32], init: i64, f: impl Fn(i64, i32) -> i64) -> i64 {
    s.iter().fold(init, |acc, &v| f(acc, v))
}

fn doubled(v: i32) -> i32 {
    v * 2
}

fn is_even(v: i32) -> bool {
    v % 2 == 0
}

fn add(acc: i64, v: i32) -> i64 {
    acc + i64::from(v)
}
```

listing 172: Rust, the trio as free functions over an int slice, std iterators inside, an empty Vec out of empty (source: patterns-concurrency-distributed/samples-rust/src/Ch05/trio.rs, lines 13-35)

```go
func Map[T, U any](s []T, f func(T) U) []U {
  out := make([]U, len(s))
  for i, v := range s {
    out[i] = f(v)
  }
  return out
}

func Filter[T any](s []T, keep func(T) bool) []T {
  out := []T{}
  for _, v := range s {
    if keep(v) {
      out = append(out, v)
    }
  }
  return out
}

func Reduce[T, U any](s []T, init U, f func(U, T) U) U {
  acc := init
  for _, v := range s {
    acc = f(acc, v)
  }
  return acc
}
```

listing 173: Go, map, filter, reduce over any slice (source: patterns-concurrency-distributed/samples/ch05/functional.go, lines 6-30)

```
  // (long acc, int v) -> long, the reducer's shape
  interface LongIntFolder {
    long fold(long acc, int v);
  }

  static int[] mapInts(int[] src, IntUnaryOperator f) {
    int[] out = new int[src.length];
    for (int i = 0; i < src.length; i++)
      out[i] = f.applyAsInt(src[i]);
    return out;
  }

  static String[] mapStrs(int[] src, java.util.function.IntFunction<String> f) {
    String[] out = new String[src.length];
    for (int i = 0; i < src.length; i++)
      out[i] = f.apply(src[i]);
    return out;
  }

  static int[] filterInts(int[] src, IntPredicate keep) {
    int[] scratch = new int[src.length];
    int n = 0;
    for (int v : src)
      if (keep.test(v))
        scratch[n++] = v;
    int[] out = new int[n];
    System.arraycopy(scratch, 0, out, 0, n);
    return out;
  }

  static long reduceInts(int[] src, long init, LongIntFolder f) {
    long acc = init;
    for (int v : src)
      acc = f.fold(acc, v);
    return acc;
  }
```

listing 174: Java, the trio hand-rolled over int arrays, the shapes borrowed from java.util.function (source: patterns-concurrency-distributed/samples-java/src/Ch05/Trio.java, lines 20-55)

```
public static class Trio
{
    public static U[] Map<T, U>(T[] s, Func<T, U> f)
    {
        var outArr = new U[s.Length];
        for (int i = 0; i < s.Length; i++)
        {
            outArr[i] = f(s[i]);
        }
        return outArr;
    }

    public static List<T> Filter<T>(T[] s, Func<T, bool> keep)
    {
        var outList = new List<T>();
        foreach (var v in s)
        {
            if (keep(v))
            {
                outList.Add(v);
            }
        }
        return outList;
    }

    public static U Reduce<T, U>(T[] s, U init, Func<U, T, U> f)
    {
        U acc = init;
        foreach (var v in s)
        {
            acc = f(acc, v);
        }
        return acc;
    }
```

listing 175: C#, generics say it once, the same shape go writes (source: patterns-concurrency-distributed/samples-cs/src/Ch05/Trio.cs, lines 4-37)

```
export function map(seq, f) {
  const out = new Array(seq.length);
  for (let i = 0; i < seq.length; i++) out[i] = f(seq[i]);
  return out;
}

export function filter(seq, keep) {
  const out = [];
  for (const v of seq) {
    if (keep(v)) out.push(v);
  }
  return out;
}

export function reduce(seq, init, f) {
  let acc = init;
  for (const v of seq) acc = f(acc, v);
  return acc;
}
```

listing 176: JavaScript, the standalone functions beside the array methods (source: patterns-concurrency-distributed/samples-js/src/ch05-trio.mjs, lines 6-24)

```
def map_over(seq, f):
    return [f(v) for v in seq]


def filter_by(seq, keep):
    return [v for v in seq if keep(v)]


def reduce_with(seq, init, f):
    acc = init
    for v in seq:
        acc = f(acc, v)
    return acc
```

listing 177: Python, comprehensions are the native map and filter, the fold is a plain loop (source: patterns-concurrency-distributed/samples-py/src/Ch05/trio.py, lines 13-25)

```lua
local function Map(s, f)
  local out = {}
  for i, v in ipairs(s) do
    out[i] = f(v)
  end
  return out
end

local function Filter(s, keep)
  local out = {}
  for _, v in ipairs(s) do
    if keep(v) then
      out[#out + 1] = v
    end
  end
  return out
end

local function Reduce(s, init, f)
  local acc = init
  for _, v in ipairs(s) do
    acc = f(acc, v)
  end
  return acc
end
```

listing 178: Lua, the trio as table walks, empty in keeps a table out (source: patterns-concurrency-distributed/samples-lua/ch05_trio.lua, lines 6-30)

These are one to three lines each and read fine, which is why the stdlib never grew them: `Map` and `Filter` are usually clearer inlined as a range loop, and the language even gave `for i := range n` the integer form in 1.22 to trim the most common loop of all. The moment composition, not iteration, is the point, chapter 4's `iter.Seq` adapters are the better substrate because they fuse instead of allocating intermediate slices. The sibling trees land where their grain puts them: go and C# write the generic shapebook 4, ch 4 (go 1.27, generics and generic methods) once and reuse it for every element type, rust's stdlib is already the trio, `iter().map().collect()`, so its sample writes the three as plain functions over `&[i32]` to pin the contracts the adapters assume, C specializes to int arrays with function pointers because its generics are macros and the honest sample is just loops, java hand-rolls `mapInts`, `filterInts`, and `reduceInts` over `int[]`, borrowing `IntPredicate` and `IntUnaryOperator` from `java.util.function`, javascript arrays already carry `map`, `filter`, and `reduce` as methods so the standalone functions exist to pin the contracts, python's comprehensions are the native trio with `functools.reduce` the fold nobody imports, and lua walks tables because that is all a lua sequence is.



figure 26: map keeps the shape, filter shrinks it, reduce collapses to one

5.2. composition and partial application

`Pipe(f, g)` applies f then g, reading order matching writing order. `AddTo` fixes one argument at application time. `Counter` is the essential closure demonstration: `n` is captured by reference, unreachable from outside, and two counters from the same constructor share nothing:

The dry run: a pipe of trim then upper answers ABC for the padded input, the empty pipe answers with its input untouched, and AddTo(10) answers 15 at 5 and 0 at −10. Two counters built from 0 answer 1 and 1, then 2. The go lane's frozen test walks its own counter literals, one counter climbing 1 2 3 4 beside a second from 100 reading 101 and 102, both verified against the same implementation.

1. The empty pipe is the identity, and its return type is the input type, which is what makes it a lawful default.
2. Partial application closes over the base, not the call site, so the returned function carries its configuration.
3. State with an audience of zero is what closures sell and what chapter 7's mutexes are for when the audience grows.

```c
typedef struct {
  int (*fn)(void *state, int x);
  void *state;
} fn;

typedef struct {
  fn steps[MAXPIPE];
  int n;
} pipe;

// pipe(f, g)(x) = g(f(x)): apply in list order
static int pipe_apply(const pipe *p, int x) {
  for (int i = 0; i < p->n; i++)
    x = p->steps[i].fn(p->steps[i].state, x);
  return x;
}

static int inc(void *state, int x) {
  (void)state;
  return x + 1;
}

static int double_it(void *state, int x) {
  (void)state;
  return x * 2;
}

static int add_base(void *state, int x) { return *(int *)state + x; }

// partial application: fix base, hand back the rest of the call
static fn add_to(int *base) { return (fn){add_base, base}; }

// the counter closure: its n lives in the state struct, two counters
// from the same constructor share nothing
typedef struct {
  int n;
} counter_state;

static int counter_next(void *state, int x) {
  counter_state *c = state;
  (void)x;
  c->n++;
  return c->n;
}

static fn counter(counter_state *st) {
  st->n = 0;
  return (fn){counter_next, st};
}
```

listing 179: C, a closure is a struct plus a function pointer, the pipe a list of them (source: patterns-concurrency-distributed/samples-c/src/Ch05/compose.c, lines 21-69)

```rust
struct Pipe {
    steps: Vec<Box<dyn Fn(i32) -> i32>>,
}

impl Pipe {
    fn new() -> Pipe {
        Pipe { steps: Vec::new() }
    }

    fn then(mut self, f: Box<dyn Fn(i32) -> i32>) -> Pipe {
        self.steps.push(f);
        self
    }

    fn apply(&self, x: i32) -> i32 {
        self.steps.iter().fold(x, |v, f| f(v))
    }
}

fn inc(x: i32) -> i32 {
    x + 1
}

fn double_it(x: i32) -> i32 {
    x * 2
}

fn add_to(base: i32) -> impl Fn(i32) -> i32 {
    move |x| base + x
}

fn counter() -> impl FnMut(i32) -> i32 {
    let mut n = 0;
    move |_| {
        n += 1;
        n
    }
}
```

listing 180: Rust, the pipe a Vec of boxed steps, the counter a cell moved into its closure alone (source: patterns-concurrency-distributed/samples-rust/src/Ch05/compose.rs, lines 13-50)

```
func Pipe[T any](fns ...func(T) T) func(T) T {
  return func(in T) T {
    for _, f := range fns {
      in = f(in)
    }
    return in
  }
}

// AddTo is partial application: fix one argument, return the rest.
func AddTo(base int) func(int) int {
  return func(n int) int { return base + n }
}

// Counter is a closure owning mutable state nobody else can reach.
func Counter(start int) func() int {
  n := start
  return func() int {
    n++
    return n
  }
}

// Captured keeps one handler per loop iteration to demonstrate the
```

listing 181: Go, pipe left to right, partial application, a closure with private state (source: patterns-concurrency-distributed/samples/ch05/functional.go, lines 33-56)

```java
// pipe(f, g)(x) = g(f(x)): apply in list order
static Function<String, String> pipe(List<Function<String, String>> fns) {
  return x -> {
    String v = x;
    for (Function<String, String> f : fns)
      v = f.apply(v);
    return v;
  };
}

static String trim(String s) {
  return s.replace(" ", "");
}

static String upper(String s) {
  return s.toUpperCase();
}

// partial application: fix base, hand back the rest of the call
static Function<Integer, Integer> addTo(int base) {
  return n -> base + n;
}

// the counter closure: its n lives in the array, two counters from
// the same constructor share nothing
static Supplier<Integer> counter(int start) {
  int[] count = {start};
  return () -> ++count[0];
}
```

listing 182: Java, Function composition, the counter's cell a one-element array because lambdas capture values only (source: patterns-concurrency-distributed/samples-java/src/Ch05/Compose.java, lines 22-50)

```
// Pipe composes unary functions left to right: Pipe(f, g)(x) = g(f(x)).
// AddTo is partial application: fix one argument, return the rest.
// Counter is a closure owning mutable state nobody else can reach.
public static class Compose
{
    public static Func<T, T> Pipe<T>(params Func<T, T>[] fns)
    {
        return input =>
        {
            var current = input;
            foreach (var f in fns)
            {
                current = f(current);
            }
            return current;
        };
    }

    public static Func<int, int> AddTo(int basement) => n => basement + n;

    public static Func<int> Counter(int start)
    {
        int n = start;
        return () => ++n;
    }
```

listing 183: C#, lambdas and a params array, the direct mirror (source: patterns-concurrency-distributed/samples-cs/src/Ch05/Compose.cs, lines 3-27)

```
export function pipe(...fns) {
  return (input) => {
    let v = input;
    for (const f of fns) v = f(v);
    return v;
  };
}

export function addTo(base) {
  return (n) => base + n;
}

export function counter(start) {
  let n = start;
  return () => {
    n++;
    return n;
  };
}
```

listing 184: JavaScript, rest arguments fold into arrows (source: patterns-concurrency-distributed/samples-js/src/ch05-compose.mjs, lines 6-24)

```
def add_to(base):
    def add(n):
        return base + n

    return add


add10 = add_to(10)
check(add10(5) == 15, "partial application fixes the base")
check(add10(-10) == 0, "and closes over it, not the call site")
check(add_to(100)(1) == 101, "each application is its own closure")


def counter(start):
    count = start

    def bump():
        nonlocal count
        count += 1
        return count

    return bump


a = counter(0)
b = counter(0)
check(a() == 1 and b() == 1, "two counters each return 1 first")
check(a() == 2, "the second bump of a is 2")
check(b() == 2 and a() == 3, "the counters share nothing")
```

listing 185: Python, nested defs, nonlocal for the writable cell, the checks pin 1, 1 then 2 (source: patterns-concurrency-distributed/samples-py/src/Ch05/compose.py, lines 41-69)

```lua
local function Pipe(...)
  local fns = { ... }
  return function(x)
    for _, f in ipairs(fns) do
      x = f(x)
    end
    return x
  end
end

local function AddTo(base)
  return function(n)
    return base + n
  end
end

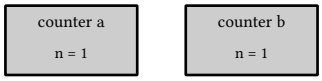
local function Counter(start)
  local n = start
  return function()
    n = n + 1
    return n
  end
end
```

listing 186: Lua, closures over upvalues, the same three shapes (source: patterns-concurrency-distributed/samples-lua/ch05_compose.lua, lines 6-28)

C spells the closure out loud, a function pointer plus a state pointer, because it has nothing to hide it behind, and the counter closure becomes a `counter_state` struct the caller allocates. The other seven get the closure for free, C# lambdas, javascript arrows, python nested `def`s where `nonlocal` marks the writable cell, lua upvalues, rust moving the counter's cell into the closure so two counters share nothing because nothing is reachable, with java one step aside: its lambdas capture values, not variables, so the counter's writable cell is a one-element array each constructor call allocates. The pipe itself is the same fold everywhere, apply the list in order and return the last output, and the empty list folds to the identity by construction.



pipe(f, g), the empty pipe is the identity



n is captured by reference and unreachable, the two share nothing

figure 27: pipe reads left to right, a closure is a private cell

5.3. the loop variable, fixed

Go 1.22 changed the oldest closure trap in the language: loop variables are now created per iteration instead of shared. Before, every closure capturing `i` in a loop observed the final value, after, each observes its own:

The dry run: the fixture is 5 handlers, and the fixed lane pins handler k returning k in all eight trees. The trap lanes are per-language truth: javascript's `var` lane returns 5 for every handler, python's late-binding lambdas return 4 for every handler, lua's deliberately shared local behind a while loop returns 6, C's shared-pointer lane returns 4, rust's shared `Rc<Cell>` lane answers 4 because a move capture cannot dangle the binding at all, and java's smuggled static field answers 4 five times because capturing the reassigned local does not compile at all. The go lane's frozen test pins the fixed lane only.

1. `let` in javascript and the numeric for in lua create a fresh binding per iteration, the 1.22 semantics native and free.
2. `var` in javascript is one binding for the whole loop, the pre-1.22 semantics still alive whenever someone types it.
3. Python closures bind names, not values, so the default argument `i=i` is the hand-rolled per-iteration copy, comprehensions included.

```c
typedef struct {
  int (*call)(void *state);
  void *state;
} handler;

// per-iteration capture: each slot stores its own copy of i
static int return_state(void *state) { return *(const int *)state; }

// the trap lane: every handler reads the one shared variable
typedef struct {
  int *shared;
} shared_state;

static int return_shared(void *state) {
  shared_state *s = state;
  return *s->shared;
}
```

listing 187: C, no closures, the per-slot copy beside the shared-pointer trap, both written out (source: patterns-concurrency-distributed/samples-c/src/Ch05/loopvar.c, lines 20-36)

```rust
fn main() {
    let mut handlers: Vec<Box<dyn Fn() -> i32>> = Vec::new();
    for i in 0..5 {
        let own = i;
        handlers.push(Box::new(move || own));
    }
    ok(
        handlers
            .iter()
            .enumerate()
            .all(|(k, h)| h() == k as i32),
        "a move capture gives each handler its own iteration value",
    );
    ok(handlers[0]() == 0, "slot zero kept the first iteration");
    ok(handlers[4]() == 4, "slot four kept the last iteration");

    let shared = Rc::new(Cell::new(0));
    let mut trapped: Vec<Box<dyn Fn() -> i32>> = Vec::new();
    for i in 0..5 {
        shared.set(i);
        let cell = Rc::clone(&shared);
        trapped.push(Box::new(move || cell.get()));
    }
    ok(
        trapped.iter().all(|h| h() == 4),
        "closures sharing one cell all answer the final value",
    );
    ok(shared.get() == 4, "the shared cell still holds the last write");
}
```

listing 188: Rust, move capture is per-iteration by default, the trap rebuilt as one shared Rc<Cell> (source: patterns-concurrency-distributed/samples-rust/src/Ch05/loopvar.rs, lines 16-44)

```go
func Captured(n int) []func() int {
  handlers := make([]func() int, n)
  for i := 0; i < n; i++ {
    handlers[i] = func() int { return i }
  }
  return handlers
}
```

listing 189: Go, each handler keeps the i of its own iteration (source: patterns-concurrency-distributed/samples/ch05/functional.go, lines 59-66)

```java
// the trap lane's shared cell: fields are mutable and every lambda
// reads the same one at call time, unlike locals
static int loopVar;

public static void main(String[] args) {
  // per-iteration capture: capturing i itself does not compile
  // because the loop reassigns it, so the effectively-final rule
  // forces a fresh copy per iteration, the hand-rolled form of go
  // 1.22's loop variable rule
  List<IntSupplier> own = new ArrayList<>();
  for (int i = 0; i < N; i++) {
    int copy = i;
    own.add(() -> copy);
  }
  List<Integer> got = own.stream().map(IntSupplier::getAsInt).toList();
  ok(got.equals(List.of(0, 1, 2, 3, 4)), "each iteration captures its own i");

  // the shared-cell bug, reproduced on purpose: every supplier reads
  // the field after the loop, so all answer 4. this is the pre-1.22
  // go behavior and the python late-binding lane
  List<IntSupplier> late = new ArrayList<>();
  for (int i = 0; i < N; i++) {
    loopVar = i;
    late.add(() -> loopVar);
  }
  List<Integer> lateGot = late.stream().map(IntSupplier::getAsInt).toList();
  ok(lateGot.equals(List.of(4, 4, 4, 4, 4)), "one shared cell answers 4 five times");
```

listing 190: Java, capturing the reassigned i does not compile, the copy is forced, the trap smuggled through a field (source: patterns-concurrency-distributed/samples-java/src/Ch05/Loopvar.java, lines 25-51)

```
// Captured keeps one handler per loop iteration to demonstrate the
// capture rules. C# foreach has been per-iteration since C# 5, and a
// copied local fixes the classic for-loop capture: each closure sees
// the i of its own iteration. CapturedShared is the unfixed for loop:
// one variable shared by every iteration, so every closure sees the
// final value.
public static class Loopvar
{
    public static Func<int>[] Captured(int n)
    {
        var handlers = new Func<int>[n];
        for (int i = 0; i < n; i++)
        {
            int iteration = i;
            handlers[i] = () => iteration;
        }
        return handlers;
    }

    public static Func<int>[] CapturedShared(int n)
    {
        var handlers = new Func<int>[n];
        for (int i = 0; i < n; i++)
        {
            handlers[i] = () => i;
        }
        return handlers;
    }
```

listing 191: C#, foreach per-iteration since C# 5, a copied local fixing the for loop (source: patterns-concurrency-distributed/samples-cs/src/Ch05/Loopvar.cs, lines 3-30)

```
export function capturedLet(n) {
  const handlers = [];
  for (let i = 0; i < n; i++) {
    handlers.push(() => i);
  }
  return handlers;
}

export function capturedVar(n) {
  const handlers = [];
  for (var i = 0; i < n; i++) {
    handlers.push(() => i);
  }
  return handlers;
}
```

listing 192: JavaScript, let versus var, the identical history in one file (source: patterns-concurrency-distributed/samples-js/src/ch05-loopvar.mjs, lines 7-21)

```python
def captured_late(count):
    # one shared cell: every lambda reads the same i at call time
    handlers = []
    for i in range(count):
        handlers.append(lambda: i)
    return handlers


def captured_fixed(count):
    # default arguments evaluate at definition time, one cell per
    # iteration
    handlers = []
    for i in range(count):
        handlers.append(lambda i=i: i)
    return handlers
```

listing 193: Python, late binding and the default-argument fix (source: patterns-concurrency-distributed/samples-py/src/Ch05/loopvar.py, lines 15-29)

```lua
local function Captured(n)
  local handlers = {}
  for i = 1, n do
    handlers[i] = function() return i end
  end
  return handlers
end

-- the trap, built on purpose: one local mutated by the loop
local function SharedCaptured(n)
  local handlers = {}
  local i = 1
  while i ≤ n do
    handlers[i] = function() return i end
    i = i + 1
  end
  return handlers
end

-- the fix for the trap: copy the shared local per iteration
local function FixedCaptured(n)
  local handlers = {}
  local i = 1
  while i ≤ n do
    local k = i
    handlers[i] = function() return k end
    i = i + 1
  end
  return handlers
end
```

listing 194: Lua, the numeric for is per-iteration, the trap rebuilt with a while loop (source: patterns-concurrency-distributed/samples-lua/ch05_loopvar.lua, lines 8-37)

The test is the proof: five handlers, handler k returns k, where pre-1.22 semantics would have returned 5 for all of them. The semantics are gated by the module's `go` directive, so this module's `go 1.27` line is what buys the fix. Code that must be correct under both regimes still writes `i := i` at the loop head, and reviewers still flag naked captures in old modules. The sibling trees turn the trap into a museum: javascript's `var` lane is go's history word for wordbook 7, ch 3 (javascript es2026 and typescript 7, functions, scope, and closures), python's lambda list is

the same trap through late binding with the default argument as the museum's exit, lua's numeric for never had the bug so the sample builds it on purpose with a while loop over one local, C stores a copy per slot because a pointer to the loop variable dies with the iteration, rust sits beside java at the compile-error exit from the other side, a closure must own or borrow what it captures so `move` gives the per-iteration copy and the trap has to be rebuilt as an `Rc<Cell>` all five handlers share, java makes the bug a compile error, a lambda cannot capture a local the loop reassigns, so the effectively-final copy is the sibling's per-slot fix made mandatory and the trap lane has to smuggle its cell through a static field, and C# fixed `foreach` captures in C# 5 while the raw `for` loop still shares, so the sample shows the copied local beside it.

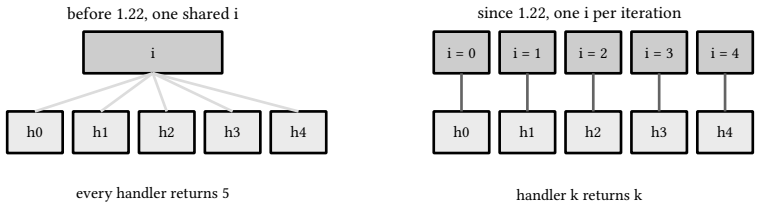


figure 28: before 1.22 one shared i, since then one i per iteration

5.4. memoization

A memoized function is a decorator that trades memory for time, and it generalizes cleanly:

The dry run: the wrapper holds a squaring function, 9 answers 81 twice and 12 answers 144 with the underlying function running exactly 2 times. Recursive fib lands fib(5) = 5 at exactly 9 calls, fib(10) = 55 at 19, and fib(40) = 102334155 at 79, the 2n-1 spine the seven new trees pin exactly. The go lane's frozen test pins the fib(40) value, bounds the calls under 2n, and checks the count is deterministic across instances, the 9 and 19 counts are the seven new trees' lane.

1. The wrapper guards only the map, the wrapped call runs outside the lock, and last write wins because the function is pure.
2. A memoized fib from an empty cache costs exactly 2n-1 calls, the spine down to the base cases plus one hit per level.
3. Concurrent callers hammer one wrapper and every answer is the function's, 50 goroutines in go, 4 threads in C, 4 scoped threads in rust, 20 virtual threads in java, 20 threads in python, 50 coroutines in lua.

```c
typedef struct {
  mtx_t mu;
  long cache[DOMAIN];
  bool have[DOMAIN];
  long (*f)(int);
} memo;

static void memo_init(memo *m, long (*f)(int)) {
  mtx_init(&m->mu, mtx_plain);
  for (int i = 0; i < DOMAIN; i++)
    m->have[i] = false;
  m->f = f;
}

static long memo_at(memo *m, int k) {
  mtx_lock(&m->mu);
  if (k ≥ 0 && k < DOMAIN && m->have[k]) {
    long v = m->cache[k];
    mtx_unlock(&m->mu);
    return v;
  }
  mtx_unlock(&m->mu);
  long v = m->f(k);
  mtx_lock(&m->mu);
  if (k ≥ 0 && k < DOMAIN) {
    m->cache[k] = v;
    m->have[k] = true;
  }
  mtx_unlock(&m->mu);
  return v;
}
```

listing 195: C, fixed-array cache behind one mutex, the call runs outside the lock (source: patterns-concurrency-distributed/samples-c/src/Ch05/memoize.c, lines 23-53)

```rust
struct Memo {
    state: Mutex<Vec<Option<i64>>>,
    f: fn(i32) -> i64,
}

impl Memo {
    fn new(f: fn(i32) -> i64) -> Memo {
        Memo {
            state: Mutex::new(vec![None; DOMAIN]),
            f,
        }
    }

    fn at(&self, k: i32) -> i64 {
        {
            let cache = self.state.lock().unwrap();
            if k ≥ 0 && (k as usize) < DOMAIN {
                if let Some(v) = cache[k as usize] {
                    return v;
                }
            }
        }
        let v = (self.f)(k);
        let mut cache = self.state.lock().unwrap();
        if k ≥ 0 && (k as usize) < DOMAIN {
            cache[k as usize] = Some(v);
        }
        v
    }
}
```

listing 196: Rust, the cache a Mutex over a Vec of Options, the wrapped call runs between the two holds (source: patterns-concurrency-distributed/samples-rust/src/Ch05/memoize.rs, lines 19-48)

```go
func Memoize[T comparable, U any](f func(T) U) func(T) U {
  var mu sync.Mutex
  cache := map[T]U{}
  return func(k T) U {
    mu.Lock()
    v, ok := cache[k]
    mu.Unlock()
    if ok {
      return v
    }
    v = f(k)
    mu.Lock()
    cache[k] = v
    mu.Unlock()
    return v
  }
}
```

listing 197: Go, generic memoize, lock guarded, last write wins (source: patterns-concurrency-distributed/samples/ch05/functional.go, lines 68-85)

```
  static final class Memo {
    private final Object lock = new Object();
    private final Map<Integer, Long> cache = new HashMap<>();
    private final IntToLongFunction f;

    Memo(IntToLongFunction f) {
      this.f = f;
    }

    long at(int k) {
      synchronized (lock) {
        Long hit = cache.get(k);
        if (hit != null)
          return hit;
      }
      long v = f.applyAsLong(k);
      synchronized (lock) {
        cache.put(k, v);
      }
      return v;
    }
  }
```

listing 198: Java, the synchronized cache, the wrapped call runs outside the lock (source: patterns-concurrency-distributed/samples-java/src/Ch05/Memoize.java, lines 25-46)

```
  public static Func<T, U> Wrap<T, U>(Func<T, U> f) where T : notnull
  {
      var gate = new object();
      var cache = new Dictionary<T, U>();
      return k =>
      {
          U v;
          lock (gate)
          {
              if (cache.TryGetValue(k, out v!))
              {
                  return v;
              }
          }
          v = f(k);
          lock (gate)
          {
              cache[k] = v;
          }
          return v;
      };
  }
```

listing 199: C#, the same shape, lock over a Dictionary (source: patterns-concurrency-distributed/samples-cs/src/Ch05/Memoize.cs, lines 6-27)

```javascript
export function memoize(fn) {
  const cache = new Map();
  return (k) => {
    if (cache.has(k)) return cache.get(k);
    const v = fn(k);
    cache.set(k, v);
    return v;
  };
}
```

listing 200: JavaScript, a Map memo closure, no lock under one realm (source: patterns-concurrency-distributed/samples-js/src/ch05-memoize.mjs, lines 7-15)

```python
def memoize(f):
    lock = threading.Lock()
    cache = {}

    def fast(k):
        with lock:
            if k in cache:
                return cache[k]
        v = f(k)
        with lock:
            cache[k] = v
        return v

    return fast
```

listing 201: Python, a lock-guarded dict closure, the counted lane the stdlib decorator cannot express (source: patterns-concurrency-distributed/samples-py/src/Ch05/memoize.py, lines 17-30)

```lua
-- cache lookup, compute, store: no yield anywhere in the section, so
-- however the callers interleave, each key is computed once
local function Memoize(f)
  local cache = {}
  return function(k)
    local v = cache[k]
    if v ~= nil then
      return v
    end
    v = f(k)
    cache[k] = v
    return v
  end
end
```

listing 202: Lua, the memo section never yields, so each key computes exactly once (source: patterns-concurrency-distributed/samples-lua/ch05_memoize.lua, lines 31-44)

The lock is coarse by design: it protects the map, not the wrapped call, so concurrent misses may both compute, last write wins, and correctness holds because the function is pure. The test hammers it from 50 goroutines under `-race`, and rust's `Memo` splits the same seam, the lookup and the store each take the lock while the wrapped call runs between the two holds, four scoped threads hammering it for every answer `plus_one`. The dynamic lanes state their own boundary: javascript needs no lock because one realm runs one thread, python's lock is real because the GIL switches threads between bytecodes, and lua's cooperative scheduler cannot interrupt the check, compute, store section at all, which is why its concurrent lane also pins exactly 100 underlying calls for 100 keys, a determinism the preemptive lanes cannot promise.

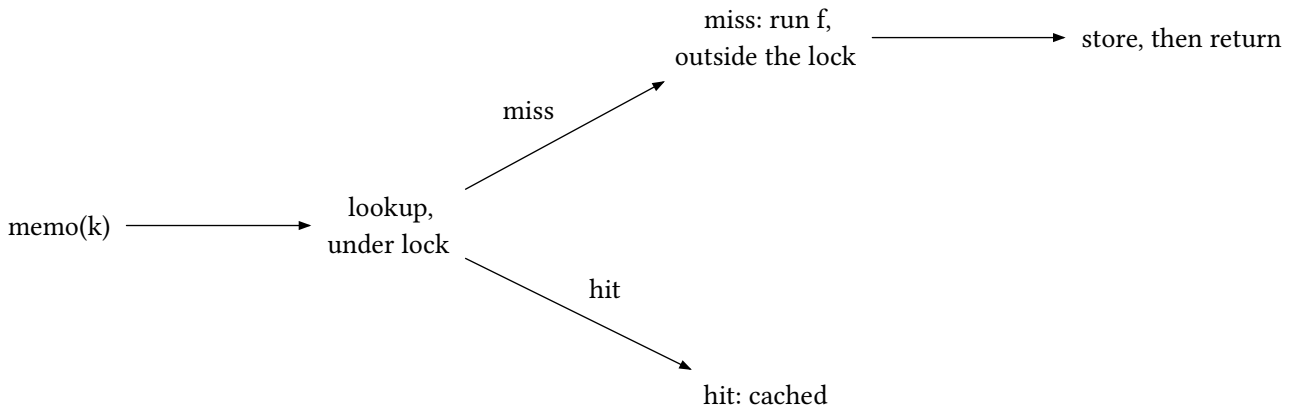


figure 29: hit or miss, the wrapped call always runs outside the lock

Recursive memoization exposes a real limit of closures: a function cannot call its own memoized name, only the raw one, and raw recursion skips the cache. Every tree reaches the same answer, a named holder whose fields are the cache and the call counter, rust fusing both into one locked tuple so the count and the cache move together:

```c
// fib the honest way: named struct, cache and call counter inside, calls
// counted at entry including the hits
typedef struct {
  mtx_t mu;
  long cache[48];
  bool have[48];
  int calls;
} fib;

static void fib_init(fib *f) {
  mtx_init(&f->mu, mtx_plain);
  for (int i = 0; i < 48; i++)
    f->have[i] = false;
  f->cache[0] = 0;
  f->have[0] = true;
  f->cache[1] = 1;
  f->have[1] = true;
  f->calls = 0;
}

static long fib_at(fib *f, int x) {
  mtx_lock(&f->mu);
  f->calls++;
  if (f->have[x]) {
    long v = f->cache[x];
    mtx_unlock(&f->mu);
    return v;
  }
  mtx_unlock(&f->mu);
  long v = fib_at(f, x - 1) + fib_at(f, x - 2);
  mtx_lock(&f->mu);
  f->cache[x] = v;
  f->have[x] = true;
  mtx_unlock(&f->mu);
  return v;
}
```

listing 203: C, the named struct holds the cache array and the call counter (source: patterns-concurrency-distributed/samples-c/src/Ch05/memoize.c, lines 73-108)

```rust
struct Fib {
    state: Mutex<(Vec<Option<i64>>, i32)>,
}

impl Fib {
    fn new() -> Fib {
        let mut cache = vec![None; FIB_MAX];
        cache[0] = Some(0);
        cache[1] = Some(1);
        Fib {
            state: Mutex::new((cache, 0)),
        }
    }

    fn at(&self, x: usize) -> i64 {
        let hit = {
            let mut g = self.state.lock().unwrap();
            g.1 += 1;
            g.0[x]
        };
        if let Some(v) = hit {
            return v;
        }
        let v = self.at(x - 1) + self.at(x - 2);
        self.state.lock().unwrap().0[x] = Some(v);
        v
    }

    fn calls(&self) -> i32 {
        self.state.lock().unwrap().1
    }
}
```

listing 204: Rust, the named Fib owns cache and counter in one locked tuple, the lock dropped before each recursion (source: patterns-concurrency-distributed/samples-rust/src/Ch05/memoize.rs, lines 50-81)

```
type Fib struct {
  mu    sync.Mutex
  cache map[int]int
  calls int
}

func NewFib() *Fib { return &Fib{cache: map[int]int{0: 0, 1: 1}} }

func (f *Fib) Calls() int {
  f.mu.Lock()
  defer f.mu.Unlock()
  return f.calls
}

func (f *Fib) At(n int) int {
  f.mu.Lock()
  f.calls++
  if v, ok := f.cache[n]; ok {
    f.mu.Unlock()
    return v
  }
  f.mu.Unlock()
  v := f.At(n-1) + f.At(n-2)
  f.mu.Lock()
  f.cache[n] = v
  f.mu.Unlock()
  return v
```

listing 205: Go, fib as a type with its own cache, calls counted (source: patterns-concurrency-distributed/samples/ch05/functional.go, lines 90-116)

```
  static final class Fib {
    private final Object lock = new Object();
    private final long[] cache = new long[48];
    private final boolean[] have = new boolean[48];
    private int calls;

    Fib() {
      cache[0] = 0;
      have[0] = true;
      cache[1] = 1;
      have[1] = true;
    }

    long at(int x) {
      synchronized (lock) {
        calls++;
        if (have[x])
          return cache[x];
      }
      long v = at(x - 1) + at(x - 2);
      synchronized (lock) {
        cache[x] = v;
        have[x] = true;
      }
      return v;
    }

    int calls() {
      synchronized (lock) {
        return calls;
      }
    }
  }
```

listing 206: Java, fib a named class, cache array and call counter inside, counted under the lock (source: patterns-concurrency-distributed/samples-java/src/Ch05/Memoize.java, lines 50-82)

```
// Fib is recursive memoization the honest way: a class with the cache
// as a field, because a closure cannot reference itself through the
// memoized name, only the raw one, and raw recursion defeats the cache.
public sealed class Fib
{
    private readonly object _gate = new();
    private readonly Dictionary<int, int> _cache = new() { [0] = 0, [1] = 1 };
    private int _calls;

    public int Calls()
    {
        lock (_gate)
        {
            return _calls;
        }
    }

    public int At(int n)
    {
        int v;
        lock (_gate)
        {
            _calls++;
            if (_cache.TryGetValue(n, out v))
            {
                return v;
            }
        }
        v = At(n - 1) + At(n - 2);
        lock (_gate)
        {
            _cache[n] = v;
        }
        return v;
    }
```

listing 207: C#, the Fib class, cache and counter as fields (source: patterns-concurrency-distributed/samples-cs/src/Ch05/Memoize.cs, lines 30-64)

```
export class Fib {
  #cache = new Map([
    [0, 0],
    [1, 1],
  ]);
  #calls = 0;

  calls() {
    return this.#calls;
  }

  at(n) {
    this.#calls++;
    if (this.#cache.has(n)) return this.#cache.get(n);
    const v = this.at(n - 1) + this.at(n - 2);
    this.#cache.set(n, v);
    return v;
  }
```

listing 208: JavaScript, private fields for the cache and the call count (source: patterns-concurrency-distributed/samples-js/src/ch05-memoize.mjs, lines 17-34)

```
class Fib:
    def __init__(self):
        self.lock = threading.Lock()
        self.cache = {0: 0, 1: 1}
        self.call_count = 0

    def at(self, k):
        with self.lock:
            self.call_count += 1
            if k in self.cache:
                return self.cache[k]
        v = self.at(k - 1) + self.at(k - 2)
        with self.lock:
            self.cache[k] = v
        return v
```

listing 209: Python, lock, cache, and counter as attributes (source: patterns-concurrency-distributed/samples-py/src/Ch05/memoize.py, lines 49-63)

```lua
-- the named holder: the cache is a field, At recurses through the
-- holder so every recursive call passes the cache
local function NewFib()
  local f = { cache = { [0] = 0, [1] = 1 }, calls = 0 }
  function f:At(n)
    self.calls = self.calls + 1
    local v = self.cache[n]
    if v ~= nil then
      return v
    end
    v = self:At(n - 1) + self:At(n - 2)
    self.cache[n] = v
    return v
  end
  return f
end
```

listing 210: Lua, the holder table, At recurses through it (source: patterns-concurrency-distributed/samples-lua/ch05_memoize.lua, lines 46-61)

`Fib.At` checks the cache first, recurses on miss, stores before returning. The test pins both correctness and economy, `fib(40)` completes in 79 calls where the naive version needs `fib(40)` calls for the value alone, and the call count is deterministic across instances, which is what makes the test a test.

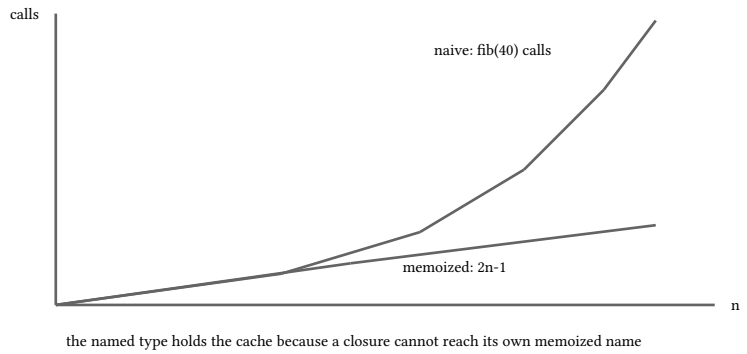


figure 30: naive fib calls explode, the cache holds the line at 2n-1

PITFALL: where to stop

The line this book draws: functions as values, composition, and memoization pay rent. Simulating sum types with interfaces plus `IsLeft` methods, writing `FlatMap` over error, or building lazy evaluator frameworks fights the language for aesthetics. Idiomatic go keeps error handling in `if err ≠ nil` and reaches for a goroutine when evaluation must be deferred, which is the next chapter's subject in full.

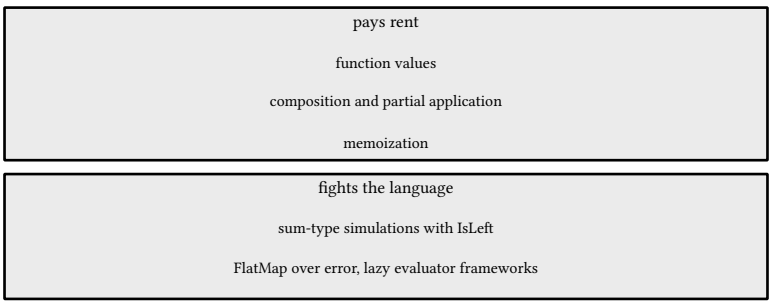


figure 31: where the functional style pays and where it fights

5.5. across the eight languages

The build sizes count non-comment source lines, the go column the chapter's one frozen file:

language	build SLOC	dependency	boundary note
c	327	libc plus threads.h	closures become state structs, the trio specialized to int arrays, fib's counter inside the named struct
rust	267	std only, no crates	move-captured counter cells, the shared `Rc<Cell>` trap rebuilt by hand, mutex-guarded memo with the call outside the lock
go	92	stdlib	frozen reference lane, the generic trio and wrapper, fib's under-2n bound with the 2n-1 exactness landing in the six new trees
java	263	jdk 27 stdlib	one-element arrays as closure cells, the effectively-final copy made mandatory, 20 virtual threads on the memo
c#	136	bcl	lambdas and generics mirror go, foreach fixed since C# 5 with the for loop shown beside
javascript	74	node stdlib	var versus let in one file, Map memo with no lock under one realm, private-field fib
python	167	stdlib only	late binding plus the default-argument fix, nonlocal cells, a real lock over the dict cache
lua	284	lib.lua harness	the numeric for never had the trap, the while-loop rebuild, a scheduler lane pinning exactly-once compute

sources: go.dev/doc/go1.22 for per-iteration loop variables and integer range, go.dev/ref/spec function literals and capture, go.dev/blog/intro-generics for the type parameter syntax, accessed 2026-09-08. Verified by the eight chapter legs: 4 Ch05 C programs with 35 embedded checks, 4 Ch05 rust programs in `samples-rust/src` with 28 embedded checks under `run-rust-samples`, `go test -race` at 7 tests in `patternsbook/ch05`, 4 Ch05 java programs with 30 checks under `run-java-samples`, 15 C# facts over `PatternsBook.slnx`, 10 `node --test` cases, 30 Python checks across 4 files, and 18 Lua rows under `run.lua`.

6. concurrency fundamentals and the memory model

Book 4 toured the syntax of goroutines and channels. This chapter is about what the language actually promises when two goroutines touch the same memory, because the promise is narrower than most programmers assume and everything later in this book, the sync primitives, the hazards chapter, the raft capstone, is built on its edges. The question now runs in eight voices, and the voices disagree about the substrate before they agree about anything else: real operating system threads in C, the same real threads with the borrow checker auditing every share in rust, goroutines in go, virtual threads on the jvm in java, pool tasks in C#, one event loop plus message-passing workers in javascript, threads under the GIL plus asyncio in python, coroutines in lua. The fixtures stay identical anyway, exact totals, exact counts, bounds forced by gates, because those are the claims a memory model can actually keep.

6.1. the model in one rule

A data race is a write to a memory location concurrent with another read or write of the same location, unless every access is atomic through `sync/atomic`. Racy programs are broken programs, not programs with unspecified timing. What separates go from c and c++ is the floor under the wreckage: a racing read must observe some preceding or concurrent write, never an out of thin air value, and the race detector can flag the program, at runtime when a racy interleaving actually executes, never at build time. But races on multiword values, interfaces, slices, maps, strings, can tear pointer and length pairs and corrupt memory, so the floor is not a place to stand.

The guarantee for correct programs is DRF-SC, data race free programs execute as if multiplexed on a single processor, sequentially consistent. The whole engineering discipline of go concurrency is getting into that category and staying there, and the memory model names exactly which constructs are doors in.

Each sibling tree faces its own model instead. C's is the C11 one: plain racy accesses are undefined, and the doors are thread creation, join, mutex and condition variables, and the stdatomic ordering ladder. book 1, ch 15 (c23: the language, the machine, and the cloud, threads and synchronization) puts C23 `<threads.h>` and the win32 layer beneath it side by side. Rust moves the floor earlier, its safe subset rules data races out at compile time, `Send` and `Sync` deciding what may cross a thread boundary, so the lane writes every fixture in this chapter without a single `unsafe`. C#, go, and java promise the same DRF-SC shape over tasks, goroutines, and virtual threads, and java names the same ordering ladder stdatomic.h walks through `VarHandle` access modes. Javascript runs one thread per realm, so its model question is queue ordering, not memory tearing, until `worker_threads` or a `SharedArrayBuffer` enters. Python's GIL keeps bytecode-granular atomicity but switches threads between bytecodes, so compound reads and writes still tear. Lua has no preemption at all, and its lanes demonstrate races by yielding on purpose inside a read-modify-write.

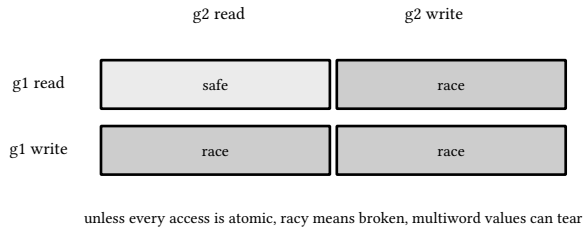


figure 32: one memory cell, which concurrent access pairs race

6.2. happens-before, the doors

Happens before is the transitive closure of program order within a goroutine plus the synchronization edges between them. The edges that matter:

NOTE: the synchronization edges

> `go f()`: the statement is synchronized before the goroutine starts, so data written before `go` is visible inside it. The reverse is false, a goroutine's exit orders nothing. Channel send: a send is synchronized before the completion of the corresponding receive. Unbuffered, the reverse also holds: the receive is synchronized before the send completes, a full rendezvous. Buffered capacity C: the kth receive is synchronized before the k+Cth send completes, which makes a buffered channel a counting semaphore. Close: closing is synchronized before a receive that returns zero because of it. Mutex: for n under m, the nth Unlock is synchronized before the mth Lock returns. Once: the completion of f in once.Do is synchronized before any once.Do returns. Atomics: if B observes A's effect, A is synchronized before B, and all atomics behave as one sequentially consistent order.

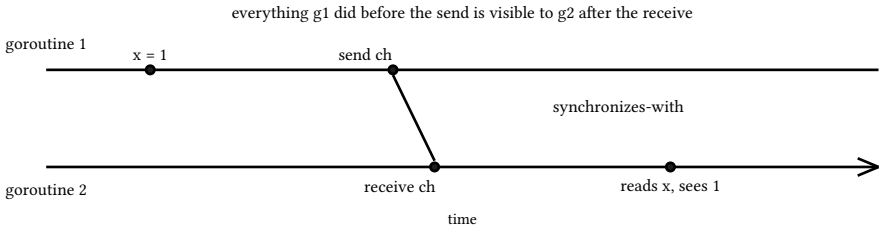


figure 33: one synchronizes-with edge carrying visibility across goroutines

the edge	what it orders
go f()	before the goroutine starts
send	before the matching receive completes
unbuffered receive	before the send completes, the rendezvous
buffered cap C	kth receive before the (k+C)th send
close	before the zero receive it causes
mutex	nth Unlock before the mth Lock
once.Do	f completes before any Do returns
atomics	observing A's effect puts A before B

figure 34: all eight synchronizes-with edges and what each one orders

The go list is go's, and the four sections below walk one fixture through each tree's own doors. The C lane answers with C11: thread creation and `thrd_join` are edges, mutexes and condition variables are edges, and the `stdatomic.h` ordering ladder is the fine-grained one, book 1, ch 16 (c23: the language, the machine, and the cloud, atomics and memory ordering) traces that ladder and the single-producer ring it is asked to justify.

The rust row leans on the same edges C names, a joined `thread::scope` is an edge, `Mutex` and `Condvar` are edges, and the atomics carry the same ladder with `Relaxed` and `SeqCst` named at each call, while the borrow checker keeps plain access off shared cells entirely.

The C# row keeps the channel shape native: `System.Threading.Channels` makes completion an edge and bounded capacity a counting rule, book 6, ch 16 (c# 15, f# 11, and .net 11, stdlib tour 2: io, text, net, channels, diagnostics) is where the BCL surface lives.

The python row splits by substrate: real threads order through `threading` locks and queues, the GIL underneath making each bytecode atomic but nothing more, book 8, ch 17 (python 3.14, threads and the gil) is the locks-and-queues tour this lane rides.

The javascript row has no shared memory to order, one thread per realm, so every edge is a queue boundary instead: an `await` is a park and a resolve is a wake, book 7, ch 6 (javascript es2026 and typescript 7, async javascript) tours the loop every javascript goroutine rides. Lua's rows are cooperative, a yield is the only place an interleaving can happen, which is why its races are scripted, never suffered.

6.3. the wait edge

SumParallel splits the input, sums each part concurrently, and totals the partials. The wait is the whole lesson, the edge that makes every unlocked partial write visible:

The dry run: 1 through 10001 totals 50015001 under all 5 partitionings, parts 1, 2, 7, 100, and 5000, and the empty input totals 0. The chunk ladder pins at 10001, 5001, 1429, 101, and 3, asserted by the seven new trees, the go lane's frozen test pins the totals and the nil case without the ladder.

1. Each partial slot has exactly one writer, and the wait is ordered after every done, itself ordered after the write, so the unlocked writes are safe by construction.
2. The partitioning is ceil-div chunks with trailing empties skipped, every element in exactly one part across all 5 shapes.
3. The wait is a join in every substrate: the zero-count condition broadcast in C and python, the 15-line synchronized waitgroup in java, `WhenAll` in C#, the drained promise in javascript, the last done waking the parked total lane in lua.

```c
typedef struct {
  mtx_t mu;
  cnd_t cv;
  int count;
} waitgroup;

static void wg_init(waitgroup *wg) {
  mtx_init(&wg->mu, mtx_plain);
  cnd_init(&wg->cv);
  wg->count = 0;
}

static void wg_add(waitgroup *wg, int delta) {
  mtx_lock(&wg->mu);
  wg->count += delta;
  if (wg->count == 0)
    cnd_broadcast(&wg->cv);
  mtx_unlock(&wg->mu);
}

static void wg_done(waitgroup *wg) { wg_add(wg, -1); }

static void wg_wait(waitgroup *wg) {
  mtx_lock(&wg->mu);
  while (wg->count != 0)
    cnd_wait(&wg->cv, &wg->mu);
  mtx_unlock(&wg->mu);
}

static void wg_free(waitgroup *wg) {
  mtx_destroy(&wg->mu);
  cnd_destroy(&wg->cv);
}
```

listing 211: C, the waitgroup by hand, one mutex and one condition, the wait loop rechecking the count (source: patterns-concurrency-distributed/samples-c/src/Ch06/waitgroup.c, lines 20-52)

```
struct WaitGroup {
    state: Mutex<i32>,
    cv: Condvar,
}

impl WaitGroup {
    fn new() -> WaitGroup {
        WaitGroup {
            state: Mutex::new(0),
            cv: Condvar::new(),
        }
    }

    fn add(&self, delta: i32) {
        let mut count = self.state.lock().unwrap();
        *count += delta;
        if *count == 0 {
            self.cv.notify_all();
        }
    }

    fn done(&self) {
        self.add(-1);
    }

    fn wait(&self) {
        let mut count = self.state.lock().unwrap();
        while *count != 0 {
            count = self.cv.wait(count).unwrap();
        }
    }
}
```

listing 212: Rust, the waitgroup by hand over a Mutex and one Condvar, wait hands the guard back for the recheck (source: patterns-concurrency-distributed/samples-rust/src/Ch06/waitgroup.rs, lines 15-46)

```
func SumParallel(nums []int, parts int) int {
  if parts < 1 {
    parts = 1
  }
  partials := make([]int, parts)
  var wg sync.WaitGroup
  chunk := (len(nums) + parts - 1) / parts
  for p := 0; p < parts; p++ {
    lo := p * chunk
    hi := lo + chunk
    if hi > len(nums) {
      hi = len(nums)
    }
    if lo ≥ hi {
      continue
    }
    wg.Add(1)
    go func(slot, lo, hi int) {
      defer wg.Done()
      s := 0
      for _, v := range nums[lo:hi] {
        s += v
      }
      partials[slot] = s
    }(p, lo, hi)
  }
  wg.Wait()
  total := 0
  for _, v := range partials {
    total += v
  }
  return total
}

// PingPong bounces a token between two goroutines through
// unbuffered channels n times. Every ping is causally ordered
```

listing 213: Go, waitgroup parallel sum: partial writes become visible at wait (source: patterns-concurrency-distributed/samples/ch06/memorymodel.go, lines 11-46)

```
  static final class WaitGroup {
    private final Object lock = new Object();
    private int count;

    void add(int delta) {
      synchronized (lock) {
        count += delta;
        if (count == 0)
          lock.notifyAll();
      }
    }

    void done() {
      add(-1);
    }

    void waitZero() throws InterruptedException {
      synchronized (lock) {
        while (count != 0)
          lock.wait();
      }
    }
  }

  // chunk ceil(len/parts), trailing empty parts skipped, wait joins the
  // partials into the total
  static long sumParallel(int[] nums, int parts) throws InterruptedException {
    if (parts < 1)
      parts = 1;
    int chunk = (nums.length + parts - 1) / parts;
    WaitGroup wg = new WaitGroup();
    List<long[]> partials = new ArrayList<>();
    for (int lo = 0; lo < nums.length; lo += chunk) {
      int hi = Math.min(lo + chunk, nums.length);
      long[] slot = new long[1];
      partials.add(slot);
      wg.add(1);
      int from = lo, to = hi;
      Thread.ofVirtual().start(() -> {
        long s = 0;
        for (int i = from; i < to; i++)
          s += nums[i];
        slot[0] = s;
        wg.done();
      });
    }
    wg.waitZero();
    long total = 0;
    for (long[] p : partials)
      total += p[0];
    return total;
  }
```

listing 214: Java, the waitgroup hand-rolled on synchronized and notifyAll, CountDownLatch being one-shot only, the parts on virtual threads (source: patterns-concurrency-distributed/samples-java/src/Ch06/Waitgroup.java, lines 23-74)

```
// SumParallel splits an array into parts, sums each concurrently, and
// totals the partials. Task.WhenAll is the happens-before edge that
// makes every partial write visible after the await: the go lane's
// WaitGroup translated to the task world.
public static class Waitgroup
{
    public static async Task<int> SumParallel(int[] nums, int parts)
    {
        if (parts < 1)
        {
            parts = 1;
        }
        (int lo, int hi)[] bounds = PlanChunks(nums.Length, parts);
        var partials = new int[bounds.Length];
        var tasks = new Task[bounds.Length];
        for (int p = 0; p < bounds.Length; p++)
        {
            int slot = p;
            (int lo, int hi) = bounds[slot];
            tasks[slot] = Task.Run(() =>
            {
                int s = 0;
                for (int i = lo; i < hi; i++)
                {
                    s += nums[i];
                }
                partials[slot] = s;
            });
        }
        await Task.WhenAll(tasks);
        int total = 0;
        foreach (var v in partials)
        {
            total += v;
        }
        return total;
    }
```

listing 215: C#, Task.Run per part, Task.WhenAll is the wait edge (source: patterns-concurrency-distributed/samples-cs/src/Ch06/Waitgroup.cs, lines 3-39)

```
export class WaitGroup {
  #pending = 0;
  #waiters = [];

  add(delta = 1) {
    this.#pending += delta;
  }

  done() {
    this.#pending--;
    if (this.#pending === 0) {
      const ws = this.#waiters;
      this.#waiters = [];
      for (const w of ws) w();
    }
  }

  wait() {
    if (this.#pending === 0) return Promise.resolve();
    return new Promise((resolve) => this.#waiters.push(resolve));
  }
}
```

listing 216: JavaScript, the waitgroup as a pending counter that wakes parked promise resolvers (source: patterns-concurrency-distributed/samples-js/src/ch06-waitgroup.mjs, lines 7-28)

```
class WaitGroup:
    def __init__(self):
        self.cond = threading.Condition()
        self.count = 0

    def add(self, k=1):
        with self.cond:
            self.count += k

    def done(self):
        with self.cond:
            self.count -= 1
            if self.count == 0:
                self.cond.notify_all()

    def wait(self):
        with self.cond:
            while self.count > 0:
                self.cond.wait()
```

listing 217: Python, the waitgroup over a Condition, notify_all when the count drains (source: patterns-concurrency-distributed/samples-py/src/Ch06/waitgroup.py, lines 18-36)

```lua
-- the waitgroup: Add before spawn, Done at exit, Wait parks the
-- caller until the count drains, and the last Done is the wake edge
local function SumParallel(S, nums, parts, probe)
  if parts < 1 then
    parts = 1
  end
  local partials = {}
  local chunk = ChunkSize(#nums, parts)
  local doneOrder = {}
  local wg = { n = 0, waiters = {} }
  for p = 0, parts - 1 do
    local lo = p * chunk + 1
    local hi = math.min(lo + chunk - 1, #nums)
    if lo ≤ hi then
      wg.n = wg.n + 1
      spawn(S, function()
        local s = 0
        for i = lo, hi do
          if i == (lo + hi) // 2 then
            coroutine.yield() -- a cooperative step mid sum
          end
          s = s + nums[i]
        end
        partials[p + 1] = s
        doneOrder[#doneOrder + 1] = p + 1
        wg.n = wg.n - 1
        if wg.n == 0 then
          for _, co in ipairs(wg.waiters) do
            S.ready[#S.ready + 1] = co
          end
          wg.waiters = {}
        end
      end)
    end
  end
  spawn(S, function()
    if wg.n > 0 then
      wg.waiters[#wg.waiters + 1] = coroutine.running()
      coroutine.yield("park")
    end
    local total = 0
    for _, v in pairs(partials) do
      total = total + v
    end
    probe.total = total
    probe.partialsAtTotal = #doneOrder
  end)
  return probe
end
```

listing 218: Lua, Add before spawn, Done at exit, the last Done wakes the parked total lane (source: patterns-concurrency-distributed/samples-lua/ch06_waitgroup.lua, lines 36-84)

Each slot has exactly one writer, and `wg.Wait` is ordered after every `wg.Done`, itself ordered after the write. The test drives 5 partitionings over 10001 elements plus the nil case and demands the exact total every time. That determinism is DRF-SC paying off: under a race the same code is wrong in unbounded ways. The sibling trees build the wait out of their own substrate, mutex plus condition in C and python, the same mutex and condvar in rust with `thread::scope` lending the partials by reference so no Arc climbs into the fixture, `synchronized` plus

`notifyAll` in java, task completion in C#, a promise registry in javascript, and a parked-coroutine list in lua, the same 20-line scheduler shape every lua lane in this chapter carries.

6.4. the rendezvous edge

PingPong bounces a token between two workers n times, and every bounce is causally ordered, the count can be neither lost nor doubled:

The dry run: 1000 round trips count exactly 1000 and a single round trip counts 1, all eight lanes pinning both counts. The degenerate shapes are per-tree lanes, zero rounds counting zero and an odd 17 holding in java and python, and the logged walk strictly alternating sent and received, 16 entries for 8 trips, in lua.

1. Each exchange costs 2 synchronization edges, the receive completing before the send returns, the unbuffered rule made visible.
2. The pinger only counts a trip after its pong came back, so no interleaving can lose or double one.
3. The lane shapes differ, two threads in C and go, two virtual threads in java, two tasks in C#, two async closures in javascript, two asyncio tasks in python, two coroutines in lua, and the counts agree anyway.

```
enum { TURN_NONE, TURN_PING, TURN_PONG };

typedef struct {
  mtx_t mu;
  cnd_t ping_cv; // the responder sleeps here until a ping is outstanding
  cnd_t pong_cv; // the initiator sleeps here until a pong is outstanding
  int turn;
  int rounds;
} pingpong;

static void pp_init(pingpong *pp) {
  mtx_init(&pp->mu, mtx_plain);
  cnd_init(&pp->ping_cv);
  cnd_init(&pp->pong_cv);
  pp->turn = TURN_NONE;
  pp->rounds = 0;
}

static void pp_free(pingpong *pp) {
  mtx_destroy(&pp->mu);
  cnd_destroy(&pp->ping_cv);
  cnd_destroy(&pp->pong_cv);
}

// the initiator: send a ping, wait for its pong, count one round trip
static int initiator(void *arg) {
  pingpong *pp = arg;
  for (int i = 0; i < pp->rounds; i++) {
    mtx_lock(&pp->mu);
    pp->turn = TURN_PING;
    cnd_signal(&pp->ping_cv);
    while (pp->turn != TURN_PONG)
      cnd_wait(&pp->pong_cv, &pp->mu);
    mtx_unlock(&pp->mu);
  }
  return pp->rounds;
}

// the responder: consume a ping, send a pong back
static int responder(void *arg) {
  pingpong *pp = arg;
  for (int i = 0; i < pp->rounds; i++) {
    mtx_lock(&pp->mu);
    while (pp->turn != TURN_PING)
      cnd_wait(&pp->ping_cv, &pp->mu);
    pp->turn = TURN_PONG;
    cnd_signal(&pp->pong_cv);
    mtx_unlock(&pp->mu);
  }
  return pp->rounds;
}
```

listing 219: C, two threads over one mutex and two condition variables, every wait guaranteed a partner (source: patterns-concurrency-distributed/samples-c/src/Ch06/rendezvous.c, lines 20-70)

```
struct PingPong {
    state: Mutex<Turn>,
    ping_cv: Condvar,
    pong_cv: Condvar,
}

impl PingPong {
    fn new() -> PingPong {
        PingPong {
            state: Mutex::new(Turn::None),
            ping_cv: Condvar::new(),
            pong_cv: Condvar::new(),
        }
    }
}

fn initiator(pp: &PingPong, rounds: usize) -> usize {
    for _ in 0..rounds {
        let mut turn = pp.state.lock().unwrap();
        *turn = Turn::Ping;
        pp.ping_cv.notify_one();
        while *turn != Turn::Pong {
            turn = pp.pong_cv.wait(turn).unwrap();
        }
    }
    rounds
}

fn responder(pp: &PingPong, rounds: usize) -> usize {
    for _ in 0..rounds {
        let mut turn = pp.state.lock().unwrap();
        while *turn != Turn::Ping {
            turn = pp.ping_cv.wait(turn).unwrap();
        }
        *turn = Turn::Pong;
        pp.pong_cv.notify_one();
    }
    rounds
}

fn ping_pong(rounds: usize) -> (usize, usize) {
    let pp = PingPong::new();
    let pp_ref = &pp;
    std::thread::scope(|s| {
        let responder_handle = s.spawn(move || responder(pp_ref, rounds));
        let initiator_handle = s.spawn(move || initiator(pp_ref, rounds));
        (
            initiator_handle.join().unwrap(),
            responder_handle.join().unwrap(),
        )
    })
}
```

listing 220: Rust, C's exact shape, one Mutex over the turn and two Condvars, scope joining both lanes (source: patterns-concurrency-distributed/samples-rust/src/Ch06/rendezvous.rs, lines 22-73)

```
func PingPong(n int) int {
  ping := make(chan struct{})
  pong := make(chan struct{})
  count := make(chan int)

  go func() {
    for i := 0; i < n; i++ {
      <-ping
      pong <- struct{}{}
    }
  }()

  go func() {
    hits := 0
    for i := 0; i < n; i++ {
      ping <- struct{}{}
      <-pong
      hits++
    }
    count <- hits
  }()

  return <-count
}
```

listing 221: Go, strict alternation through two unbuffered channels (source: patterns-concurrency-distributed/samples/ch06/memorymodel.go, lines 49-73)

```
  static int pingPong(int rounds) throws InterruptedException {
    SynchronousQueue<Integer> ping = new SynchronousQueue<>();
    SynchronousQueue<Integer> pong = new SynchronousQueue<>();
    SynchronousQueue<Integer> count = new SynchronousQueue<>();

    // the responder: consume a ping, answer with a pong, n times over
    Thread responder = Thread.ofVirtual().start(() -> {
      try {
        for (int i = 0; i < rounds; i++) {
          ping.take();
          pong.put(1);
        }
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
    });

    // the initiator: send a ping, wait for its pong, count one round
    // trip
    Thread initiator = Thread.ofVirtual().start(() -> {
      try {
        int hits = 0;
        for (int i = 0; i < rounds; i++) {
          ping.put(1);
          pong.take();
          hits++;
        }
        count.put(hits);
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
    });

    int hits = count.take();
    initiator.join();
    responder.join();
    return hits;
  }
```

listing 222: Java, SynchronousQueue is the unbuffered channel native, put returns only when a take takes the value (source: patterns-concurrency-distributed/samples-java/src/Ch06/Rendezvous.java, lines 21-58)

```
// PingPong bounces a token between two workers. The go lane uses two
// unbuffered channels; C# channels have no unbuffered mode, so the
// strict rendezvous is a pair of binary semaphores: the waiter blocks
// until the releaser acts, every ping is causally ordered against its
// pong, and the final count cannot be lost or doubled.
public static class Rendezvous
{
    public static async Task<int> PingPong(int n)
    {
        using var ping = new SemaphoreSlim(0, 1);
        using var pong = new SemaphoreSlim(0, 1);
        int hits = 0;

        var bouncer = Task.Run(async () =>
        {
            for (int i = 0; i < n; i++)
            {
                await ping.WaitAsync();
                pong.Release();
            }
        });

        var driver = Task.Run(async () =>
        {
            for (int i = 0; i < n; i++)
            {
                ping.Release();
                await pong.WaitAsync();
                hits++;
            }
        });

        await Task.WhenAll(bouncer, driver);
        return hits;
    }
```

listing 223: C#, a pair of binary SemaphoreSlims, no unbuffered channel exists (source: patterns-concurrency-distributed/samples-cs/src/Ch06/Rendezvous.cs, lines 3-37)

```
export class Handoff {
  #recvWaiters = [];
  #sendQ = [];

  send(v) {
    return new Promise((resolve) => {
      const taker = this.#recvWaiters.shift();
      if (taker) {
        taker(v);
        resolve();
      } else {
        this.#sendQ.push({ v, resolve });
      }
    });
  }

  recv() {
    return new Promise((resolve) => {
      const parked = this.#sendQ.shift();
      if (parked) {
        parked.resolve();
        resolve(parked.v);
      } else {
        this.#recvWaiters.push(resolve);
      }
    });
  }
}
```

listing 224: JavaScript, the Handoff class, send parks until a receiver takes (source: patterns-concurrency-distributed/samples-js/src/ch06-rendezvous.mjs, lines 7-34)

```
async def ping_pong(rounds):
    ping = asyncio.Queue()
    pong = asyncio.Queue()
    count = asyncio.Queue()

    async def bouncer():
        # receive on ping, answer on pong: the get must complete
        # before the put, n times over
        for _ in range(rounds):
            await ping.get()
            await pong.put(None)

    async def driver():
        hits = 0
        for _ in range(rounds):
            await ping.put(None)
            await pong.get()
            hits += 1
        await count.put(hits)

    b = asyncio.create_task(bouncer())
    d = asyncio.create_task(driver())
    got = await count.get()
    await asyncio.gather(b, d)
    return got
```

listing 225: Python, two asyncio tasks over plain queues, the get before the put (source: patterns-concurrency-distributed/samples-py/src/Ch06/rendezvous.py, lines 18-42)

```lua
-- an unbuffered channel: a sender with no receiver parked, a
-- receiver with no sender parked, and the handoff wakes the parked
-- side. the value moves before the sender continues: that is the
-- synchronization edge
local function chan(S)
  return { S = S, senders = {}, receivers = {}, closed = false }
end

local function send(ch, v)
  if ch.closed then
    error("send on closed channel")
  end
  while true do
    if #ch.receivers > 0 then
      local r = table.remove(ch.receivers, 1)
      r.v, r.ok = v, true
      ch.S.ready[#ch.S.ready + 1] = r.co
      return true
    end
    local w = { co = coroutine.running(), v = v }
    ch.senders[#ch.senders + 1] = w
    coroutine.yield("park")
    if w.ok then
      return true
    end
    if ch.closed then
      return false
    end
  end
end
```

listing 226: Lua, the unbuffered channel as parked sender and receiver lists (source: patterns-concurrency-distributed/samples-lua/ch06_rendezvous.lua, lines 30-59)

Go's unbuffered channel is the native shape and C#'s channels have no unbuffered mode, so its rendezvous is two binary semaphores, the waiter blocking until the releaser acts. Java's `SynchronousQueue` is the unbuffered channel native, `put` returning only when a `take` takes the value, the one stdlib in this chapter that needs no rebuilding. C builds the same binary handoff from a mutex and two condition variables, where the zero-waiter forgetfulness trap from chapter 15 of the C book cannot bite because every wait is guaranteed a partner. Rust builds C's exact shape, one `Mutex` over a turn enum and two `Condvars`, and `thread::scope` joins both lanes before the counts are read. The dynamic three park continuations instead of threads, javascript resolvers, python asyncio waits, lua parked coroutines, and the alternation comes from the queue edges themselves.

6.5. atomics, the word-sized escape hatch

100 workers adding 1000 each into one word must land on exactly 100000, and only the atomic add guarantees it:

The dry run: the exact 100000 total pins in every lane with real shared memory, go's goroutines, C's threads over an `_Atomic` cell, rust's scoped threads over an `AtomicI64`, java's threads over an `AtomicLong`, C#'s workers over `Interlocked`, javascript's workers over one `SharedArrayBuffer` cell. The two coroutine lanes state their boundary the honest way: a scripted 2 by 5 read, pause, write walk loses 5 of 10 updates in python and lua alike, and the locked or non-yielding versions reach 10, the same code reaching 100000 when the whole section is atomic.

1. The add must be one linearizable step, an interleaved read then write loses updates, pinned at 5 of 10 in the scripted lanes.

2. Atomics are the model's smallest door, correct for single words with no invariants spanning them, two-field invariants need the mutex of chapter 7.
3. The javascript lane's inline-eval worker over a `SharedArrayBuffer` is the one sanctioned SAB home in this corpus, the only place two javascript heaps share memory.

```c
typedef struct {
  _Atomic long long count;
} atomic_counter;

static void ac_init(atomic_counter *c) { atomic_init(&c->count, 0); }

static void ac_add(atomic_counter *c, long long delta) {
  atomic_fetch_add_explicit(&c->count, delta, memory_order_relaxed);
}

static long long ac_load(const atomic_counter *c) {
  return atomic_load_explicit(&c->count, memory_order_relaxed);
}

#define THREADS 100
#define PER_THREAD 1000

static int adder(void *arg) {
  atomic_counter *c = arg;
  for (int i = 0; i < PER_THREAD; i++)
    ac_add(c, 1);
  return 0;
}
```

listing 227: C, an atomic long long with fetch_add, the relaxed ordering stated where it is chosen (source: patterns-concurrency-distributed/samples-c/src/Ch06/atomics.c, lines 20-42)

```rust
struct AtomicCounter {
    count: AtomicI64,
}

impl AtomicCounter {
    fn new() -> AtomicCounter {
        AtomicCounter {
            count: AtomicI64::new(0),
        }
    }

    fn add(&self, delta: i64) {
        self.count.fetch_add(delta, Ordering::Relaxed);
    }

    fn load(&self) -> i64 {
        self.count.load(Ordering::Relaxed)
    }
}
```

listing 228: Rust, fetch_add under Relaxed where only the arithmetic needs atomicity, the same rung C names (source: patterns-concurrency-distributed/samples-rust/src/Ch06/atomics.rs, lines 18-36)

```
type AtomicCounter struct {
  n atomic.Int64
}

func (c *AtomicCounter) Add(delta int64) { c.n.Add(delta) }

func (c *AtomicCounter) Load() int64 { return c.n.Load() }

// ClosingRange demonstrates the close edge: closing is synchronized
// before the receive that returns because of it, so every value
```

listing 229: Go, atomic adds are linearizable, lost updates impossible (source: patterns-concurrency-distributed/samples/ch06/memorymodel.go, lines 76-85)

```
  // the ladder carrier: one long field, every access mode pointed at
  // it through the same VarHandle
  static final class Cell {
    long value;

    static final VarHandle VALUE;

    static {
      try {
        VALUE = MethodHandles.lookup().findVarHandle(Cell.class, "value", long.class);
      } catch (ReflectiveOperationException e) {
        throw new ExceptionInInitializerError(e);
      }
    }
  }

  // the publication pair: plain payload, release flag. the release
  // write to flag is the edge that makes the plain payload visible to
  // whichever acquire read observes it
  static final class Publish {
    int payload;
    int flag;

    static final VarHandle PAYLOAD, FLAG;

    static {
      try {
        var lookup = MethodHandles.lookup();
        PAYLOAD = lookup.findVarHandle(Publish.class, "payload", int.class);
        FLAG = lookup.findVarHandle(Publish.class, "flag", int.class);
      } catch (ReflectiveOperationException e) {
        throw new ExceptionInInitializerError(e);
      }
    }
  }
```

listing 230: Java, VarHandle aims every access mode at one field, the plain payload published by a release flag (source: patterns-concurrency-distributed/samples-java/src/Ch06/Atomics.java, lines 25-59)

```
// AtomicCounter shows the word-sized escape hatch: Interlocked adds
// are linearizable, so N workers adding M each yields exactly N*M.
// The C# analog of the go lane's sync/atomic.Int64.
public sealed class AtomicCounter
{
    private long _n;

    public void Add(long delta) => Interlocked.Add(ref _n, delta);

    public long Load() => Interlocked.Read(ref _n);
}

// TornCounter is the control group: the same workload through a plain
// read-modify-write loses updates whenever the scheduler interleaves.
public sealed class TornCounter
{
    private long _n;

    public void Add(long delta) => _n += delta;

    public long Load() => _n;
}
```

listing 231: C#, Interlocked.Add, with the plain += control group beside it (source: patterns-concurrency-distributed/samples-cs/src/Ch06/Atomics.cs, lines 3-24)

```
import { Worker } from "node:worker_threads";

const WORKER_SRC = `
const { parentPort } = require("node:worker_threads");
parentPort.on("message", ({ sab, each }) => {
  const cell = new Int32Array(sab);
  for (let i = 0; i < each; i++) Atomics.add(cell, 0, 1);
  parentPort.postMessage("done");
});
`;

export async function atomicTotal(workers, each) {
  const sab = new SharedArrayBuffer(4);
  const cell = new Int32Array(sab);
  const exits = [];
  const pool = [];
  for (let w = 0; w < workers; w++) {
    const wk = new Worker(WORKER_SRC, { eval: true });
    pool.push(wk);
    exits.push(
      new Promise((resolve) => {
        wk.once("message", resolve);
        wk.postMessage({ sab, each });
      }),
    );
  }
  await Promise.all(exits); // join on the exit messages
  await Promise.all(pool.map((wk) => wk.terminate()));
  return Atomics.load(cell, 0);
}
```

listing 232: JavaScript, worker_threads over one SharedArrayBuffer, Atomics.add into the shared cell (source: patterns-concurrency-distributed/samples-js/src/ch06-atomics.mjs, lines 9-38)

```python
def scripted_total(interleaved):
    # two workers, five read-pause-write increments each. the pause
    # is a scheduler checkpoint driven by hand, so the outcome is a
    # property of the interleaving, never of machine timing
    cell = {"v": 0}

    def worker():
        for _ in range(5):
            cur = cell["v"]
            yield
            cell["v"] = cur + 1

    workers = [worker(), worker()]
    if not interleaved:
        for w in workers:
            while True:
                try:
                    next(w)
                except StopIteration:
                    break
        return cell["v"]
    while True:
        alive = False
        for w in workers:
            try:
                next(w)
                alive = True
            except StopIteration:
                pass
        if not alive:
            break
    return cell["v"]
```

listing 233: Python, the scripted tear, read, pause, write, 5 of 10 updates lost on display (source: patterns-concurrency-distributed/samples-py/src/Ch06/atomics.py, lines 18-49)

```lua
local function AtomicCounter()
  return {
    n = 0,
    -- one statement, no yield inside: atomic under any interleaving
    Add = function(self, d) self.n = self.n + d end,
    Load = function(self) return self.n end,
  }
end

-- the torn version: read, yield, write. the window is the lesson
local function torn_rmw(box, reads)
  local cur = box.n
  reads[#reads + 1] = cur
  coroutine.yield()
  box.n = cur + 1
end
```

listing 234: Lua, the non-yielding add is the atomic, the torn version opens the window with a yield (source: patterns-concurrency-distributed/samples-lua/ch06_atomics.lua, lines 30-45)

C names the ordering it wants, `memory_order_relaxed` for the count where only the arithmetic needs to be atomic, the ladder chapter 16 of the C book walks in full. Rust names the same rung, the counter's `fetch_add` carries

`Ordering::Relaxed` at the call site, one of the five orderings the atomic types demand you choose every time. Java walks the same ladder through `VarHandle` access modes, plain, opaque, release and acquire, volatile, with `AtomicLong` as the counter and `Thread.onSpinWait` the hint the acquire spin rides. C#'s `Interlocked` is the same facility with one default. The javascript lane is the corpus's one real shared-memory lane, a worker pool inside an inline-eval module adding into the same `Int32Array` cell, joined on exit messages, and it is the only such lane because a second heap is a cost this tree pays once and points at. Python has no word-sized escape hatch: `+=` is a load, an add, and a store the GIL may split, so the sample shows the tear with a hand-driven interleaving and buys the exact total with a lock, the honest row. Lua has no preemption, so a read-modify-write with no yield inside is atomic however the lanes interleave, and the sample tears it on purpose with a yield to show what the preemptive worlds suffer, the lost-update walk on the cooperative schedulerbook 9, ch 7 (lua 5.5, coroutines) this whole book scripts.

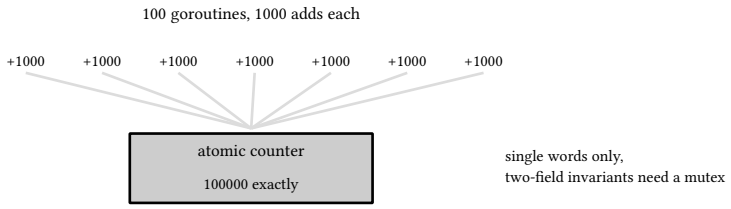


figure 35: many adders, one word, an exact total every run

6.6. close and capacity

The closing edge and the counting rule, one channel carrying both lessons:

The dry run: the values 5, 3, 9, 1, 7 fill a channel, close it, and the drain totals 25, the empty input closes and drains to 0, and sending on a closed channel is a loud error where a tree pins it, lua's row the explicit one. 50 jobs through a gate of 3 all finish, the observed peak never passes 3.

1. Close is ordered after every send before it, so the range sees all values, and close never erases the buffer, the lagging receiver still drains.
2. The kth receive admits the k+Cth send, so capacity C bounds C workers at once, the counting semaphore rule.
3. The peak probe asserts a bound forced by the gate, an invariant, never a timing.

```c
typedef struct {
  mtx_t mu;
  cnd_t cv;
  int buf[CH_CAP];
  int count;
  bool closed;
} chan;

static void chan_init(chan *c) {
  mtx_init(&c->mu, mtx_plain);
  cnd_init(&c->cv);
  c->count = 0;
  c->closed = false;
}

static void chan_free(chan *c) {
  mtx_destroy(&c->mu);
  cnd_destroy(&c->cv);
}

static bool chan_send(chan *c, int v) {
  mtx_lock(&c->mu);
  while (c->count == CH_CAP && !c->closed)
    cnd_wait(&c->cv, &c->mu);
  if (c->closed) {
    mtx_unlock(&c->mu);
    return false;
  }
  c->buf[(c->count)++] = v;
  cnd_signal(&c->cv);
  mtx_unlock(&c->mu);
  return true;
}

static void chan_close(chan *c) {
  mtx_lock(&c->mu);
  c->closed = true;
  cnd_broadcast(&c->cv);
  mtx_unlock(&c->mu);
}
```

listing 235: C, the bounded channel over one mutex and one condition, close broadcasts (source: patterns-concurrency-distributed/samples-c/src/Ch06/channels.c, lines 24-63)

```rust
struct Chan {
    state: Mutex<ChanState>,
    cv: Condvar,
}

struct ChanState {
    buf: VecDeque<i32>,
    closed: bool,
}

impl Chan {
    fn new() -> Chan {
        Chan {
            state: Mutex::new(ChanState {
                buf: VecDeque::new(),
                closed: false,
            }),
            cv: Condvar::new(),
        }
    }

    fn send(&self, v: i32) -> bool {
        let mut st = self.state.lock().unwrap();
        while st.buf.len() == CH_CAP && !st.closed {
            st = self.cv.wait(st).unwrap();
        }
        if st.closed {
            return false;
        }
        st.buf.push_back(v);
        self.cv.notify_all();
        true
    }

    fn close(&self) {
        self.state.lock().unwrap().closed = true;
        self.cv.notify_all();
    }

    fn recv(&self) -> Option<i32> {
        let mut st = self.state.lock().unwrap();
        while st.buf.is_empty() && !st.closed {
            st = self.cv.wait(st).unwrap();
        }
        let v = st.buf.pop_front();
        self.cv.notify_all();
        v
    }
}
```

listing 236: Rust, the same channel over a Mutex and one Condvar, a VecDeque for the ring, close wakes every waiter (source: patterns-concurrency-distributed/samples-rust/src/Ch06/channels.rs, lines 20-68)

```go
func ClosingRange(values []int) int {
  ch := make(chan int, len(values))
  for _, v := range values {
    ch <- v
  }
  close(ch)
  sum := 0
  for v := range ch {
    sum += v
  }
  return sum
}

// Semaphore bounds concurrency using the buffered channel rule: the
// kth receive is synchronized before the (k+C)th send completes, so
// a channel of capacity C admits at most C workers at once.
func Semaphore(jobs []func(), capacity int) int {
  gate := make(chan struct{}, capacity)
  var wg sync.WaitGroup
  for _, j := range jobs {
    wg.Add(1)
    go func(job func()) {
      defer wg.Done()
      gate <- struct{}{}
      job()
      <-gate
    }(j)
  }
  wg.Wait()
  return len(jobs)
}
```

listing 237: Go, close orders the terminal receive, capacity C bounds C workers (source: patterns-concurrency-distributed/samples/ch06/memorymodel.go, lines 87-117)

```
  static final class Chan {
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition cv = lock.newCondition();
    private final ArrayDeque<Integer> buf = new ArrayDeque<>();
    private boolean closed;

    boolean send(int v) throws InterruptedException {
      lock.lock();
      try {
        while (buf.size() == CH_CAP && !closed)
          cv.await();
        if (closed)
          return false;
        buf.addLast(v);
        cv.signalAll();
        return true;
      } finally {
        lock.unlock();
      }
    }

    void close() {
      lock.lock();
      try {
        closed = true;
        cv.signalAll();
      } finally {
        lock.unlock();
      }
    }

    boolean recv(int[] out) throws InterruptedException {
      lock.lock();
      try {
        while (buf.isEmpty() && !closed)
          cv.await();
        if (buf.isEmpty())
          return false;
        out[0] = buf.pollFirst();
        cv.signalAll();
        return true;
      } finally {
        lock.unlock();
      }
    }
  }
```

listing 238: Java, the channel hand-rolled over ReentrantLock and one Condition, close broadcasts, recv answers false once drained (source: patterns-concurrency-distributed/samples-java/src/Ch06/Channels.java, lines 32-77)

```
// ClosingRange demonstrates the completion edge: completing a channel
// is synchronized before the read that returns because of it, so
// every value written before Complete is visible to the draining
// reader.
public static class Channels
{
    public static async Task<int> ClosingRange(int[] values)
    {
        var ch = Channel.CreateBounded<int>(Math.Max(1, values.Length));
        var feed = Task.Run(async () =>
        {
            foreach (var v in values)
            {
                await ch.Writer.WriteAsync(v);
            }
            ch.Writer.Complete();
        });
        int sum = 0;
        await foreach (var v in ch.Reader.ReadAllAsync())
        {
            sum += v;
        }
        await feed;
        return sum;
    }

    // Semaphore bounds concurrency using the bounded channel rule: a
    // channel of capacity C admits at most C workers at once, because
    // the kth read frees the (k+C)th write.
    public static async Task<int> Semaphore(Action[] jobs, int capacity)
    {
        var gate = Channel.CreateBounded<object>(capacity);
        var tasks = new Task[jobs.Length];
        for (int i = 0; i < jobs.Length; i++)
        {
            Action job = jobs[i];
            tasks[i] = Task.Run(async () =>
            {
                await gate.Writer.WriteAsync(new object()); // acquire
                job();
                await gate.Reader.ReadAsync(); // release
            });
        }
        await Task.WhenAll(tasks);
        return jobs.Length;
    }
```

listing 239: C#, Complete before ReadAllAsync, the bounded channel doubling as the gate (source: patterns-concurrency-distributed/samples-cs/src/Ch06/Channels.cs, lines 5-50)

```
export class Chan {
  #buf = [];
  #cap;
  #closed = false;
  #recvWaiters = [];
  #sendWaiters = [];

  constructor(capacity) {
    if (capacity < 1) throw new Error("capacity must be at least 1");
    this.#cap = capacity;
  }

  get closed() {
    return this.#closed;
  }

  async send(v) {
    for (;;) {
      if (this.#closed) throw new Error("send on closed channel");
      if (this.#buf.length < this.#cap) {
        this.#buf.push(v);
        for (const w of this.#recvWaiters.splice(0)) w();
        return;
      }
      await new Promise((resolve) => this.#sendWaiters.push(resolve));
    }
  }

  async recv() {
    for (;;) {
      if (this.#buf.length > 0) {
        const v = this.#buf.shift();
        const s = this.#sendWaiters.shift();
        if (s) s();
        return { value: v, ok: true };
      }
      if (this.#closed) return { value: undefined, ok: false };
      await new Promise((resolve) => this.#recvWaiters.push(resolve));
    }
  }

  close() {
    if (this.#closed) throw new Error("close of closed channel");
    this.#closed = true;
    for (const w of this.#recvWaiters.splice(0)) w();
    for (const s of this.#sendWaiters.splice(0)) s();
  }
}
```

listing 240: JavaScript, the Chan class, send waits when full, close wakes every parked side (source: patterns-concurrency-distributed/samples-js/src/ch06-channels.mjs, lines 7-54)

```
class Chan:
    def __init__(self, capacity):
        self.cond = asyncio.Condition()
        self.items = []
        self.cap = capacity
        self.closed = False

    async def send(self, v):
        async with self.cond:
            while len(self.items) ≥ self.cap and not self.closed:
                await self.cond.wait()
            if self.closed:
                raise ValueError("send on closed channel")
            self.items.append(v)
            self.cond.notify_all()

    async def recv(self):
        async with self.cond:
            while not self.items and not self.closed:
                await self.cond.wait()
            if self.items:
                v = self.items.pop(0)
                self.cond.notify_all()
                return v
            return CLOSED

    async def close(self):
        async with self.cond:
            self.closed = True
            self.cond.notify_all()

    def __aiter__(self):
        return self

    async def __anext__(self):
        v = await self.recv()
        if v is CLOSED:
            raise StopAsyncIteration
        return v
```

listing 241: Python, the Chan over one asyncio Condition, a CLOSED sentinel ending the drain (source: patterns-concurrency-distributed/samples-py/src/Ch06/channels.py, lines 21-59)

```lua
-- a channel with a buffer: send fills it, parks when full, and the
-- kth receive is the edge that admits the (k+C)th send. close wakes
-- every parked receiver without erasing buffered values
local function chan(S, cap)
  return {
    S = S, cap = cap or 0, buf = {},
    senders = {}, receivers = {}, closed = false,
  }
end

local function close_ch(ch)
  ch.closed = true
  for _, w in ipairs(ch.receivers) do
    ch.S.ready[#ch.S.ready + 1] = w.co
  end
  ch.receivers = {}
  for _, w in ipairs(ch.senders) do
    ch.S.ready[#ch.S.ready + 1] = w.co
  end
  ch.senders = {}
end

local function send(ch, v)
  if ch.closed then
    error("send on closed channel")
  end
  while true do
    if #ch.receivers > 0 then
      local r = table.remove(ch.receivers, 1)
      r.v, r.ok = v, true
      ch.S.ready[#ch.S.ready + 1] = r.co
      return true
    end
    if #ch.buf < ch.cap then
      ch.buf[#ch.buf + 1] = v
      return true
    end
    local w = { co = coroutine.running(), v = v }
    ch.senders[#ch.senders + 1] = w
    coroutine.yield("park")
    if w.ok then
      return true
    end
    if ch.closed then
      return false
    end
  end
end
```

listing 242: Lua, buffered tables with parked senders, close without erasing (source: patterns-concurrency-distributed/samples-lua/ch06_channels.lua, lines 30-77)

C's channel is one mutex, one condition, a ring of integers, and a closed flag, the chapter 7 bounded queue arriving one chapter early in miniature. Rust rolls the same shape over `Mutex` and `Condvar` with a `VecDeque` for the ring, and its gate is the same permits count under its own condvar. Java rolls the same channel by hand on `ReentrantLock` and one `Condition`, and parks its stdlib lanes beside it: `Semaphore` is the gate and `ArrayBlockingQueue` the buffered channel minus the close. C# gets both edges from the BCL channel, `Writer.Complete` is the close and bounded capacity is the

gate. The dynamic three park continuations, javascript on promise resolvers, python on the condition, lua on the scheduler's parked list, and go's semaphore is the idiom the memory model document itself suggests, a buffered channel of capacity C where sending acquires and receiving releases.

6.7. broken idioms, named

The memory model document closes with broken idioms worth naming because every go codebase grows one eventually. Double checked locking, checking a plain flag before `once.Do`, fails because the racing read may observe the flag set without observing the writes sequenced before it. Busy waiting on a plain bool may spin forever, since nothing forces the loop to see the update. Publishing a pointer through a plain field may surface a non-nil pointer with stale contents. The document's own advice ends the argument: if you must read the rest of the model to understand your program, you are being too clever. Don't be clever. The sibling trees inherit the same graves: C's data race is undefined behavior with no floor, rust's safe code cannot write the race at all, the borrow checker rejecting the aliasing before the program runs, and the cooperative lanes can only exhibit the bug by building it, which is the strongest argument for never shipping it.

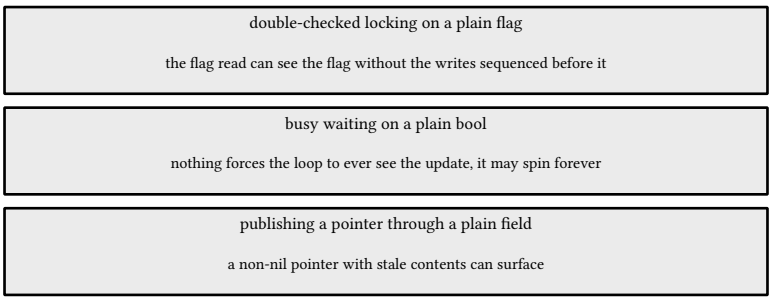


figure 36: three idioms the memory model document names as broken

6.8. across the eight languages

The build sizes count non-comment source lines, the go column the chapter's one frozen file:

language	build SLOC	dependency	boundary note
c	411	libc plus threads.h, stdatomic.h	waitgroup, channel, and semaphore by hand over mtx and cnd, relaxed ordering named at the add
rust	376	std only, no crates	waitgroup, rendezvous, channel, and gate rolled over Mutex and Condvar, thread::scope lending without Arc, Relaxed named at the add
go	91	stdlib	frozen reference lane, the five fixtures this chapter's contracts derive from
java	354	jdk 27 stdlib	the VarHandle ladder named rung by rung, SynchronousQueue the native rendezvous, waitgroup and channel rolled by hand
c#	134	bcl	WhenAll waits, SemaphoreSlim rendezvous, Interlocked counters, Channels for close and capacity
javascript	217	node stdlib	promise-resolver waits, Handoff rendezvous, the corpus's one SharedArrayBuffer lane
python	269	stdlib only	Condition waitgroup, asyncio queues, the scripted tear where atomics do not exist
lua	529	lib.lua harness	one 20-line scheduler under every lane, races built on purpose with yields, the torn 5 beside the serialized 10

sources: go.dev/ref/mem for the definition of data race, every synchronization edge quoted above, DRF-SC, and the broken idioms, accessed 2026-09-08. Verified by the eight chapter legs: 4 Ch06 C programs with 327 embedded checks, 4 Ch06 rust programs in `samples-rust/src` with 25 embedded checks under `run-rust-samples`, `go test -race` at 6 tests in `patternsbook/ch06`, 4 Ch06 java programs with 37 checks under `run-java-samples`, 10 C# facts over `PatternsBook.slnx`, 9 `node --test` cases, 23 Python checks across 4 files, and 17 Lua rows under `run.lua`.

7. sync primitives in depth

Chapter 6 established that synchronization is the only currency the memory model accepts. This chapter is the price list, read in eight voices: what each primitive in go's `sync` and `sync/atomic` buys in its home grain, and what the same guarantee costs where the language hands you more or less. C carries the locks of C23 `threads.h` raw and pays for every convenience go wraps around them. Rust keeps the same raw shapes one call away in std, `Mutex`, `Condvar`, `OnceLock`, and the atomic types, with the borrow checker auditing who may hold what. Java splits the family across three shelves, `synchronized` blocks, the `java.util.concurrent` toolbox, and `VarHandle` fences. C# finds half the family native in the bcl, `Lazy`, `Interlocked`, `ConcurrentDictionary`. Javascript runs one thread per realm and pays in queue discipline where go pays in kernel waits, python threads under the GIL with `threading.Condition` a near 1:1 mirror of `Cond`, and lua schedules coroutines by hand, so its waits are parking lists the driver owns. One artifact per primitive, eight trees, and the go tests stay the frozen reference the seven siblings cross-check.

7.1. mutex and rwmutex

The mutex is the default tool because it is the tool that composes: it guards an invariant, not a variable. The convention that keeps guarded code auditable is on display in `SafeMap`, spelled eight ways.

The dry run: 50 concurrent writers each set 100 values over 20 keys and the map holds exactly 20 keys afterward, never a torn row, in every tree that spawns real workers. The read-modify-write lane then demands exact totals under the guard: 400 in C, 400 in rust, 400 in java beside 2000 more through its lock-free `ConcurrentHashMap.compute`, 2000 in C#, 10 in javascript, 2000 in python, 5000 in lua, each tree at its own fan-in. The frozen go test pins the key set and the under-lock count, C# matches that shape and pins the concurrent total, and go alone leaves the increment totals to the siblings.

```c
typedef struct {
  mtx_t mu;
  row rows[KEYS];
  int n;
} safemap;

static void sm_init(safemap *m) {
  mtx_init(&m->mu, mtx_plain);
  m->n = 0;
}

static void sm_free(safemap *m) { mtx_destroy(&m->mu); }

static row *sm_find(safemap *m, const char *key) {
  for (int i = 0; i < m->n; i++)
    if (strcmp(m->rows[i].key, key) == 0)
      return &m->rows[i];
  return NULL;
}

static void sm_set(safemap *m, const char *key, long val) {
  mtx_lock(&m->mu);
  row *r = sm_find(m, key);
  if (r) {
    r->val = val;
  } else if (m->n < KEYS) {
    snprintf(m->rows[m->n].key, 8, "%s", key);
    m->rows[m->n].val = val;
    m->n++;
  }
  mtx_unlock(&m->mu);
}

static bool sm_get(safemap *m, const char *key, long *out) {
  mtx_lock(&m->mu);
  row *r = sm_find(m, key);
  bool ok = r != NULL;
  if (ok)
    *out = r->val;
  mtx_unlock(&m->mu);
  return ok;
}
```

listing 243: C, the mutex is a field in the struct, find runs under it, and threads.h ships one flavor so the reader-writer split collapses (source: patterns-concurrency-distributed/samples-c/src/Ch07/safemap.c, lines 29-70)

```rust
struct SafeMap {
    rows: Mutex<HashMap<String, i64>>,
}

impl SafeMap {
    fn new() -> SafeMap {
        SafeMap {
            rows: Mutex::new(HashMap::new()),
        }
    }

    fn set(&self, key: &str, val: i64) {
        self.rows.lock().unwrap().insert(key.to_string(), val);
    }

    fn get(&self, key: &str) -> Option<i64> {
        self.rows.lock().unwrap().get(key).copied()
    }

    fn len(&self) -> usize {
        self.rows.lock().unwrap().len()
    }

    fn under_lock(&self, f: impl FnOnce(&mut HashMap<String, i64>)) {
        f(&mut self.rows.lock().unwrap());
    }
}
```

listing 244: Rust, the lock a field wrapping the whole map, under_lock hands the guard to the closure (source: patterns-concurrency-distributed/samples-rust/src/Ch07/safemap.rs, lines 20-46)

```
type SafeMap[K comparable, V any] struct {
  mu sync.RWMutex
  m  map[K]V
}

func NewSafeMap[K comparable, V any]() *SafeMap[K, V] {
  return &SafeMap[K, V]{m: map[K]V{}}
}

func (s *SafeMap[K, V]) Set(k K, v V) {
  s.mu.Lock()
  defer s.mu.Unlock()
  s.m[k] = v
}

func (s *SafeMap[K, V]) Get(k K) (V, bool) {
  s.mu.RLock()
  defer s.mu.RUnlock()
  v, ok := s.m[k]
  return v, ok
}

func (s *SafeMap[K, V]) Len() int {
  s.mu.RLock()
  defer s.mu.RUnlock()
  return len(s.m)
}

// UnderLock runs f while the write lock is held, the shape for
// read-modify-write that must not tear.
func (s *SafeMap[K, V]) UnderLock(f func(m map[K]V)) {
  s.mu.Lock()
  defer s.mu.Unlock()
  f(s.m)
}
```

listing 245: Go, lock as a field beside its data, defer on every path, rwlock for reads (source: patterns-concurrency-distributed/samples/ch07/syncdeep.go, lines 12-47)

```
  static final class SafeMap {
    private final Object lock = new Object();
    private final Map<String, Long> m = new HashMap<>();

    void set(String key, long val) {
      synchronized (lock) {
        m.put(key, val);
      }
    }

    Long get(String key) {
      synchronized (lock) {
        return m.get(key);
      }
    }

    int len() {
      synchronized (lock) {
        return m.size();
      }
    }

    // f runs with the lock held, the shape for updates that must not
    // tear against each other
    <R> R underLock(Function<Map<String, Long>, R> f) {
      synchronized (lock) {
        return f.apply(m);
      }
    }
  }
```

listing 246: Java, the lock an object field beside its map, synchronized the release, underLock the read-modify-write shape (source: patterns-concurrency-distributed/samples-java/src/Ch07/Safemap.java, lines 24-53)

```
public sealed class SafeMap<K, V> where K : notnull
{
    private readonly ReaderWriterLockSlim _rw = new();
    private readonly Dictionary<K, V> _m = [];

    public void Set(K k, V v)
    {
        _rw.EnterWriteLock();
        try
        {
            _m[k] = v;
        }
        finally
        {
            _rw.ExitWriteLock();
        }
    }

    public (V value, bool ok) Get(K k)
    {
        _rw.EnterReadLock();
        try
        {
            if (_m.TryGetValue(k, out V? v))
            {
                return (v!, true);
            }
            return (default!, false);
        }
        finally
        {
            _rw.ExitReadLock();
        }
    }
```

listing 247: C#, ReaderWriterLockSlim beside the dictionary, try and finally on every path (source: patterns-concurrency-distributed/samples-cs/src/Ch07/Safemap.cs, lines 6-39)

```
export class SafeMap {
  #m = new Map();
  #tail = Promise.resolve();

  set(k, v) {
    this.#m.set(k, v);
  }

  get(k) {
    return [this.#m.get(k), this.#m.has(k)];
  }

  len() {
    return this.#m.size;
  }

  // f may await; every caller's f runs alone.
  underLock(f) {
    const run = this.#tail.then(() => f(this.#m));
    this.#tail = run.then(
      () => {},
      () => {},
    );
    return run;
  }
}
```

listing 248: JavaScript, one thread cannot tear the map, underLock chains a promise tail so awaiting sections run alone (source: patterns-concurrency-distributed/samples-js/src/ch07-safemap.mjs, lines 7-32)

```
class SafeMap:
    def __init__(self):
        self.lock = threading.RLock()
        self.m = {}

    def set(self, k, v):
        with self.lock:
            self.m[k] = v

    def get(self, k):
        with self.lock:
            return self.m.get(k)

    def len(self):
        with self.lock:
            return len(self.m)

    def under_lock(self, f):
        # the shape for read-modify-write that must not tear
        with self.lock:
            return f(self.m)
```

listing 249: Python, the lock beside a plain dict, the with statement is the unlock (source: patterns-concurrency-distributed/samples-py/src/Ch07/safemap.py, lines 16-37)

```lua
local function NewSafeMap()
  local m = { m = {} }
  function m:Set(k, v) self.m[k] = v end
  function m:Get(k) return self.m[k], self.m[k] ~= nil end
  function m:Len()
    local n = 0
    for _ in pairs(self.m) do n = n + 1 end
    return n
  end
  -- read-modify-write that must not tear runs under the lock: the
  -- callback receives the raw map and yields nothing
  function m:UnderLock(f) f(self.m) end
  return m
end
```

listing 250: Lua, a cooperative world makes the lock nominal, UnderLock is the promise the callback yields nothing (source: patterns-concurrency-distributed/samples-lua/ch07_safemap.lua, lines 31-44)

Three rules do most of the work and every lane keeps them. The lock lives in the same struct as the data, so the pairing is visible. Every method locks and releases on every path, go through `defer`, C# through `finally`, java through the `synchronized` block's own release, rust through the guard dropping at the closing brace, the lock's lifetime readable as a scope, C through paired calls because it has neither. And `RWMutex` is paid for only when readers outnumber writers and hold the lock long enough for the read lock's bookkeeping to amortize, a getter on a map is usually faster under a plain `Mutex` because `RLock` is itself atomic work. `UnderLock` shows the read-modify-write shape: when the operation spans check then act, the caller gets the whole thing under one hold instead of two acquisitions with a window between.

The lanes that lack a kernel split say so instead of faking one. C has one mutex flavor in `threads.h`, so the whole structure rides it and the row reads honest. Rust ships both `Mutex` and `RwLock` in std and the sample still rides the mutex, one lookup too short to amortize the read lock's bookkeeping, go's own economy argument. Java parks the sample on plain `synchronized` blocks, its `ReadWriteLock` left on the shelf, and the file's divergence lane runs the same increments through `ConcurrentHashMap.compute`, the atomic read-modify-write with no lock in sight. C# has the split and a documented writer preference, chapter 9 cashes that in. Javascript cannot tear a map on one thread, so its real hazard is the critical section that awaits: control returns to the loop mid-update and other tasks interleave, which is why `underLock` serializes through a promise chain and the test pins unlocked interleaving losing 10 increments down to 1. Python puts a plain `threading.RLock` beside a plain dict and leans on the with statement. Lua's lock is nominal, there is no preemption to exclude, so `UnderLock` buys a different guarantee, the callback yields nothing, and the lua file pins the counterpoint: a callback that yields under it tears exactly like go code that slept holding the mutex.

The violation worth naming in the reference lane: copying a struct that contains a mutex copies the lock's state, and `go vet` flags it, which is why guarded types are always handled by pointer.

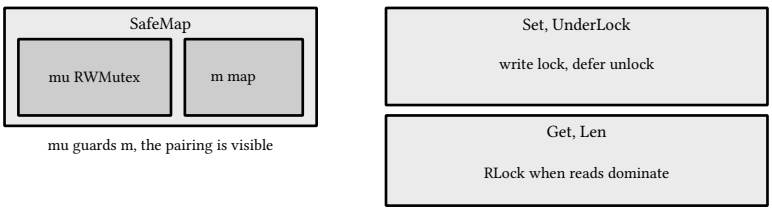


figure 37: the lock lives in the struct, beside the data it guards

7.2. once, the lazy family

Chapter 2 used `OnceValue` for lazy creation. The family has three members, `OnceFunc` for effects, `OnceValue` and `OnceValues` for results, and the memory model gives them their teeth: the completion of the one call to f is synchronized before any call returns, which is precisely the edge double checked locking lacks.

The dry run: `Lazy("a1b22c333")` folds every digit into one running number and returns 122333, the parse runs exactly once, and 20 concurrent callers all read the value. The seven new trees count the parse and pin exactly 1. The go lane's frozen file carries no counter, its test pins the value across the same 20 callers.

```c
typedef struct {
  once_flag flag;
  long value;
  int parses;
  const char *raw;
} lazy;

static lazy *target;

static void lazy_init(lazy *l, const char *raw) {
  l->flag = (once_flag)ONCE_FLAG_INIT;
  l->value = 0;
  l->parses = 0;
  l->raw = raw;
  target = l;
}

static void run_parse(void) {
  lazy *l = target;
  l->parses++;
  long acc = 0;
  for (const char *p = l->raw; *p; p++)
    if (*p ≥ '0' && *p ≤ '9')
      acc = acc * 10 + (*p - '0');
  l->value = acc;
}

static long lazy_get(lazy *l) {
  call_once(&l->flag, run_parse);
  return l->value;
}
```

listing 251: C, call_once over a once_flag, the target pointer installed at init because call_once carries no context argument (source: patterns-concurrency-distributed/samples-c/src/Ch07/once.c, lines 24-54)

```rust
struct Lazy {
    cell: OnceLock<i64>,
    parses: AtomicI32,
    raw: &'static str,
}

impl Lazy {
    fn new(raw: &'static str) -> Lazy {
        Lazy {
            cell: OnceLock::new(),
            parses: AtomicI32::new(0),
            raw,
        }
    }

    fn parse(raw: &str) -> i64 {
        raw.bytes()
            .filter(|b| b.is_ascii_digit())
            .fold(0i64, |acc, b| acc * 10 + i64::from(b - b'0'))
    }

    fn get(&self) -> i64 {
        *self.cell.get_or_init(|| {
            self.parses.fetch_add(1, Ordering::SeqCst);
            Lazy::parse(self.raw)
        })
    }

    fn parses(&self) -> i32 {
        self.parses.load(Ordering::SeqCst)
    }
}
```

listing 252: Rust, OnceLock::get_or_init takes the closure inline, no flag, no context pointer to install (source: patterns-concurrency-distributed/samples-rust/src/Ch07/once.rs, lines 18-49)

```go
func Lazy(raw string) func() int {
  return sync.OnceValue(func() int {
    n := 0
    for _, c := range raw {
      if c ≥ '0' && c <= '9' {
        n = n*10 + int(c-'0')
      }
    }
    return n
  })
}

// BoundedQueue is Cond in its natural habitat: producers sleep when
```

listing 253: Go, one parse, concurrent callers, no flag anywhere (source: patterns-concurrency-distributed/samples/ch07/syncdeep.go, lines 49-61)

```
  static final class OnceBox<V> {
    private final Supplier<V> fn;
    private final Object lock = new Object();
    private volatile boolean done;
    private V value;

    OnceBox(Supplier<V> fn) {
      this.fn = fn;
    }

    V call() {
      if (!done) {
        synchronized (lock) {
          if (!done) {
            value = fn.get();
            done = true;
          }
        }
      }
      return value;
    }
  }

  static final AtomicInteger parseCalls = new AtomicInteger();

  static long parseDigits(String raw) {
    parseCalls.incrementAndGet();
    long digits = 0;
    for (int i = 0; i < raw.length(); i++) {
      char c = raw.charAt(i);
      if (c ≥ '0' && c ≤ '9')
        digits = digits * 10 + (c - '0');
    }
    return digits;
  }
```

listing 254: Java, double-checked locking on a volatile flag, the value published before the flag flips (source: patterns-concurrency-distributed/samples-java/src/Ch07/Once.java, lines 24-58)

```
public sealed class DigitOnce
{
    private readonly Lazy<int> _lazy;
    private int _parses;

    public DigitOnce(string raw)
    {
        _lazy = new Lazy<int>(
            () =>
            {
                Interlocked.Increment(ref _parses);
                int n = 0;
                foreach (char c in raw)
                {
                    if (c is ≥ '0' and ≤ '9')
                    {
                        n = n * 10 + (c - '0');
                    }
                }
                return n;
            },
            LazyThreadSafetyMode.ExecutionAndPublication);
    }

    public int Value() => _lazy.Value;

    public int Parses => Volatile.Read(ref _parses);
}
```

listing 255: C#, Lazy with ExecutionAndPublication is the once family, the counter makes exactly-once observable (source: patterns-concurrency-distributed/samples-cs/src/Ch07/Once.cs, lines 6-33)

```
export function once(fn) {
  let done = false;
  let value;
  return () => {
    if (!done) {
      value = fn();
      done = true;
    }
    return value;
  };
}

// digitsOf concatenates every decimal digit in the input.
export function digitsOf(raw) {
  let n = 0;
  for (const c of raw) {
    if (c ≥ "0" && c ≤ "9") n = n * 10 + Number(c);
  }
  return n;
}

export function lazy(raw) {
  let calls = 0;
  const compute = () => {
    calls++;
    return digitsOf(raw);
  };
  const get = once(compute);
  get.parseCalls = () => calls;
  return get;
}
```

listing 256: JavaScript, the check-and-fill is one synchronous block no other task can enter, the memo closes the race by construction (source: patterns-concurrency-distributed/samples-js/src/ch07-once.mjs, lines 6-36)

```
class Once:
    def __init__(self, fn):
        self.fn = fn
        self.lock = threading.Lock()
        self.done = False
        self.value = None

    def call(self):
        with self.lock:
            if not self.done:
                self.value = self.fn()
                self.done = True
            return self.value


parse_calls = 0


def parse_digits(raw):
    global parse_calls
    parse_calls += 1
    digits = 0
    for c in raw:
        if "0" ≤ c ≤ "9":
            digits = digits * 10 + int(c)
    return digits


digits = Once(lambda: parse_digits("a1b22c333"))

check(digits.call() == 122333, "the lazy parse digs the digits out")
check(digits.call() == 122333, "and the second call reuses it")
check(parse_calls == 1, "the parse ran exactly once")
```

listing 257: Python, the double-checked flag lives under the lock, the value publishes before the flag flips (source: patterns-concurrency-distributed/samples-py/src/Ch07/once.py, lines 16-48)

```lua
local function Lazy(S, raw)
  local state, value = "idle", nil
  local calls = 0
  local waiters = {}
  local fn = function()
    if state == "done" then
      return value
    end
    if state == "running" then
      waiters[#waiters + 1] = coroutine.running()
      coroutine.yield("park")
      return value
    end
    state = "running"
    calls = calls + 1
    local n = 0
    for i = 1, #raw do
      local c = raw:byte(i)
      if c ≥ 48 and c ≤ 57 then
        coroutine.yield() -- the window
        n = n * 10 + (c - 48)
      end
    end
    value = n
    state = "done"
    for i = 1, #waiters do
      S.ready[#S.ready + 1] = waiters[i]
    end
    waiters = {}
    return value
  end
  return fn, function() return calls end
end
```

listing 258: Lua, a running state with a waiter list, callers that overlap the parse park and wake holding the value (source: patterns-concurrency-distributed/samples-lua/ch07_once.lua, lines 32-64)

Each tree spends the guarantee differently. Go's `OnceValue` ships the edge in the standard library, pre-generics go needed a page of boilerplate for it. C finds the direct standard analog in `call_once` and `once_flag`, paying one indirection, the init function reads its target through a pointer installed before any concurrent caller exists. Rust's `OnceLock::get_or_init` takes the initializer as a closure, so the parse closure captures the counter and the raw input directly and the indirection C pays does not exist. Java writes double-checked locking out longhand, the fast path reading a `volatile` flag and the slow path rechecking under the lock, publication safe because the flag's write is the last step. C# buys the whole thing off the shelf with `Lazy<T>` in `ExecutionAndPublication` mode, publication serialized by the lazy machinery itself. Javascript closes the window by construction, the memoizing check-and-fill never awaits, so no other task can enter it, and the 20 callers is a walk, not a race. Python writes the classic shape out longhand, flag and value under one lock, because that shape is the lesson. Lua has no preemption, so once is a cooperative protocol: the first caller flips to running, callers that arrive inside the parse window park as waiters and wake holding the finished value, which the scheduler drives deterministically.

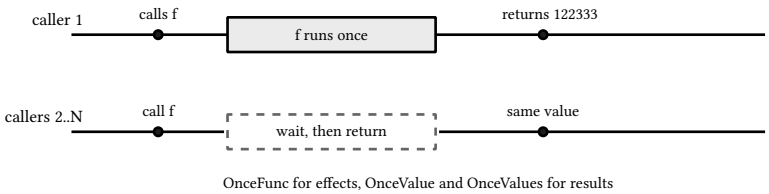


figure 38: one caller runs f, the rest wait and replay the value

7.3. cond, waiting for a predicate

`sync.Cond` is the primitive every hand rolled wait loop wants to be. A `Wait` atomically releases the lock and sleeps, a `Signal` wakes one waiter, `Broadcast` wakes all, and the rule that separates correct code from deadlocks is the recheck loop, because a wake means the predicate changed once, not that it is still true.

The dry run: capacity 3, puts 1, 2, 3 land, a fourth producer blocks until a take frees a slot, two consumers drain, close unblocks both, and the 4 puts total survive as the multiset 1, 2, 3, 4 summing to 10, with a take after close reporting false. C, rust, java, javascript, python, and lua pin the drained multiset 2, 3, 4, the go and C# lanes pin the sum and the closed take.

```c
static void bq_put(bqueue *q, int v) {
  mtx_lock(&q->mu);
  while (q->n == q->cap)
    cnd_wait(&q->not_full, &q->mu);
  q->items[q->n++] = v;
  cnd_signal(&q->not_empty);
  mtx_unlock(&q->mu);
}

static bool bq_take(bqueue *q, int *out) {
  mtx_lock(&q->mu);
  while (q->n == 0) {
    if (q->closed) {
      mtx_unlock(&q->mu);
      return false;
    }
    cnd_wait(&q->not_empty, &q->mu);
  }
  *out = q->items[0];
  for (int i = 1; i < q->n; i++)
    q->items[i - 1] = q->items[i];
  q->n--;
  cnd_signal(&q->not_full);
  mtx_unlock(&q->mu);
  return true;
}

static void bq_close(bqueue *q) {
  mtx_lock(&q->mu);
  q->closed = true;
  cnd_broadcast(&q->not_empty);
  mtx_unlock(&q->mu);
}
```

listing 259: C, two cnd_t and one mutex, while loops around every wait, broadcast on close (source: patterns-concurrency-distributed/samples-c/src/Ch07/cond.c, lines 47-79)

```
impl BQueue {
    fn new(cap: usize) -> BQueue {
        BQueue {
            state: Mutex::new(QState {
                items: VecDeque::new(),
                cap,
                closed: false,
            }),
            not_empty: Condvar::new(),
            not_full: Condvar::new(),
        }
    }

    fn len(&self) -> usize {
        self.state.lock().unwrap().items.len()
    }

    fn put(&self, v: i32) {
        let mut st = self.state.lock().unwrap();
        while st.items.len() == st.cap {
            st = self.not_full.wait(st).unwrap();
        }
        st.items.push_back(v);
        self.not_empty.notify_one();
    }

    fn take(&self) -> Option<i32> {
        let mut st = self.state.lock().unwrap();
        loop {
            if let Some(v) = st.items.pop_front() {
                self.not_full.notify_one();
                return Some(v);
            }
            if st.closed {
                return None;
            }
            st = self.not_empty.wait(st).unwrap();
        }
    }

    fn close(&self) {
        self.state.lock().unwrap().closed = true;
        self.not_empty.notify_all();
    }
}
```

listing 260: Rust, two Condvars over one Mutexed state, while loops around every wait, close notifies all (source: patterns-concurrency-distributed/samples-rust/src/Ch07/cond.rs, lines 28-72)

```
type BoundedQueue struct {
  mu       sync.Mutex
  notEmpty sync.Cond
  notFull  sync.Cond
  items    []int
  cap      int
  closed   bool
}

func NewBoundedQueue(capacity int) *BoundedQueue {
  q := &BoundedQueue{cap: capacity}
  q.notEmpty = *sync.NewCond(&q.mu)
  q.notFull = *sync.NewCond(&q.mu)
  return q
}

func (q *BoundedQueue) Put(v int) {
  q.mu.Lock()
  defer q.mu.Unlock()
  for len(q.items) == q.cap {
    q.notFull.Wait()
  }
  q.items = append(q.items, v)
  q.notEmpty.Signal()
}

func (q *BoundedQueue) Take() (int, bool) {
  q.mu.Lock()
  defer q.mu.Unlock()
  for len(q.items) == 0 {
    if q.closed {
      return 0, false
    }
    q.notEmpty.Wait()
  }
  v := q.items[0]
  q.items = q.items[1:]
  q.notFull.Signal()
  return v, true
}

func (q *BoundedQueue) Close() {
  q.mu.Lock()
  defer q.mu.Unlock()
  q.closed = true
  q.notEmpty.Broadcast()
}
```

listing 261: Go, bounded queue: two conds, one mutex, while loops around every wait (source: patterns-concurrency-distributed/samples/ch07/syncdeep.go, lines 64-111)

```
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notEmpty = lock.newCondition();
    private final Condition notFull = lock.newCondition();
    private final int[] items = new int[QITEMS];
    private int n;
    private boolean closed;

    void put(int v) throws InterruptedException {
      lock.lock();
      try {
        while (n == QCAP && !closed)
          notFull.await();
        if (closed)
          return;
        items[n++] = v;
        notEmpty.signalAll();
      } finally {
        lock.unlock();
      }
    }

    boolean take(int[] out) throws InterruptedException {
      lock.lock();
      try {
        while (n == 0) {
          if (closed)
            return false;
          notEmpty.await();
        }
        out[0] = items[0];
        for (int i = 1; i < n; i++)
          items[i - 1] = items[i];
        n--;
        notFull.signalAll();
        return true;
      } finally {
        lock.unlock();
      }
    }

    void close() {
      lock.lock();
      try {
        closed = true;
        notEmpty.signalAll();
      } finally {
        lock.unlock();
      }
    }
```

listing 262: Java, ReentrantLock.newCondition twice, one condition per predicate, the go layout native (source: patterns-concurrency-distributed/samples-java/src/Ch07/Cond.java, lines 26-74)

```
public sealed class BoundedQueue(int capacity)
{
    private readonly object _gate = new();
    private readonly List<int> _items = [];
    private bool _closed;

    public void Put(int v)
    {
        lock (_gate)
        {
            while (_items.Count == capacity)
            {
                Monitor.Wait(_gate);
            }
            _items.Add(v);
            Monitor.PulseAll(_gate);
        }
    }

    public bool Take(out int v)
    {
        lock (_gate)
        {
            while (_items.Count == 0)
            {
                if (_closed)
                {
                    v = 0;
                    return false;
                }
                Monitor.Wait(_gate);
            }
            v = _items[0];
            _items.RemoveAt(0);
            Monitor.PulseAll(_gate);
            return true;
        }
    }

    public void Close()
    {
        lock (_gate)
        {
            _closed = true;
            Monitor.PulseAll(_gate);
        }
    }
}
```

listing 263: C#, one monitor, one wait set, PulseAll everywhere because every sleeper rechecks its own predicate (source: patterns-concurrency-distributed/samples-cs/src/Ch07/Cond.cs, lines 8-55)

```
export class BoundedQueue {
  #items = [];
  #cap;
  #closed = false;
  #notEmpty = [];
  #notFull = [];

  constructor(capacity) {
    this.#cap = capacity;
  }

  async put(v) {
    while (this.#items.length === this.#cap) {
      await new Promise((resolve) => this.#notFull.push(resolve));
    }
    this.#items.push(v);
    const w = this.#notEmpty.shift();
    if (w) w();
  }

  async take() {
    for (;;) {
      if (this.#items.length > 0) {
        const v = this.#items.shift();
        const s = this.#notFull.shift();
        if (s) s();
        return { value: v, ok: true };
      }
      if (this.#closed) return { value: 0, ok: false };
      await new Promise((resolve) => this.#notEmpty.push(resolve));
    }
  }

  close() {
    this.#closed = true;
    for (const w of this.#notEmpty.splice(0)) w();
    for (const s of this.#notFull.splice(0)) s();
  }
}
```

listing 264: JavaScript, waiters are promise resolvers, signal wakes one, close resolves everyone on both sides (source: patterns-concurrency-distributed/samples-js/src/ch07-cond.mjs, lines 6-44)

```
class BoundedQueue:
    def __init__(self, capacity):
        self.lock = asyncio.Lock()
        self.not_empty = asyncio.Condition(self.lock)
        self.not_full = asyncio.Condition(self.lock)
        self.items = []
        self.cap = capacity
        self.closed = False

    async def put(self, v):
        async with self.lock:
            while len(self.items) == self.cap and not self.closed:
                await self.not_full.wait()
            if self.closed:
                return False
            self.items.append(v)
            self.not_empty.notify_all()
            return True

    async def take(self):
        async with self.lock:
            while not self.items:
                if self.closed:
                    return None, False
                await self.not_empty.wait()
            v = self.items.pop(0)
            self.not_full.notify_all()
            return v, True

    async def close(self):
        async with self.lock:
            self.closed = True
            self.not_empty.notify_all()
```

listing 265: Python, two asyncio conditions over one lock, the walk parks puts and consumers on queue state (source: patterns-concurrency-distributed/samples-py/src/Ch07/cond.py, lines 17-49)

```lua
function BoundedQueue.new(S, capacity)
  return setmetatable({
    S = S, items = {}, cap = capacity, closed = false,
    notEmptyWaiters = {}, notFullWaiters = {},
  }, BoundedQueue)
end

function BoundedQueue:Put(v)
  while #self.items == self.cap do
    self.notFullWaiters[#self.notFullWaiters + 1] = coroutine.running()
    coroutine.yield("park")
  end
  self.items[#self.items + 1] = v
  self:signalOne(self.notEmptyWaiters)
end

function BoundedQueue:Take()
  while #self.items == 0 do
    if self.closed then
      return nil, false
    end
    self.notEmptyWaiters[#self.notEmptyWaiters + 1] = coroutine.running()
    coroutine.yield("park")
    -- woken: loop and recheck, the wake is not the answer
  end
  local v = table.remove(self.items, 1)
  self:signalOne(self.notFullWaiters)
  return v, true
end

function BoundedQueue:Close()
  self.closed = true
  for _, co in ipairs(self.notEmptyWaiters) do
    self.S.ready[#self.S.ready + 1] = co
  end
  self.notEmptyWaiters = {}
end

function BoundedQueue:signalOne(waiters)
  local co = table.remove(waiters, 1)
  if co then
    self.S.ready[#self.S.ready + 1] = co
  end
end
```

listing 266: Lua, waiter lists per predicate, park is a yield to the scheduler, Close wakes every parked consumer (source: patterns-concurrency-distributed/samples-lua/ch07_cond.lua, lines 47-90)

`BoundedQueue` is the canonical shape, producers sleep on `notFull`, consumers on `notEmpty`, both conds share the queue's mutex because the predicates read the same state. `Close` broadcasts so every sleeping consumer wakes, rechecks, sees the closed flag, and exits. The scripted walks drive the interesting interleavings directly: the producer past capacity parks until a take frees a slot, then two consumers drain everything and unblock on close, and the accounting lands on exactly the values that were ever put.

The eight idioms all enforce the same wake discipline, that a wake only means look again. C spends `cnd_wait` with the mutex held and rechecks in a while loop, and the same zero-waiter forgetfulness that makes a naked signal a losing bet is documented for win32 condition variables in the systems book, book 1, ch 15 (c23: the language,

the machine, and the cloud, threads and synchronization). Rust's `Condvar::wait` hands the guard back as its return value, so the recheck loop re-acquires through the type, and its two-predicate shape is two `Condvars` over one `Mutex<QState>`. Java builds the two-condition shape native, `ReentrantLock.newCondition` called twice, one wait set per predicate, so its `signalAll` reaches only the sleepers that share the waker's predicate. C# has one wait set per monitor where go used two conds, so every wake is `PulseAll` and every sleeper rechecks its own predicate, the queue works with one wait set because the predicates share one lock. Javascript parks waiters as promise resolvers, signal resolves one, close resolves everyone, and the recheck is the loop the await sits in. Python's `threading.Condition` in its threaded form is the near 1:1 mirror of go's cond, wait, notify, notify_all under one lock, the asyncio form the sample rides swaps the parking for the event loop, book 8, ch 17 (python 3.14, threads and the gil) covers both. Lua parks the running coroutine on a waiter list the scheduler owns, and close walks the list back into the ready queue.

The honest comparison: channels can express this queue too, and for a queue, channels usually win on clarity. `Cond` earns its keep when the wakeup condition is a predicate over shared state, with no arriving value to hand off, and the pool of waiters must re-evaluate it.

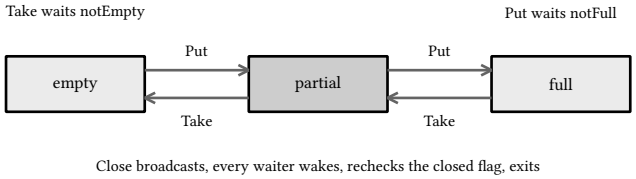


figure 39: queue states, waits at the edges, wake means look again

7.4. sync.map

`sync.Map` is not a faster map, it is a map for two specific access shapes: write-once-read-many caches with disjoint keys, and append-only sets. Its superpower over `Mutex` plus `map` is the atomic read-mostly path plus fused operations like `LoadOrStore`, which answer whether this caller created the entry.

The dry run: one key through 1000 racing canonical calls creates exactly once, every caller walks away holding the same canonical value, and a distinct key creates a second time. All eight trees pin created 1 on the 1000, the siblings also pin the identity of the returned handle.

```c
static const char *canonical(interned *in, const char *key) {
  mtx_lock(&in->mu);
  for (int i = 0; i < in->n; i++)
    if (strcmp(in->canon[i], key) == 0) {
      const char *hit = in->canon[i];
      mtx_unlock(&in->mu);
      return hit;
    }
  size_t z = strlen(key) + 1;
  char *stored = malloc(z);
  memcpy(stored, key, z);
  if (in->n < SLOTS) {
    in->canon[in->n++] = stored;
  } else {
    free(stored);
    stored = NULL;
  }
  mtx_unlock(&in->mu);
  atomic_fetch_add_explicit(&in->created, 1, memory_order_relaxed);
  return stored;
}
```

listing 267: C, find-or-insert under the mutex by hand, only the caller that performed the store counts as the creator (source: patterns-concurrency-distributed/samples-c/src/Ch07/intern.c, lines 48-68)

```rust
struct Interned {
    canon: Mutex<Vec<Arc<String>>>,
    created: AtomicI64,
}

impl Interned {
    fn new() -> Interned {
        Interned {
            canon: Mutex::new(Vec::new()),
            created: AtomicI64::new(0),
        }
    }

    fn canonical(&self, key: &str) -> Option<Arc<String>> {
        let mut canon = self.canon.lock().unwrap();
        for c in canon.iter() {
            if c.as_str() == key {
                return Some(Arc::clone(c));
            }
        }
        if canon.len() ≥ SLOTS {
            return None;
        }
        let stored = Arc::new(key.to_string());
        canon.push(Arc::clone(&stored));
        drop(canon);
        self.created.fetch_add(1, Ordering::Relaxed);
        Some(stored)
    }

    fn created(&self) -> i64 {
        self.created.load(Ordering::Relaxed)
    }
}
```

listing 268: Rust, find-or-install under the mutex, an Arc of String the handle, `Arc::ptr_eq` the identity (source: patterns-concurrency-distributed/samples-rust/src/Ch07/intern.rs, lines 20-53)

```go
type Interned struct {
  m       sync.Map
  created atomic.Int64
}

func (in *Interned) Canonical(key string) string {
  if v, loaded := in.m.LoadOrStore(key, key); loaded {
    return v.(string)
  }
  in.created.Add(1)
  return key
}

func (in *Interned) Created() int64 { return in.created.Load() }

// CasMax raises a shared maximum with a compare-and-swap loop: read,
// compute, try to install, retry if someone moved it first.
func CasMax(a *atomic.Int64, candidate int64) {
```

listing 269: Go, load or store reports the winner (source: patterns-concurrency-distributed/samples/ch07/syncdeep.go, lines 114-131)

```
  static final class Interned {
    private final ConcurrentHashMap<String, String> m = new ConcurrentHashMap<>();
    private final AtomicInteger created = new AtomicInteger();

    String canonical(String key) {
      return m.computeIfAbsent(key, k -> {
        created.incrementAndGet();
        return k;
      });
    }

    int distinct() {
      return m.size();
    }
  }
```

listing 270: Java, computeIfAbsent is the native load-or-store, the mapping function runs at most once per key (source: patterns-concurrency-distributed/samples-java/src/Ch07/Intern.java, lines 26-40)

```
public sealed class Interned
{
    private readonly ConcurrentDictionary<string, string> _m = new();
    private long _created;

    public string Canonical(string key)
    {
        if (_m.TryAdd(key, key))
        {
            Interlocked.Increment(ref _created);
            return key;
        }
        return _m[key];
    }

    public long Created() => Interlocked.Read(ref _created);
}
```

listing 271: C#, ConcurrentDictionary.TryAdd is LoadOrStore, the winner is the only caller that counts as a creation (source: patterns-concurrency-distributed/samples-cs/src/Ch07/Intern.cs, lines 8-24)

```
export function loadOrStore(m, key) {
  if (m.has(key)) {
    return { value: m.get(key), loaded: true };
  }
  m.set(key, key);
  return { value: key, loaded: false };
}

export class Interned {
  #m = new Map();
  #created = 0;

  canonical(key) {
    const { value, loaded } = loadOrStore(this.#m, key);
    if (!loaded) this.#created++;
    return value;
  }

  created() {
    return this.#created;
  }
}
```

listing 272: JavaScript, loadOrStore as one synchronous block, the loaded flag is the winner report (source: patterns-concurrency-distributed/samples-js/src/ch07-intern.mjs, lines 6-27)

```
class Interned:
    def __init__(self):
        self.lock = threading.Lock()
        self.m = {}
        self.created = 0

    def canonical(self, key):
        with self.lock:
            if key in self.m:
                return self.m[key]
            self.created += 1
            self.m[key] = key
            return key

    def distinct(self):
        with self.lock:
            return len(self.m)
```

listing 273: Python, check-then-install under the lock, the created counter increments only on the insert path (source: patterns-concurrency-distributed/samples-py/src/Ch07/intern.py, lines 16-32)

```lua
local function Interned()
  local in_ = { m = {}, created = 0 }
  -- LoadOrStore as one step: the loser of the race reads the winner's
  -- value, only the winner counts a creation
  function in_:Canonical(key)
    local v = self.m[key]
    if v == nil then
      self.created = self.created + 1
      v = { key = key }
      self.m[key] = v
    end
    return v
  end
  return in_
end

-- the torn version: check, yield, install. every lane that overlaps
-- the window believes it created the entry
local function TornInterned()
  local in_ = { m = {}, created = 0 }
  function in_:Canonical(key)
    local v = self.m[key]
    if v == nil then
      coroutine.yield() -- the race window
      self.created = self.created + 1
      v = { key = key }
      self.m[key] = v
    end
    return v
  end
  return in_
end
```

listing 274: Lua, the lookup-or-install as one non-yielding step, and the torn check-yield-store that loses it (source: patterns-concurrency-distributed/samples-lua/ch07_intern.lua, lines 30-61)

The fused check-then-act is the whole value: the same code with a mutex map needs the lock held across the check, and the same code with a plain map is a race. C writes find-or-insert under the mutex and returns the canonical pointer, only the inserting caller counts a creation, which is `LoadOrStore` disassembled to its parts. Rust performs the same disassembly with an `Arc<String>` per entry, the identity pinned by `Arc::ptr_eq` across all thousand callers, and the handle outliving the lock because its count says who holds it. Java's `computeIfAbsent` is the fused operation itself, the mapping function guaranteed at most one run per key, and the file pins the identity with == across 1000 concurrent callers each passing a freshly built string. C# reaches for `ConcurrentDictionary.TryAdd`, the bcl row of the same contract, and the wider concurrent-collection surface it belongs to is the stdlib tour's territory, book 6, ch 16 (c# 15, f# 11, and .net 11, stdlib tour 2: io, text, net, channels, diagnostics). Javascript has one thread per realm, so check-then-set inside one synchronous block is already atomic and the lane says so, no concurrent variant exists to build. Python runs the same shape under its lock, the GIL would make the plain dict version mostly safe but the lock is what makes it stated. Lua demonstrates both sides: the non-yielding canonical wins exactly once across 10 interleaved lanes, and the torn version that yields between check and store hands every lane its own creation, 10 winners where 1 is correct.

Outside those two shapes sync.Map loses to the guarded map, the builtin map got swiss tables in go 1.24 and the spread only widened.

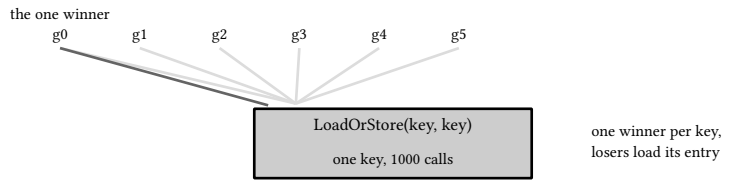


figure 40: 1000 racing calls, exactly one created = true

7.5. atomics, beyond the counter

Chapter 6 used atomics as a counter. The general form is the compare-and-swap loop, read, compute, try to install, retry if the world moved.

The dry run: 64 racers each offer their racer number times 7 and the shared maximum lands on the true max 448 in every tree, the loop terminates because every failed exchange means someone else installed a larger or equal candidate. The scripted lanes pin forced retries, 2 failed exchanges in C, rust, java, and C#, 1 scripted move in javascript, 2 lost installs over 3 loads in python, 3 retries in lua. A candidate at or below the current value returns without ever attempting the swap.

```c
static void cas_max(_Atomic long long *a, long long candidate) {
  for (;;) {
    long long cur = atomic_load_explicit(a, memory_order_relaxed);
    if (candidate ≤ cur)
      return;
    if (atomic_compare_exchange_strong(a, &cur, candidate))
      return;
  }
}

// the scripted lane: an interloper raises the target exactly `bumps`
// times, each strictly between our load and our compare_exchange, so
// the loop fails exactly `bumps` times before it succeeds
static long long cas_max_scripted(_Atomic long long *a, long long candidate,
                                  int bumps, int *retries) {
  *retries = 0;
  for (;;) {
    long long cur = atomic_load_explicit(a, memory_order_relaxed);
    if (candidate ≤ cur)
      return cur;
    if (bumps > 0) {
      atomic_fetch_add_explicit(a, 10, memory_order_relaxed);
      bumps--;
      (*retries)++;
      continue;
    }
    long long expected = cur;
    if (atomic_compare_exchange_strong(a, &expected, candidate))
      return candidate;
    (*retries)++;
  }
}
```

listing 275: C, atomic_compare_exchange_strong in the loop, the scripted interloper raises the target between load and exchange (source: patterns-concurrency-distributed/samples-c/src/Ch07/cas.c, lines 21-52)

```rust
fn cas_max(a: &AtomicI64, candidate: i64) {
    let mut cur = a.load(Ordering::Relaxed);
    while candidate > cur {
        match a.compare_exchange(cur, candidate, Ordering::SeqCst, Ordering::Relaxed) {
            Ok(_) => return,
            Err(actual) => cur = actual,
        }
    }
}

fn cas_max_scripted(a: &AtomicI64, candidate: i64, mut bumps: i32) -> (i64, i64, i32) {
    let mut retries = 0;
    loop {
        let cur = a.load(Ordering::Relaxed);
        if candidate ≤ cur {
            return (cur, cur, retries);
        }
        if bumps > 0 {
            a.fetch_add(10, Ordering::Relaxed);
            bumps -= 1;
            retries += 1;
            continue;
        }
        match a.compare_exchange(cur, candidate, Ordering::SeqCst, Ordering::Relaxed) {
            Ok(_) => return (candidate, candidate, retries),
            Err(_) => retries += 1,
        }
    }
}
```

listing 276: Rust, compare_exchange in the loop, the Err hands back the observed value so the retry needs no re-read (source: patterns-concurrency-distributed/samples-rust/src/Ch07/cas.rs, lines 17-45)

```go
func CasMax(a *atomic.Int64, candidate int64) {
  for {
    cur := a.Load()
    if candidate <= cur {
      return
    }
    if a.CompareAndSwap(cur, candidate) {
      return
    }
  }
}
```

listing 277: Go, cas loop raising a shared maximum (source: patterns-concurrency-distributed/samples/ch07/syncdeep.go, lines 131-142)

```
  static final class Cell {
    long value;

    static final VarHandle VALUE;

    static {
      try {
        VALUE = MethodHandles.lookup().findVarHandle(Cell.class, "value", long.class);
      } catch (ReflectiveOperationException e) {
        throw new ExceptionInInitializerError(e);
      }
    }
  }

  static void casMax(Cell cell, long candidate) {
    for (;;) {
      long cur = (long) Cell.VALUE.get(cell);
      if (candidate ≤ cur)
        return;
      if (Cell.VALUE.compareAndSet(cell, cur, candidate))
        return;
    }
  }

  // the scripted lane: an interloper raises the target exactly `bumps`
  // times, each strictly between our load and our compare-and-set, so
  // the loop fails exactly `bumps` times before it succeeds
  static long casMaxScripted(Cell cell, long candidate, int bumps, int[] retries) {
    retries[0] = 0;
    for (;;) {
      long cur = (long) Cell.VALUE.get(cell);
      if (candidate ≤ cur)
        return cur;
      if (bumps > 0) {
        Cell.VALUE.getAndAdd(cell, 10);
        bumps--;
        retries[0]++;
        continue;
      }
      if (Cell.VALUE.compareAndSet(cell, cur, candidate))
        return candidate;
      retries[0]++;
    }
  }
```

listing 278: Java, VarHandle.compareAndSet in the loop, the scripted interloper raising the target between load and install (source: patterns-concurrency-distributed/samples-java/src/Ch07/Cas.java, lines 22-65)

```
// CompareAndSwap so a scripted test can force retries.
public class AtomicLong(long initial)
{
    private long _n = initial;

    public virtual long Load() => Interlocked.Read(ref _n);

    public virtual bool CompareAndSwap(long expected, long next) =>
        Interlocked.CompareExchange(ref _n, next, expected) == expected;

    public void Store(long v) => Interlocked.Exchange(ref _n, v);
}

public static class Cas
{
    // CasMax raises a shared maximum with a compare-and-swap loop:
    // read, compute, try to install, retry if someone moved it first.
    public static void Max(AtomicLong a, long candidate)
    {
        while (true)
        {
            long cur = a.Load();
            if (candidate ≤ cur)
            {
                return;
            }
            if (a.CompareAndSwap(cur, candidate))
            {
                return;
            }
        }
    }
}
```

listing 279: C#, Interlocked.CompareExchange behind a virtual swap so a scripted test can force retries (source: patterns-concurrency-distributed/samples-cs/src/Ch07/Cas.cs, lines 4-36)

```
export class ScriptedCell extends AtomicCell {
  #script;

  constructor(v, script) {
    super(v);
    this.#script = [...script];
  }

  // each scripted move pretends another racer installed it first:
  // the swap fails and the caller must re-read.
  compareExchange(expected, next) {
    const move = this.#script.shift();
    if (move !== undefined) {
      this.value = move;
      this.swapFails++;
      return false;
    }
    return super.compareExchange(expected, next);
  }
}

export function casMax(cell, candidate) {
  for (;;) {
    const cur = cell.load();
    if (candidate <= cur) return;
    if (cell.compareExchange(cur, candidate)) return;
  }
}
```

listing 280: JavaScript, the scripted cell moves the value on cue so the swap fails and the caller must re-read (source: patterns-concurrency-distributed/samples-js/src/ch07-cas.mjs, lines 27-54)

```
def scripted_racer(target):
    rig = Rigged(0, [5, 7])
    loads = 0
    failed = 0
    while True:
        cur = rig.load()
        loads += 1
        if target <= cur:
            return loads, failed, rig.v
        rig.interfere()
        if rig.try_install(cur, target):
            return loads, failed, rig.v
        failed += 1


loads, failed, final = scripted_racer(10)
check(loads == 3, "the scripted lane pins 3 loads")
check(failed == 2, "two installs lost the race")
check(final == 10, "the third install wins")

stale_loads, stale_failed, unchanged = scripted_racer(0)
check(stale_loads == 1, "a candidate at or below current costs one load")
check(stale_failed == 0 and unchanged == 0, "and never installs")
```

listing 281: Python, the rigged cell sabotages between load and install, the retry count is the pinned lesson (source: patterns-concurrency-distributed/samples-py/src/Ch07/cas.py, lines 37-59)

```lua
local function cas(a, cur, cand)
  if a.n == cur then
    a.n = cand
    return true
  end
  return false
end

-- the CAS loop: read, try to install, retry on a moved value. the
-- retry counter and the optional cooperative window are injected so
-- the same loop serves both lanes
local function CasMax(a, candidate, slow, retries)
  while true do
    local cur = a.n
    if candidate ≤ cur then
      return
    end
    if slow then
      coroutine.yield()
    end
    if cas(a, cur, candidate) then
      return
    end
    if retries then
      retries.n = retries.n + 1
    end
  end
end
```

listing 282: Lua, the loop with an injected cooperative window, retries counted only when the lane is slow (source: patterns-concurrency-distributed/samples-lua/ch07_cas.lua, lines 30-57)

The lanes earn the loop's atomicity differently. C and C# hold the real primitive, `atomic_compare_exchange_strong` and `Interlocked.CompareExchange`, and pin the retry count by moving the target inside the window, an interloper in C, a flaky subclass in C#. Rust holds it too, and its `compare_exchange` failure returns the observed value inside the `Err`, so the retry's re-read is the match arm itself. Go writes the same loop over `atomic.Int64`. Java holds the primitive twice over, `VarHandle.compareAndSet` over a plain field for the ladder lane and `AtomicLong` for the 64 racers, its scripted interloper built exactly like C's. Javascript has `Atomics.compareExchange` for shared-array-buffer cells, and this lane's cell is plain, atomic by event-loop construction, with a scripted subclass that pretends another racer installed first. Python exposes no public compare-and-swap on ints, so the honest row runs the identical loop under a lock and says the GIL is the cas, the scripted rig still measures the retries. Lua has no atomics at all, the cooperative lane injects the yield between load and swap and counts the retries it causes, the lesson made visible where other languages prevent it. The shape fails everywhere for the same reasons: when the retry can livelock or the invariant spans two words, it is a mutex wearing a disguise.

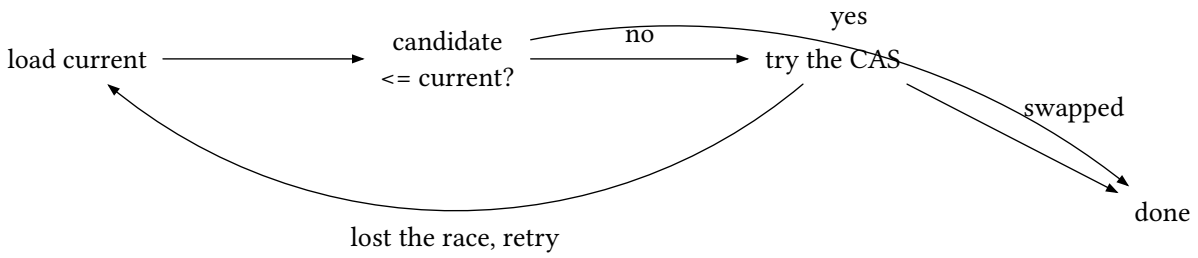


figure 41: read, compute, try to install, retry if the world moved

7.6. waitgroup and the error group

`WaitGroup.Go`, added in go 1.25, folds the add, the goroutine, and the done into one call and removes the classic miscount where `Add` runs after `Wait` has already passed zero. The pinned error group keeps the classic `wg.Add` + `go` + `defer wg.Done` deliberately: its tasks return errors, `WaitGroup.Go` accepts a plain `func()`, so each task would need a wrapping closure anyway, and the three-line form puts the error append where the goroutine starts. Under the same wait the pattern grows an error slice, and this sample closes with a compact error group.

The dry run: a clean task, a task failing alpha, a task failing beta. Waiting joins both names into one error, the first-error view matches, and a clean group waits out to nothing. C, rust, java, C#, python, and lua pin alpha as the first error through a causal edge, a second condvar the beta task waits on, a first-error latch, a submission-order slot, gather order, and a scripted yield, the go and javascript lanes accept either failure as first because their landing order is genuinely free.

```c
static int task_thread(void *arg) {
  struct task_wrap *w = arg;
  const char *err = w->task(w->arg);
  mtx_lock(&w->g->mu);
  if (err != NULL && w->g->n_errs < MAXERRS)
    w->g->errs[w->g->n_errs++] = err;
  w->g->running--;
  cnd_broadcast(&w->g->done_cv);
  cnd_broadcast(&w->g->err_cv);
  mtx_unlock(&w->g->mu);
  return 0;
}

// spawn one thread per task, exactly the go lane's Go. the wrap outlives
// the thread start because the group frees it only after wait
static bool eg_go(errgroup *g, task_fn task, void *arg) {
  struct task_wrap *w = malloc(sizeof *w);
  w->g = g;
  w->task = task;
  w->arg = arg;
  mtx_lock(&g->mu);
  g->running++;
  bool stored = g->n_wraps < MAXTASKS;
  if (stored)
    g->wraps[g->n_wraps++] = w;
  mtx_unlock(&g->mu);
  thrd_t t;
  if (thrd_create(&t, task_thread, w) != thrd_success) {
    mtx_lock(&g->mu);
    g->running--;
    cnd_broadcast(&g->done_cv);
    mtx_unlock(&g->mu);
    if (!stored)
      free(w);
    else
      g->n_wraps--;
    return false;
  }
  thrd_detach(t);
  return true;
}

static void eg_wait(errgroup *g) {
  mtx_lock(&g->mu);
  while (g->running > 0)
    cnd_wait(&g->done_cv, &g->mu);
  mtx_unlock(&g->mu);
}
```

listing 283: C, one thread per task, the done and error condvars, and the causal edge that makes the second failure observably second (source: patterns-concurrency-distributed/samples-c/src/Ch07/errgroup.c, lines 63-110)

```
    fn go(self: &Arc<Self>, task: impl FnOnce() -> Option<&'static str> + Send + 'static) {
        self.state.lock().unwrap().running += 1;
        let g = Arc::clone(self);
        std::thread::spawn(move || {
            let err = task();
            let mut st = g.state.lock().unwrap();
            if let Some(e) = err {
                st.errs.push(e);
            }
            st.running -= 1;
            g.done_cv.notify_all();
            g.err_cv.notify_all();
        });
    }

    fn wait(&self) {
        let mut st = self.state.lock().unwrap();
        while st.running > 0 {
            st = self.done_cv.wait(st).unwrap();
        }
    }

    fn wait_for_error(&self) {
        let mut st = self.state.lock().unwrap();
        while st.errs.is_empty() {
            st = self.err_cv.wait(st).unwrap();
        }
    }

    fn joined(&self) -> Option<String> {
        self.wait();
        let st = self.state.lock().unwrap();
        if st.errs.is_empty() {
            return None;
        }
        Some(st.errs.join("; "))
    }

    fn first(&self) -> Option<&'static str> {
        self.wait();
        self.state.lock().unwrap().errs.first().copied()
    }
}
```

listing 284: Rust, Arc over the group, a thread per task, the err_cv the beta task parks on is the causal edge (source: patterns-concurrency-distributed/samples-rust/src/Ch07/errgroup.rs, lines 38-80)

```
type ErrGroup struct {
  wg   sync.WaitGroup
  mu   sync.Mutex
  errs []error
}

func (g *ErrGroup) Go(task func() error) {
  g.wg.Add(1)
  go func() {
    defer g.wg.Done()
    if err := task(); err != nil {
      g.mu.Lock()
      g.errs = append(g.errs, err)
      g.mu.Unlock()
    }
  }()
}

func (g *ErrGroup) Wait() error {
  g.wg.Wait()
  g.mu.Lock()
  defer g.mu.Unlock()
  return errors.Join(g.errs...)
}

func (g *ErrGroup) WaitFirst() error {
  g.wg.Wait()
  g.mu.Lock()
  defer g.mu.Unlock()
  if len(g.errs) == 0 {
    return nil
  }
  return g.errs[0]
```

listing 285: Go, go and wait, join every error or surface the first (source: patterns-concurrency-distributed/samples/ch07/syncdeep.go, lines 145-177)

```
  static final class Group {
    private final List<String> errs = new ArrayList<>();
    private final CountDownLatch firstError = new CountDownLatch(1);

    void go(ExecutorService pool, Task task) {
      pool.submit(() -> {
        String err = task.run();
        synchronized (this) {
          if (err != null) {
            errs.add(err);
            firstError.countDown();
          }
        }
        return (Void) null;
      });
    }

    // the causal edge: the second failure waits until some error has
    // landed, so the first recorded error is deterministic
    void waitForError() throws InterruptedException {
      firstError.await();
    }

    synchronized List<String> errs() {
      return new ArrayList<>(errs);
    }
  }

  static String waitJoined(List<String> errs) {
    if (errs.isEmpty())
      return null;
    return String.join("; ", errs);
  }

  static String waitFirst(List<String> errs) {
    if (errs.isEmpty())
      return null;
    return errs.get(0);
  }
```

listing 286: Java, tasks on the virtual-thread executor, pool.close the wait edge, a first-error latch behind the group (source: patterns-concurrency-distributed/samples-java/src/Ch07/Errgroup.java, lines 30-68)

```
public sealed class ErrGroup
{
    private readonly List<Task> _tasks = [];
    private readonly object _gate = new();
    private string?[] _errs = [];
    private int _slots;

    public void Go(Func<string?> task)
    {
        int slot = _slots++;
        lock (_gate)
        {
            if (_errs.Length < _slots)
            {
                Array.Resize(ref _errs, _slots);
            }
        }
        _tasks.Add(Task.Run(() =>
        {
            string? err = task();
            lock (_gate)
            {
                _errs[slot] = err;
            }
        }));
    }

    public async Task<string?> WaitAsync()
    {
        await Task.WhenAll(_tasks);
        lock (_gate)
        {
            return JoinErrors();
        }
    }
```

listing 287: C#, a slot per task at Go time, WhenAll is the barrier, both views deterministic in submission order (source: patterns-concurrency-distributed/samples-cs/src/Ch07/Errgroup.cs, lines 7-41)

```
export class ErrGroup {
  #inflight = [];
  #errs = [];

  go(task) {
    const p = (async () => {
      try {
        const err = await task();
        if (err instanceof Error) this.#errs.push(err);
      } catch (e) {
        this.#errs.push(e);
      }
    })();
    this.#inflight.push(p);
  }

  async #settled() {
    await Promise.all(this.#inflight);
  }

  async wait() {
    await this.#settled();
    if (this.#errs.length === 0) return null;
    return new Error(this.#errs.map((e) => e.message).join("\n"));
  }

  async waitFirst() {
    await this.#settled();
    if (this.#errs.length === 0) return null;
    return this.#errs[0];
  }
}
```

listing 288: JavaScript, tasks are promises, allSettled semantics by hand, the joined error one per line (source: patterns-concurrency-distributed/samples-js/src/ch07-errgroup.mjs, lines 5-36)

```python
class GroupError(Exception):
    # the errors.Join analog: one error carrying every failure
    def __init__(self, errors):
        self.errors = list(errors)
        super().__init__("\n".join(str(e) for e in self.errors))


class ErrGroup:
    def __init__(self):
        self.tasks = []

    def go(self, coro_fn):
        self.tasks.append(asyncio.ensure_future(coro_fn()))

    async def wait(self):
        results = await asyncio.gather(*self.tasks, return_exceptions=True)
        errs = [r for r in results if isinstance(r, Exception)]
        return GroupError(errs) if errs else None

    async def wait_first(self):
        results = await asyncio.gather(*self.tasks, return_exceptions=True)
        for r in results:
            if isinstance(r, Exception):
                return r
        return None
```

listing 289: Python, gather with return_exceptions keeps submission order, GroupError is the errors.Join analog (source: patterns-concurrency-distributed/samples-py/src/Ch07/errgroup.py, lines 17-42)

```lua
local function ErrGroup(S)
  local g = { S = S, wg = 0, errs = {}, waiters = {} }
  function g:Go(task)
    self.wg = self.wg + 1
    spawn(S, function()
      local err = task()
      if err then
        self.errs[#self.errs + 1] = err
      end
      self.wg = self.wg - 1
      if self.wg == 0 then
        for _, co in ipairs(self.waiters) do
          self.S.ready[#self.S.ready + 1] = co
        end
        self.waiters = {}
      end
    end)
  end
  function g:Wait()
    if self.wg > 0 then
      self.waiters[#self.waiters + 1] = coroutine.running()
      coroutine.yield("park")
    end
    if #self.errs == 0 then
      return nil
    end
    return table.concat(self.errs, "\n")
  end
  function g:WaitFirst()
    if self.wg > 0 then
      self.waiters[#self.waiters + 1] = coroutine.running()
      coroutine.yield("park")
    end
    return self.errs[1]
  end
  return g
end
```

listing 290: Lua, the last Done wakes the waiter, failures keep their landing order, the walk scripts it (source: patterns-concurrency-distributed/samples-lua/ch07_errgroup.lua, lines 33-69)

Every lane builds the same 3 parts, spawn, barrier, collect, and differs in what makes the first error deterministic. Go and javascript race the landing, so their honest assertion accepts either failure as first. C makes the order causal, the beta task blocks until alpha's error has landed, an edge in place of a sleep. Rust makes it causal C's way, one `Arc<ErrGroup>` shared into the spawned tasks, the beta task parked on the group's error condvar until the first failure lands. Java plays the same trick with a `CountDownLatch`, its beta task waiting for the first error to land, and its barrier is `pool.close()`, the virtual-thread executor waiting out every submitted task. C# assigns each task its slot at submission, so the collected array is ordered no matter how the pool schedules. Python's `gather` preserves submission order in its results even with `return_exceptions`, and lua scripts the yields so the landing order is part of the fixture. The x/sync `errgroup` packages go's version with context cancellation, and the capstone's test harness will reuse this shape to run a 5 node cluster to quiescence.

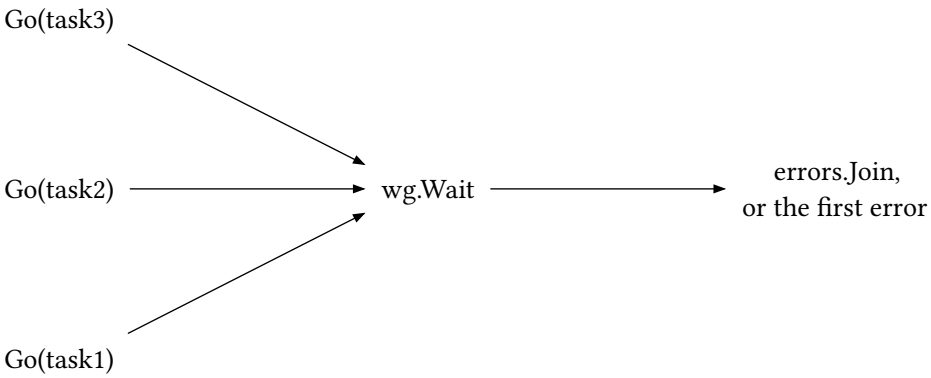


figure 42: spawn per task, one barrier, every error joined

PITFALL: pool

`sync.Pool` round trips temporary objects, buffers mostly, and two facts bound its usefulness: the GC may drop pool contents at any allocation cycle, so a pool is a cache, never a store, and `Put` hands out no guarantee the same object comes back. Pools earn their keep inside hot allocation loops measured by chapter 9′s profiling, not as a general object recycling facility. The idea needs a garbage collector to be safe, so the other seven lanes have nothing to pool: C would be a free list you manage, rust's std ships no pool type either, java's jdk ships no pool type of its own, and the dynamic three hand allocation to their runtimes.

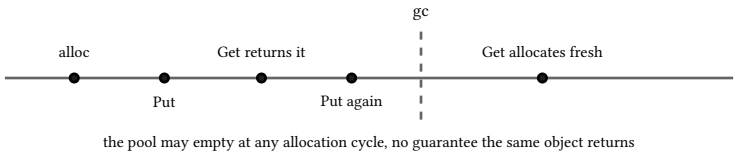


figure 43: a pool lives between gc cycles, a cache and never a store

7.7. across the eight languages

The build sizes count non-comment source lines, the go column the chapter's one frozen file:

language	build SLOC	dependency	boundary note
c	677	libc plus threads.h, stdatomic.h	one mutex flavor so the rw split collapses, call_once reads through a pointer
rust	525	std only, no crates	OnceLock carries the initializer inline, two Condvars per queue, `Arc` identity for the intern handles, `compare_exchange` errors handing back the observed value
go	142	stdlib	frozen reference lane, OnceValue off the shelf, errgroup keeps classic wg.Add
java	517	jdk 27 stdlib	double-checked locking on a volatile flag, two conditions from newCondition, computeIfAbsent as the load-or-store
c#	236	bcl	Lazy in ExecutionAndPublication, TryAdd is LoadOrStore, a virtual swap for cas
javascript	179	node stdlib	one thread per realm, underLock chains a promise tail, the memo never awaits
python	341	stdlib only	double-checked flag under the lock, the GIL is the cas, gather preserves order
lua	777	lib.lua harness	the lock nominal, once a waiter-list protocol, the cas window an injected yield

sources: go.dev/pkg/sync for `RWMutex`, the `Once` family, `Cond`, `Map`, `Pool`, and `WaitGroup.Go` including its memory model note, go.dev/pkg/sync/atomic for CAS semantics, go.dev/ref/mem for the Once and mutex edges quoted, accessed 2026-09-08. Verified by the eight chapter legs: 6 Ch07 C programs with 339 embedded checks, 6 Ch07 rust programs in `samples-rust/src` with 33 embedded checks under `run-rust-samples`, `go test` at 6 tests in `patternsbook/ch07`, 6 Ch07 java programs with 42 checks under `run-java-samples`, 14 xunit facts, node's 13 cases in `test/ch07.test.mjs`, 6 python modules with 43 embedded checks, and the lua runner's 25 ch07 rows.

8. concurrency patterns

The primitives compose into a small set of recurring structures, and code that feels idiomatic in a language is code using those structures instead of inventing per-file choreography. This chapter builds the five that carry most production load, generator, pipeline stage, fan-in, worker pool, and bounded parallel map, plus the racing-first shape with a deadline, all threaded through a cancellation carrier, all proven leak free by each tree's own instrument.

8.1. cancellation first

Every pattern in this chapter takes a stop signal as its first argument, and the reason is structural: a worker that cannot be told to stop is a worker that will, someday, be a leak. Go gives the signal a tree shape, cancel a parent and the children observe it, and the observation point is the same select everywhere:

```
select {
case out <- v: // hand off
case <-ctx.Done(): // stop everything
}
```

fragment, illustrative, not executed

Since go 1.20 a canceled context carries a cause, `WithCancelCause` plus `context.Cause`, so pipeline stages can distinguish "user hung up" from "deadline exceeded" from "shutting down", and `AfterFunc` registers post-cancellation work without a goroutine parked in select. The sample sticks to plain cancellation, which is the common case.

The seven sibling lanes thread their own carrier as the first parameter, each the native spelling of the same contract:

lane	the stop signal
c	a `cancel_ctx`, an atomic done flag plus the channels registered with it, cancel broadcasts into every blocked send
rust	a `CancelCtx`, an atomic done flag plus a watch list of rendezvous channels, cancel marks each canceled and wakes the parked sends
java	a `CancelCtx`, an atomic done flag plus a watch list of channels, cancel pokes every blocked send awake
c#	`CancellationToken`, accepted by every `WriteAsync` and `ReadAllAsync`, cancellation throws out of the wait
javascript	a `CancelToken`, a canceled flag with a waiter list, checked between yields
python	a `Cancel` wrapper over an `asyncio.Event`, polled at loop heads
lua	a ctx table whose cancel aborts every parked channel operation at once

Javascript's native carrier is `AbortSignal`, the signal fetch, timers, and streams already accept, book 7, ch 6 (javascript es2026 and typescript 7, async javascript) tours the loop it rides, and the sample's `CancelToken` hand-spells the same contract so the waiter list stays inspectable. Java hand-spells its carrier too, an `AtomicBoolean` done flag whose cancel walks a watch list of channels and wakes every blocked send. Python's row is the flag-plus-Event carrier in these samples and asyncio task cancellation in production shape, book 8, ch 21 (python 3.14, asyncio) covers the family. C's lane is the one that builds the whole mechanism by hand, and its broadcast-on-cancel is the same discipline the condition variables of chapter 7 enforce. Rust hand-spells the same carrier, an `AtomicBool` done flag whose cancel walks the watch list and flips each channel's canceled bit under its own lock.

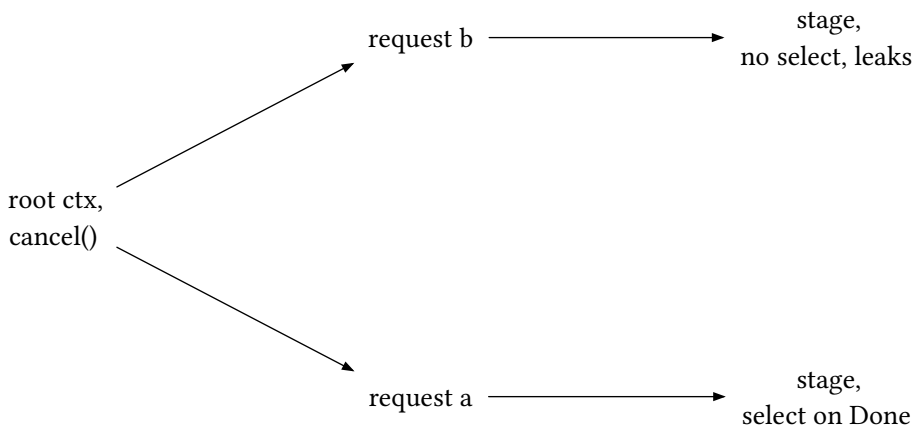


figure 44: cancel the root, every select observes it, the one that cannot is the leak

8.2. generator and stage

A generator converts a finite source into a stream, a stage transforms one stream into another, and in go both are one goroutine with a defer close:

The dry run: squares of 1 through 10 sum to 385, an empty generator sums to 0, the first value off the chain is 1, and cancel mid-stream stops the producer with every stage unwound. The seven new trees pin the first value 1, the go lane's frozen cancel test feeds an endless counter from 0 and pins first square 0, both verified against the same implementation.

```c
// the select analog, strict rendezvous: the send completes only when a
// receiver has taken the value, and cancel aborts it. one sender per
// channel, the shape every lane here uses, so the retract below can
// only drop this sender's own value
static bool xc_send(xchan *c, int v) {
  mtx_lock(&c->mu);
  while (c->full && !c->closed && !c->canceled)
    cnd_wait(&c->space, &c->mu);
  if (c->closed || c->canceled) {
    mtx_unlock(&c->mu);
    return false;
  }
  c->slot = v;
  c->full = true;
  cnd_signal(&c->items);
  while (c->full && !c->closed && !c->canceled)
    cnd_wait(&c->space, &c->mu);
  if (c->full) {
    c->full = false; // canceled while holding: retract the value
    mtx_unlock(&c->mu);
    return false;
  }
  mtx_unlock(&c->mu);
  return true;
}

// false once closed and drained, the range analog
static bool xc_recv(xchan *c, int *out) {
  mtx_lock(&c->mu);
  while (!c->full && !c->closed)
    cnd_wait(&c->items, &c->mu);
  if (!c->full) {
    mtx_unlock(&c->mu);
    return false;
  }
  *out = c->slot;
  c->full = false;
  cnd_signal(&c->space);
  mtx_unlock(&c->mu);
  return true;
}
```

listing 291: C, the select analog as a strict rendezvous, the send completes when a receiver takes the value and cancel aborts it mid-handoff (source: patterns-concurrency-distributed/samples-c/src/Ch08/pipeline.c, lines 84-124)

```
    fn close(&self) {
        let mut st = self.state.lock().unwrap();
        st.closed = true;
        self.items.notify_all();
        self.space.notify_all();
    }

    fn send(&self, v: i32) -> bool {
        let mut st = self.state.lock().unwrap();
        while st.full && !st.closed && !st.canceled {
            st = self.space.wait(st).unwrap();
        }
        if st.closed || st.canceled {
            return false;
        }
        st.slot = v;
        st.full = true;
        self.items.notify_one();
        while st.full && !st.closed && !st.canceled {
            st = self.space.wait(st).unwrap();
        }
        if st.full {
            st.full = false;
            return false;
        }
        true
    }

    fn recv(&self) -> Option<i32> {
        let mut st = self.state.lock().unwrap();
        while !st.full && !st.closed {
            st = self.items.wait(st).unwrap();
        }
        if !st.full {
            return None;
        }
        let v = st.slot;
        st.full = false;
        self.space.notify_one();
        Some(v)
    }
}
```

listing 292: Rust, the same strict rendezvous over a Mutex and two Condvars, a canceled sender holding the slot retracts it (source: patterns-concurrency-distributed/samples-rust/src/Ch08/pipeline.rs, lines 47-88)

```
func Generator(ctx context.Context, values ...int) <-chan int {
  out := make(chan int)
  go func() {
    defer close(out)
    for _, v := range values {
      select {
      case out <- v:
      case <-ctx.Done():
        return
      }
    }
  }()
  return out
}

// Square is a pipeline stage: one goroutine, select on both
// directions, close on exit so the stage after it can finish.
func Square(ctx context.Context, in <-chan int) <-chan int {
  out := make(chan int)
  go func() {
    defer close(out)
    for v := range in {
      select {
      case out <- v * v:
      case <-ctx.Done():
        return
      }
    }
  }()
  return out
}

// FanIn merges many channels into one. One forwarder goroutine per
// input, a WaitGroup, and a closer goroutine: the merge closes only
```

listing 293: Go, generator and square stage, identical shape, select on both directions (source: patterns-concurrency-distributed/samples/ch08/patterns.go, lines 12-45)

```
    // the select analog, strict rendezvous: the send completes only
    // when a receiver has taken the value, and cancel aborts it
    boolean send(int v) throws InterruptedException {
      lock.lock();
      try {
        while (full && !closed && !canceled)
          space.await();
        if (closed || canceled)
          return false;
        slot = v;
        full = true;
        items.signalAll();
        while (full && !closed && !canceled)
          space.await();
        if (full) {
          full = false; // canceled while holding: retract the value
          return false;
        }
        return true;
      } finally {
        lock.unlock();
      }
    }

    // false once closed and drained, the range analog
    boolean recv(int[] out) throws InterruptedException {
      lock.lock();
      try {
        while (!full && !closed)
          items.await();
        if (!full)
          return false;
        out[0] = slot;
        full = false;
        space.signalAll();
        return true;
      } finally {
        lock.unlock();
      }
    }
```

listing 294: Java, the select analog a strict one-slot rendezvous over two Conditions, cancel aborts mid-handoff and the sender retracts (source: patterns-concurrency-distributed/samples-java/src/Ch08/Pipeline.java, lines 57-96)

```
// Generator streams values until the input ends or the token fires.
// The WriteAsync(ct) is what makes it leak-free: cancellation throws
// out of the write and the finally completes the channel.
public static class Pipeline
{
    public static ChannelReader<int> Generator(CancellationToken ct, params int[] values)
    {
        var outCh = Channel.CreateUnbounded<int>();
        _ = Task.Run(async () =>
        {
            StageTelemetry.Enter();
            try
            {
                foreach (var v in values)
                {
                    await outCh.Writer.WriteAsync(v, ct);
                }
            }
            catch (OperationCanceledException)
            {
            }
            finally
            {
                StageTelemetry.Exit();
                outCh.Writer.Complete();
            }
        });
        return outCh.Reader;
    }
```

listing 295: C#, WriteAsync with the token is the select, cancellation throws out of the wait, finally completes the channel (source: patterns-concurrency-distributed/samples-cs/src/Ch08/Pipeline.cs, lines 20-48)

```
export class CancelToken {
  #canceled = false;
  #waiters = [];

  get canceled() {
    return this.#canceled;
  }

  cancel() {
    if (this.#canceled) return;
    this.#canceled = true;
    for (const w of this.#waiters.splice(0)) w();
  }

  get done() {
    if (this.#canceled) return Promise.resolve();
    return new Promise((resolve) => this.#waiters.push(resolve));
  }
}

let stages = 0;

// leak probe: how many generator bodies are alive right now. the
// leak test asserts this returns to its baseline after every cycle.
export function liveStages() {
  return stages;
}

export async function* generator(token, values) {
  stages++;
  try {
    for (const v of values) {
      if (token.canceled) return;
      yield v;
    }
  } finally {
    stages--;
  }
}

export async function* square(token, inSeq) {
  stages++;
  try {
    for await (const v of inSeq) {
      if (token.canceled) return;
      yield v * v;
    }
  } finally {
    stages--;
  }
}
```

listing 296: JavaScript, async generators with a finally head count, the consumer's return propagates down the chain (source: patterns-concurrency-distributed/samples-js/src/ch08-pipeline.mjs, lines 7-57)

```
class Cancel:
    # the context analog: a flag tasks can poll, plus an event for
    # anyone that wants to park on it
    def __init__(self):
        self.event = asyncio.Event()

    @property
    def done(self):
        return self.event.is_set()

    def cancel(self):
        self.event.set()


async def generator_stage(ctx, values, probe=None):
    try:
        alive["stages"] += 1
        for v in values:
            if ctx.done:
                if probe is not None:
                    probe["stopped_by"] = "cancel"
                return
            yield v
    finally:
        alive["stages"] -= 1
        if probe is not None:
            probe["closed"] = True


async def square(ctx, src):
    try:
        alive["stages"] += 1
        async for v in src:
            if ctx.done:
                return
            yield v * v
    finally:
        alive["stages"] -= 1
        await src.aclose()
```

listing 297: Python, async generators, cancellation checked at the loop head, aclose runs the finally (source: patterns-concurrency-distributed/samples-py/src/Ch08/pipeline.py, lines 22-60)

```lua
-- Generator streams values and closes behind itself
local function Generator(S, c, values, probe)
  local out = chan(S, 0)
  spawn(S, function()
    for _, v in ipairs(values) do
      if not send(out, v, c) then
        break
      end
      if probe then
        probe.sent = probe.sent + 1
      end
    end
    close_ch(out)
  end)
  return out
end

-- Square is one stage: receive, square, forward, close on exit
local function Square(S, c, in_, probe)
  local out = chan(S, 0)
  spawn(S, function()
    while true do
      local v, ok = recv(in_, c)
      if not ok then
        break
      end
      if not send(out, v * v, c) then
        break
      end
      if probe then
        probe.staged = probe.staged + 1
      end
    end
    close_ch(out)
  end)
  return out
end
```

listing 298: Lua, generator and stage lanes over hand channels, send and recv abort on a canceled ctx, close on exit (source: patterns-concurrency-distributed/samples-lua/ch08_pipeline.lua, lines 170-206)

The shape is the contract in every lane. The producer checks the carrier or selects on it, so a consumer that quits unblocks it. The close behind the producer means the stage downstream can drain its input and terminate when the chain empties. Go writes it as one select covering both directions. C builds the select by hand as a strict rendezvous, the send parks until a receiver takes the value, cancel wakes it, and a sender canceled while holding the slot retracts its own value. Java's `XChan` is the same strict rendezvous over a `ReentrantLock` and two conditions, its cancel broadcast retracting the held slot exactly like C's. Rust's `XChan` is the same shape over `Mutex` and two `Condvars`, and the sender canceled while holding the slot clears `full` itself before returning, the retraction spelled in the type's own state. C# writes the stages over `System.Threading.Channels`, `WriteAsync` and `ReadAllAsync` each taking the token, the channel pipeline idiom the stdlib tour builds, book 6, ch 16 (c# 15, f# 11, and .net 11, stdlib tour 2: io, text, net, channels, diagnostics). Javascript and python lean on async generators, where exhaustion and the consumer's early return both run the finally, and the js lane counts stage bodies alive to make unwinding assertable. Lua spawns one lane per stage over its hand channels and teaches the cancel watcher list, the same mechanism chapter 9's guard reuses.

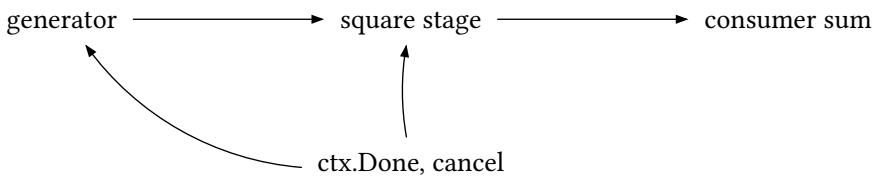


figure 45: the pipeline spine

8.3. fan-in, merge

Fan-out is free, starting a consumer per producer is a loop. Fan-in, merging many streams back into one, needs one forwarder per source and a closer that waits:

The dry run: sources (1,2,3), (10,20), and (100) merge to exactly the sorted six 1, 2, 3, 10, 20, 100, and an empty fan-in drains 0 values. Interleaving is the one property the pattern does not promise, so every lane sorts before asserting.

```c
static bool all_closed(const chan *srcs, int nsrc) {
  for (int i = 0; i < nsrc; i++)
    if (!srcs[i].closed)
      return false;
  return true;
}

// one scripted merge pass: round robin, one value from each source that
// has one right now
static int merge_pass(chan *srcs, int nsrc, int *out, int maxout) {
  int got = 0;
  for (int i = 0; i < nsrc; i++) {
    int v;
    if (chan_try_recv(&srcs[i], &v) && got < maxout)
      out[got++] = v;
  }
  return got;
}
```

listing 299: C, the closer's predicate, one open source keeps the merge alive, scripted round-robin passes replace racing forwarders (source: patterns-concurrency-distributed/samples-c/src/Ch08/fanin.c, lines 77-94)

```rust
fn all_closed(srcs: &[Chan]) -> bool {
    srcs.iter().all(|c| c.is_closed())
}

fn merge_pass(srcs: &[Chan], out: &mut Vec<i32>) {
    for src in srcs {
        if let Some(v) = src.try_recv() {
            out.push(v);
        }
    }
}
```

listing 300: Rust, the scripted round-robin merge, all_closed the closer's predicate, one value per source per pass (source: patterns-concurrency-distributed/samples-rust/src/Ch08/fanin.rs, lines 58-68)

```go
func FanIn[T any](ctx context.Context, ins ...<-chan T) <-chan T {
  out := make(chan T)
  var wg sync.WaitGroup
  for _, ch := range ins {
    wg.Add(1)
    go func(c <-chan T) {
      defer wg.Done()
      for v := range c {
        select {
        case out <- v:
        case <-ctx.Done():
          return
        }
      }
    }(ch)
  }
  go func() {
    wg.Wait()
    close(out)
  }()
  return out
}

// WorkerPool keeps n workers pulling from one queue. Every job
```

listing 301: Go, forwarder per input, waitgroup, close after the last source closes (source: patterns-concurrency-distributed/samples/ch08/patterns.go, lines 47-70)

```
  static final class Chan {
    private final int[] buf = new int[CAP];
    private int count;
    boolean closed;

    void send(int v) {
      if (count < CAP)
        buf[count++] = v;
    }

    void close() {
      closed = true;
    }

    boolean tryRecv(int[] out) {
      if (count == 0)
        return false;
      out[0] = buf[0];
      for (int i = 1; i < count; i++)
        buf[i - 1] = buf[i];
      count--;
      return true;
    }
  }

  // the closer's predicate: the merge may close only when every source
  // has closed, one open source keeps it alive
  static boolean allClosed(Chan[] srcs) {
    for (Chan src : srcs)
      if (!src.closed)
        return false;
    return true;
  }

  // one scripted merge pass: round robin, one value from each source
  // that has one right now
  static int mergePass(Chan[] srcs, int[] out, int offset, int maxout) {
    int got = 0;
    int[] v = new int[1];
    for (Chan src : srcs)
      if (src.tryRecv(v) && offset + got < maxout)
        out[offset + got++] = v[0];
    return got;
  }
```

listing 302: Java, the scripted round-robin merge, allClosed the closer's predicate, the trace recorded instead of raced (source: patterns-concurrency-distributed/samples-java/src/Ch08/Fanin.java, lines 24-67)

```
public static class Fanin
{
    public static ChannelReader<T> Merge<T>(CancellationToken ct, params ChannelReader<T>[] ins)
    {
        var outCh = Channel.CreateUnbounded<T>();
        var forwarders = new Task[ins.Length];
        for (int i = 0; i < ins.Length; i++)
        {
            ChannelReader<T> src = ins[i];
            forwarders[i] = Task.Run(async () =>
            {
                StageTelemetry.Enter();
                try
                {
                    await foreach (var v in src.ReadAllAsync(ct))
                    {
                        await outCh.Writer.WriteAsync(v, ct);
                    }
                }
                catch (OperationCanceledException)
                {
                }
                finally
                {
                    StageTelemetry.Exit();
                }
            });
        }
        _ = Task.Run(async () =>
        {
            await Task.WhenAll(forwarders);
            outCh.Writer.Complete();
        });
        return outCh.Reader;
    }
}
```

listing 303: C#, one forwarder task per input, a WhenAll over them, the closer task completes the output last (source: patterns-concurrency-distributed/samples-cs/src/Ch08/Fanin.cs, lines 8-43)

```
export async function* fanIn(token, sources) {
  const q = new MergeQueue();
  void (async () => {
    await Promise.all(
      sources.map(async (src) => {
        forwarders++;
        try {
          for await (const v of src) {
            if (token.canceled || q.closed) break;
            q.push(v);
          }
        } finally {
          forwarders--;
        }
      }),
    );
    q.close();
  })();
  try {
    for (;;) {
      const r = await q.next();
      if (r.done) return;
      yield r.value;
    }
  } finally {
    q.close(); // an abandoned consumer stops the forwarders too
  }
}
```

listing 304: JavaScript, one forwarder per source pushing into a merge queue, close only after every source finished (source: patterns-concurrency-distributed/samples-js/src/ch08-fanin.mjs, lines 41-68)

```
async def fan_in_collect(sources):
    out = asyncio.Queue()

    async def forward(src):
        async for v in src:
            await out.put(v)

    forwarders = [asyncio.ensure_future(forward(s)) for s in sources]

    async def drain_all():
        got = []
        while True:
            v = await out.get()
            if v is SENTINEL:
                return got
            got.append(v)

    drainer = asyncio.ensure_future(drain_all())
    await asyncio.gather(*forwarders)
    await out.put(SENTINEL)
    return await drainer
```

listing 305: Python, forwarders into one queue, a sentinel after the gather ends the drain (source: patterns-concurrency-distributed/samples-py/src/Ch08/fanin.py, lines 24-45)

```lua
-- FanIn runs one forwarder per input and a closer lane behind a
-- waitgroup: out closes only after every forwarder exited
local function FanIn(S, ins)
  local out = chan(S, 0)
  local wg = WaitGroup(S)
  for _, ch in ipairs(ins) do
    wg:Add(1)
    spawn(S, function()
      while true do
        local v, ok = recv(ch)
        if not ok then
          break
        end
        send(out, v)
      end
      wg:Done()
    end)
  end
  spawn(S, function()
    wg:Wait()
    close_ch(out)
  end)
  return out
end
```

listing 306: Lua, one forwarder lane per input, a waitgroup, a closer lane that closes out after the last Done (source: patterns-concurrency-distributed/samples-lua/ch08_fanin.lua, lines 141-164)

The closer after the barrier is the subtle part in every lane: the merge's output must close only when every input has closed, and the barrier is the cheapest edge that orders "all forwarders returned" before close. Go and lua spend a waitgroup, C# a `WhenAll`, javascript a `Promise.all` before `q.close()`, python a sentinel enqueued after the gather. The C, rust, and java lanes script the whole interleave, round-robin passes with sources closing at chosen points, so the pinned pass order 1, 10, 100, 2, 20, 3 is a fixture, an interleaving the racer lanes leave free.

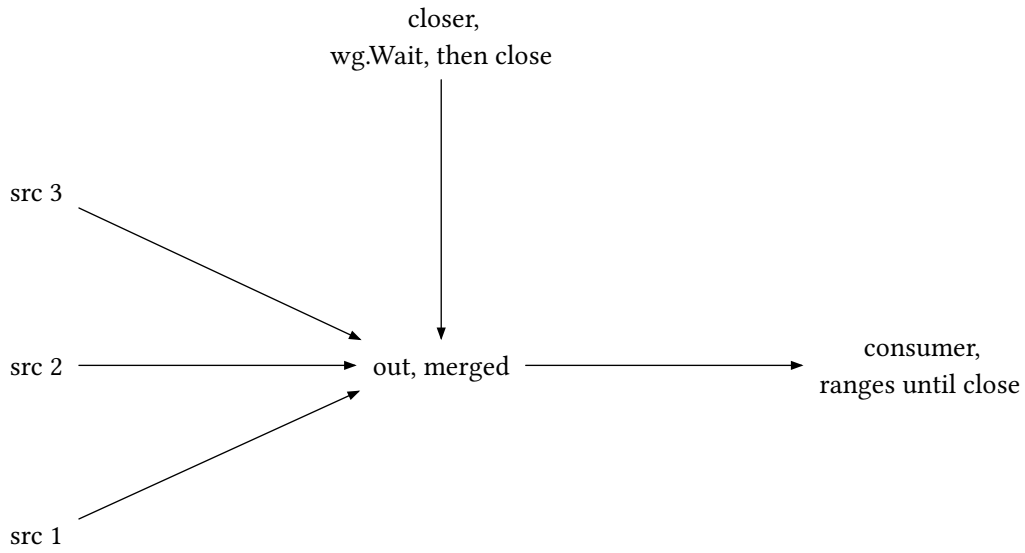


figure 46: one forwarder per source, the closer waits then closes out

8.4. worker pool

A pool fixes the concurrency count instead of the workload: n workers share one job queue, which is a queue with no configuration:

The dry run: 50 jobs through 4 workers. Every job processed exactly once, no more than 4 distinct worker ids ever appear, and each job computes a positive sum. The java, javascript, python, and lua lanes pin the sum itself, 147, the total of i modulo 7 over 50 ops, the go and C# lanes assert positivity, and C and rust check every result against their own grind function.

```
static int worker(void *arg) {
  pool *p = arg;
  atomic_fetch_add(&live_threads, 1);
  // worker id from the pool's start counter, taken under the same lock
  // the results use
  mtx_lock(&p->res_mu);
  int id = p->worker_seq++;
  mtx_unlock(&p->res_mu);
  int j;
  while (queue_take(p->jobs, &j)) {
    int sum = job_sum(OPS);
    mtx_lock(&p->res_mu);
    if (p->n_results < NJOBS) {
      p->results[p->n_results].id = j;
      p->results[p->n_results].worker = id;
      p->results[p->n_results].sum = sum;
      p->n_results++;
    }
    if (id ≥ 0 && id < NWORKERS)
      p->used_worker[id] = true;
    mtx_unlock(&p->res_mu);
  }
  atomic_fetch_sub(&live_threads, 1);
  return 0;
}

static void pool_reset(pool *p, queue *q) {
  p->jobs = q;
  mtx_init(&p->res_mu, mtx_plain);
  p->n_results = 0;
  for (int i = 0; i < NWORKERS; i++)
    p->used_worker[i] = false;
  p->worker_seq = 0;
}
```

listing 307: C, 4 real threads ranging one queue, worker ids taken under the results lock, every job lands exactly once (source: patterns-concurrency-distributed/samples-c/src/Ch08/workerpool.c, lines 107-140)

```
fn run_round() -> PoolState {
    let q = Queue::new();
    for i in 0..NJOBS as i32 {
        q.push(i);
    }
    q.close();
    let state = Mutex::new(PoolState {
        results: Vec::new(),
        used_worker: [false; NWORKERS],
        worker_seq: 0,
    });
    std::thread::scope(|s| {
        for _ in 0..NWORKERS {
            let q = &q;
            let state = &state;
            s.spawn(move || {
                LIVE.fetch_add(1, Ordering::SeqCst);
                let id = {
                    let mut st = state.lock().unwrap();
                    let id = st.worker_seq;
                    st.worker_seq += 1;
                    id
                };
                while let Some(j) = q.take() {
                    let sum = job_sum(OPS);
                    let mut st = state.lock().unwrap();
                    st.results.push((j as usize, id, sum));
                    st.used_worker[id] = true;
                }
                LIVE.fetch_sub(1, Ordering::SeqCst);
            });
        }
    });
    state.into_inner().unwrap()
}
```

listing 308: Rust, four scoped workers ranging one hand queue, ids taken under the results lock, landing order kept (source: patterns-concurrency-distributed/samples-rust/src/Ch08/workerpool.rs, lines 83-117)

```
func WorkerPool(ctx context.Context, jobs <-chan Job, workers int) <-chan Result {
  out := make(chan Result)
  var wg sync.WaitGroup
  for w := 0; w < workers; w++ {
    wg.Add(1)
    go func(id int) {
      defer wg.Done()
      for j := range jobs {
        sum := 0
        for i := 0; i < j.Ops; i++ {
          sum += i % 7
        }
        select {
        case out <- Result{ID: j.ID, Worker: id, Sum: sum}:
        case <-ctx.Done():
          return
        }
      }
    }(w)
  }
  go func() {
    wg.Wait()
    close(out)
  }()
  return out
}

// OrderedMap fans out f over inputs with bounded parallelism and
```

listing 309: Go, n goroutines ranging one channel, results unordered (source: patterns-concurrency-distributed/samples/ch08/patterns.go, lines 83-110)

```
  static List<Result> runRound() throws Exception {
    ExecutorService pool = Executors.newFixedThreadPool(NWORKERS);
    Set<Long> workers = ConcurrentHashMap.newKeySet();
    List<Callable<Result>> jobs = new ArrayList<>();
    for (int i = 0; i < NJOBS; i++) {
      int id = i;
      jobs.add(() -> {
        long worker = Thread.currentThread().threadId();
        workers.add(worker);
        return new Result(id, worker, jobSum(OPS));
      });
    }
    List<Result> results = new ArrayList<>();
    for (Future<Result> f : pool.invokeAll(jobs))
      results.add(f.get());
    pool.close();
    lastWorkers.set(workers.size());
    return results;
  }
```

listing 310: Java, the pool over a fixed ExecutorService, invokeAll hands futures back in submission order (source: patterns-concurrency-distributed/samples-java/src/Ch08/Workerpool.java, lines 44-62)

```
public static class Workerpool
{
    public static ChannelReader<Result> Run(
        CancellationToken ct, ChannelReader<Job> jobs, int workers)
    {
        var outCh = Channel.CreateUnbounded<Result>();
        var tasks = new Task[workers];
        for (int w = 0; w < workers; w++)
        {
            int id = w;
            tasks[w] = Task.Run(async () =>
            {
                StageTelemetry.Enter();
                try
                {
                    await foreach (var j in jobs.ReadAllAsync(ct))
                    {
                        int sum = 0;
                        for (int i = 0; i < j.Ops; i++)
                        {
                            sum += i % 7;
                        }
                        await outCh.Writer.WriteAsync(new Result(j.Id, id, sum), ct);
                    }
                }
                catch (OperationCanceledException)
                {
                }
                finally
                {
                    StageTelemetry.Exit();
                }
            });
        }
        _ = Task.Run(async () =>
        {
            await Task.WhenAll(tasks);
            outCh.Writer.Complete();
        });
        return outCh.Reader;
    }
}
```

listing 311: C#, n tasks ranging one channel through ReadAllAsync, the closer task completes after WhenAll (source: patterns-concurrency-distributed/samples-cs/src/Ch08/Workerpool.cs, lines 11-52)

```
export async function* workerPool(jobs, workers) {
  const pending = [...jobs];
  const out = new MergeQueue();

  const run = async (id) => {
    for (;;) {
      const j = pending.shift();
      if (j === undefined) break;
      let sum = 0;
      for (let i = 0; i < j.ops; i++) sum += i % 7;
      out.push({ id: j.id, worker: id, sum });
      await null; // hand the turn to the other workers
    }
  };

  void (async () => {
    await Promise.all(Array.from({ length: workers }, (_, i) => run(i)));
    out.close();
  })();

  for (;;) {
    const r = await out.next();
    if (r.done) return;
    yield r.value;
  }
}
```

listing 312: JavaScript, n runners sharing one job list, the synchronous shift is the exactly-once guarantee (source: patterns-concurrency-distributed/samples-js/src/ch08-workerpool.mjs, lines 8-33)

```
async def worker_pool(jobs, workers):
    queue = asyncio.Queue()
    out = asyncio.Queue()
    used = set()
    for j in jobs:
        queue.put_nowait(j)
    for _ in range(workers):
        queue.put_nowait(STOP)

    async def laborer(wid):
        while True:
            j = await queue.get()
            if j is STOP:
                return
            used.add(wid)
            await out.put(Result(j.id, wid, grind(j.ops)))

    crew = [asyncio.ensure_future(laborer(w)) for w in range(workers)]
    await asyncio.gather(*crew)
    results = []
    while not out.empty():
        results.append(out.get_nowait())
    return results, used
```

listing 313: Python, n laborers over one queue, stop sentinels drain the crew, results in completion order (source: patterns-concurrency-distributed/samples-py/src/Ch08/workerpool.py, lines 39-61)

```lua
local function WorkerPool(S, jobs, workers)
  local out = chan(S, 0)
  local wg = WaitGroup(S)
  for w = 1, workers do
    wg:Add(1)
    spawn(S, function()
      while true do
        local j, ok = recv(jobs)
        if not ok then
          break
        end
        local sum = 0
        for i = 0, j.ops - 1 do
          sum = sum + i % 7
        end
        coroutine.yield() -- the work interleaves
        send(out, { id = j.id, worker = w, sum = sum })
      end
      wg:Done()
    end)
  end
  spawn(S, function()
    wg:Wait()
    close_ch(out)
  end)
  return out
end
```

listing 314: Lua, n lanes pulling one channel, a yield inside the loop interleaves the work (source: patterns-concurrency-distributed/samples-lua/ch08_workerpool.lua, lines 129-155)

Ordering is explicitly not among the guarantees, results arrive in completion order, and when the caller needs input order anyway the cheaper shape is the next pattern. Java is the one lane that gets order for free: `ExecutorService.invokeAll` hands futures back in submission order, so its results are ordered without any indexing, a real divergence from the siblings. The lanes differ only in what enforces exactly-once: go, C, and lua share one channel or queue that hands each job to one receiver, rust's hand queue pops each job under the lock, the mutex itself the single hand-off, java's executor owns the queue, C# ranges one channel through `ReadAllAsync`, javascript relies on the single-threaded `shift`, and python's queue plus sentinels keeps every job with one laborer.

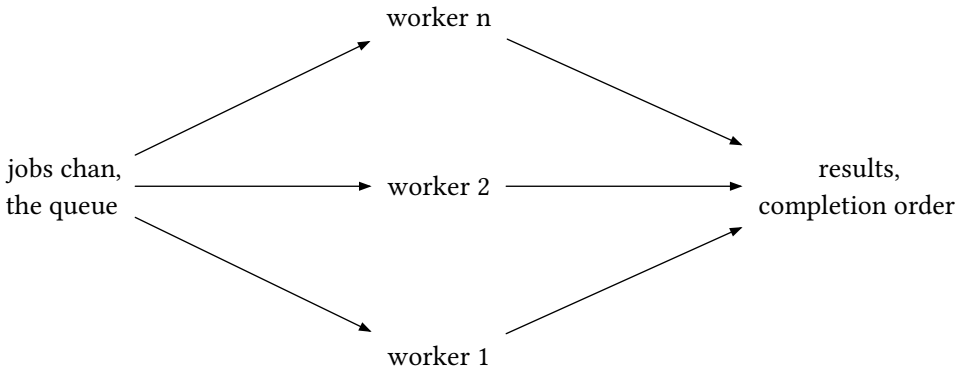


figure 47: n workers range one queue, exactly once each, results unordered

8.5. bounded parallel map, ordered

Fan-out with a result array indexed by input position gives parallelism with order preservation, and the gate bounds the concurrency, chapter 6's counting semaphore at work:

The dry run: 8 inputs through a gate of 3, f maps each input to its letter. The output reads back a through h in input order while completion order scrambles by construction, and the peak in-flight count never exceeds 3. The scrambles differ per lane: go sleeps real 60 minus v milliseconds, C, rust, and java drive a discrete-event scheduler with a pinned completion order, C# releases countdown events in reverse input order, javascript scales the same shape to microtask turns, python and lua run the sleeps on virtual clocks, the only real milliseconds spent anywhere are the go lane's.

```
static int ordered_map(const int *inputs, int n_inputs, int gate, char *out,
                       int *completion) {
  task t[INPUTS];
  for (int i = 0; i < n_inputs; i++) {
    t[i].index = i;
    t[i].duration = map_duration(inputs[i]);
    t[i].value = map_value(inputs[i]);
    t[i].started = t[i].done = false;
  }

  long now = 0;
  int next_admit = 0, running = 0, completed = 0, peak = 0;
  while (completed < n_inputs) {
    while (running < gate && next_admit < n_inputs) {
      task *k = &t[next_admit];
      k->started = true;
      k->start = now;
      k->finish = now + k->duration;
      next_admit++;
      running++;
      if (running > peak)
        peak = running;
    }
    // earliest finish, admission order breaks ties
    int pick = -1;
    for (int i = 0; i < next_admit; i++)
      if (t[i].started && !t[i].done &&
          (pick < 0 || t[i].finish < t[pick].finish))
        pick = i;
    now = t[pick].finish;
    t[pick].done = true;
    t[pick].started = false;
    running--;
    completion[completed++] = t[pick].index;
  }
  for (int i = 0; i < n_inputs; i++)
    out[i] = t[i].value;
  return peak;
}
```

listing 315: C, admit in input order, always advance to the earliest finish, ties by admission order, no thread waits (source: patterns-concurrency-distributed/samples-c/src/Ch08/orderedmap.c, lines 41-79)

```
fn ordered_map(inputs: &[i32], gate: usize, out: &mut [char], completion: &mut [usize]) -> usize {
    let mut t: Vec<Task> = inputs
        .iter()
        .enumerate()
        .map(|(i, &v)| Task {
            index: i,
            finish: 0,
            duration: map_duration(v),
            value: map_value(v),
            started: false,
            done: false,
        })
        .collect();
    let n = inputs.len();

    let mut now: i64 = 0;
    let mut next_admit = 0;
    let mut running = 0;
    let mut completed = 0;
    let mut peak = 0;
    while completed < n {
        while running < gate && next_admit < n {
            let k = &mut t[next_admit];
            k.started = true;
            k.finish = now + k.duration;
            next_admit += 1;
            running += 1;
            if running > peak {
                peak = running;
            }
        }
        let mut pick: Option<usize> = None;
        for i in 0..next_admit {
            let k = &t[i];
            if k.started && !k.done && pick.is_none_or(|p| k.finish < t[p].finish) {
                pick = Some(i);
            }
        }
        let p = pick.unwrap();
        now = t[p].finish;
        t[p].done = true;
        t[p].started = false;
        running -= 1;
        completion[completed] = t[p].index;
        completed += 1;
    }
    for (i, k) in t.iter().enumerate() {
        out[i] = k.value;
    }
    peak
}
```

listing 316: Rust, C's discrete-event scheduler, admit in input order, advance to the earliest finish, the gate a running count (source: patterns-concurrency-distributed/samples-rust/src/Ch08/orderedmap.rs, lines 33-83)

```
func OrderedMap[T, U any](ctx context.Context, inputs []T, parallelism int, f func(T) U) []U {
  out := make([]U, len(inputs))
  gate := make(chan struct{}, parallelism)
  var wg sync.WaitGroup
  for i, in := range inputs {
    wg.Add(1)
    go func(slot int, v T) {
      defer wg.Done()
      select {
      case gate <- struct{}{}:
      case <-ctx.Done():
        return
      }
      defer func() { <-gate }()
      out[slot] = f(v)
    }(i, in)
  }
  wg.Wait()
  return out
}

// First answers from whichever source replies sooner, with a
// deadline. The loser's write is discarded, so it must tolerate an
```

listing 317: Go, no result channels: one slot per input, a gate for parallelism (source: patterns-concurrency-distributed/samples/ch08/patterns.go, lines 112-134)

```
  static int orderedMap(int[] inputs, int gate, char[] out, int[] completion,
      long[] probe) {
    Task[] t = new Task[inputs.length];
    for (int i = 0; i < inputs.length; i++) {
      t[i] = new Task();
      t[i].index = i;
      t[i].duration = mapDuration(inputs[i]);
      t[i].value = mapValue(inputs[i]);
    }

    long now = 0;
    int nextAdmit = 0, running = 0, completed = 0, peak = 0;
    while (completed < inputs.length) {
      while (running < gate && nextAdmit < inputs.length) {
        Task k = t[nextAdmit];
        k.started = true;
        k.start = now;
        k.finish = now + k.duration;
        nextAdmit++;
        running++;
        if (running > peak)
          peak = running;
      }
      // earliest finish, admission order breaks ties
      int pick = -1;
      for (int i = 0; i < nextAdmit; i++)
        if (t[i].started && !t[i].done && (pick < 0 || t[i].finish < t[pick].finish))
          pick = i;
      now = t[pick].finish;
      t[pick].done = true;
      t[pick].started = false;
      running--;
      completion[completed++] = t[pick].index;
    }
    for (int i = 0; i < inputs.length; i++)
      out[i] = t[i].value;
    probe[0] = now;
    return peak;
  }
```

listing 318: Java, C's discrete-event scheduler, admit in input order, advance to the earliest finish, ties by admission (source: patterns-concurrency-distributed/samples-java/src/Ch08/Orderedmap.java, lines 49-87)

```
public static class Orderedmap
{
    public static async Task<U[]> Map<T, U>(T[] inputs, int parallelism, Func<T, U> f)
    {
        var outArr = new U[inputs.Length];
        using var gate = new SemaphoreSlim(parallelism, parallelism);
        var tasks = new Task[inputs.Length];
        for (int i = 0; i < inputs.Length; i++)
        {
            int slot = i;
            T v = inputs[i];
            tasks[slot] = Task.Run(async () =>
            {
                await gate.WaitAsync();
                try
                {
                    outArr[slot] = f(v);
                }
                finally
                {
                    gate.Release();
                }
            });
        }
        await Task.WhenAll(tasks);
        return outArr;
    }
}
```

listing 319: C#, a SemaphoreSlim of size parallelism, every task writes its own slot, WhenAll publishes (source: patterns-concurrency-distributed/samples-cs/src/Ch08/Orderedmap.cs, lines 7-34)

```
export class Gate {
  #cap;
  #held = 0;
  #waiters = [];

  constructor(capacity) {
    this.#cap = capacity;
  }

  async acquire() {
    while (this.#held ≥ this.#cap) {
      await new Promise((resolve) ⇒ this.#waiters.push(resolve));
    }
    this.#held++;
  }

  release() {
    this.#held--;
    const w = this.#waiters.shift();
    if (w) w();
  }
}

export async function orderedMap(inputs, parallelism, f) {
  const out = new Array(inputs.length);
  const gate = new Gate(parallelism);
  await Promise.all(
    inputs.map(async (v, i) ⇒ {
      await gate.acquire();
      try {
        out[i] = await f(v);
      } finally {
        gate.release();
      }
    }),
  );
  return out;
}
```

listing 320: JavaScript, a promise-count gate bounds the in-flight tasks, every slot writes itself (source: patterns-concurrency-distributed/samples-js/src/ch08-orderedmap.mjs, lines 6-43)

```python
async def ordered_map(clock, inputs, parallelism, f, probe):
    out = [None] * len(inputs)
    gate = asyncio.Semaphore(parallelism)

    async def one(slot, v):
        async with gate:
            probe["active"] += 1
            probe["peak"] = max(probe["peak"], probe["active"])
            out[slot] = await f(v)
            probe["active"] -= 1

    crew = [asyncio.ensure_future(one(i, v)) for i, v in enumerate(inputs)]
    await asyncio.gather(*crew)
    return out


INPUTS = [10, 20, 30, 40, 50, 60, 70, 80]


async def scenario():
    clock = VirtualClock()
    probe = {"active": 0, "peak": 0, "order": []}

    async def work(v):
        # later slots sleep less, so completion order scrambles
        await clock.sleep(60 - v)
        probe["order"].append(v)
        return chr(ord("a") + v // 10 - 1)

    async def play():
        return await ordered_map(clock, INPUTS, 3, work, probe)

    got = await clock.drive(play())
    return got, probe, clock.now
```

listing 321: Python, an asyncio semaphore gates the fan-out, sleeps resolve through the heap clock, later slots finish first (source: patterns-concurrency-distributed/samples-py/src/Ch08/orderedmap.py, lines 64-97)

```lua
-- the gate: at most cap lanes inside, waiters park in arrival order
local function gate(C, cap)
  local g = { C = C, cap = cap, inflight = 0, waiters = {}, peak = 0 }
  function g:Take()
    while self.inflight ≥ self.cap do
      self.waiters[#self.waiters + 1] = coroutine.running()
      coroutine.yield("park")
    end
    self.inflight = self.inflight + 1
    if self.inflight > self.peak then
      self.peak = self.inflight
    end
  end
  -- release the slot first, then wake the head waiter: its recheck
  -- must find inflight below the cap or it would park again
  function g:Give()
    self.inflight = self.inflight - 1
    local co = table.remove(self.waiters, 1)
    if co then
      wake(self.C, co)
    end
  end
  return g
end

-- runs f over inputs with bounded parallelism; results land in input
-- order whatever the completion order was
local function OrderedMap(C, inputs, parallelism, f, observe)
  local out = {}
  local g = gate(C, parallelism)
  for i, v in ipairs(inputs) do
    spawn(C, function()
      g:Take()
      out[i] = f(v)
      g:Give()
    end)
  end
  return out, g
end
```

listing 322: Lua, the gate parks waiters in arrival order, vsleep orders completions, slots fill in input order (source: patterns-concurrency-distributed/samples-lua/ch08_orderedmap.lua, lines 59-97)

No mutex on the array: each worker writes exactly one distinct slot, and the barrier is the happens-before edge that publishes them all, `Wait` in go, the `join` over the virtual-thread crew in java's live lane, `WhenAll` in C#, the gather in python, the resolved promise chain in javascript, the joined lanes in lua, the completed loop in C's, rust's, and java's scheduler, rust running the whole fixture with no threads at all, the gate a running count the loop enforces on itself and the completion order pinned as 2, 1, 0, then the tail in input order. Java runs the fan-out twice, the recorded scheduler beside a live `Semaphore`-gated crew whose slot-indexed writes keep input order whatever the landing order. The test makes later inputs finish first and the output still comes back in input order. This is the shape to reach for when someone reaches for an error group with an index map on the side.

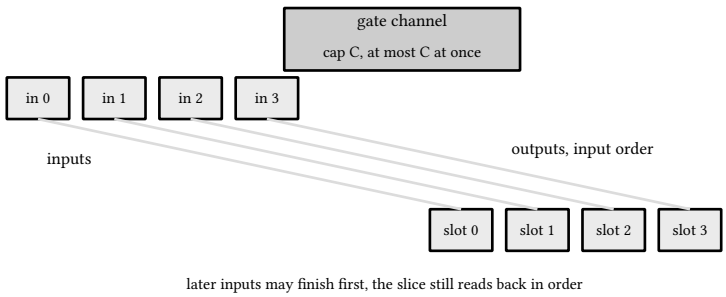


figure 48: every goroutine writes its own slot, wait publishes them all

8.6. racing sources, first wins

Select over answers and deadline together gives hedged requests their core:

The dry run: a source ready at 10ms races one at 1s inside a 2s budget and the caller receives `fast: quick`. Two sources stalled past 5s under a 30ms deadline produce the timeout error naming neither source. The mechanism split is the honesty of this section: the frozen go test sleeps real milliseconds with margins around 100x, the seven new trees inject the clock, so their runs spend no wall time and C, rust, java, javascript, python, and lua also assert where the clock lands, 10 on the win and 30 on the timeout.

```c
static const char *first_answer(vclock *ck, long fallback_ms, source_fn fast,
                                void *fast_ctx, source_fn slow, void *slow_ctx,
                                char *out, int outsz, char *err, int errsz) {
  timed_answer a = fast(fast_ctx);
  timed_answer b = slow(slow_ctx);
  long deadline = ck->now_ms + fallback_ms;

  long best = deadline;
  int which = -1; // -1: the deadline fires first
  if (a.ready_at >= 0 && a.ready_at < best) {
    best = a.ready_at;
    which = 0;
  }
  if (b.ready_at >= 0 && b.ready_at < best) {
    best = b.ready_at;
    which = 1;
  }
  ck->now_ms = best;

  if (which < 0) {
    snprintf(err, errsz, "neither source answered: %ld ms deadline",
             fallback_ms);
    return NULL;
  }
  const char *name = which == 0 ? "fast" : "slow";
  const char *val = which == 0 ? a.val : b.val;
  snprintf(out, outsz, "%s: %s", name, val);
  return out;
}
```

listing 323: C, both producers report a ready instant, the earliest edge wins unless the deadline is earlier still (source: patterns-concurrency-distributed/samples-c/src/Ch08/first.c, lines 47-75)

```rust
fn first_answer(
    ck: &mut VClock,
    fallback_ms: i64,
    fast: TimedAnswer,
    slow: TimedAnswer,
) -> Result<String, String> {
    let deadline = ck.now_ms + fallback_ms;

    let mut best = deadline;
    let mut which = -1;
    if fast.ready_at ≥ 0 && fast.ready_at < best {
        best = fast.ready_at;
        which = 0;
    }
    if slow.ready_at ≥ 0 && slow.ready_at < best {
        best = slow.ready_at;
        which = 1;
    }
    ck.now_ms = best;

    if which < 0 {
        return Err(format!("neither source answered: {fallback_ms} ms deadline"));
    }
    let (name, val) = if which == 0 {
        ("fast", fast.val)
    } else {
        ("slow", slow.val)
    };
    Ok(format!("{name}: {val}"))
}
```

listing 324: Rust, the deadline one more instant on the virtual clock, the earliest edge wins, an exact tie loses to the deadline (source: patterns-concurrency-distributed/samples-rust/src/Ch08/first.rs, lines 34-63)

```go
func First(ctx context.Context, fallbackDelay time.Duration, fast, slow func() string) (string, error) {
  ctx, cancel := context.WithTimeout(ctx, fallbackDelay)
  defer cancel()

  type answer struct {
    from string
    val  string
  }
  pick := make(chan answer, 2)
  go func() { pick <- answer{"fast", fast()} }()
  go func() { pick <- answer{"slow", slow()} }()

  select {
  case a := <-pick:
    return a.from + ": " + a.val, nil
  case <-ctx.Done():
    return "", fmt.Errorf("neither source answered: %w", ctx.Err())
  }
}
```

listing 325: Go, first answer wins, loser writes discarded into a buffered channel (source: patterns-concurrency-distributed/samples/ch08/patterns.go, lines 136-154)

```java
// at. readyAt < 0 means the source never answers
record TimedAnswer(String val, long readyAt) {}

interface Source {
  TimedAnswer answer();
}

static final class Stall implements Source {
  final long readyAt;
  final String val;

  Stall(long readyAt, String val) {
    this.readyAt = readyAt;
    this.val = val;
  }

  public TimedAnswer answer() {
    return new TimedAnswer(val, readyAt);
  }
}

// first: both producers run, the pick buffer holds both answers (the
// loser's write simply sits there unread), the winner is the earliest
// ready edge unless the deadline is earlier still
static String firstAnswer(VClock ck, long fallbackMs, Source fast, Source slow) {
  TimedAnswer a = fast.answer();
  TimedAnswer b = slow.answer();
  long deadline = ck.nowMs + fallbackMs;

  long best = deadline;
  int which = -1; // -1: the deadline fires first
  if (a.readyAt() ≥ 0 && a.readyAt() < best) {
    best = a.readyAt();
    which = 0;
  }
  if (b.readyAt() ≥ 0 && b.readyAt() < best) {
    best = b.readyAt();
    which = 1;
  }
  ck.nowMs = best;

  if (which < 0)
    return null;
  String name = which == 0 ? "fast" : "slow";
  String val = which == 0 ? a.val() : b.val();
  return name + ": " + val;
}
```

listing 326: Java, the sources answer with their ready instants, the earliest edge wins unless the deadline is earlier still (source: patterns-concurrency-distributed/samples-java/src/Ch08/First.java, lines 26-72)

```
public static class First
{
    public static async Task<(string answer, string? error)> Answer(
        TimeProvider time, TimeSpan fallbackDelay, Func<Task<string>> fast, Func<Task<string>> slow)
    {
        using var cts = new CancellationTokenSource();
        Task deadline = Task.Delay(fallbackDelay, time, cts.Token);

        var pick = Channel.CreateBounded<string>(2);
        var producers = new Task[2];
        producers[0] = Task.Run(async () =>
        {
            string v = await fast();
            await pick.Writer.WriteAsync("fast: " + v);
        });
        producers[1] = Task.Run(async () =>
        {
            string v = await slow();
            await pick.Writer.WriteAsync("slow: " + v);
        });

        var read = pick.Reader.ReadAsync().AsTask();
        var winner = await Task.WhenAny(read, deadline);
        cts.Cancel();
        if (winner == read)
        {
            return (await read, null);
        }
        return ("", "neither source answered: deadline elapsed");
    }
}
```

listing 327: C#, WhenAny over the read and a Task.Delay on the injected TimeProvider, no real interval slept (source: patterns-concurrency-distributed/samples-cs/src/Ch08/First.cs, lines 10-40)

```
export class Clock {
  #now = 0;
  #timers = [];

  get now() {
    return this.#now;
  }

  sleep(ms) {
    const due = this.#now + ms;
    return new Promise((resolve) => this.#timers.push({ due, resolve }));
  }

  async advance(ms) {
    const target = this.#now + ms;
    for (;;) {
      let best = -1;
      for (let i = 0; i < this.#timers.length; i++) {
        if (this.#timers[i].due > target) continue;
        if (best === -1 || this.#timers[i].due < this.#timers[best].due) best = i;
      }
      if (best === -1) break;
      const t = this.#timers.splice(best, 1)[0];
      this.#now = t.due;
      t.resolve();
      for (let i = 0; i < 8; i++) await Promise.resolve();
    }
    this.#now = target;
  }
}

export async function first(clock, fallbackDelay, fast, slow) {
  const timeout = clock.sleep(fallbackDelay).then(() => ({ timeout: true }));
  const run = async (from, fn) => ({ from, val: await fn() });
  const winner = await Promise.race([
    run("fast", fast),
    run("slow", slow),
    timeout,
  ]);
  if (winner.timeout) {
    throw new Error("neither source answered: deadline elapsed");
  }
  return winner.from + ": " + winner.val;
}
```

listing 328: JavaScript, the fake timer queue, sleep parks on a due instant, advance fires the due ones in order (source: patterns-concurrency-distributed/samples-js/src/ch08-first.mjs, lines 9-52)

```
async def first(clock, fallback_delay, fast, slow):
    bomb = clock.timer(fallback_delay)

    async def run(name, fn):
        return name, await fn()

    producers = [
        asyncio.ensure_future(run("fast", fast)),
        asyncio.ensure_future(run("slow", slow)),
    ]
    bomb_task = asyncio.ensure_future(bomb.wait())
    done, _pending = await asyncio.wait(
        [*producers, bomb_task], return_when=asyncio.FIRST_COMPLETED
    )
    for t in (*producers, bomb_task):
        if not t.done():
            t.cancel()
    if bomb_task in done:
        raise DeadlineReached("neither source answered before the deadline")
    winner = next(t for t in producers if t in done)
    name, val = winner.result()
    return f"{name}: {val}"
```

listing 329: Python, the deadline is one more timer on the heap, asyncio.wait takes the first completion, losers are canceled (source: patterns-concurrency-distributed/samples-py/src/Ch08/first.py, lines 66-87)

```lua
local function First(C, deadlineMs, fast, slow)
  local out = {}
  local picked = false
  local c = ctx()
  local raceWaiter = coroutine.running()

  local function deliver(from, fn)
    return function()
      local v = fn(C, c)
      if not picked and not c.canceled then
        picked = true
        out.val = from .. ": " .. v
        out.at = C.now
        wake(C, raceWaiter)
      end
    end
  end

  spawn(C, deliver("fast", fast))
  spawn(C, deliver("slow", slow))
  spawn(C, function()
    vsleep(C, deadlineMs, c)
    if not picked then
      picked = true
      out.err = "neither source answered: context deadline exceeded"
      out.at = C.now
      wake(C, raceWaiter)
    end
  end)

  coroutine.yield("park")
  cancel(c, C)
  return out
end
```

listing 330: Lua, the deadline is one more racer, the first fire wins and cancels the sleeps still pending (source: patterns-concurrency-distributed/samples-lua/ch08_first.lua, lines 88-121)

The answer channel is buffered to 2 in the go lane so the loser's send never blocks after the winner is chosen, a small instance of the general rule that every producer must tolerate an unread result. The injected-clock lanes each rebuild that tolerance their own way: C lets the loser's answer sit in the pick buffer unread, rust's sources are pure records answering with their ready instant, so there is no loser send to absorb and the sample pins the tie the others leave open, a source ready exactly at the deadline loses to the deadline, java does the same and parks its native shape beside the clock, `anyOf` over two `CompletableFutures` answering from whichever future completed, C# writes into a bounded channel of 2, javascript's race drops the losing branch, python cancels the losing tasks, lua aborts the pending sleeps through the deadline's context. The winner is whichever edge fires first, the names are labels, and the C, rust, java, and lua lanes pin the counter-lane where the slow source answers first and wins anyway.

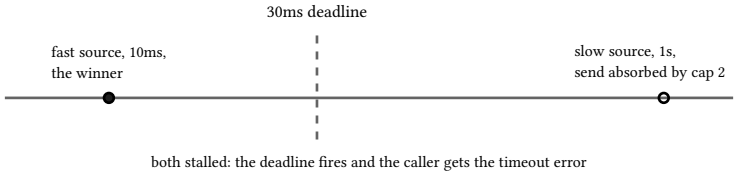


figure 49: first answer wins, the loser drains into the buffer

8.7. the leak test

The chapter's last test is the one worth copying into real projects. It runs the whole composite, generator into square into fan-in, 10 times, cancels, and asks each tree's own instrument whether anything stayed behind:

The dry run: 10 rounds of build, consume, cancel, drain. The go lane polls the goroutine count back to baseline after a GC. The seven new lanes read their instrument directly, C joins every thread and counts the live ones back to 0, rust reads its LIVE stage counter back to 0 inside each joined scope, java reads its live-thread counter back to 0 each round, C# reads the in-flight stage counter to 0 past the merged channel's completion, javascript reads the stage and forwarder probes to 0, python reads the alive probe to 0, lua asserts no lane stays parked. Only the go lane samples a runtime counter, so only it needs a polling window, and none of the eight asserts a timing.

```c
// the leak instrument, ten rounds of build consume cancel drain
for (int round = 0; round < 10; round++) {
  cancel_ctx cxr;
  ctx_init(&cxr);
  xchan m, o;
  xc_init(&m);
  xc_init(&o);
  ctx_watch(&cxr, &m);
  ctx_watch(&cxr, &o);
  struct gen_arg gr = {.out = &m, .cx = &cxr, .values = vals, .n = 10};
  struct stage_arg sr = {.in = &m, .out = &o, .cx = &cxr};
  CHECK(thrd_create(&gt, generator, &gr) == thrd_success);
  CHECK(thrd_create(&st, squarer, &sr) == thrd_success);
  CHECK(xc_recv(&o, &v));
  ctx_cancel(&cxr);
  while (xc_recv(&o, &v)) {
  }
  CHECK(thrd_join(gt, &rc) == thrd_success);
  CHECK(thrd_join(st, &rc) == thrd_success);
  CHECK(atomic_load(&live_threads) == 0);
  xc_free(&m);
  xc_free(&o);
}
```

listing 331: C, the check region: 10 rounds of build, cancel, drain, the live-thread count returns to 0 each round (source: patterns-concurrency-distributed/samples-c/src/Ch08/pipeline.c, lines 279-301)

```
    for round in 0..10 {
        let m = XChan::new();
        let o = XChan::new();
        let cxr = CancelCtx::new(vec![&m, &o]);
        std::thread::scope(|s| {
            let gt = s.spawn(|| generator(&VALS, &m));
            let st = s.spawn(|| squarer(&m, &o));
            let first = o.recv();
            cxr.cancel();
            while o.recv().is_some() {}
            gt.join().unwrap();
            st.join().unwrap();
            ok(
                first.is_some()
                    && LIVE.load(Ordering::SeqCst) == 0,
                &format!("round {round} builds, consumes, cancels and drains clean"),
            );
        });
    }
}
```

listing 332: Rust, the check region: ten rounds of build, consume, cancel, drain, the LIVE counter reads 0 each round (source: patterns-concurrency-distributed/samples-rust/src/Ch08/pipeline.rs, lines 240-259)

```
func TestNoGoroutineLeak(t *testing.T) {
  before := runtime.NumGoroutine()
  for i := 0; i < 10; i++ {
    ctx, cancel := context.WithCancel(context.Background())
    src := Generator(ctx, 1, 2, 3, 4, 5, 6, 7, 8)
    mid := Square(ctx, src)
    merged := FanIn(ctx, mid, Square(ctx, Generator(ctx, 9, 10)))
    for range merged {
    }
    cancel()
  }
  runtime.GC()
  deadline := time.Now().Add(2 * time.Second)
  for runtime.NumGoroutine() > before && time.Now().Before(deadline) {
    time.Sleep(10 * time.Millisecond)
    runtime.GC()
  }
  if now := runtime.NumGoroutine(); now > before {
    t.Fatalf("goroutines before=%d after=%d, leak", before, now)
  }
}
```

listing 333: Go, goroutine count as a leak detector (source: patterns-concurrency-distributed/samples/ch08/patterns_test.go, lines 148-168)

```
    // the leak instrument, ten rounds of build consume cancel drain
    for (int round = 0; round < 10; round++) {
      CancelCtx cxr = new CancelCtx();
      XChan m = new XChan();
      XChan o = new XChan();
      cxr.watch(m);
      cxr.watch(o);
      GenArg gr = new GenArg();
      gr.out = m;
      gr.values = vals;
      StageArg sr = new StageArg();
      sr.in = m;
      sr.out = o;
      Thread a = generator(gr);
      Thread b = squarer(sr);
      ok(o.recv(v), "round " + round + " produced a value");
      cxr.cancel();
      while (o.recv(v)) {
      }
      a.join();
      b.join();
      ok(liveThreads.get() == 0, "round " + round + " leaked no threads");
    }
```

listing 334: Java, the check region: ten rounds of build, consume, cancel, drain, the live-thread counter returns to 0 each round (source: patterns-concurrency-distributed/samples-java/src/Ch08/Pipeline.java, lines 275-297)

```
    [Fact]
    public async Task NoStageLeaks()
    {
        Assert.Equal(0, StageTelemetry.InFlight);
        for (int round = 0; round < 10; round++)
        {
            using var cts = new CancellationTokenSource();
            var src = Pipeline.Generator(cts.Token, 1, 2, 3, 4, 5, 6, 7, 8);
            var mid = Pipeline.Square(cts.Token, src);
            var merged = Fanin.Merge(
                cts.Token, mid,
                Pipeline.Square(cts.Token, Pipeline.Generator(cts.Token, 9, 10)));
            await SumOf(merged);
            cts.Cancel();
            // completion is the edge past which every stage ran its
            // Exit, so the counter reads true, not approximately
            await merged.Completion;
        }
        Assert.Equal(0, StageTelemetry.InFlight);
    }
```

listing 335: C#, xunit: completion is the edge past which every stage ran its Exit, the counter reads true, not approximately (source: patterns-concurrency-distributed/samples-cs/tests/Ch08/PipelineTests.cs, lines 81-100)

```
describe("ch08 leak lane", () => {
  it("stage and forwarder probes return to baseline after 10 cycles", async () => {
    assert.equal(liveStages(), 0);
    assert.equal(liveForwarders(), 0);
    for (let i = 0; i < 10; i++) {
      const tok = new CancelToken();
      const merged = fanIn(tok, [
        square(tok, generator(tok, [1, 2, 3, 4, 5, 6, 7, 8])),
        square(tok, generator(tok, [9, 10])),
      ]);
      for await (const _ of merged) {
        // drain
      }
      tok.cancel();
    }
    assert.equal(liveStages(), 0);
    assert.equal(liveForwarders(), 0);
  });
});
```

listing 336: JavaScript, node:test: stage and forwarder probes return to baseline after 10 cycles (source: patterns-concurrency-distributed/samples-js/test/ch08.test.mjs, lines 152-170)

```
async def leak_probe():
    totals = []
    for _ in range(10):
        clean = await sum_squares(list(range(1, 9)))
        totals.append(clean)
        if alive["stages"] != 0:
            return totals, False
    return totals, True


def main():
    check(
        asyncio.run(sum_squares(list(range(1, 11)))) == 385,
        "squares of 1..10 sum to 385",
    )
    check(asyncio.run(sum_squares([])) == 0, "an empty pipeline sums to 0")

    first, rest, probe = asyncio.run(cancel_scenario())
    check(first == 1, "the first square off the feeder is 1")
    check(rest == [], "cancel ends the chain with nothing left over")
    check(probe["stopped_by"] == "cancel", "the producer observed the cancel")
    check(probe["closed"], "and its finally ran")
    check(alive["stages"] == 0, "both stages unwound")

    totals, clean = asyncio.run(leak_probe())
    check(clean, "ten runs, every stage left the alive probe at zero")
    check(totals == [204] * 10, "each run computed the same total")
```

listing 337: Python, the check region: 10 runs, the alive probe stays at 0 and each run computes the same total (source: patterns-concurrency-distributed/samples-py/src/Ch08/pipeline.py, lines 87-115)

```lua
{name = "the leak instrument: no lane stays parked after the walk", fn = function()
  for _ = 1, 10 do
    local S = new_sched()
    local c = ctx()
    local probe = {}
    spawn(S, function()
      local sq = Square(S, c, Generator(S, c, { 1, 2, 3, 4 }))
      local seen = 0
      while true do
        local _, ok = recv(sq, c)
        if not ok then
          break
        end
        seen = seen + 1
        if seen == 2 then
          cancel(c) -- abandon the pipeline mid drain
        end
      end
      probe.seen = seen
    end)
    run(S)
    T.eq(S.live, 0, "a lane stayed parked, the pipeline leaks")
  end
end},
```

listing 338: Lua, the row table: cancel mid drain, no lane stays parked, the scheduler drains to live 0 (source: patterns-concurrency-distributed/samples-lua/ch08_pipeline.lua, lines 293-316)

Two seconds of polling absorbs scheduler noise in the go lane, and a count that never returns to baseline is a worker that ignored cancellation, exactly the bug every select and every token check in this chapter exists to prevent. Go 1.27 ships a sharper tool for the same job, the `goroutineleak` profile in `runtime/pprof` flags goroutines blocked on channels or mutexes that GC reachability says can never wake, covered with the other detectors in chapter 9.

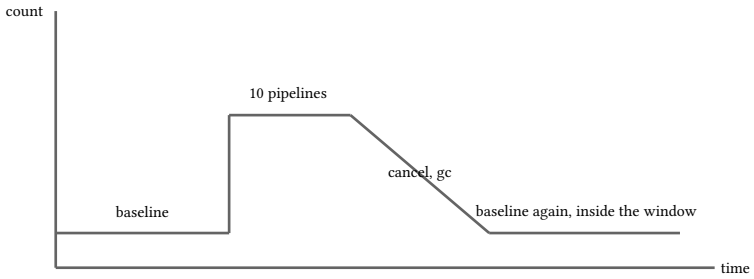


figure 50: goroutine count climbs, cancels, and returns to baseline

NOTE: pattern selection

Need results in input order with bounded parallelism: the ordered map. Need unordered throughput on a stream: the worker pool. Need to converge many producers: fan-in. Need to stop everything from one place: the cancellation carrier, threaded into all of them from the start. Retrofitting cancellation onto a finished pipeline is a rewrite.

8.8. across the eight languages

The build sizes count non-comment source lines, the go column the chapter's one frozen file:

language	build SLOC	dependency	boundary note
c	724	libc plus threads.h, stdatomic.h	the select analog a strict rendezvous, cancel broadcasts into blocked sends
rust	713	std only, no crates	the strict-rendezvous XChan with cancel retraction, scripted fan-in passes, the discrete-event ordered map, a virtual clock for the racers
go	129	stdlib	frozen reference lane, select on both directions, goroutine count as leak probe
java	599	jdk 27 stdlib	the XChan rendezvous and CancelCtx rolled by hand, the scripted fan-in trace, invokeAll handing back submission order
c#	204	bcl	the token into every channel wait, WhenAll the barrier, WhenAny over Task.Delay
javascript	196	node stdlib, one sibling module	a hand-spelled CancelToken, the finally head count, the fake timer queue
python	351	stdlib only	a Cancel over an Event, the sentinel after the gather, the heap clock
lua	1150	lib.lua harness	cancel aborts every parked operation, lanes per stage, the deadline a racer

sources: go.dev/pkg/context for cancellation trees, `Cause`, `AfterFunc`, go.dev/blog/pipelines for the generator and stage vocabulary, go.dev/doc/go1.27 for the goroutine leak profile, accessed 2026-09-08. Verified by the eight chapter legs: 5 Ch08 C programs with 270 embedded checks, 5 Ch08 rust programs in `samples-rust/src` with 111 embedded checks under `run-rust-samples`, `go test` at 8 tests in `patternsbook/ch08`, 5 Ch08 java programs with 69 checks under `run-java-samples`, 11 xunit facts, node's 8 cases in `test/ch08.test.mjs`, 5 python modules with 33 embedded checks, and the lua runner's 22 ch08 rows.

9. hazards and detectors

Four failure modes own concurrent code: races, deadlocks, starvation, and leaks. Chapter 6 defined the first one and its cure in the large, and chapter 8 closed the leak account with one instrument per tree. This chapter makes the middle two concrete across eight trees, shows the code that survives each, and equips the reader with the detectors each toolchain actually ships: the go race detector, the contention instruments of the other lanes, the goroutine leak profile new in go 1.27, and `testing/synctest`, the fake-clock bubble that turns time dependent concurrency tests deterministic, plus the seven portable rebuilds of that bubble the sibling trees carry.

9.1. deadlock, and the cycle rule

A deadlock is a cycle of workers where each waits for the next. The go runtime detects only the total version, every goroutine in the program asleep with no possibility of waking, and reports it as `fatal error: all goroutines are asleep - deadlock!`. A partial deadlock, 2 of a thousand workers circling each other while the rest work, produces no message at all, just a counter that stops moving, and none of the eight runtimes here detects that one for you.

The classic generator of the partial kind is dining philosophers with the naive rule, everyone picks up the left fork, then the right:

```
// the bug: a cycle of left-then-right waits
t.forks[left].Lock()
t.forks[right].Lock()  // holds left, waits for right
```

fragment, illustrative, not executed

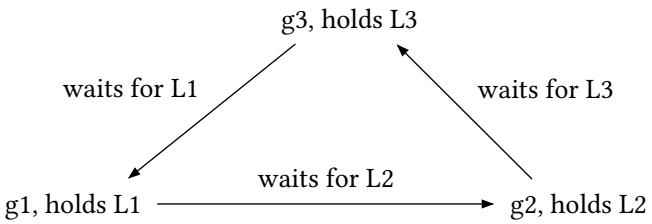


figure 51: the wait cycle, each goroutine holds one lock and waits for the next

5 diners, 5 forks, everyone holds their left and waits for their right, the wait graph is a cycle. The fix is total resource ordering: assign every lock a number and always acquire in ascending order, which makes a cycle impossible because the last holder of the highest lock cannot be waiting for a lower one.

The dry run: 5 diners x 100 bites each under the ordering rule and the guard, every seat eats exactly 100, the table total is 500. The counter-lanes pin the bug side by hand: lua runs the same scheduler on the naive left-then-right rule and all 5 diners park at 0 bites, python proves the naive rule closes a cycle in the fork graph and the ordered rule leaves it acyclic, java runs the same proof as a coloring DFS over the rule itself before the live table eats its 500.

```c
typedef struct {
  mtx_t forks[SEATS];
  int bites[SEATS];
} table;

static void table_init(table *t) {
  for (int i = 0; i < SEATS; i++) {
    mtx_init(&t->forks[i], mtx_plain);
    t->bites[i] = 0;
  }
}

static void table_free(table *t) {
  for (int i = 0; i < SEATS; i++)
    mtx_destroy(&t->forks[i]);
}

struct dine_arg {
  table *t;
  int seat;
  int rounds;
};

static int philosopher(void *a) {
  struct dine_arg *d = a;
  int left = d->seat;
  int right = (d->seat + 1) % SEATS;
  int first = left < right ? left : right;
  int second = left < right ? right : left;
  for (int r = 0; r < d->rounds; r++) {
    mtx_lock(&d->t->forks[first]);
    mtx_lock(&d->t->forks[second]);
    d->t->bites[d->seat]++;
    mtx_unlock(&d->t->forks[second]);
    mtx_unlock(&d->t->forks[first]);
  }
  return 0;
}
```

listing 339: C, 5 mtx_t forks, first and second by fork number, the run wrapped in the guard below (source: patterns-concurrency-distributed/samples-c/src/Ch09/dining.c, lines 70-107)

```rust
fn dinner(forks: &[Mutex<()>], bites: &[AtomicU32]) {
    std::thread::scope(|scope| {
        for seat in 0..SEATS {
            scope.spawn(move || {
                let left = seat;
                let right = (seat + 1) % SEATS;
                let (first, second) = if left < right {
                    (left, right)
                } else {
                    (right, left)
                };
                for _ in 0..ROUNDS {
                    let _lower = forks[first].lock().unwrap();
                    let _higher = forks[second].lock().unwrap();
                    bites[seat].fetch_add(1, Ordering::SeqCst);
                }
            });
        }
    });
}
```

listing 340: Rust, 5 Mutex forks over a zero-sized unit, scoped threads borrow the table, first and second by fork number, each fork back by drop (source: patterns-concurrency-distributed/samples-rust/src/Ch09/dining.rs, lines 56-75)

```go
func (t *Table) Dine(seat, rounds int) {
  left := seat
  right := (seat + 1) % len(t.forks)
  first, second := left, right
  if right < left {
    first, second = right, left
  }
  for r := 0; r < rounds; r++ {
    t.forks[first].Lock()
    t.forks[second].Lock()
    t.bites[seat]++
    t.forks[second].Unlock()
    t.forks[first].Unlock()
  }
}

func (t *Table) Bites() []int { return t.bites }
```

listing 341: Go, first and second by fork number, the cycle cannot form (source: patterns-concurrency-distributed/samples/ch09/hazards.go, lines 22-39)

```
  interface Rule {
    int[] pick(int seat, int count);
  }

  static int[] naiveRule(int seat, int count) {
    return new int[] {seat, (seat + 1) % count};
  }

  static int[] orderedRule(int seat, int count) {
    int left = seat;
    int right = (seat + 1) % count;
    return right < left ? new int[] {right, left} : new int[] {left, right};
  }

  // a diner can stall only on its second fork, and only while the other
  // sharer of that fork holds it as a first: that one sits holding a fork
  // while waiting. edges diner -> that sharer, and a cycle in the graph is
  // a possible deadlock
  static boolean hasWaitCycle(Rule rule, int count) {
    int[] first = new int[count];
    int[] second = new int[count];
    for (int seat = 0; seat < count; seat++) {
      int[] picks = rule.pick(seat, count);
      first[seat] = picks[0];
      second[seat] = picks[1];
    }
    int[] edges = new int[count];
    Arrays.fill(edges, -1);
    for (int seat = 0; seat < count; seat++) {
      for (int other = 0; other < count; other++) {
        if (other != seat && first[other] == second[seat]) {
          edges[seat] = other;
          break;
        }
      }
    }
    int[] color = new int[count]; // 0 unvisited, 1 on the stack, 2 done
    for (int seat = 0; seat < count; seat++) {
      if (color[seat] == 0 && visit(seat, edges, color)) {
        return true;
      }
    }
    return false;
  }
```

listing 342: Java, both rules as data, a coloring DFS over the wait graph proves the naive order cyclic and the ordered one acyclic (source: patterns-concurrency-distributed/samples-java/src/Ch09/Dining.java, lines 24-67)

```
public sealed class Table
{
    private readonly SemaphoreSlim[] _forks;
    private readonly int[] _bites;

    public Table(int n)
    {
        _forks = new SemaphoreSlim[n];
        for (int i = 0; i < n; i++)
        {
            _forks[i] = new SemaphoreSlim(1, 1);
        }
        _bites = new int[n];
    }

    public void Dine(int seat, int rounds)
    {
        int left = seat;
        int right = (seat + 1) % _forks.Length;
        int first = Math.Min(left, right);
        int second = Math.Max(left, right);
        for (int r = 0; r < rounds; r++)
        {
            _forks[first].Wait();
            _forks[second].Wait();
            _bites[seat]++;
            _forks[second].Release();
            _forks[first].Release();
        }
    }

    public int[] Bites() => (int[])_bites.Clone();
}
```

listing 343: C#, a fork is a binary SemaphoreSlim, the owned mutual exclusion shape, acquired ascending (source: patterns-concurrency-distributed/samples-cs/src/Ch09/Dining.cs, lines 8-40)

```
export class Table {
  #bites;

  constructor(n) {
    this.forks = Array.from({ length: n }, () => new AMutex());
    this.#bites = new Array(n).fill(0);
  }

  async dine(seat, rounds) {
    const left = seat;
    const right = (seat + 1) % this.forks.length;
    const first = Math.min(left, right);
    const second = Math.max(left, right);
    for (let r = 0; r < rounds; r++) {
      await this.forks[first].lock();
      await this.forks[second].lock();
      this.#bites[seat]++;
      this.forks[second].unlock();
      this.forks[first].unlock();
    }
  }

  bites() {
    return [...this.#bites];
  }
}
```

listing 344: JavaScript, forks are promise-queue mutexes, a naive rule would park every diner on a promise that never resolves (source: patterns-concurrency-distributed/samples-js/src/ch09-dining.mjs, lines 37-62)

```
class Table:
    def __init__(self, count):
        self.forks = [threading.Lock() for _ in range(count)]
        self.bites = [0] * count

    def dine(self, seat, rounds):
        first, second = ordered_rule(seat, len(self.forks))
        for _ in range(rounds):
            with self.forks[first]:
                with self.forks[second]:
                    self.bites[seat] += 1

    def bite_counts(self):
        return list(self.bites)
```

listing 345: Python, 5 real locks under the GIL, the cycle logic applies even where threads cannot tear memory (source: patterns-concurrency-distributed/samples-py/src/Ch09/dining.py, lines 65-79)

```lua
local Table = {}
Table.__index = Table

function Table.new(S, n)
  local t = { S = S, forks = {}, bites = {} }
  for i = 1, n do
    t.forks[i] = new_fork(S)
    t.bites[i] = 0
  end
  return setmetatable(t, Table)
end

-- the ordering rule is the whole lesson: lower number first. seat 5
-- reaches across for fork 1 before touching its own fork 5
function Table:order(seat)
  local left, right = seat, seat % #self.forks + 1
  if right < left then
    return right, left
  end
  return left, right
end

-- dine runs rounds bite cycles. window inserts a cooperative yield
-- between the two takes, standing in for the preemption window a real
-- scheduler would inject at exactly the wrong moment
function Table:dine(seat, rounds, ordered, window)
  local first, second = seat, seat % #self.forks + 1
  if ordered then
    first, second = self:order(seat)
  end
  for _ = 1, rounds do
    take(self.forks[first])
    if window then coroutine.yield() end
    take(self.forks[second])
    if window then coroutine.yield() end
    self.bites[seat] = self.bites[seat] + 1
    drop(self.forks[second])
    drop(self.forks[first])
  end
end
```

listing 346: Lua, forks hand ownership to the head waiter, the ordering rule is the whole lesson, the window stands in for preemption (source: patterns-concurrency-distributed/samples-lua/ch09_dining.lua, lines 59-98)

The eight lanes all carry the same rule and differ in what a fork is. C, rust, go, java, and python hold real kernel locks, C through `mtx_t`, rust through `std::sync::Mutex` over a zero-sized unit, go through `sync.Mutex`, java through `synchronized` on a plain object per fork, python through `threading.Lock`, and python's row is the honest one about scope: the GIL stops torn memory, it does nothing to a wait cycle, the graph logic is identical. Rust's scoped threads borrow the fork vector outright, no Arc indirection and no manual join, the borrow checker proving every seat outlives the table. C# reaches for a binary `SemaphoreSlim` because C# locks are monitor-bound to objects and a counting semaphore of 1 is the clean owned-exclusion primitive. Javascript builds the mutex as a promise queue, one thread but many coroutines, so the deadlock would be just as real, every diner parked on a resolver nobody calls. Lua's scheduler makes the hazard visible instead of preventing it: the naive run is a pinned fixture, 5 diners parked, 0 bites, and the ordered run feeds everyone.

9.2. the guard, hangs become failures

Any test that could hang needs a deadline, because a suite that freezes reports nothing. `MustFinishWithin` converts a would-be hang into a failure with a stack.

The dry run: an instant body passes everywhere. A stuck body trips the deadline in go, C, rust, java, javascript, and python, each with a real timer of a few tens of milliseconds, the one real-time primitive those trees allow in an asserted path, and java's stuck body announces entry on its own latch, so a fired deadline is provably a stuck body, never one that never started. Rust's `Condvar::wait_timeout` carries a relative span, so the guard re-derives the remaining budget from an `Instant` deadline on every wake, where C's `cnd_timedwait` hands the kernel one absolute timespec. C# pins only the pass path, tripping its guard would demand asserting on a real timed wait, which the tree's determinism policy refuses. Lua runs the whole contract on the virtual clock, a 30ms budget against a 1000ms hang trips at clock 30 in zero real time.

```c
// start the body and wait for it under a deadline: true when the body
// returned inside it. the caller joins h->t afterwards either way. the
// launch arg is static: the thread reads it after this frame is gone
static bool must_finish_within(guarded *h, long deadline_ms, body_fn body,
                               void *ctx) {
  guard_init(&h->g);
  static struct guarded_arg ga;
  ga.g = &h->g;
  ga.body = body;
  ga.ctx = ctx;
  if (thrd_create(&h->t, guarded_run, &ga) != thrd_success)
    return false;
  return guard_wait(&h->g, deadline_ms);
}
```

listing 347: C, the body on its own thread, a timed cond wait for the deadline, true only when the body returned (source: patterns-concurrency-distributed/samples-c/src/Ch09/guard.c, lines 84-97)

```rust
fn must_finish_within<F>(body: F, limit: Duration) -> (bool, std::thread::JoinHandle<()>)
where
    F: FnOnce() + Send + 'static,
{
    let guard = Arc::new(Guard::new());
    let trip = Arc::clone(&guard);
    let handle = std::thread::spawn(move || {
        body();
        trip.finish();
    });
    (guard.wait_within(limit), handle)
}
```

listing 348: Rust, the body a moved closure on its own thread, the guard returns the JoinHandle so a tripped deadline can still release the body (source: patterns-concurrency-distributed/samples-rust/src/Ch09/guard.rs, lines 53-64)

```go
func MustFinishWithin(d time.Duration, f func()) (ok bool) {
  done := make(chan struct{})
  go func() {
    f()
    close(done)
  }()
  select {
  case <-done:
    return true
  case <-time.After(d):
    return false
  }
}

// Backoff is the synctest target: retry with exponential delay.
```

listing 349: Go, timeout guard: deadlocks fail the test instead of freezing the suite (source: patterns-concurrency-distributed/samples/ch09/hazards.go, lines 114-128)

```java
  // the worker is returned with the answer: a false answer means the
  // deadline fired, and the caller can still release the body afterwards
  record Guarded(boolean finished, Thread worker) {}

  static Guarded mustFinishWithin(long deadlineMs, Runnable body) {
    CountDownLatch done = new CountDownLatch(1);
    Thread worker = new Thread(() -> {
      body.run();
      done.countDown();
    });
    worker.start();
    boolean finished = false;
    try {
      finished = done.await(deadlineMs, TimeUnit.MILLISECONDS);
    } catch (InterruptedException e) {
      throw new IllegalStateException(e);
    }
    return new Guarded(finished, worker);
  }
```

listing 350: Java, the body on its own thread, the caller awaits a done latch with the deadline, the returned worker lets a tripped guard still release it (source: patterns-concurrency-distributed/samples-java/src/Ch09/Guard.java, lines 24-42)

```
public static class Guard
{
    public static async Task<bool> MustFinishWithin(TimeSpan deadline, Action f)
    {
        var work = Task.Run(f);
        return await Task.WhenAny(work, Task.Delay(deadline)) == work;
    }

    public static async Task<bool> MustFinishWithin(TimeSpan deadline, Func<Task> f)
    {
        var work = Task.Run(f);
        return await Task.WhenAny(work, Task.Delay(deadline)) == work;
    }
}
```

listing 351: C#, the body on the pool, WhenAny against a Task.Delay, the one sanctioned real-time wait (source: patterns-concurrency-distributed/samples-cs/src/Ch09/Guard.cs, lines 9-22)

```
export function mustFinishWithin(ms, f) {
  return new Promise((resolve, reject) => {
    let finished = false;
    const timer = setTimeout(() => {
      if (finished) return;
      finished = true;
      resolve(false);
    }, ms);
    Promise.resolve(f()).then(
      (v) => {
        if (finished) return;
        finished = true;
        clearTimeout(timer);
        resolve(true);
      },
      (err) => {
        if (finished) return; // the guard already reported the hang
        finished = true;
        clearTimeout(timer);
        reject(err);
      },
    );
  });
}
```

listing 352: JavaScript, one real setTimeout races the body, a body that rejects surfaces its own error instead of the guard's false (source: patterns-concurrency-distributed/samples-js/src/ch09-guard.mjs, lines 9-32)

```python
def must_finish_within(limit, body):
    # body runs in a worker; the caller gets (finished, worker). a
    # False answer means the deadline fired, and the worker is
    # returned so the caller can still release it afterwards.
    done = threading.Event()

    def run():
        body()
        done.set()

    worker = threading.Thread(target=run, daemon=True)
    worker.start()
    return done.wait(limit), worker
```

listing 353: Python, the body in a daemon worker, Event.wait with the limit, the worker returned so a tripped guard can still release it (source: patterns-concurrency-distributed/samples-py/src/Ch09/guard.py, lines 18-30)

```lua
local function Guard(C, budgetMs, f)
  local out = {}
  local settled = false
  local c = ctx()
  local waiter = coroutine.running()
  spawn(C, function()
    f(C, c)
    if not settled then
      settled = true
      out.finished = true
      cancel(c, C) -- retire the deadline so it cannot advance the clock
      wake(C, waiter)
    end
  end)
  spawn(C, function()
    -- check the context before parking: a body that already won must
    -- not leave a stray deadline event behind to advance the clock
    if not c.canceled then
      vsleep(C, budgetMs, c)
    end
    if not settled then
      settled = true
      out.timedOut = true
      cancel(c, C)
      wake(C, waiter)
    end
  end)
  coroutine.yield("park")
  return out
end
```

listing 354: Lua, the deadline is one more racer on the virtual clock, cancel aborts the body's pending sleeps (source: patterns-concurrency-distributed/samples-lua/ch09_guard.lua, lines 86-115)

Every concurrency test in the capstone wraps its cluster in this guard. The general rule the pattern encodes: any wait that could plausibly never end must have a deadline, in production code as a timeout context, in tests as this helper, and the guarded body must be fast enough that a green run never trips it.

9.3. starvation, and the rwmutex decision

Starvation is a worker making progress while another never does. The sample pins the instance go programmers actually meet, a writer behind an unbroken stream of readers:

The dry run: 8 readers in an unbroken flood, 1 writer taking the write side 50 times, the writer completes 50 of 50 in every tree. Java pins the policy predicates pure before the flood runs, a waiting writer blocks new readers while active readers keep the writer out, and rust pins the same policy pre-flood, 3 serialized write acquisitions each holding the lock alone. The counter-lanes pin the failure mode: lua runs the same flood without the pending-writer rule and the writer's single acquisition lands only after all 1600 reader sections.

```c
// the policy lives here: a pending writer fences new readers out
static void rw_rlock(rwlock *l) {
  mtx_lock(&l->mu);
  while (l->writer_inside || l->writers_waiting > 0)
    cnd_wait(&l->quiet, &l->mu);
  l->readers++;
  mtx_unlock(&l->mu);
}

static void rw_runlock(rwlock *l) {
  mtx_lock(&l->mu);
  l->readers--;
  if (l->readers == 0)
    cnd_broadcast(&l->quiet);
  mtx_unlock(&l->mu);
}

static void rw_wlock(rwlock *l) {
  mtx_lock(&l->mu);
  l->writers_waiting++;
  while (l->writer_inside || l->readers > 0)
    cnd_wait(&l->quiet, &l->mu);
  l->writers_waiting--;
  l->writer_inside = true;
  mtx_unlock(&l->mu);
}

static void rw_wunlock(rwlock *l) {
  mtx_lock(&l->mu);
  l->writer_inside = false;
  cnd_broadcast(&l->quiet);
  mtx_unlock(&l->mu);
}
```

listing 355: C, the hand-rolled writer-preferring rwlock, readers wait while a writer is active or pending, the policy is the while condition (source: patterns-concurrency-distributed/samples-c/src/Ch09/starvation.c, lines 45-77)

```
    fn read(&self) -> ReadGuard<'_> {
        let mut s = self.state.lock().unwrap();
        while s.writer_inside || s.writers_waiting > 0 {
            s = self.quiet.wait(s).unwrap();
        }
        s.readers += 1;
        ReadGuard { lock: self }
    }

    fn write(&self) -> WriteGuard<'_> {
        let mut s = self.state.lock().unwrap();
        s.writers_waiting += 1;
        while s.writer_inside || s.readers > 0 {
            s = self.quiet.wait(s).unwrap();
        }
        s.writers_waiting -= 1;
        s.writer_inside = true;
        WriteGuard { lock: self }
    }

    fn snapshot(&self) -> (i64, bool, i64) {
        let s = self.state.lock().unwrap();
        (s.readers, s.writer_inside, s.writers_waiting)
    }
}

struct ReadGuard<'a> {
    lock: &'a RwPref,
}

impl Drop for ReadGuard<'_> {
    fn drop(&mut self) {
        let mut s = self.lock.state.lock().unwrap();
        s.readers -= 1;
        if s.readers == 0 {
            self.lock.quiet.notify_all();
        }
    }
}

struct WriteGuard<'a> {
    lock: &'a RwPref,
}

impl Drop for WriteGuard<'_> {
    fn drop(&mut self) {
        let mut s = self.lock.state.lock().unwrap();
        s.writer_inside = false;
        self.lock.quiet.notify_all();
    }
}
```

listing 356: Rust, the while condition is the same policy, ReadGuard and WriteGuard release in Drop so the paired unlock cannot be skipped (source: patterns-concurrency-distributed/samples-rust/src/Ch09/starvation.rs, lines 42-92)

```
func WriterProgress(readers, readsEach, writes int) int {
  var mu sync.RWMutex
  var wg sync.WaitGroup
  stop := make(chan struct{})

  for r := 0; r < readers; r++ {
    wg.Add(1)
    go func() {
      defer wg.Done()
      for {
        select {
        case <-stop:
          return
        default:
        }
        mu.RLock()
        mu.RUnlock()
        _ = readsEach
      }
    }()
  }

  done := make(chan int)
  go func() {
    completed := 0
    for i := 0; i < writes; i++ {
      mu.Lock()
      completed++
      mu.Unlock()
    }
    close(stop)
    done <- completed
  }()
  wg.Wait()
  return <-done
}
```

listing 357: Go, 8 readers in a tight loop, one writer counting acquisitions (source: patterns-concurrency-distributed/samples/ch09/hazards.go, lines 44-80)

```
    // the policy lives here: a pending writer fences new readers out
    boolean readAllowed() {
      synchronized (mu) {
        return !writer && waitingWriters == 0;
      }
    }

    boolean writeAllowed() {
      synchronized (mu) {
        return !writer && readers == 0;
      }
    }

    void acquireRead() {
      synchronized (mu) {
        while (writer || waitingWriters > 0) {
          waitQuietly();
        }
        readers++;
      }
    }

    void releaseRead() {
      synchronized (mu) {
        readers--;
        if (readers == 0) {
          mu.notifyAll();
        }
      }
    }

    void acquireWrite() {
      synchronized (mu) {
        waitingWriters++;
        while (writer || readers > 0) {
          waitQuietly();
        }
        waitingWriters--;
        writer = true;
      }
    }
```

listing 358: Java, the rwlock hand-rolled on a monitor because ReentrantReadWriteLock documents no reader fencing, the while condition is the policy (source: patterns-concurrency-distributed/samples-java/src/Ch09/Starvation.java, lines 31-71)

```
public static class Starvation
{
    public static async Task<int> WriterProgress(int readers, int writes)
    {
        var rw = new ReaderWriterLockSlim();
        using var stop = new ManualResetEventSlim(false);

        var readerTasks = new Task[readers];
        for (int r = 0; r < readers; r++)
        {
            readerTasks[r] = Task.Run(() =>
            {
                while (!stop.IsSet)
                {
                    rw.EnterReadLock();
                    rw.ExitReadLock();
                }
            });
        }

        int completed = 0;
        await Task.Run(() =>
        {
            for (int i = 0; i < writes; i++)
            {
                rw.EnterWriteLock();
                completed++;
                rw.ExitWriteLock();
            }
            stop.Set();
        });
        await Task.WhenAll(readerTasks);
        rw.Dispose();
        return completed;
    }
}
```

listing 359: C#, ReaderWriterLockSlim, the documented fairness policy that favors writers (source: patterns-concurrency-distributed/samples-cs/src/Ch09/Starvation.cs, lines 8-43)

```
export class RWLock {
  #readers = 0;
  #writing = false;
  #pendingWriters = 0;
  #queue = []; // { write, resolve }

  async readLock() {
    if (this.#writing || this.#pendingWriters > 0 || this.#queue.length > 0) {
      await new Promise((resolve) => this.#queue.push({ write: false, resolve }));
      return;
    }
    this.#readers++;
  }

  readUnlock() {
    this.#readers--;
    if (this.#readers === 0) this.#grantNext();
  }

  async writeLock() {
    this.#pendingWriters++;
    if (!this.#writing && this.#readers === 0 && this.#queue.length === 0) {
      this.#writing = true;
      this.#pendingWriters--;
      return;
    }
    await new Promise((resolve) => this.#queue.push({ write: true, resolve }));
  }

  writeUnlock() {
    this.#writing = false;
    this.#grantNext();
  }

  #grantNext() {
    while (this.#queue.length > 0) {
      const next = this.#queue[0];
      if (next.write) {
        if (this.#writing || this.#readers > 0) return;
        this.#queue.shift();
        this.#writing = true;
        this.#pendingWriters--;
        next.resolve();
        return;
      }
      if (this.#writing || this.#pendingWriters > 0) return;
      this.#queue.shift();
      this.#readers++;
      next.resolve();
    }
  }
}
```

listing 360: JavaScript, the rw lock as one fifo of requests, the pending-writer count is what stops read grants from cutting in line (source: patterns-concurrency-distributed/samples-js/src/ch09-starvation.mjs, lines 9-60)

```
class RWLock:
    def __init__(self):
        self.cv = threading.Condition()
        self.readers = 0
        self.writer = False
        self.waiting_writers = 0

    def read_allowed(self):
        return not self.writer and self.waiting_writers == 0

    def write_allowed(self):
        return not self.writer and self.readers == 0

    def acquire_read(self):
        with self.cv:
            while not self.read_allowed():
                self.cv.wait()
            self.readers += 1

    def release_read(self):
        with self.cv:
            self.readers -= 1
            if self.readers == 0:
                self.cv.notify_all()

    def acquire_write(self):
        with self.cv:
            self.waiting_writers += 1
            while not self.write_allowed():
                self.cv.wait()
            self.waiting_writers -= 1
            self.writer = True

    def release_write(self):
        with self.cv:
            self.writer = False
            self.cv.notify_all()
```

listing 361: Python, the rwlock built on a Condition, read_allowed excludes readers once a writer waits (source: patterns-concurrency-distributed/samples-py/src/Ch09/starvation.py, lines 18-54)

```lua
function RWMutex:RLock()
  while self.writer or (self.priority and #self.writeWaiters > 0) do
    self.parkedReads = self.parkedReads + 1
    local w = { co = coroutine.running() }
    self.readWaiters[#self.readWaiters + 1] = w
    coroutine.yield("park")
  end
  self.readers = self.readers + 1
end

function RWMutex:RUnlock()
  self.readers = self.readers - 1
  if self.readers == 0 then
    self:handOff()
  end
end

function RWMutex:Lock()
  while self.writer or self.readers > 0 do
    local w = { co = coroutine.running() }
    self.writeWaiters[#self.writeWaiters + 1] = w
    coroutine.yield("park")
    if w.claimed then
      return -- ownership was handed to us, writer stays held
    end
  end
  self.writer = true
end

function RWMutex:Unlock()
  self.writer = false
  self:handOff()
end

-- ownership transfer: prefer the oldest writer, and hand it the lock
-- outright; otherwise wake every parked reader to recheck
function RWMutex:handOff()
  local w = table.remove(self.writeWaiters, 1)
  if w then
    w.claimed = true
    self.writer = true
    self.S.ready[#self.S.ready + 1] = w.co
    return
  end
  for _, r in ipairs(self.readWaiters) do
    self.S.ready[#self.S.ready + 1] = r.co
  end
  self.readWaiters = {}
end
```

listing 362: Lua, the priority rule behind a flag so one walk shows both worlds, handoff prefers the oldest writer (source: patterns-concurrency-distributed/samples-lua/ch09_starvation.lua, lines 59-107)

The `sync.RWMutex` documentation states the guarantee being exercised: if a goroutine calls `Lock` while readers hold it, concurrent calls to `RLock` block until that writer has acquired and released. The reader flood cannot starve the writer, and the price is printed in the same paragraph: recursive read locking is prohibited, because a reentrant reader could wedge a pending writer forever. Choosing `RWMutex` over `Mutex` is choosing this scheduling policy as much as any performance. The policy is exactly what the hand-rolled lanes encode in their wait conditions, C and

python because `threads.h` and the python stdlib ship no rwlock at all, rust and java because `std::sync::RwLock` and `ReentrantReadWriteLock` document no fencing of new readers behind a pending writer, so fairness becomes a policy you implement and can read off the while loop. C# documents the same writer preference for `ReaderWriterLockSlim`. Javascript and lua have no kernel to blame, so their floods are coroutine choreography, and the js lane keeps one fifo of requests where the pending-writer count is the priority, a read request enqueued while a writer waits is granted only after it.

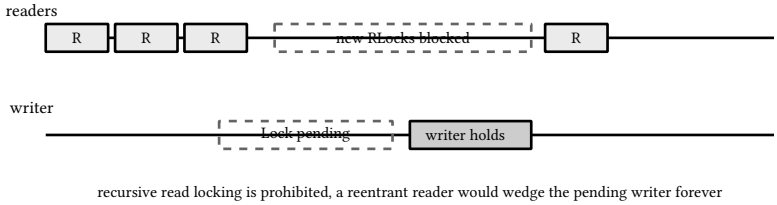


figure 52: the writer announces, new readers queue behind it

Livelock, the third cousin, is workers that keep waking and retrying without progressing, two `TryLock` spinners politely backing off forever. The detector story is the same as starvation, a counter that never advances, and the cure is always the same family: randomize backoff or serialize through a tie breaker.

9.4. the detectors

The race detector remains the primary instrument. `go test -race` instruments every memory access through shadow memory and vector clocks, reports races the moment they happen at runtime, and the documented cost is 2 to 20 times the execution time and 5 to 10 times the memory. Two limits to keep straight: it finds races that execute, a racy path the tests never walk is invisible to it, and it is unavailable on platforms without the instrumentation, `wasm` among them. This platform adds a third limit for the C lane: the clang that builds the samples has no thread sanitizer binary for the msvc target, the asan staging exists and tsan does not, the same wall the icpc book documents for its C toolbox's sanitizer legs, book 11, ch 2 (the icpc world finals, the c toolbox). The C tree earns its concurrency claims by construction instead, every observed interleaving either scripted or invariant checked, the rule chapter 1 stated, restated here because this is the chapter where a detector would be missed most.

Rust needs no runtime detector for the class: safe code rules data races out at compile time, `Send` and `Sync` gate what may cross a thread boundary and the borrow checker refuses two live writers to one location, so a race would have to come from `unsafe`, which no sample in the rust tree uses.

The dry run: 8 workers x 5000 increments through one lock total exactly 40000, and every lane's contention instrument records at least one event. The seven new trees pin the total itself, while the frozen go test enables the mutex profile at rate 1 and asserts only a nonzero sample count. The seven siblings count contention on an instrumented lock because their runtimes expose nothing finer: C counts trylocks that found the lock held, rust counts try_lock failures before blocking, its start window opened by the std `Barrier` the C lane rolls by hand, java counts tryLock failures on a ReentrantLock before blocking, C# counts Monitor.TryEnter failures before blocking, javascript counts acquisitions that queued on the promise-queue mutex, python counts failed non-blocking acquires, lua counts parks and the waiter queue's peak depth. The python, C#, and java lanes add a forced window, a holder announces while holding and a waiter probes after, recording exactly 1 contended acquisition, and java's window is airtight by construction, the release latch opens only after the probe latch proves the waiter already found the lock held.

```c
static int hammer(void *a) {
  hammer_arg *h = a;
  bar_wait(h->start);
  for (int i = 0; i < h->loops; i++) {
    if (mtx_trylock(h->mu) == thrd_busy) {
      // the lock was held: this is the contention event the go lane's
      // profiler would record, counted here by hand
      atomic_fetch_add(h->contended, 1);
      mtx_lock(h->mu);
    }
    (*h->counter)++;
    mtx_unlock(h->mu);
  }
  return 0;
}
```

listing 363: C, the instrumented acquire, a trylock that finds the lock held counts one contention event before blocking (source: patterns-concurrency-distributed/samples-c/src/Ch09/contention.c, lines 65-79)

```rust
fn contended_section(counter: &Mutex<usize>, contended: &AtomicUsize) {
    let start = Barrier::new(THREADS);
    std::thread::scope(|scope| {
        for _ in 0..THREADS {
            scope.spawn(|| {
                start.wait();
                for _ in 0..LOOPS {
                    let mut guard = counter.try_lock().unwrap_or_else(|_| {
                        contended.fetch_add(1, Ordering::SeqCst);
                        counter.lock().unwrap()
                    });
                    *guard += 1;
                }
            });
        }
    });
}
```

listing 364: Rust, the instrumented acquire, a try_lock that comes back with the lock held counts one contention event before blocking, a std Barrier opens the window (source: patterns-concurrency-distributed/samples-rust/src/Ch09/contention.rs, lines 19-35)

```
func ContendedSection(n, loops int) int {
  var mu sync.Mutex
  counter := 0
  var wg sync.WaitGroup
  for g := 0; g < n; g++ {
    wg.Add(1)
    go func() {
      defer wg.Done()
      for i := 0; i < loops; i++ {
        mu.Lock()
        counter++
        mu.Unlock()
      }
    }()
  }
  wg.Wait()
  return counter
}

// EnableContentionProfiles turns on the two contention profilers
// and returns a restore function.
func EnableContentionProfiles() (restore func()) {
  oldMutex := runtime.SetMutexProfileFraction(1)
  return func() {
    runtime.SetMutexProfileFraction(oldMutex)
  }
}

// MustFinishWithin fails the current test if f does not return in
```

listing 365: Go, contended section as bait, mutex profile enabled at rate 1 (source: patterns-concurrency-distributed/samples/ch09/hazards.go, lines 83-111)

```
  // the instrument: an acquisition that finds the lock held counts as
  // contended before it blocks. onContended fires on that path, so a test
  // can observe the bounce itself before the parking starts
  static final class ContendedLock {
    private final ReentrantLock lock = new ReentrantLock();
    long contended;
    Runnable onContended = () -> {};

    void acquire() {
      if (!lock.tryLock()) {
        contended++;
        onContended.run();
        lock.lock();
      }
    }

    void release() {
      lock.unlock();
    }
  }
```

listing 366: Java, the instrumented acquire, tryLock finds the lock held, count one contention then block, onContended reports the bounce (source: patterns-concurrency-distributed/samples-java/src/Ch09/Contention.java, lines 26-45)

```
public class ContendedLock
{
    private readonly object _gate = new();

    public long Contended { get; private set; }

    protected virtual void OnContended()
    {
    }

    public void Run(Action critical)
    {
        if (!Monitor.TryEnter(_gate))
        {
            Contended++;
            OnContended();
            Monitor.Enter(_gate);
        }
        try
        {
            critical();
        }
        finally
        {
            Monitor.Exit(_gate);
        }
    }
}
```

listing 367: C#, TryEnter fails, count, then block, OnContended fires on the bounce itself (source: patterns-concurrency-distributed/samples-cs/src/Ch09/Contention.cs, lines 8-35)

```
export async function contendedSection(n, loops) {
  const mu = new AMutex();
  let counter = 0;
  const worker = async () => {
    for (let i = 0; i < loops; i++) {
      await mu.lock();
      counter++;
      mu.unlock();
    }
  };
  await Promise.all(Array.from({ length: n }, () => worker()));
  return { total: counter, contended: mu.contended };
}
```

listing 368: JavaScript, the mutex's own contended counter, how many acquisitions arrived while the lock was held (source: patterns-concurrency-distributed/samples-js/src/ch09-contention.mjs, lines 10-23)

```python
class ProfiledLock:
    def __init__(self):
        self.lock = threading.Lock()
        self.contended = 0

    def acquire(self):
        if not self.lock.acquire(blocking=False):
            self.contended += 1
            self.lock.acquire()

    def release(self):
        self.lock.release()
```

listing 369: Python, the profiled lock, a non-blocking acquire that fails counts one contended event (source: patterns-concurrency-distributed/samples-py/src/Ch09/contention.py, lines 19-31)

```lua
local function new_lock(S)
  return { S = S, held = false, waiters = {}, parks = 0, maxQueue = 0 }
end

-- the claim flag is the handoff: a woken head waiter must not requeue
-- behind a lock it already owns
local function lock(m)
  while m.held do
    local w = { co = coroutine.running() }
    m.waiters[#m.waiters + 1] = w
    m.parks = m.parks + 1
    if #m.waiters > m.maxQueue then
      m.maxQueue = #m.waiters
    end
    coroutine.yield("park")
    if w.claimed then return end
  end
  m.held = true
end

local function unlock(m)
  local w = table.remove(m.waiters, 1)
  if w then
    w.claimed = true
    m.S.ready[#m.S.ready + 1] = w.co -- the head waiter takes over, held stays
  else
    m.held = false
  end
end
```

listing 370: Lua, parks and queue depth on the lock itself, the counter exact under a scripted interleaving (source: patterns-concurrency-distributed/samples-lua/ch09_contention.lua, lines 30-58)

The test enables the mutex profile, runs 8 goroutines through 5000 contended lock cycles, and asserts `pprof.Lookup("mutex").Count()` is nonzero, the profile is really wired. In a live service the same profile, plus `block` for channel and `goroutine` for dumps, rides the `/debug/pprof` endpoint of `net/http/pprof` on the default mux, and the profile family's home in this corpus is the runtime chapter, book 4, ch 11 (go 1.27, profiling: pprof, the tracer, benchstat, p99).

The python row points at its own runtime's instruments: cprofile and, since 3.12, sys.monitoring, either of which shows the hot function but neither of which surfaces per-acquisition contention, which is why the lane counts, book 8, ch 14 (python 3.14, profiling: cprofile, sys.monitoring, and p99).

The lua row reaches for its real sampler, `debug.sethook` counting hook fires, the counting profiler its book builds, the instrument to reach for when stacks matter, while this lane's counters answer the question the fixture asks, book 9, ch 10 (lua 5.5, profiling: hooks, clocks, and p99).

The C row has no runtime profiler at all, so measurement discipline replaces one, count the events you care about and read percentiles off them, the habit the systems book's measurement chapter teaches, book 1, ch 20 (c23: the language, the machine, and the cloud, measurement, profiling, and p99).

Go 1.27 promotes the sharpest of the family to general availability: the `goroutineleak` profile detects goroutines blocked on channels, mutexes, or conds that garbage collection reachability proves can never wake, the difference between "many goroutines" and "many dead goroutines". Chapter 8's poll-the-count test catches leaks eventually, this profile names them with stacks.

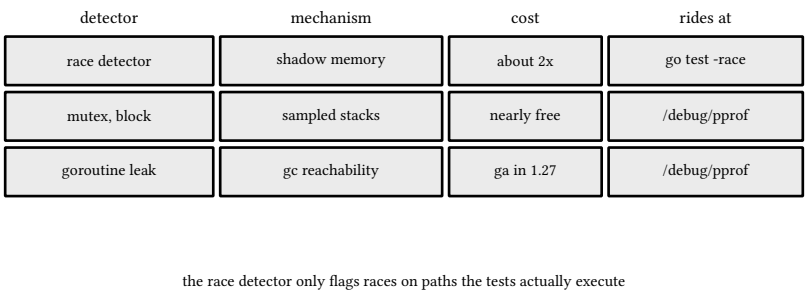


figure 53: three detectors, three mechanisms, one endpoint

9.5. synctest, deterministic time

`testing/synctest` runs a test inside a bubble where the `time` package is fake and the clock advances only when every goroutine in the bubble is durably blocked. Timers fire in order, instantly, and a deadlock inside the bubble fails the test instead of hanging it, an extension of the toolchain's own testing discipline, book 4, ch 14 (go 1.27, testing).

The dry run: `Backoff(6, 10ms)` walks 6 tries, the ladder doubles 10, 20, 40, 80, 160, 320, cumulative 10, 30, 70, 150, 310, 630, total 630ms, and the run costs zero real sleep. The contract is identical in all 8 trees. The bubble is go's tool and only go's: the go lane runs inside `synctest.Test`, where the runtime refuses to advance the clock while any participant can still run. The seven siblings rebuild the same rule by construction, C's discrete-event scheduler jumps time to the earliest wake only once every participant parks, rust scripts each participant as a typed enum of Work and Wait steps swept by the same discrete clock, java has no bubble and no coroutine to park mid-function, so participants run as scripts of work and sleep steps and a discrete scheduler sweeps one step per participant while anyone still has work, jumping time to the earliest wake only once everybody parks, C#'s fake TimeProvider fires the earliest due timer while its driver holds the clock for the continuations to drain, javascript's timer queue fires only when nothing else can run and reports a deadlock when nothing can, python advances its heap clock only when the event loop's ready deque is empty, lua parks lanes on a time-ordered event queue the runner pops in order.

```
static void bubble_run(bubble *b) {
  long total = 0;
  for (;;) {
    bool all_done = true, any_awake = false;
    for (int i = 0; i < b->n_parts; i++) {
      participant *p = &b->ps[i];
      if (p->at ≥ p->n_steps && !p->parked)
        continue; // finished: script exhausted and awake
      all_done = false;
      if (p->parked)
        continue; // still owes its wake edge
      any_awake = true;
      step s = p->script[p->at++];
      if (s.kind == STEP_WORK) {
        snprintf(b->events[b->n_events++], 32, "%s %s", p->name, s.tag);
      } else {
        // the sleep edge: park now, wake at now + delay
        p->parked = true;
        p->wake = b->ck->now_ms + s.ms;
        total += s.ms;
        b->waits[b->n_waits] = s.ms;
        b->cumulative[b->n_waits] = total;
        b->n_waits++;
      }
    }
    if (all_done)
      return;
    if (any_awake)
      continue; // someone still has work: the bubble holds time still
    // everyone is parked: advance to the earliest wake, the bubble move
    long best = -1;
    for (int i = 0; i < b->n_parts; i++)
      if (b->ps[i].parked && (best < 0 || b->ps[i].wake < best))
        best = b->ps[i].wake;
    b->ck->now_ms = best;
    b->ck->advances++;
    for (int i = 0; i < b->n_parts; i++)
      if (b->ps[i].parked && b->ps[i].wake == best) {
        b->ps[i].parked = false;
        snprintf(b->events[b->n_events++], 32, "%s wakes", b->ps[i].name);
      }
  }
}
```

listing 371: C, the bubble runner, someone still working holds time still, everyone parked jumps to the earliest wake (source: patterns-concurrency-distributed/samples-c/src/Ch09/synctest.c, lines 60-102)

```
        loop {
            let mut all_done = true;
            let mut any_awake = false;
            for p in parts.iter_mut() {
                if p.at ≥ p.script.len() && !p.parked {
                    continue;
                }
                all_done = false;
                if p.parked {
                    continue;
                }
                any_awake = true;
                match p.script[p.at] {
                    Step::Work(tag) ⇒ {
                        p.at += 1;
                        events.push(format!("{} {}", p.name, tag));
                    }
                    Step::Wait(ms) ⇒ {
                        p.at += 1;
                        p.parked = true;
                        p.wake = clock.now_ms + ms;
                        total += ms;
                        waits.push(ms);
                        cumulative.push(total);
                    }
                }
            }
            if all_done {
                return;
            }
            if any_awake {
                continue;
            }
            let mut best = u64::MAX;
            for p in parts.iter() {
                if p.parked && p.wake < best {
                    best = p.wake;
                }
            }
            clock.now_ms = best;
            clock.advances += 1;
            for p in parts.iter_mut() {
                if p.parked && p.wake == best {
                    p.parked = false;
                    events.push(format!("{} wakes", p.name));
                }
            }
        }
```

listing 372: Rust, the same bubble runner over a typed Step enum, someone still working holds time still, everyone parked jumps to the earliest wake (source: patterns-concurrency-distributed/samples-rust/src/Ch09/synctest.rs, lines 68-115)

```
func Backoff(attempts int, base time.Duration) (tries int, total time.Duration) {
  delay := base
  for t := 0; t < attempts; t++ {
    tries++
    total += delay
    time.Sleep(delay)
    delay *= 2
  }
  return tries, total
}
```

listing 373: Go, exponential backoff, 6 attempts, all schedule and no sleep (source: patterns-concurrency-distributed/samples/ch09/hazards.go, lines 131-140)

```
    void run() {
      long total = 0;
      while (true) {
        boolean allDone = true;
        boolean anyAwake = false;
        for (Participant p : ps) {
          if (p.finished()) {
            continue; // script exhausted and awake
          }
          allDone = false;
          if (p.parked) {
            continue; // still owes its wake edge
          }
          anyAwake = true;
          Step s = p.script[p.at++];
          if (s.kind() == WORK) {
            events.add(p.name + " " + s.tag());
          } else {
            // the sleep edge: park now, wake at now + delay
            p.parked = true;
            p.wake = clock.nowMs + s.ms();
            total += s.ms();
            waits.add(s.ms());
            cumulative.add(total);
          }
        }
        if (allDone) {
          return;
        }
        if (anyAwake) {
          continue; // someone still has work: the bubble holds time still
        }
        // everyone is parked: advance to the earliest wake, the bubble move
        long best = -1;
        for (Participant p : ps) {
          if (p.parked && (best < 0 || p.wake < best)) {
            best = p.wake;
          }
        }
        clock.nowMs = best;
        clock.advances++;
        for (Participant p : ps) {
          if (p.parked && p.wake == best) {
            p.parked = false;
            events.add(p.name + " wakes");
            wakeLog.add(new Wake(p.name, best));
          }
        }
      }
    }
```

listing 374: Java, no coroutine to park mid-function, participants run as work and sleep scripts, time jumps to the earliest wake only once everybody parks (source: patterns-concurrency-distributed/samples-java/src/Ch09/Synctest.java, lines 74-124)

```
public sealed record BackoffResult(int Tries, TimeSpan Total, IReadOnlyList<TimeSpan> Ladder);

// Backoff is the bubble-semantics target: retry with exponential
// delay. Go runs it inside a testing/synctest bubble where the clock
// advances only when every participant blocks. The portable
// discipline is the injected clock: each sleep is a Task.Delay on
// the TimeProvider, the test fires due timers one jump at a time,
// and the whole ladder costs zero real time while virtual time
// moves exactly as far as the schedule says.
public static class Synctest
{
    public static async Task<BackoffResult> Backoff(
        int attempts, TimeSpan baseDelay, TimeProvider time)
    {
        var ladder = new List<TimeSpan>();
        var delay = baseDelay;
        TimeSpan total = TimeSpan.Zero;
        for (int t = 0; t < attempts; t++)
        {
            ladder.Add(delay);
            total += delay;
            await Task.Delay(delay, time);
            delay = TimeSpan.FromTicks(delay.Ticks * 2);
        }
        return new BackoffResult(attempts, total, ladder);
    }
}
```

listing 375: C#, every delay is a Task.Delay on the injected TimeProvider, the ladder is schedule and no sleep (source: patterns-concurrency-distributed/samples-cs/src/Ch09/Synctest.cs, lines 3-29)

```
// runInBubble is synctest.Test: start f on the virtual clock, let it
// run, and whenever it can no longer progress, fire the earliest
// timer. resolves with f's value and the clock, so the test reads
// how much virtual time the bubble consumed.
export async function runInBubble(fn) {
  const clock = new Clock();
  let state = "running";
  let out;
  const done = Promise.resolve()
    .then(() => fn(clock))
    .then(
      (v) => {
        state = "ok";
        out = v;
      },
      (e) => {
        state = "err";
        out = e;
      },
    );
  for (;;) {
    for (let i = 0; i < 16; i++) await Promise.resolve();
    if (state !== "running") break;
    if (!(await clock.fireEarliest())) {
      throw new Error(
        "bubble deadlock: all participants blocked and no timer to fire",
      );
    }
  }
  await done;
  if (state === "err") throw out;
  return { value: out, clock };
}

// Backoff is the bubble's target: retry with exponential delay.
// inside the bubble the sleeps cost nothing and the ladder is
// deterministic: 10, 20, 40, 80, 160, 320, cumulative 630.
export async function backoff(clock, attempts, base) {
  let tries = 0;
  let total = 0;
  let delay = base;
  const ladder = [];
  const cumulative = [];
  for (let t = 0; t < attempts; t++) {
    tries++;
    total += delay;
    ladder.push(delay);
    cumulative.push(total);
    await clock.sleep(delay);
    delay *= 2;
  }
  return { tries, total, ladder, cumulative };
}
```

listing 376: JavaScript, runInBubble is the bubble port, fire the earliest timer when nothing else can run, deadlock says so (source: patterns-concurrency-distributed/samples-js/src/ch09-synctest.mjs, lines 60-112)

```
class VirtualClock:
    def __init__(self):
        self.now = 0.0
        self.seq = 0
        self.timers = []

    def timer(self, delay):
        ev = asyncio.Event()
        heapq.heappush(self.timers, (self.now + max(delay, 0.0), self.seq, ev))
        self.seq += 1
        return ev

    async def sleep(self, delay):
        if delay ≤ 0:
            await asyncio.sleep(0)
            return
        await self.timer(delay).wait()

    async def drive(self, main):
        # the bubble rule: after each yield, an empty _ready (the
        # loop's pending callback deque) means no task can move
        # without virtual time, so advancing to the earliest deadline
        # is safe and loses no interleaving.
        task = asyncio.ensure_future(main)
        loop = asyncio.get_running_loop()
        while not task.done():
            await asyncio.sleep(0)
            if task.done():
                break
            if loop._ready:
                continue
            if not self.timers:
                continue
            deadline, _, ev = heapq.heappop(self.timers)
            if deadline > self.now:
                self.now = deadline
            ev.set()
        for _, _, ev in self.timers:
            ev.set()
        self.timers.clear()
        return await task
```

listing 377: Python, the heap clock, advance only when the loop's ready deque is empty, the bubble rule (source: patterns-concurrency-distributed/samples-py/src/Ch09/synctest.py, lines 22-62)

```lua
local function vrun(C)
  while true do
    while #C.ready > 0 do
      local co = table.remove(C.ready, 1)
      local ok, why = coroutine.resume(co)
      if not ok then error(why, 0) end
      if coroutine.status(co) == "dead" then
        C.live = C.live - 1
      elseif why ~= "park" then
        C.ready[#C.ready + 1] = co
      end
    end
    if #C.events == 0 then
      break
    end
    table.sort(C.events, function(a, b)
      if a.at ~= b.at then
        return a.at < b.at
      end
      return a.seq < b.seq
    end)
    local ev = table.remove(C.events, 1)
    C.now = ev.at
    ev.woken = true
    wake(C, ev.co)
  end
end

-- Backoff retries with exponential delay: base, then doubling. on the
-- virtual clock the sleeps resolve in order and cost nothing real
local function Backoff(C, attempts, base, probe)
  local delay = base
  local tries, total = 0, 0
  for _ = 1, attempts do
    tries = tries + 1
    total = total + delay
    probe.ladder[#probe.ladder + 1] = delay
    probe.cumulative[#probe.cumulative + 1] = total
    vsleep(C, delay)
    delay = delay * 2
  end
  return tries, total
end
```

listing 378: Lua, run every ready lane to its next block, then advance to the earliest pending event (source: patterns-concurrency-distributed/samples-lua/ch09_synctest.lua, lines 32-74)

The test side is half the fixture, and each tree answers with its own test artifact: the frozen go test's bubble, the xunit and node test files, and the check regions of the C, rust, java, python, and lua modules.

```
  // the ladder: 6 attempts at base 10ms run as one participant, every sleep
  // through the clock, zero real time spent
  vclock ck2 = {.now_ms = 0, .advances = 0};
  step ladder[MAX_STEPS];
  int n_steps = backoff_script(ladder, 6, 10);
  CHECK(n_steps == 6);
  participant solo[1] = {{.name = "backoff", .script = ladder, .n_steps = 6}};
  bubble b2 = {.ck = &ck2, .ps = solo, .n_parts = 1};
  bubble_run(&b2);

  CHECK(b2.n_waits == 6); // tries
  const long want_ladder[6] = {10, 20, 40, 80, 160, 320};
  const long want_cum[6] = {10, 30, 70, 150, 310, 630};
  for (int i = 0; i < 6; i++) {
    CHECK(b2.waits[i] == want_ladder[i]);
    CHECK(b2.cumulative[i] == want_cum[i]);
  }
  CHECK(b2.cumulative[5] == 630); // total scheduled delay
  CHECK(ck2.now_ms == 630);       // the bubble clock advanced by exactly it
  CHECK(ck2.advances == 6);
```

listing 379: C, the check region: the ladder and cumulative arrays exact, the clock advanced by exactly 630ms in 6 jumps (source: patterns-concurrency-distributed/samples-c/src/Ch09/synctest.c, lines 146-165)

```
    let mut clock = VClock {
        now_ms: 0,
        advances: 0,
    };
    let ladder = backoff_script(6, 10);
    ok(ladder.len() == 6, "the backoff ladder holds 6 attempts");
    let mut bubble = Bubble {
        clock: &mut clock,
        parts: vec![Participant::new("backoff", &ladder)],
        events: Vec::new(),
        waits: Vec::new(),
        cumulative: Vec::new(),
    };
    bubble.run();

    ok(bubble.waits.len() == 6, "every attempt slept once");
    let want_ladder = [10u64, 20, 40, 80, 160, 320];
    let want_cumulative = [10u64, 30, 70, 150, 310, 630];
    for i in 0..6 {
        ok(
            bubble.waits[i] == want_ladder[i],
            &format!("attempt {i} sleeps {}ms", want_ladder[i]),
        );
        ok(
            bubble.cumulative[i] == want_cumulative[i],
            &format!("attempt {i} carries {}ms cumulative", want_cumulative[i]),
        );
    }
    ok(bubble.cumulative[5] == 630, "the total scheduled delay is 630ms");
    ok(
        bubble.clock.now_ms == 630,
        "the bubble clock advanced by exactly the total",
    );
    ok(
        bubble.clock.advances == 6,
        "the clock moved once per parked wake",
    );
}
```

listing 380: Rust, the check region: ladder and cumulative matched element for element against want arrays, the clock at 630 in 6 advances (source: patterns-concurrency-distributed/samples-rust/src/Ch09/synctest.rs, lines 166-203)

```
func TestBackoffDeterministicInBubble(t *testing.T) {
  synctest.Test(t, func(t *testing.T) {
    start := time.Now()
    tries, total := Backoff(6, 10*time.Millisecond)

    if tries ≠ 6 {
      t.Fatalf("tries = %d", tries)
    }
    // 10+20+40+80+160+320 = 630ms of scheduled delay
    if total ≠ 630*time.Millisecond {
      t.Fatalf("total = %s", total)
    }
    if elapsed := time.Since(start); elapsed ≠ 630*time.Millisecond {
      t.Fatalf("bubble clock advanced %s, want 630ms", elapsed)
    }
  })
}
```

listing 381: Go, inside the bubble the full backoff ladder runs and the clock reads exactly 630ms (source: patterns-concurrency-distributed/samples/ch09/hazards_test.go, lines 65-81)

```
    // two sleepers in one bubble: a works, sleeps 30; b sleeps 10, works.
    // time must hold at 0 while a still has work, then jump to 10 for b,
    // then to 30 for a, exactly the ordering the bubble forces
    Vclock ck = new Vclock();
    Bubble b = new Bubble(ck, new Participant[] {
        new Participant("a", new Step[] {Step.work("starts"), Step.sleep(30),
            Step.work("finishes")}),
        new Participant("b", new Step[] {Step.sleep(10), Step.work("finishes")})});
    b.run();
    ok(ck.nowMs == 30, "the bubble clock ends at 30");
    ok(ck.advances == 2, "0 -> 10 -> 30, never a partial move");
    ok(b.events.equals(
            List.of("a starts", "b wakes", "b finishes", "a wakes", "a finishes")),
        "the interleaving is pinned, byte for byte");
    ok(b.waits.equals(List.of(10L, 30L)),
        "b's 10ms parked first, a's 30ms second");
    ok(b.cumulative.equals(List.of(10L, 40L)), "the waits accumulate 10 then 40");

    // the ladder: 6 attempts at base 10ms run as one participant, every
    // sleep through the clock, zero real time spent
    Vclock ck2 = new Vclock();
    Bubble b2 = new Bubble(ck2,
        new Participant[] {new Participant("backoff", backoffScript(6, 10))});
    b2.run();
    ok(b2.waits.equals(List.of(10L, 20L, 40L, 80L, 160L, 320L)),
        "the ladder walked 6 doubling attempts");
    ok(b2.cumulative.equals(List.of(10L, 30L, 70L, 150L, 310L, 630L)),
        "cumulative delay reaches 630ms on the ladder");
    ok(ck2.nowMs == 630, "the bubble clock advanced exactly 630ms");
    ok(ck2.advances == 6, "one advance per parked attempt");
```

listing 382: Java, the check region: the two-sleeper interleaving pinned byte for byte, ladder and cumulative exact, the clock at 630 in 6 advances (source: patterns-concurrency-distributed/samples-java/src/Ch09/Synctest.java, lines 139-168)

```
public class SynctestTests
{
    [Fact]
    public async Task BackoffLadderIsExactUnderTheVirtualClock()
    {
        var fake = new FakeTime();
        var result = await VirtualClock.Run(
            fake, Synctest.Backoff(6, TimeSpan.FromMilliseconds(10), fake));

        Assert.Equal(6, result.Tries);
        // 10+20+40+80+160+320 = 630ms of scheduled delay
        Assert.Equal(TimeSpan.FromMilliseconds(630), result.Total);
        Assert.Equal(
            new[]
            {
                TimeSpan.FromMilliseconds(10), TimeSpan.FromMilliseconds(20),
                TimeSpan.FromMilliseconds(40), TimeSpan.FromMilliseconds(80),
                TimeSpan.FromMilliseconds(160), TimeSpan.FromMilliseconds(320),
            },
            result.Ladder);
    }

    [Fact]
    public async Task BubbleClockAdvancesExactlyAsFarAsTheSchedule()
    {
        var fake = new FakeTime();
        var start = fake.GetUtcNow();
        await VirtualClock.Run(fake, Synctest.Backoff(6, TimeSpan.FromMilliseconds(10), fake));

        // the bubble analog: virtual time moved 630ms because six
        // sleeps said so, and no further. FireNext jumps only to due
        // times, so there is no scheduling slop in the elapsed.
        Assert.Equal(TimeSpan.FromMilliseconds(630), fake.GetUtcNow() - start);

        var cumulative = new List<TimeSpan>();
        var running = TimeSpan.Zero;
        for (int i = 0; i < 6; i++)
        {
            running += TimeSpan.FromMilliseconds(10 << i);
            cumulative.Add(running);
        }
        Assert.Equal(
            new[]
            {
                TimeSpan.FromMilliseconds(10), TimeSpan.FromMilliseconds(30),
                TimeSpan.FromMilliseconds(70), TimeSpan.FromMilliseconds(150),
                TimeSpan.FromMilliseconds(310), TimeSpan.FromMilliseconds(630),
            },
            cumulative);
    }
}
```

listing 383: C#, xunit: the ladder exact, virtual time moved 630ms and no further, no scheduling slop (source: patterns-concurrency-distributed/samples-cs/tests/Ch09/SynctestTests.cs, lines 6-56)

```
describe("ch09 synctest", () => {
  it("backoff(6, 10) in the bubble: ladder 10..320, cumulative to 630", async () => {
    const { value, clock } = await runInBubble((c) => backoff(c, 6, 10));
    assert.equal(value.tries, 6);
    assert.equal(value.total, 630);
    assert.deepEqual(value.ladder, [10, 20, 40, 80, 160, 320]);
    assert.deepEqual(value.cumulative, [10, 30, 70, 150, 310, 630]);
    assert.equal(clock.now, 630, "bubble clock must advance exactly 630");
  });
});
```

listing 384: JavaScript, node:test: backoff in the bubble, ladder and cumulative deep-equal, clock at 630 (source: patterns-concurrency-distributed/samples-js/test/ch09.test.mjs, lines 48-57)

```
def main():
    async def ladder():
        clock = VirtualClock()
        backoff = Backoff(clock)
        tries, total = await clock.drive(backoff.run(6, 10.0))
        return clock, backoff, tries, total

    wall = time.perf_counter()
    clock, backoff, tries, total = asyncio.run(ladder())
    elapsed = time.perf_counter() - wall
    check(tries == 6, "the ladder walked 6 attempts")
    check(backoff.ladder == [10.0, 20.0, 40.0, 80.0, 160.0, 320.0], "delays double")
    check(
        backoff.cumulative == [10.0, 30.0, 70.0, 150.0, 310.0, 630.0],
        "cumulative delay reaches 630ms on the ladder",
    )
    check(total == 630.0, "the reported total is 630ms")
    check(clock.now == 630.0, "the bubble clock advanced exactly 630ms")
    check(elapsed < 0.5, "no real sleep happened: the run is a guard, not a wait")
```

listing 385: Python, the check region: ladder and cumulative exact, the bubble clock at 630, wall cost under half a second (source: patterns-concurrency-distributed/samples-py/src/Ch09/synctest.py, lines 100-118)

```
return {
  {name = "the fixture: 6 attempts from 10ms, ladder and cumulative exact", fn = function()
    local C = clock()
    local probe = { ladder = {}, cumulative = {} }
    local out = {}
    spawn(C, function()
      out.tries, out.total = Backoff(C, 6, 10, probe)
    end)
    vrun(C)
    T.eq(out.tries, 6)
    T.eq(out.total, 630)
    T.eq(probe.ladder, { 10, 20, 40, 80, 160, 320 })
    T.eq(probe.cumulative, { 10, 30, 70, 150, 310, 630 })
  end},

  {name = "the bubble clock advanced exactly 630ms, in zero real time", fn = function()
    local C = clock()
    local probe = { ladder = {}, cumulative = {}, sleeps = 0 }
    spawn(C, function()
      local w = C.seq -- count events as they queue
      local tries, _ = Backoff(C, 6, 10, probe)
      probe.tries = tries
      probe.events = C.seq - w
    end)
    vrun(C)
    T.eq(C.now, 630, "the virtual clock is the only thing that moved")
    T.eq(probe.tries, 6)
    T.eq(probe.events, 6)
    T.eq(C.live, 0)
  end},
```

listing 386: Lua, the row table: ladder and cumulative exact, the clock advanced exactly 630 in zero real time (source: patterns-concurrency-distributed/samples-lua/ch09_synctest.lua, lines 76-105)

6 retries with a doubling base execute in microseconds of wall time in every tree, and the bubble's clock reads exactly 630 milliseconds, the sum of the ladder, asserted as equality rather than a bounded interval. That equality is what fake time buys: timing behavior becomes value behavior. The boundaries are as documented for the go lane: the test must be self contained, no network, no goroutines outside the bubble, which is why the in-memory transport of the raft capstone is channels instead of sockets, it fits a bubble if the tests ever want one. The sibling lanes inherit the same demand in their own grain, everything the walk touches must resolve through the injected clock, one real sleep hiding anywhere breaks the equality.

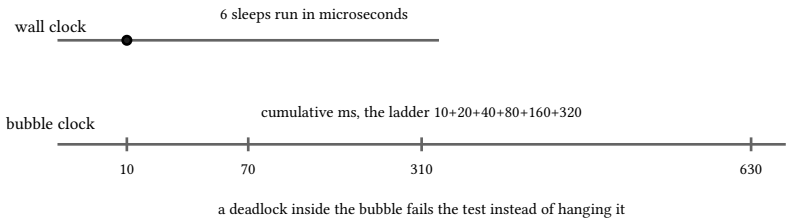


figure 54: wall clock microseconds, bubble clock exactly 630ms

PITFALL: detectors are the gate, not decoration

A concurrency suite without `-race` in its CI line is testing luck. The book's own chain splits the work: the pre-commit hook is gofmt and vet for speed, and the Makefile's `verify-go` runs the plain suite and then the race pass over every go module, so each chapter's race claim is machine-checked. The deeper repetition, `go test -race -count=2`, stays a workstation step run before a chapter ships. The sibling trees have no race detector

> to lean on, so their suites earn the same claims the other way, every interleaving scripted, every total exact, and the guards convert anything that might hang into a failure. Rust's answer sits upstream of any suite, the borrow checker refuses the racy program itself.

9.6. across the eight languages

The build sizes count non-comment source lines, the go column the chapter's one frozen file:

language	build SLOC	dependency	boundary note
c	707	libc plus threads.h, stdatomic.h	fork order under mtx_t, trylock failures counted, a discrete-event bubble
rust	624	stdlib	Mutex forks on scoped threads, the rwlock hand-rolled because std documents no fencing, an enum-scripted bubble
go	110	stdlib	frozen reference lane, synctest the bubble only go has, mutex profile at rate 1
java	575	jdk 27 stdlib	monitor forks, the rwlock hand-rolled because ReentrantReadWriteLock documents no reader fencing, a script-swept bubble
c#	147	bcl	a binary SemaphoreSlim fork, documented writer preference, TimeProvider delays
javascript	238	node stdlib, one sibling module	promise-queue mutex forks, one real setTimeout guard, a timer-queue bubble
python	382	stdlib only	real locks under the GIL, the rwlock a Condition, the bubble a heap clock
lua	687	lib.lua harness	the naive fork order a pinned 0-bite fixture, the guard on the virtual clock

sources: go.dev/pkg/sync for the RWMutex writer guarantee and the recursive read prohibition, go.dev/pkg/runtime for `SetMutexProfileFraction`, go.dev/pkg/runtime/pprof for profile lookup, go.dev/doc/go1.27 for the goroutine leak profile GA, go.dev/pkg/testing/synctest for bubble semantics and `Sleep`, go.dev/doc/articles/race_detector for the detector's mechanics and limits, accessed 2026-09-08. Verified by the eight chapter legs: 5 C programs with 55 embedded checks, 5 rust programs with 54 embedded checks under `samples-rust/src/Ch09`, `go test` at 5 tests in `patternsbook/ch09`, 5 java programs with 42 embedded checks under `samples-java/src/Ch09`, 7 xunit facts, node's 5 cases in `test/ch09.test.mjs`, 5 python modules with 38 embedded checks, and the lua runner's 19 ch09 rows.

10. distributed fundamentals

A distributed system is one where a machine you do not control can slow you down, reorder you, or vanish, and where the speed of light is a latency floor you bill for. Everything in the second half of this book, consensus, partitioning, messaging, observability, resilience, is engineering around those three facts. This chapter fixes the vocabulary and builds the two algorithmic primitives every later chapter leans on: logical clocks and quorum arithmetic. The three samples, lamport, vector, and quorum, are pure functions in all eight trees, no concurrency at all, and the shared fixtures land as the same literals in every lane.

10.1. the fallacies, as constraints

The old list of eight fallacies of distributed computing, the network is reliable, latency is zero, bandwidth is infinite, the network is secure, topology does not change, there is one administrator, transport cost is zero, the network is homogeneous, reads as a checklist of assumptions production systems violate weekly. Each tree's native error surface encodes the answers: go's `error` values, rust's `Result`, and c's status codes carry network failure back to the caller without crashing anyone, c# and python raise, javascript rejects a promise, lua returns nil plus a message, latency gets its deadline through the context objects of chapter 8, and transient loss meets the jittered retries of chapter 15. The list is not trivia, it is a requirements document.

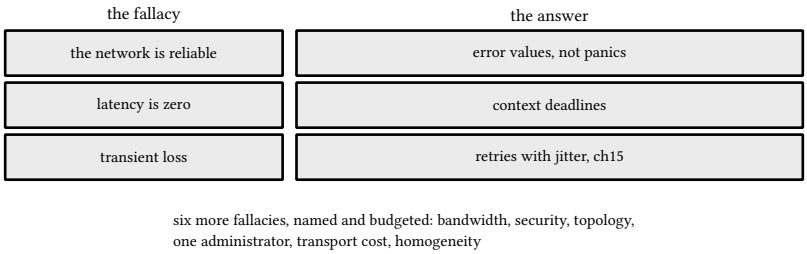


figure 55: the eight fallacies mapped to the answers the book builds on

The number that disciplines intuition: a round trip within a datacenter is around 0.5 milliseconds, across a continent around 30, and reading an nvme drive is around 100 microseconds. A protocol that adds 10 sequential cross-continent round trips costs 300 milliseconds before any work happens, which is why the raft chapter counts messages and why every resilience pattern in chapter 15 trades round trips for retries.

10.2. why wall clocks are not clocks

Every machine's `time.Now()` disagrees, ntp keeps the disagreement bounded but never zero, and the clock can jump backward during correction. Any ordering or timeout logic built on comparing two machines' wall clocks inherits all of it. The rule this book follows: wall clocks are for humans reading logs, monotonic clocks (`time.Since`, read from the same machine) are for measuring durations, and logical clocks are for ordering events. Every tree keeps the same split with its own primitive, `clock_gettime` over a monotonic clock id in c, `std::time::Instant` in rust, `Stopwatch` in c#, `performance.now` in javascript, `time.monotonic` in python, and nothing in lua, whose stdlib offers only wall `os.time` and cpu-time `os.clock`, so the lua lanes in this book carry time through the scheduler instead.

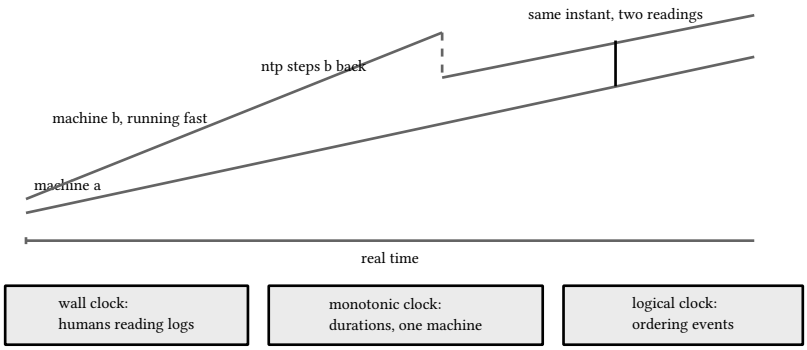


figure 56: two wall clocks drift, ntp steps one back, no ordering survives

10.3. lamport clocks

Lamport's 1978 insight: order does not have to come from time, it can come from communication. Each process keeps a counter, ticks it on every event, and folds the maximum on receive:

The dry run: the same walk in all eight lanes.

- a leader's 100 local ticks leave its stamp at exactly 100, and a lagging clock receiving that stamp lands on 101, max then tick
- along a send and receive chain every later event strictly out-stamps the earlier one, the one-way property
- an independent clock can hold a smaller stamp than a message it never saw, the false converse, and every lane pins that falsehood as expected behavior

```c
typedef struct {
  const char *id;
  uint64_t count;
} lamport;

static void lamport_init(lamport *l, const char *id) {
  l->id = id;
  l->count = 0;
}

static uint64_t tick(lamport *l) { return ++l->count; }

// a local tick that also yields the stamp to attach to a message
static uint64_t send(lamport *l) { return tick(l); }

// merge a remote stamp and tick
static uint64_t receive(lamport *l, uint64_t remote) {
  if (remote > l->count)
    l->count = remote;
  return tick(l);
}
```

listing 387: C, the clock is a struct, tick, send, and receive are plain functions over it (source: patterns-concurrency-distributed/samples-c/src/Ch10/lamport.c, lines 21-41)

```rust
struct Lamport {
    count: u64,
}

impl Lamport {
    fn new() -> Lamport {
        Lamport { count: 0 }
    }

    fn tick(&mut self) -> u64 {
        self.count += 1;
        self.count
    }

    fn send(&mut self) -> u64 {
        self.tick()
    }

    fn receive(&mut self, remote: u64) -> u64 {
        if remote > self.count {
            self.count = remote;
        }
        self.tick()
    }
}
```

listing 388: Rust, the clock is a bare u64 behind `&mut self` methods, the process name c carries is gone, receive folds the max then ticks (source: patterns-concurrency-distributed/samples-rust/src/Ch10/lamport.rs, lines 15-39)

```go
type Lamport struct {
  id    string
  count uint64
}

func NewLamport(id string) *Lamport { return &Lamport{id: id} }

func (l *Lamport) Tick() uint64 {
  l.count++
  return l.count
}

// Send is a local tick that also yields the stamp to attach.
func (l *Lamport) Send() uint64 { return l.Tick() }

// Receive merges a remote stamp and ticks.
func (l *Lamport) Receive(remote uint64) uint64 {
  if remote > l.count {
    l.count = remote
  }
  return l.Tick()
}

// VectorClock tracks the latest stamp each process has observed.
```

listing 389: Go, tick, send, receive: the whole scheme in three methods (source: patterns-concurrency-distributed/samples/ch10/clocks.go, lines 10-33)

```
  static final class Clock {
    final String pid;
    long count;

    Clock(String pid) {
      this.pid = pid;
    }

    long tick() {
      return ++count;
    }

    // a local tick that also yields the stamp to attach to a message
    long send() {
      return tick();
    }

    // merge a remote stamp and tick
    long receive(long remote) {
      if (remote > count) {
        count = remote;
      }
      return tick();
    }
  }
```

listing 390: Java, the clock is a small final class around a long, receive folds the max then ticks (source: patterns-concurrency-distributed/samples-java/src/Ch10/Lamport.java, lines 20-44)

```
// Lamport clock: a counter per process. The rule is one line,
// tick on local events, and on receiving a stamp take max(mine,
// theirs) then increment. The property: if a happens-before b,
// clock(a) < clock(b). The converse is false, which is the honest
// boundary of the scheme.
public sealed class Lamport(string id)
{
    private ulong _count;

    public string Id { get; } = id;

    public ulong Count => _count;

    public ulong Tick() => ++_count;

    // Send is a local tick that also yields the stamp to attach.
    public ulong Send() => Tick();

    // Receive merges a remote stamp and ticks.
    public ulong Receive(ulong remote)
    {
        if (remote > _count)
        {
            _count = remote;
        }
        return Tick();
    }
}
```

listing 391: C#, the clock is a class, Receive folds the max then ticks (source: patterns-concurrency-distributed/samples-cs/src/Ch10/Lamport.cs, lines 3-30)

```
export class Lamport {
  #count = 0;

  constructor(id) {
    this.id = id;
  }

  get count() {
    return this.#count;
  }

  tick() {
    this.#count++;
    return this.#count;
  }

  // a local tick that also yields the stamp to attach
  send() {
    return this.tick();
  }

  // merge a remote stamp and tick
  receive(remote) {
    if (remote > this.#count) {
      this.#count = remote;
    }
    return this.tick();
  }
}
```

listing 392: JavaScript, the counter sits behind a private class field, the same three methods (source: patterns-concurrency-distributed/samples-js/src/ch10-lamport.mjs, lines 7-35)

```
class Lamport:
    def __init__(self, pid):
        self.pid = pid
        self.count = 0

    def tick(self):
        self.count += 1
        return self.count

    def send(self):
        return self.tick()

    def receive(self, remote):
        if remote > self.count:
            self.count = remote
        return self.tick()
```

listing 393: Python, the clock is a small class, count starts at zero (source: patterns-concurrency-distributed/samples-py/src/Ch10/lamport.py, lines 16-32)

```lua
local Lamport = {}
Lamport.__index = Lamport

function Lamport.new(id)
  return setmetatable({ id = id, count = 0 }, Lamport)
end

function Lamport:Tick()
  self.count = self.count + 1
  return self.count
end

-- Send is a local tick that also yields the stamp to attach
function Lamport:Send()
  return self:Tick()
end

-- Receive merges a remote stamp and ticks
function Lamport:Receive(remote)
  if remote > self.count then
    self.count = remote
  end
  return self:Tick()
end
```

listing 394: Lua, the clock is a table dispatched through its metatable (source: patterns-concurrency-distributed/samples-lua/ch10_lamport.lua, lines 7-30)

The counter's carrier is the only thing the eight trees disagree on: go keeps `count uint64` in a struct, c a `uint64_t` beside the process name, rust a `u64` behind `&mut self` methods with no name at all, c# a `ulong` property, javascript a private class field, python an attribute on `self`, lua a table field reached through `__index`, java a `long` field in a final class. Receive is max then increment in every spelling, and because the scheme is pure arithmetic over one integer, all eight fixtures print the same numbers with no portability seam at all.

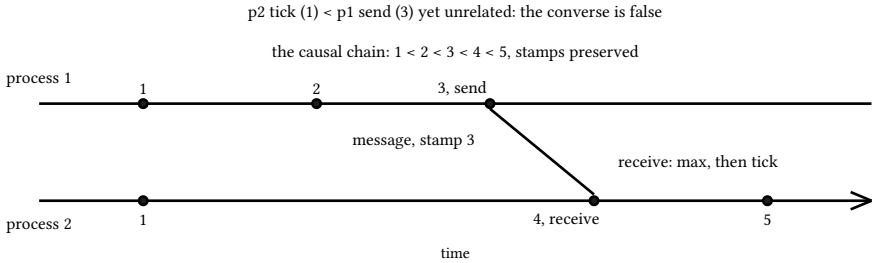


figure 57: local events tick, receive takes the max then ticks, order is one way

The guarantee is one direction only: if a happens-before b, then `stamp(a) < stamp(b)`. The converse is false, and the test pins that honestly, an independent process with a smaller stamp is not "earlier" in any causal sense, it is simply unrelated. Lamport clocks buy total order cheaply (ties broken by process id) at the price of ordering things that have no relationship, which is acceptable for choosing a lock acquisition order and unacceptable for deciding which write wins.

10.4. vector clocks

To recover the true causal relation, each process records the latest stamp it has seen from every process:

The dry run: the fork-merge fixture, identical in all eight lanes.

- 2 independent first ticks stamp {a:1} and {b:1} and compare concurrent
- after a ticks again to {a:2} and b merges it, b holds {a:2,b:2}: before one way and after the other

- b's pre-merge fork stamp stays concurrent with a's later {a:3}
- a clone of a stamp compares equal to the stamp it came from

```c
static vstamp vc_tick(vector_clock *v) {
  v->stamp.counts[slot_of(v->proc)]++;
  return v->stamp;
}

// merge folds a received stamp elementwise max, then ticks self: the
// receive itself is an event
static vstamp vc_merge(vector_clock *v, vstamp received) {
  for (int i = 0; i < SLOTS; i++)
    if (received.counts[i] > v->stamp.counts[i])
      v->stamp.counts[i] = received.counts[i];
  return vc_tick(v);
}

typedef enum { REL_EQUAL, REL_BEFORE, REL_AFTER, REL_CONCURRENT } relation;

static const char *relation_name(relation r) {
  switch (r) {
  case REL_EQUAL:
    return "equal";
  case REL_BEFORE:
    return "before";
  case REL_AFTER:
    return "after";
  default:
    return "concurrent";
  }
}

// compare recovers the causal relation between two stamps: walk every slot,
// note which side is ever smaller, and combine
static relation compare(vstamp a, vstamp b) {
  bool less = false, greater = false;
  for (int i = 0; i < SLOTS; i++) {
    if (a.counts[i] < b.counts[i])
      less = true;
    if (a.counts[i] > b.counts[i])
      greater = true;
  }
  if (!less && !greater)
    return REL_EQUAL;
  if (less && !greater)
    return REL_BEFORE;
  if (greater && !less)
    return REL_AFTER;
  return REL_CONCURRENT;
}
```

listing 395: C, merge folds elementwise max then ticks, compare walks the slots of a fixed two-process stamp (source: patterns-concurrency-distributed/samples-c/src/Ch10/vector.c, lines 41-87)

```rust
struct VectorClock {
    slot: usize,
    stamp: VStamp,
}

impl VectorClock {
    fn new(slot: usize) -> VectorClock {
        VectorClock {
            slot,
            stamp: VStamp { counts: [0; 2] },
        }
    }

    fn tick(&mut self) -> VStamp {
        self.stamp.counts[self.slot] += 1;
        self.stamp
    }

    fn merge(&mut self, received: VStamp) -> VStamp {
        for i in 0..2 {
            if received.counts[i] > self.stamp.counts[i] {
                self.stamp.counts[i] = received.counts[i];
            }
        }
        self.tick()
    }
}

fn compare(a: VStamp, b: VStamp) -> Relation {
    let mut less = false;
    let mut greater = false;
    for i in 0..2 {
        if a.counts[i] < b.counts[i] {
            less = true;
        }
        if a.counts[i] > b.counts[i] {
            greater = true;
        }
    }
    match (less, greater) {
        (false, false) => Relation::Equal,
        (true, false) => Relation::Before,
        (false, true) => Relation::After,
        (true, true) => Relation::Concurrent,
    }
}
```

listing 396: Rust, the fixed two-slot universe as a Copy [u64; 2], merge folds the max then ticks, compare a 4-arm match over the (less, greater) tuple (source: patterns-concurrency-distributed/samples-rust/src/Ch10/vector.rs, lines 37-82)

```go
type VectorClock struct {
  proc  string
  stamp map[string]uint64
}

func NewVectorClock(proc string) *VectorClock {
  return &VectorClock{proc: proc, stamp: map[string]uint64{}}
}

func (v *VectorClock) Tick() map[string]uint64 {
  v.stamp[v.proc]++
  return v.Clone()
}

// Merge folds a received stamp into the local one, elementwise max,
// then ticks self: the receive itself is an event.
func (v *VectorClock) Merge(received map[string]uint64) map[string]uint64 {
  for proc, n := range received {
    if n > v.stamp[proc] {
      v.stamp[proc] = n
    }
  }
  return v.Tick()
}

func (v *VectorClock) Clone() map[string]uint64 {
  out := make(map[string]uint64, len(v.stamp))
  for proc, n := range v.stamp {
    out[proc] = n
  }
  return out
```

listing 397: Go, stamp per process, elementwise merge, tick on the receive itself (source: patterns-concurrency-distributed/samples/ch10/clocks.go, lines 36-66)

```go
type Relation int

const (
  Equal Relation = iota
  Before
  After
  Concurrent
)

// Compare recovers the causal relation between two vector stamps.
func Compare(a, b map[string]uint64) Relation {
  procs := map[string]bool{}
  for p := range a {
    procs[p] = true
  }
  for p := range b {
    procs[p] = true
  }
  names := make([]string, 0, len(procs))
  for p := range procs {
    names = append(names, p)
  }
  slices.Sort(names)

  less, greater := false, false
  for _, p := range names {
    switch {
    case a[p] < b[p]:
      less = true
    case a[p] > b[p]:
      greater = true
    }
  }
  switch {
  case !less && !greater:
    return Equal
  case less && !greater:
    return Before
  case greater && !less:
    return After
  default:
    return Concurrent
  }
}
```

listing 398: Go, comparison: all-less is before, all-greater is after, mixed is concurrent (source: patterns-concurrency-distributed/samples/ch10/clocks.go, lines 69-112)

```
static final class VectorClock {
  final String proc;
  final long[] stamp = new long[2]; // slot 0 = "a", slot 1 = "b"

  VectorClock(String proc) {
    this.proc = proc;
  }

  static int slotOf(String proc) {
    return proc.charAt(0) == 'a' ? 0 : 1;
  }

  // tick returns a copy of the stamp after counting this event
  long[] tick() {
    stamp[slotOf(proc)]++;
    return snapshot();
  }

  // merge folds a received stamp elementwise max, then ticks self: the
  // receive itself is an event
  long[] merge(long[] received) {
    for (int i = 0; i < stamp.length; i++) {
      if (received[i] > stamp[i]) {
        stamp[i] = received[i];
      }
    }
    return tick();
  }

  long[] snapshot() {
    return stamp.clone();
  }
}
```

listing 399: Java, the fixed two-slot universe of the c tree, tick hands back a defensive copy, merge folds the max then ticks (source: patterns-concurrency-distributed/samples-java/src/Ch10/Vector.java, lines 32-64)

```
public static class VectorOrder
{
    // Compare recovers the causal relation between two vector
    // stamps: strictly smaller is Before, strictly larger is After,
    // a mix in both directions is Concurrent.
    public static Relation Compare(Dictionary<string, ulong> a, Dictionary<string, ulong> b)
    {
        var names = new List<string>();
        foreach (var p in a.Keys)
        {
            names.Add(p);
        }
        foreach (var p in b.Keys)
        {
            if (!a.ContainsKey(p))
            {
                names.Add(p);
            }
        }
        names.Sort(StringComparer.Ordinal);

        bool less = false, greater = false;
        foreach (var p in names)
        {
            a.TryGetValue(p, out ulong av);
            b.TryGetValue(p, out ulong bv);
            if (av < bv)
            {
                less = true;
            }
            else if (av > bv)
            {
                greater = true;
            }
        }
        if (!less && !greater)
        {
            return Relation.Equal;
        }
        if (less && !greater)
        {
            return Relation.Before;
        }
        if (greater && !less)
        {
            return Relation.After;
        }
        return Relation.Concurrent;
    }
}
```

listing 400: C#, the four-way compare over the sorted union of process names, missing slots read zero (source: patterns-concurrency-distributed/samples-cs/src/Ch10/Vector.cs, lines 45-94)

```
export class VectorClock {
  #stamp = new Map();

  constructor(proc) {
    this.proc = proc;
  }

  tick() {
    this.#stamp.set(this.proc, (this.#stamp.get(this.proc) ?? 0) + 1);
    return this.clone();
  }

  merge(received) {
    for (const [proc, n] of received) {
      if (n > (this.#stamp.get(proc) ?? 0)) {
        this.#stamp.set(proc, n);
      }
    }
    return this.tick();
  }

  clone() {
    return new Map(this.#stamp);
  }
}

// compare recovers the causal relation between two vector stamps.
// stamps are maps of process name to count.
export function compare(a, b) {
  const names = new Set([...a.keys(), ...b.keys()]);
  let less = false;
  let greater = false;
  for (const name of names) {
    const av = a.get(name) ?? 0;
    const bv = b.get(name) ?? 0;
    if (av < bv) less = true;
    if (av > bv) greater = true;
  }
  if (!less && !greater) return "equal";
  if (less && !greater) return "before";
  if (greater && !less) return "after";
  return "concurrent";
}
```

listing 401: JavaScript, the stamp is a Map, compare returns the relation as a string (source: patterns-concurrency-distributed/samples-js/src/ch10-vector.mjs, lines 7-49)

```
class VectorClock:
    def __init__(self, proc):
        self.proc = proc
        self.stamp = {}

    def tick(self):
        self.stamp[self.proc] = self.stamp.get(self.proc, 0) + 1
        return self.clone()

    def merge(self, received):
        for proc, count in received.items():
            if count > self.stamp.get(proc, 0):
                self.stamp[proc] = count
        return self.tick()

    def clone(self):
        return dict(self.stamp)


def compare(a, b):
    names = sorted(set(a) | set(b))
    less = False
    greater = False
    for name in names:
        av = a.get(name, 0)
        bv = b.get(name, 0)
        if av < bv:
            less = True
        elif av > bv:
            greater = True
    if not less and not greater:
        return EQUAL
    if less and not greater:
        return BEFORE
    if greater and not less:
        return AFTER
    return CONCURRENT
```

listing 402: Python, the stamp is a dict, compare walks the sorted union with zero defaults (source: patterns-concurrency-distributed/samples-py/src/Ch10/vector.py, lines 21-57)

```lua
function VectorClock.new(proc)
  return setmetatable({ proc = proc, stamp = {} }, VectorClock)
end

function VectorClock:Clone()
  local out = {}
  for p, n in pairs(self.stamp) do
    out[p] = n
  end
  return out
end

function VectorClock:Tick()
  self.stamp[self.proc] = (self.stamp[self.proc] or 0) + 1
  return self:Clone()
end

-- Merge folds a received stamp elementwise max, then ticks self: the
-- receive itself is an event
function VectorClock:Merge(received)
  for p, n in pairs(received) do
    if n > (self.stamp[p] or 0) then
      self.stamp[p] = n
    end
  end
  return self:Tick()
end

-- Compare recovers the relation between two stamps. missing entries
-- read as zero, the union of processes decides
function Compare(a, b)
  local names = {}
  for p in pairs(a) do names[#names + 1] = p end
  for p in pairs(b) do
    if a[p] == nil then names[#names + 1] = p end
  end
  table.sort(names)

  local less, greater = false, false
  for _, p in ipairs(names) do
    local av, bv = a[p] or 0, b[p] or 0
    if av < bv then
      less = true
    elseif av > bv then
      greater = true
    end
  end
  if not less and not greater then return "Equal" end
  if less and not greater then return "Before" end
  if greater and not less then return "After" end
  return "Concurrent"
end
```

listing 403: Lua, the stamp is a plain table, compare names the relation it returns (source: patterns-concurrency-distributed/samples-lua/ch10_vector.lua, lines 9-60)

The carrier widens from one integer to a map of them and the trees split by grain: go a `map[string]uint64`, c# a `Dictionary<string, ulong>`, javascript a `Map`, python a `dict`, lua a plain table, and c, rust, and java a fixed two-slot

array, honest about a fixture that only ever forks two processes, rust's carried as a usize slot index where c maps the process name to one. Compare walks the union of process names in all eight, sorted first in go, c#, python, and lua, javascript settling for Set order over its Map keys, and c, rust, and java scanning their two slots, a missing name reading zero through `TryGetValue`, `?? 0`, `.get(name, 0)`, `or 0`, and the c, rust, and java zeroed arrays. The relation itself is an enum in go, c, rust, c#, and java, java's and rust's carrying a name renderer for their check text, and a returned name in javascript, python, and lua, the same four outcomes either way.

Comparison over the union of process ids yields exactly four outcomes, and `Concurrent` is the outcome this section exists for, the formal statement "these two events have no causal order, any ordering between them is a choice we make". The test builds the fork and merge scenario: two independent ticks compare `Concurrent`, after a message flows one direction the relation flips to `Before`/`After`, and the pre-merge fork stamp stays concurrent with later independent work. Vector clocks cost o(n) per message against lamport's o(1), the standard trade of size for truth, and production systems compress them into version vectors and dotted version vectors instead of abandoning them.

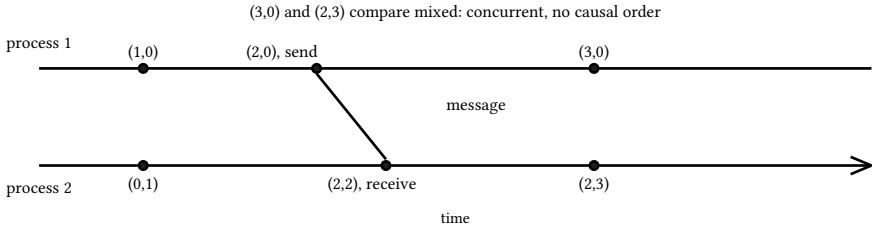


figure 58: the test's fork and merge scenario, stamps are vectors

NOTE: concurrent is not simultaneous

`Concurrent` in the vector clock sense means causally unrelated, regardless of which event physically happened first in wall time. Two clients writing the same key from opposite coasts are concurrent to each other even when one clearly typed faster, and a conflict resolution rule, last-writer-wins by wall clock, application merge, or a human, is then mandatory, not optional.

10.5. quorums

Consensus in the next chapter reduces to arithmetic: a quorum is the smallest majority, and any two quorums of the same cluster intersect:

The dry run: quorum over cluster sizes 1 through 7 reads 1, 2, 2, 3, 3, 4, 4 in every lane.

- majority(2, 3) carries, majority(1, 3) does not, majority(3, 3) carries, all eight, and the c, rust, java, and python lanes add the even cluster, where half is never enough
- quorum(n) + quorum(n) > n, the overlap the raft chapters stand on, swept for n up to 20 by the lua lane and checked at 5 and 7 by python and java, which also sweeps the failure tolerance, f+1 yes of 2f+1 for f up to 3, a sweep rust runs too

```c
// the smallest majority of n
static int quorum(int n_nodes) { return n_nodes / 2 + 1; }

// whether yes votes out of n_nodes carry a quorum
static bool majority(int yes, int n_nodes) { return yes ≥ quorum(n_nodes); }
```

listing 404: C, two functions, the whole arithmetic (source: patterns-concurrency-distributed/samples-c/src/Ch10/quorum.c, lines 18-22)

```rust
fn quorum(n_nodes: usize) -> usize {
    n_nodes / 2 + 1
}

fn majority(yes: usize, n_nodes: usize) -> bool {
    yes ≥ quorum(n_nodes)
}
```

listing 405: Rust, the same two functions over usize, integer division floors on its own (source: patterns-concurrency-distributed/samples-rust/src/Ch10/quorum.rs, lines 15-21)

```go
func Quorum(n int) int { return n/2 + 1 }

// Majority reports whether yes votes out of n carry a quorum.
func Majority(yes, n int) bool { return yes ≥ Quorum(n) }
```

listing 406: Go, quorum and majority (source: patterns-concurrency-distributed/samples/ch10/clocks.go, lines 115-118)

```java
  // the smallest majority of n
  static int quorum(int nodes) {
    return nodes / 2 + 1;
  }

  // whether yes votes out of nodes carry a quorum
  static boolean majority(int yes, int nodes) {
    return yes ≥ quorum(nodes);
  }

  // two majority sets must share at least one member
  static boolean overlaps(int left, int right, int nodes) {
    return left + right > nodes;
  }
```

listing 407: Java, three static one-liners, n / 2 + 1 verbatim like c, plus the overlap predicate (source: patterns-concurrency-distributed/samples-java/src/Ch10/Quorum.java, lines 17-30)

```csharp
public static class Quorum
{
    // Of is the smallest majority of a cluster of n.
    public static int Of(int n) => n / 2 + 1;

    // Majority reports whether yes votes out of n carry a quorum.
    public static bool Majority(int yes, int n) => yes ≥ Of(n);
}
```

listing 408: C#, Of and Majority as static one-liners (source: patterns-concurrency-distributed/samples-cs/src/Ch10/Quorum.cs, lines 3-10)

```javascript
// ch10: quorum arithmetic. the smallest majority of a cluster of n
// is floor(n/2) + 1, and every consensus vote in the rest of the
// book reduces to comparing a yes count against it.

export function quorum(n) {
  return Math.floor(n / 2) + 1;
}

// majority reports whether yes votes out of n carry a quorum
export function majority(yes, n) {
  return yes ≥ quorum(n);
}
```

listing 409: JavaScript, Math.floor spells the same formula over real division (source: patterns-concurrency-distributed/samples-js/src/ch10-quorum.mjs, lines 1-12)

```python
def quorum(cluster):
    return cluster // 2 + 1


def majority(yes, cluster):
    return yes ≥ quorum(cluster)
```

listing 410: Python, the same two functions in the script style (source: patterns-concurrency-distributed/samples-py/src/Ch10/quorum.py, lines 15-20)

```lua
-- Quorum is the smallest majority of a cluster of n
local function Quorum(n)
  return n // 2 + 1
end

-- Majority reports whether yes votes out of n carry a quorum
local function Majority(yes, n)
  return yes ≥ Quorum(n)
end
```

listing 411: Lua, floor division, same formula (source: patterns-concurrency-distributed/samples-lua/ch10_quorum.lua, lines 6-14)

The formula survives translation because every language here floors its integer division: `n / 2 + 1` verbatim in c, rust, go, java, and c#, `n // 2 + 1` in python and lua, and `Math.floor(n / 2) + 1` in javascript, where `/` alone is real division and would hand back 2.5 seats for a five-node cluster. The majority predicate is one comparison everywhere. The number itself does the work in the next chapter, where elections and log commitment both reduce to reaching it.

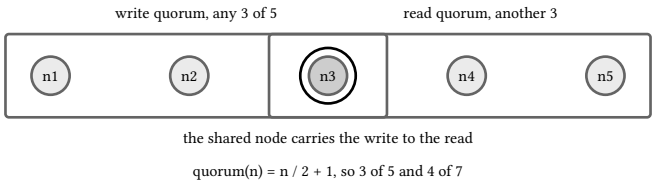


figure 59: any two majorities of five share a node, reads meet writes

The intersection property is what makes replication safe: if a write must reach a quorum and a read must consult a quorum, any read overlaps any write, so a quorum system cannot silently split into two worlds that each believe they are the whole world. Raft's elections and its log commitment are both this one property applied, and the capstone leans on it five times over.

10.6. cap, as an engineering decision

The CAP theorem states a distributed system facing a partition must choose between consistency and availability for the partitioned minority. The practical reading is narrower than the slogan: partitions are not optional, they are the network's resting state at scale, so the choice is per operation, CP operations reject minority requests to keep one truth, AP operations serve stale answers to stay up. Raft, and this book's capstone, sits on the CP side: a minority partition stops accepting writes, and chapter 16's tests exercise exactly that stoppage.

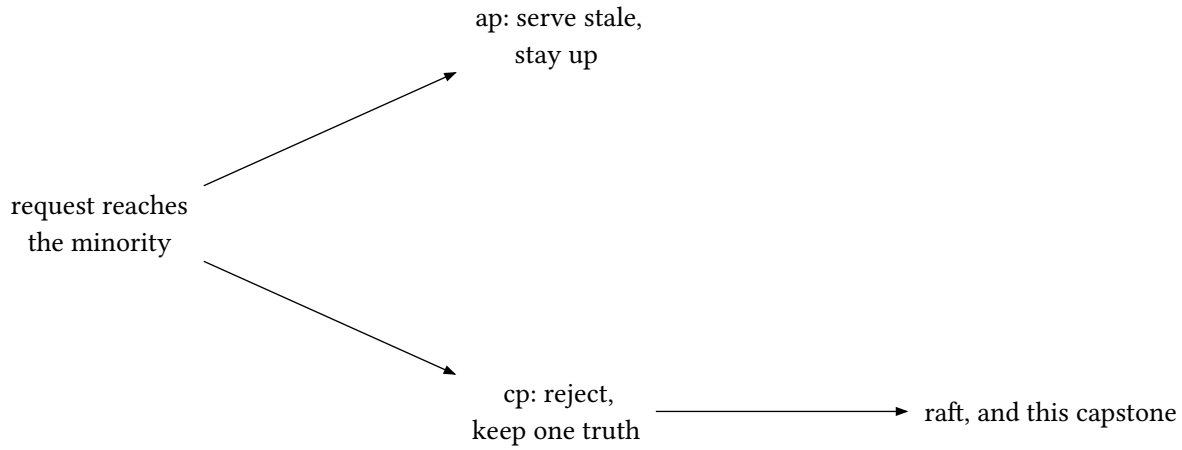


figure 60: cap as a per-operation decision, raft sits on the cp side

10.7. across the eight languages

The build sizes count non-comment source lines, the go column the chapter's one frozen file:

language	build SLOC	dependency	boundary note
c	178	libc	uint64_t clock, two-slot fixed vector stamps, an enum relation, n / 2 + 1
rust	233	stdlib	a u64 clock behind mut methods, Copy `[u64; 2]` stamps, a 4-arm match relation, n / 2 + 1
go	86	stdlib	frozen reference lane, map[string]uint64 stamps over the sorted union
java	202	jdk 27 stdlib	long-field clock, two-slot stamps with a defensive-copy tick, n / 2 + 1 verbatim
c#	102	bcl	ulong clock, Dictionary stamps, TryGetValue zeros, static quorum one-liners
javascript	64	node stdlib	private-field clock, Map stamps in Set order, Math.floor over real division
python	143	stdlib only	dict stamps with .get zeros, script style, the even-cluster probe added
lua	180	lib.lua harness	plain-table stamps with or-zero, floor division, the overlap swept to n of 20

sources: L. Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System", CACM 1978, for the logical clock construction and its one-directional guarantee, the fallacies list as compiled by Deutsch and others at Sun, Peter Norvig's latency numbers and the Berkeley interactive latency guide for the orders of magnitude, Gilbert and Lynch, "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services", 2002, for CAP, accessed 2026-09-08. Verified by the eight chapter legs: 3 C programs with 40 embedded checks, the rust runner's 40 checks across 3 programs in `samples-rust/src/Ch10`, `go test` at 5 tests in `patternsbook/ch10`, the java runner's 49 checks across 3 programs in `samples-java/src/Ch10`, 10

xunit facts, node's 4 cases across 3 suites in `test/ch10.test.mjs`, the python runner's 33 checks across 3 modules, and the lua runner's 11 ch10 rows.

11. replication and consensus: raft

Replication is copying state so failures cost capacity instead of data. Consensus is getting a cluster to agree on the order of events so the copies cannot disagree, and raft, from Ongaro and Ousterhout's 2014 paper "In Search of an Understandable Consensus Algorithm", is the design this book builds its capstone on. This chapter is the theory, distilled to the parts the capstone implements, each rule extracted as a pure function with a test before chapter 16 wires them into nodes. Four samples carry it in all eight trees, append, vote, commit, and timeout, pure functions everywhere, and every fixture below is a shared literal except where its dry run names the split.

11.1. the state machine approach

Every server runs the same deterministic state machine against the same input log. If server A applied "set a=1" as entry 1 and "set b=2" as entry 2, and server B applied anything else in those slots, the cluster has forked without crashing. Consensus is the agreement on log contents, not on values, and the safety property falls out: once a command is applied at index n by any server, no server ever applies a different command at index n.

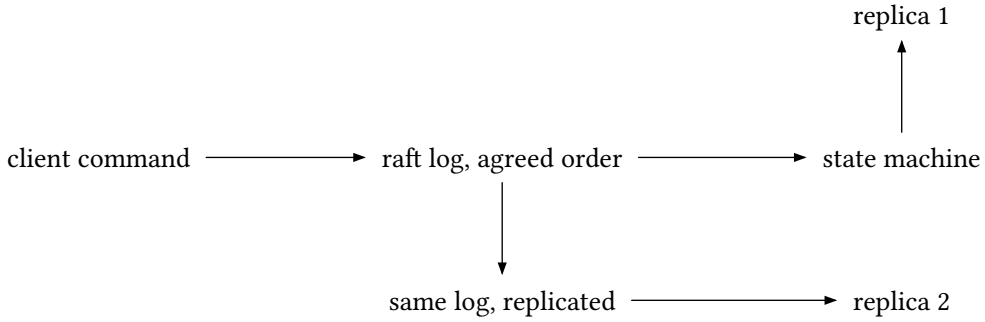


figure 61: the replicated state machine

11.2. terms and leader election

Raft divides time into terms, monotonically numbered logical epochs, each with at most one leader. A follower that hears nothing from a leader before its election timeout fires becomes a candidate, increments the term, votes for itself, and asks the rest for votes. A majority of votes makes it leader, and the very first act of leadership is a heartbeat `AppendEntries` that resets every other timeout, suppressing further elections.

Two rules carry the weight. Each server votes at most once per term, recorded in the persistent `votedFor`, so two candidates cannot both scrape a majority out of the same term. And the timeouts are randomized, in the paper's example 150 to 300 milliseconds, so split votes, everyone a candidate at once and nobody a majority, resolve on the next round because someone's timer fires first. Randomized restart doing the scheduling a smarter protocol would need is the same move the metaheuristics chapter explores, book 10, ch 41 (practical data structures and algorithms, local search and metaheuristics), and the seeded generator that chapter pinned for the whole corpus is the one the seven new lanes draw from here.

The dry run: the timeout draw is the one place this chapter splits its contract, and the split is the lesson.

- the frozen go lane draws from `rand/v2`'s pcg and asserts properties: 5 nodes land inside [150, 300), the same node redraws its own value, and at least 2 distinct values appear
- the seven new lanes draw from the corpus lcg, seed node+1, output the high 32 bits, and pin the exact vectors: nodes 0 through 4 draw 213, 265, 166, 218, 270, so node 2 fires first at 166
- the seven new trees assert the lcg warm-up vectors first, seed 1 drawing 1817669548 then 2187888307, before anything depends on the stream

```c
// the corpus lcg: uint64 state wraps mod 2^64, output is the high 32 bits
typedef struct {
  uint64_t x;
} lcg;

static uint32_t lcg_next(lcg *g) {
  g->x = 6364136223846793005ULL * g->x + 1442695040888963407ULL;
  return (uint32_t)(g->x >> 32);
}

// a uniform draw in [0, k) without the modulo bias a % k would add
static uint32_t lcg_below(uint32_t u, uint32_t k) {
  return (uint32_t)(((uint64_t)u * k) >> 32);
}

// timeout in [base, base+spread), deterministic per node id
static int election_timeout(int node_id, int base, int spread) {
  lcg g = {.x = (uint64_t)node_id + 1}; // seed node+1
  uint32_t u = lcg_next(&g);
  return base + (int)lcg_below(u, (uint32_t)spread);
}
```

listing 412: C, uint64 state wrapping mod 2^64, the draw is the high 32 bits, below(k) multiplies without modulo bias (source: patterns-concurrency-distributed/samples-c/src/Ch11/timeout.c, lines 21-41)

```rust
struct Lcg {
    x: u64,
}

impl Lcg {
    fn seeded(x: u64) -> Lcg {
        Lcg { x }
    }

    fn next_u32(&mut self) -> u32 {
        self.x = self
            .x
            .wrapping_mul(6364136223846793005)
            .wrapping_add(1442695040888963407);
        (self.x >> 32) as u32
    }
}

fn lcg_below(u: u32, k: u32) -> u32 {
    ((u as u64 * k as u64) >> 32) as u32
}

fn election_timeout(node_id: u64, base: u32, spread: u32) -> u32 {
    let mut generator = Lcg::seeded(node_id + 1);
    let u = generator.next_u32();
    base + lcg_below(u, spread)
}
```

listing 413: Rust, u64 wrapping spelled `wrapping_mul` and `wrapping_add`, plain overflow is a debug-build panic, the draw still the high 32 bits (source: patterns-concurrency-distributed/samples-rust/src/Ch11/timeout.rs, lines 15-41)

```go
func ElectionTimeout(nodeID int, base, spread int) int {
  r := rand.New(rand.NewPCG(uint64(nodeID), 0x5eed))
  return base + r.IntN(spread)
}
```

listing 414: Go, seeded draw from the base plus spread interval (source: patterns-concurrency-distributed/samples/ch11/consensus.go, lines 84-87)

```java
  // the corpus lcg: state wraps mod 2^64 in plain long arithmetic, the
  // output is the high 32 bits
  static final class Lcg {
    long x;

    Lcg(long seed) {
      x = seed;
    }

    long nextU32() {
      x = 6364136223846793005L * x + 1442695040888963407L;
      return x >>> 32; // in [0, 2^32), the unsigned high half
    }

    // a uniform draw in [0, k) without the modulo bias a % k would add
    long below(long k) {
      return nextU32() * k >>> 32;
    }
  }

  // timeout in [base, base+spread), deterministic per node id
  static int electionTimeout(int nodeId, int base, int spread) {
    return base + (int) new Lcg(nodeId + 1).below(spread);
  }
```

listing 415: Java, long arithmetic wraps mod 2^64 like c's uint64_t, the draw is the unsigned high half, no limb work anywhere (source: patterns-concurrency-distributed/samples-java/src/Ch11/Timeout.java, lines 21-44)

```csharp
// ElectionTimeout draws from [baseMs, baseMs + spreadMs),
// deterministically per node id. Randomization is what breaks
// split-vote livelock: with distinct timeouts, someone times out
// first and wins before the others retry. The draw rides the corpus
// LCG seeded with node id + 1, so every new lane prints the same
// timeouts; the frozen Go lane draws from rand/v2 PCG and asserts
// range properties instead, the split the fixtures brief records.
public static class Timeout
{
    public static int ElectionTimeout(int nodeId, int baseMs, int spreadMs)
    {
        return baseMs + new CorpusLcg((ulong)(nodeId + 1)).Below(spreadMs);
    }
}
```

listing 416: C#, the draw rides a CorpusLcg helper shared across the tree's chapters (source: patterns-concurrency-distributed/samples-cs/src/Ch11/Timeout.cs, lines 3-16)

```
export function makeLcg(seed) {
  let xh = 0;
  let xl = seed >>> 0;
  return {
    // one step of the recurrence, output is the new high half
    next() {
      const low = mul32lo(AL, xl);
      const rawLow = low.lo + CL;
      const newXL = rawLow % 4294967296;
      const carryLow = low.carry + (rawLow ≥ 4294967296 ? 1 : 0);
      const h1 = mul32lo(AL, xh);
      const h2 = mul32lo(AH, xl);
      xh = (h1.lo + h2.lo + CH + carryLow) % 4294967296;
      xl = newXL;
      return xh;
    },
    // floor(u * k / 2^32) split into 16-bit limbs so the exact
    // result holds for any 32-bit k, not only k ≤ 65536
    below(k) {
      const u = this.next();
      const uh = Math.floor(u / 65536);
      const ul = u % 65536;
      const kh = Math.floor(k / 65536);
      const kl = k % 65536;
      const mid = (uh * kl + ul * kh) * 65536 + ul * kl;
      return uh * kh + Math.floor(mid / 4294967296);
    },
  };
}

// ElectionTimeout draws deterministically per node id: node i seeds
// the generator with i + 1, the go lane's id range. distinct seeds
// give distinct ladders, someone fires first, the split vote breaks.
export function electionTimeout(nodeID, base, spread) {
  return base + makeLcg(nodeID + 1).below(spread);
}
```

listing 417: JavaScript, the 64-bit state as 32-bit halves multiplied through 16-bit limbs, no decimal literal past 2^53 anywhere (source: patterns-concurrency-distributed/samples-js/src/ch11-timeout.mjs, lines 35-70)

```python
MASK64 = (1 << 64) - 1
LCG_A = 6364136223846793005
LCG_C = 1442695040888963407


class LCG:
    def __init__(self, seed):
        self.x = seed & MASK64

    def next_u32(self):
        self.x = (LCG_A * self.x + LCG_C) & MASK64
        return self.x >> 32

    def below(self, k):
        return (self.next_u32() * k) >> 32


def election_timeout(node_id, base, spread):
    return base + LCG(node_id + 1).below(spread)
```

listing 418: Python, a masked int carries the state, the constants verbatim (source: patterns-concurrency-distributed/samples-py/src/Ch11/timeout.py, lines 16-34)

```lua
local LCG_A = 6364136223846793005
local LCG_C = 1442695040888963407

-- lcg returns a draw function: state advances by the pinned constants,
-- the output is the high 32 bits of the wrapped state
local function lcg(seed)
  local x = seed
  return function()
    x = LCG_A * x + LCG_C -- integer overflow wraps mod 2^64
    return x >> 32
  end
end

-- below maps a 32-bit draw onto [0, k): the multiply stays under 2^40,
-- the shift is the floor
local function below(u, k)
  return (u * k) >> 32
end

-- ElectionTimeout draws from [base, base+spread), deterministically
-- per node id: seed node+1
local function ElectionTimeout(node, base, spread)
  local draw = lcg(node + 1)
  return base + below(draw(), spread)
end
```

listing 419: Lua, integers wrap mod 2^64 natively, the whole draw stays integral (source: patterns-concurrency-distributed/samples-lua/ch11_timeout.lua, lines 8-32)

The carrier is where the languages disagree, and every disagreement is a number-theory fact wearing a syntax costume. C, c#, go, and lua hold 64-bit unsigned arithmetic natively, `uint64_t`, `ulong`, `uint64`, and lua's wrapping integers, so the recurrence is one line. Rust holds `u64` natively too but must name the wrap, `wrapping_mul` and `wrapping_add`, because a plain overflowing arithmetic op is a panic in a debug build. Java's signed `long` wraps through the same mod 2^64 arithmetic, one line again, and only the high-half draw needs the unsigned shift.

Python holds arbitrary precision and masks to 64 bits by hand. Javascript has no exact 64-bit `Number` at all, so its lane splits the state into high and low 32-bit halves and multiplies through 16-bit limbs, every partial product staying inside the exact double range, which is why the constants appear as hex limb pairs instead of decimals. Eight carriers, one stream, seven identical timeout vectors and one honestly unrepeatable pcg.

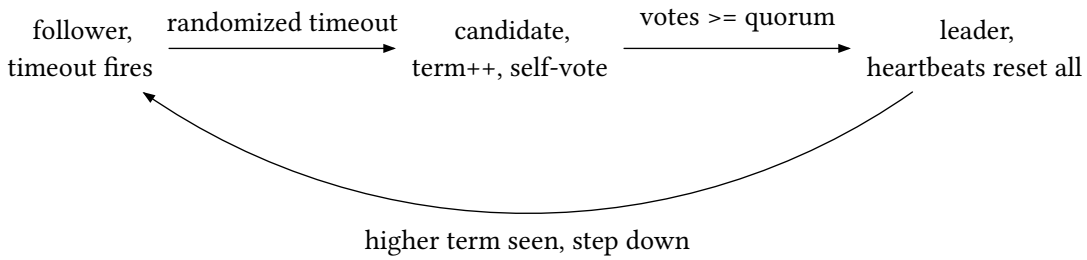


figure 62: election state machine, one vote per term, randomized timeouts break splits

The test pins the properties that matter: 5 nodes draw inside the interval, the same node draws the same value twice, and at least 2 distinct values appear, randomization actually happening.

11.3. the election restriction

Blind majority voting would let a stale log win an election and erase committed entries. Section 5.4.1 closes the hole: a voter grants its vote only if the candidate's log is at least as up-to-date as its own, where up-to-date is decided by the index and term of the last entries, larger last term wins outright, equal terms go to the longer log:

The dry run: the 5.4.1 matrix over the shared fixture log1 = [(1, "set a=1"), (1, "set b=2"), (2, "set c=3")], last term 2, last index 3, the same decisions in all eight lanes.

- a candidate ending at term 3 is up-to-date and a candidate ending at term 1 is not, at any length the lanes probe, go, java, c#, javascript, and lua probing 1 and 99, python 2 and 9, c adding 0, rust probing 1 and 0
- equal terms go to the longer log, and the exact tie goes to the candidate
- the c, rust, java, python, and lua lanes widen the matrix with the empty voter, whose last term reads 0 and who accepts anyone

```c
// is a candidate ending at (last_term, last_index) at least as up to date
// as my log?
static bool up_to_date(const entry *my_log, int my_len,
                       unsigned long long cand_last_term, int cand_last_index) {
  unsigned long long my_last_term = 0;
  if (my_len > 0)
    my_last_term = my_log[my_len - 1].term;
  int my_last_index = my_len;

  if (cand_last_term != my_last_term)
    return cand_last_term > my_last_term;
  return cand_last_index >= my_last_index;
}
```

listing 420: C, the comparison in one function, term first, length only on ties (source: patterns-concurrency-distributed/samples-c/src/Ch11/vote.c, lines 30-42)

```rust
fn up_to_date(my_log: &[Entry], cand_last_term: u64, cand_last_index: usize) -> bool {
    let my_last_term = my_log.last().map_or(0, |entry| entry.term);
    let my_last_index = my_log.len();
    if cand_last_term ≠ my_last_term {
        return cand_last_term > my_last_term;
    }
    cand_last_index ≥ my_last_index
}
```

listing 421: Rust, the empty log's last term an Option from `last()` folded to 0 by `map_or` (source: patterns-concurrency-distributed/samples-rust/src/Ch11/vote.rs, lines 35-42)

```go
func UpToDate(myLog []Entry, candidateLastTerm uint64, candidateLastIndex int) bool {
  myLastTerm := uint64(0)
  if len(myLog) > 0 {
    myLastTerm = myLog[len(myLog)-1].Term
  }
  myLastIndex := len(myLog)

  if candidateLastTerm ≠ myLastTerm {
    return candidateLastTerm > myLastTerm
  }
  return candidateLastIndex ≥ myLastIndex
}

// AdvanceCommit is the follower commit rule: trust the leader, but
```

listing 422: Go, the voter side of request vote, both rules in five lines (source: patterns-concurrency-distributed/samples/ch11/consensus.go, lines 44-57)

```java
  // is a candidate ending at (lastTerm, lastIndex) at least as up to date
  // as my log?
  static boolean upToDate(List<Entry> myLog, long candLastTerm, int candLastIndex) {
    long myLastTerm = 0;
    if (!myLog.isEmpty()) {
      myLastTerm = myLog.get(myLog.size() - 1).term();
    }
    int myLastIndex = myLog.size();

    if (candLastTerm ≠ myLastTerm) {
      return candLastTerm > myLastTerm;
    }
    return candLastIndex ≥ myLastIndex;
  }
```

listing 423: Java, term first, length only on ties, the log a list of records and the empty tail's term guarded to 0 (source: patterns-concurrency-distributed/samples-java/src/Ch11/Vote.java, lines 22-35)

```
// UpToDate is the section 5.4.1 election restriction from the
// voter's side: is a candidate ending at (lastTerm, lastIndex) at
// least as up-to-date as my log? Larger last term wins, equal terms
// go to the longer log.
public static class Vote
{
    public static bool UpToDate(List<Entry> myLog, ulong candidateLastTerm, int candidateLastIndex)
    {
        ulong myLastTerm = 0;
        if (myLog.Count > 0)
        {
            myLastTerm = myLog[myLog.Count - 1].Term;
        }
        int myLastIndex = myLog.Count;

        if (candidateLastTerm ≠ myLastTerm)
        {
            return candidateLastTerm > myLastTerm;
        }
        return candidateLastIndex ≥ myLastIndex;
    }
```

listing 424: C#, the same five decisions, the empty log reading term 0 (source: patterns-concurrency-distributed/samples-cs/src/Ch11/Vote.cs, lines 3-23)

```
// ch11: the section 5.4.1 election restriction, voter's side. is a
// candidate ending at (lastterm, lastindex) at least as up-to-date
// as my log? larger last term wins, equal terms go to the longer
// log, ties go to the candidate.

export function upToDate(myLog, candidateLastTerm, candidateLastIndex) {
  const myLastTerm = myLog.length > 0 ? myLog[myLog.length - 1].term : 0;
  const myLastIndex = myLog.length;

  if (candidateLastTerm =≠= myLastTerm) {
    return candidateLastTerm > myLastTerm;
  }
  return candidateLastIndex ≥ myLastIndex;
}
```

listing 425: JavaScript, the tail read through a conditional, falseness standing in for 0 (source: patterns-concurrency-distributed/samples-js/src/ch11-vote.mjs, lines 1-14)

```
def up_to_date(my_log, cand_last_term, cand_last_index):
    my_last_term = my_log[-1][0] if my_log else 0
    my_last_index = len(my_log)
    if cand_last_term ≠ my_last_term:
        return cand_last_term > my_last_term
    return cand_last_index ≥ my_last_index
```

listing 426: Python, the empty log's last term read through a conditional (source: patterns-concurrency-distributed/samples-py/src/Ch11/vote.py, lines 16-21)

```
-- UpToDate: is a candidate ending at (lastTerm, lastIndex) at least
-- as up-to-date as my log?
local function UpToDate(myLog, candidateLastTerm, candidateLastIndex)
  local myLastTerm = 0
  if #myLog > 0 then
    myLastTerm = myLog[#myLog].term
  end
  local myLastIndex = #myLog

  if candidateLastTerm ~= myLastTerm then
    return candidateLastTerm > myLastTerm
  end
  return candidateLastIndex ≥ myLastIndex
end
```

listing 427: Lua, the tail read at the table's end, term compared before length (source: patterns-concurrency-distributed/samples-lua/ch11_vote.lua, lines 14-27)

Every lane walks the same two comparisons in the same order, and the only spelling that differs is how an empty log yields its last term: a guarded read in go, c, java, and lua, `Count > 0` in c#, a conditional expression in javascript, python's negative-index conditional, and rust's `Option` from `last()` folded to `0` by `map_or`. The function is small because the rule is small, and the rule is small because the paper wanted implementers to get it right the first time.

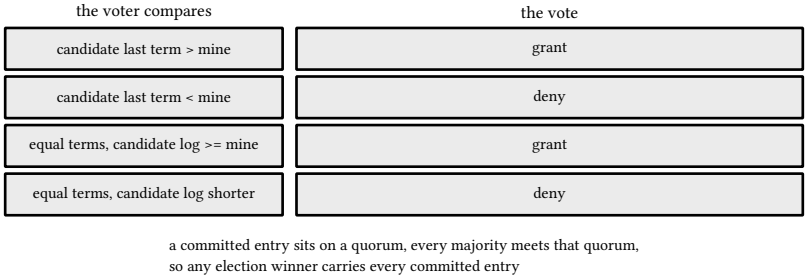


figure 63: the up-to-date matrix, larger last term first, length only breaks ties

The consequence is the paper's leader completeness property: any committed entry is present on a quorum, any majority intersects that quorum, so any election winner carries every committed entry.

11.4. log replication and the consistency check

The leader streams `AppendEntries` carrying the index and term of the entry immediately preceding the new ones. The follower rejects the whole request unless its log holds a matching entry at that position, and on acceptance overwrites conflicts and extends:

The dry run: the log1 fixture walks the four behaviors in every lane.

- prev(3, 2) extending with (2, "set d=4") accepts, 4 entries
- prev(9, 2) and prev(3, 1) both reject, the log handed back unchanged
- prev(2, 1) carrying (3, "set c=9") and (3, "set d=4") truncates the conflict, leaving [(1, a), (1, b), (3, c=9), (3, d=4)]
- re-delivering entries already present is a no-op, the second append returns a log equal to the first

```c
// append applies the consistency rules, returns the new log plus ok. the
// caller's log is copied in first so a rejection leaves it untouched
static bool append(raft_log *out, raft_log mine, int prev_index,
                   unsigned long long prev_term, const entry *entries,
                   int n_entries) {
  *out = mine;
  if (prev_index == 0) {
    // prev 0 matches the log root, always
  } else if (prev_index > mine.len ||
             mine.items[prev_index - 1].term != prev_term) {
    return false;
  }
  for (int i = 0; i < n_entries; i++) {
    int idx = prev_index + i + 1;
    if (idx ≤ out->len) {
      if (out->items[idx - 1].term != entries[i].term) {
        // conflict: delete this entry and everything after
        for (int k = i; k < n_entries; k++)
          out->items[idx - 1 + (k - i)] = entries[k];
        out->len = idx - 1 + n_entries - i;
        return true;
      }
      // already present and matching: skip, re-delivery is a no-op
    } else {
      for (int k = i; k < n_entries; k++)
        out->items[out->len++] = entries[k];
      return true;
    }
  }
  return true;
}
```

listing 428: C, a fixed-capacity log by value, the caller's copy untouched on rejection (source: patterns-concurrency-distributed/samples-c/src/Ch11/append.c, lines 49-79)

```rust
fn append(
    mine: &[Entry],
    prev_index: usize,
    prev_term: u64,
    entries: &[Entry],
) -> Option<RaftLog> {
    if prev_index > 0 {
        if prev_index > mine.len() || mine[prev_index - 1].term ≠ prev_term {
            return None;
        }
    }
    let mut out = mine.to_vec();
    for i in 0..entries.len() {
        let idx = prev_index + i + 1;
        if idx ≤ out.len() {
            if out[idx - 1].term ≠ entries[i].term {
                out.truncate(idx - 1);
                out.extend_from_slice(&entries[i..]);
                return Some(out);
            }
        } else {
            out.extend_from_slice(&entries[i..]);
            return Some(out);
        }
    }
    Some(out)
}
```

listing 429: Rust, `Option<Vec>` over `to_vec`, Some the successor and None the rejection, truncate plus extend_from_slice on conflict (source: patterns-concurrency-distributed/samples-rust/src/Ch11/append.rs, lines 40-66)

```go
func Append(log []Entry, prevIndex int, prevTerm uint64, entries []Entry) ([]Entry, bool) {
  if prevIndex == 0 {
    // prev 0 matches the log root, always
  } else if prevIndex > len(log) || log[prevIndex-1].Term ≠ prevTerm {
    return log, false
  }

  out := log
  for i, e := range entries {
    idx := prevIndex + i + 1
    switch {
    case idx <= len(out):
      if out[idx-1].Term ≠ e.Term {
        // conflict: delete this entry and everything after
        out = append(out[:idx-1], entries[i:]...)
        return out, true
      }
      // already present and matching: skip, re-delivery is a no-op
    default:
      return append(out, entries[i:]...), true
    }
  }
  return out, true
}

// UpToDate is the section 5.4.1 election restriction from the
// voter's side: is a candidate ending at (lastTerm, lastIndex) at
// least as up-to-date as my log? Larger last term wins, equal terms
```

listing 430: Go, prev check, conflict truncation, skip-already-present, extend (source: patterns-concurrency-distributed/samples/ch11/consensus.go, lines 15-42)

```
// append applies the consistency rules and returns the new log plus ok.
// the caller's log is never edited, a rejection hands it back unchanged
static Result append(List<Entry> mine, int prevIndex, long prevTerm,
    List<Entry> entries) {
  if (prevIndex ≠ 0
      && (prevIndex > mine.size() || mine.get(prevIndex - 1).term() ≠ prevTerm)) {
    return new Result(false, mine);
  }
  List<Entry> out = new ArrayList<>(mine);
  for (int i = 0; i < entries.size(); i++) {
    int idx = prevIndex + i + 1;
    if (idx ≤ out.size()) {
      if (out.get(idx - 1).term() ≠ entries.get(i).term()) {
        // conflict: this entry and everything after it is replaced by the
        // leader's tail
        List<Entry> merged = new ArrayList<>(out.subList(0, idx - 1));
        merged.addAll(entries.subList(i, entries.size()));
        return new Result(true, List.copyOf(merged));
      }
      // already present and matching: skip, re-delivery is a no-op
    } else {
      List<Entry> merged = new ArrayList<>(out);
      merged.addAll(entries.subList(i, entries.size()));
      return new Result(true, List.copyOf(merged));
    }
  }
  return new Result(true, List.copyOf(out));
}
```

listing 431: Java, append is pure over immutable lists, a new log comes back frozen, rejection hands back the caller’s own (source: patterns-concurrency-distributed/samples-java/src/Ch11/Append.java, lines 30-57)

```
    public static (List<Entry> log, bool ok) Apply(
        List<Entry> log, int prevIndex, ulong prevTerm, List<Entry> entries)
    {
        if (prevIndex == 0)
        {
            // prev 0 matches the log root, always
        }
        else if (prevIndex > log.Count || log[prevIndex - 1].Term ≠ prevTerm)
        {
            return (log, false);
        }

        var result = new List<Entry>(log);
        for (int i = 0; i < entries.Count; i++)
        {
            int idx = prevIndex + i + 1;
            if (idx ≤ result.Count)
            {
                if (result[idx - 1].Term ≠ entries[i].Term)
                {
                    // conflict: delete this entry and everything after
                    result.RemoveRange(idx - 1, result.Count - (idx - 1));
                    result.AddRange(entries.GetRange(i, entries.Count - i));
                    return (result, true);
                }
                // already present and matching: skip, re-delivery is a no-op
            }
            else
            {
                result.AddRange(entries.GetRange(i, entries.Count - i));
                return (result, true);
            }
        }
        return (result, true);
    }
```

listing 432: C#, the input list never mutated, RemoveRange truncates, AddRange extends (source: patterns-concurrency-distributed/samples-cs/src/Ch11/Append.cs, lines 13-47)

```
export function appendLog(log, prevIndex, prevTerm, entries) {
  if (prevIndex =/= 0) {
    // prev 0 matches the log root, always
    if (prevIndex > log.length || log[prevIndex - 1].term =/= prevTerm) {
      return { log, ok: false };
    }
  }

  let out = log;
  for (let i = 0; i < entries.length; i++) {
    const e = entries[i];
    const idx = prevIndex + i + 1;
    if (idx ≤ out.length) {
      if (out[idx - 1].term =/= e.term) {
        // conflict: drop this entry and everything after it
        return { log: [...out.slice(0, idx - 1), ...entries.slice(i)], ok: true };
      }
      // already present and matching: skip, re-delivery is a no-op
    } else {
      return { log: [...out, ...entries.slice(i)], ok: true };
    }
  }
  return { log: out, ok: true };
}
```

listing 433: JavaScript, slice and spread rebuild the log, the input array shared on rejection (source: patterns-concurrency-distributed/samples-js/src/ch11-append.mjs, lines 7-30)

```
def append(log, prev_index, prev_term, entries):
    if prev_index == 0:
        pass  # prev 0 matches the log root, always
    elif prev_index > len(log) or log[prev_index - 1][0] ≠ prev_term:
        return log, False

    out = list(log)
    for i, entry in enumerate(entries):
        idx = prev_index + i + 1
        if idx ≤ len(out):
            if out[idx - 1][0] ≠ entry[0]:
                # conflict: this entry and everything after goes
                return out[: idx - 1] + list(entries[i:]), True
            # already present and matching: re-delivery is a no-op
        else:
            return out + list(entries[i:]), True
    return out, True
```

listing 434: Python, list slicing spells truncate and extend, tuples carrying each entry (source: patterns-concurrency-distributed/samples-py/src/Ch11/append.py, lines 15-31)

```lua
-- Append applies the rules and returns the new log plus acceptance.
-- prevIndex 0 matches the log root always
local function Append(log, prevIndex, prevTerm, entries)
  if prevIndex > 0 then
    if prevIndex > #log or log[prevIndex].term ~= prevTerm then
      return log, false
    end
  end
  local out = copy(log)
  for i = 1, #entries do
    local idx = prevIndex + i
    if idx ≤ #out then
      if out[idx].term ~= entries[i].term then
        -- conflict: this entry and everything after it is replaced
        local t = {}
        for j = 1, idx - 1 do
          t[#t + 1] = out[j]
        end
        for j = i, #entries do
          t[#t + 1] = entries[j]
        end
        return t, true
      end
      -- already present and matching: skip, re-delivery is a no-op
    else
      local t = copy(out)
      for j = i, #entries do
        t[#t + 1] = entries[j]
      end
      return t, true
    end
  end
  return out, true
end
```

listing 435: Lua, the copy made entry by entry, the same four rules in 1-based indices (source: patterns-concurrency-distributed/samples-lua/ch11_append.lua, lines 23-56)

Purity is the pattern: go and c# copy before mutating, java copies the immutable input and freezes the successor with `List.copyOf`, javascript and python build the successor out of slices, c copies the log by value into the out parameter, rust builds the successor with `to_vec` and hands back `Some` or `None`, lua builds a fresh table. The 1-based languages pay one index shift and say so in a comment, the rule itself unchanged. Rejection returns the caller's own log, which is what lets the leader retry a lower `nextIndex` without corrupting the follower it is repairing.

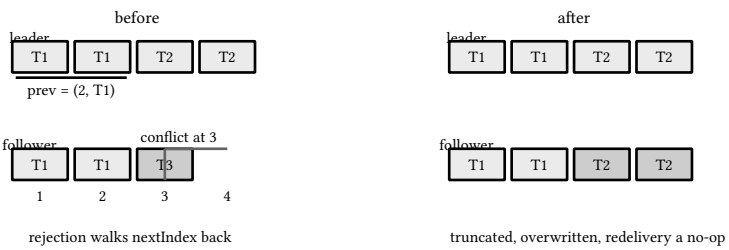


figure 64: a diverged follower, the prev anchor, the walk back, and the repair

The `prevIndex, prevTerm` pair is an induction anchor: if my entry at index n matches yours at index n, and each append proves its own prefix, the logs agree on everything up to the new entries. A rejection makes the leader step its `nextIndex` for that follower back by one and retry, walking back until the prefix matches, which is how a lagging or diverged follower is repaired. The pure function's tests cover the four behaviors the capstone's followers will

rely on: extension, rejection without mutation on both mismatch flavors, truncation on conflict, and the quiet one, re-delivery of entries already present is a no-op, because rpcs can retry and an idempotent log operation is what makes at-least-once delivery safe, chapter 13's whole subject arriving early.

11.5. commitment

An entry is committed once stored on a quorum, but the counting rule has a famous restriction, section 5.4.2: a leader only commits entries from its own current term by counting replicas. Entries from earlier terms commit indirectly, when a current-term entry above them commits. The paper's figure 8 walk-through shows why: without the restriction a re-elected leader can commit a prior-term entry and later see it overwritten. Followers learn the commit index from the leader's `leaderCommit`, clamped to their own last index:

The dry run: the commit rules over the same fixture.

- AdvanceCommit(2, 5, 3) = 3 and (1, 9, 3) = 3, the clamp, and (5, 2, 3) = 5, no regress. The seven new trees pin that triple, the go lane's frozen test pins its own, (0, 2, 3) = 2, (0, 99, 3) = 3, (2, 1, 3) = 2, against the same three rules
- CommitByCounting refuses a prior-term entry even at 3 of 3 replicas, (3, 3, term 1, current 2) is false in seven lanes, python making the same refusal at 2 of 3
- a current-term entry commits at quorum, (2, 3, 2, 2) true, and does not below it, (1, 3, 2, 2) false

```c
// trust the leader, clamped to my log, never regressing
static int advance_commit(int my_commit, int leader_commit, int my_last_index) {
  if (leader_commit > my_last_index)
    return my_last_index;
  if (leader_commit > my_commit)
    return leader_commit;
  return my_commit;
}

// counting replicas commits only current-term entries at quorum
static bool commit_by_counting(int replicas, int cluster_size,
                               unsigned long long entry_term,
                               unsigned long long current_term) {
  if (entry_term != current_term)
    return false;
  return replicas >= cluster_size / 2 + 1;
}
```

listing 436: C, the clamp and the counting restriction, two small functions (source: patterns-concurrency-distributed/samples-c/src/Ch11/commit.c, lines 19-35)

```rust
fn advance_commit(my_commit: usize, leader_commit: usize, my_last_index: usize) -> usize {
    if leader_commit > my_last_index {
        return my_last_index;
    }
    if leader_commit > my_commit {
        return leader_commit;
    }
    my_commit
}

fn commit_by_counting(
    replicas: usize,
    cluster_size: usize,
    entry_term: u64,
    current_term: u64,
) -> bool {
    if entry_term ≠ current_term {
        return false;
    }
    replicas ≥ cluster_size / 2 + 1
}
```

listing 437: Rust, the same two functions over usize, the term restriction checked before the quorum count (source: patterns-concurrency-distributed/samples-rust/src/Ch11/commit.rs, lines 15-35)

```go
func AdvanceCommit(myCommit, leaderCommit, myLastIndex int) int {
  if leaderCommit > myLastIndex {
    return myLastIndex
  }
  if leaderCommit > myCommit {
    return leaderCommit
  }
  return myCommit
}

// CommitByCounting encodes the section 5.4.2 restriction: counting
// replicas commits only entries from the leader's current term.
// Prior-term entries ride along when a current-term entry above
// them commits.
func CommitByCounting(replicas, clusterSize int, entryTerm, currentTerm uint64) bool {
  if entryTerm ≠ currentTerm {
    return false
  }
  return replicas ≥ clusterSize/2+1
}

// ElectionTimeout draws from [base, base+spread), deterministically
// per node id. Randomization is what breaks split-vote livelock:
```

listing 438: Go, clamp to what i hold, never regress, and the counting restriction (source: patterns-concurrency-distributed/samples/ch11/consensus.go, lines 59-81)

```
  // trust the leader, clamped to my log, never regressing
  static int advanceCommit(int myCommit, int leaderCommit, int myLastIndex) {
    if (leaderCommit > myLastIndex) {
      return myLastIndex;
    }
    if (leaderCommit > myCommit) {
      return leaderCommit;
    }
    return myCommit;
  }

  // counting replicas commits only current-term entries at quorum
  static boolean commitByCounting(int replicas, int clusterSize, long entryTerm,
      long currentTerm) {
    if (entryTerm ≠ currentTerm) {
      return false;
    }
    return replicas ≥ clusterSize / 2 + 1;
  }
```

listing 439: Java, the clamp never regresses, the counting check refuses any term but the current one (source: patterns-concurrency-distributed/samples-java/src/Ch11/Commit.java, lines 18-36)

```
public static class Commit
{
    // Advance is the follower commit rule: trust the leader, but
    // never past what I actually hold, and never backwards.
    public static int Advance(int myCommit, int leaderCommit, int myLastIndex)
    {
        if (leaderCommit > myLastIndex)
        {
            return myLastIndex;
        }
        if (leaderCommit > myCommit)
        {
            return leaderCommit;
        }
        return myCommit;
    }

    // ByCounting encodes the section 5.4.2 restriction: counting
    // replicas commits only entries from the leader's current term.
    // Prior-term entries ride along when a current-term entry above
    // them commits.
    public static bool ByCounting(
        int replicas, int clusterSize, ulong entryTerm, ulong currentTerm)
    {
        if (entryTerm ≠ currentTerm)
        {
            return false;
        }
        return replicas ≥ clusterSize / 2 + 1;
    }
}
```

listing 440: C#, Advance and ByCounting as static pure functions (source: patterns-concurrency-distributed/samples-cs/src/Ch11/Commit.cs, lines 3-33)

```javascript
export function advanceCommit(myCommit, leaderCommit, myLastIndex) {
  if (leaderCommit > myLastIndex) {
    return myLastIndex;
  }
  if (leaderCommit > myCommit) {
    return leaderCommit;
  }
  return myCommit;
}

export function commitByCounting(replicas, clusterSize, entryTerm, currentTerm) {
  if (entryTerm !== currentTerm) {
    return false;
  }
  return replicas >= Math.floor(clusterSize / 2) + 1;
}
```

listing 441: JavaScript, two functions, floor division rebuilding the quorum (source: patterns-concurrency-distributed/samples-js/src/ch11-commit.mjs, lines 7-22)

```python
def advance_commit(my_commit, leader_commit, my_last_index):
    if leader_commit > my_last_index:
        return my_last_index
    if leader_commit > my_commit:
        return leader_commit
    return my_commit


def commit_by_counting(replicas, cluster, entry_term, current_term):
    if entry_term != current_term:
        return False
    return replicas >= cluster // 2 + 1
```

listing 442: Python, the same pair, floor division rebuilding the quorum (source: patterns-concurrency-distributed/samples-py/src/Ch11/commit.py, lines 15-26)

```lua
-- AdvanceCommit: trust the leader, clamped to my log, never backwards
local function AdvanceCommit(myCommit, leaderCommit, myLastIndex)
  if leaderCommit > myLastIndex then
    return myLastIndex
  end
  if leaderCommit > myCommit then
    return leaderCommit
  end
  return myCommit
end

-- CommitByCounting: entries from the current term only, quorum only
local function CommitByCounting(replicas, clusterSize, entryTerm, currentTerm)
  if entryTerm ~= currentTerm then
    return false
  end
  return replicas >= clusterSize // 2 + 1
end
```

listing 443: Lua, the same pair, floor division over wrapping integers (source: patterns-concurrency-distributed/samples-lua/ch11_commit.lua, lines 6-23)

Nothing here needs a data structure at all, three integers in, one or a boolean out, and the eight listings are near transcriptions of each other. That flatness is deliberate on the paper's part: the commit rules are the safety argument, and safety arguments that depend on clever code are not arguments. The lua commit file adds a monotone envelope sweep, every (myCommit, leaderCommit) pair in a grid checked to stay never above the log and never below the current commit, a property the other lanes pin only at points.

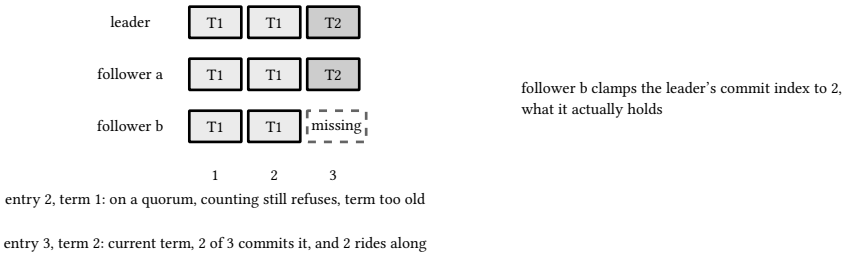


figure 65: a current-term entry commits by counting and carries the prior term with it

11.6. the state, and what the capstone keeps

Each server persists `currentTerm`, `votedFor`, and the log, everything needed to safely restart, and keeps volatile `commitIndex`, `lastApplied`, and, as leader, `nextIndex` and `matchIndex` per follower. The capstone implements exactly this shape with a wal-backed sqlite store holding the persistent triple, over an in-memory channel transport, with fixed membership, the paper's joint-consensus membership changes being the labeled door this book names and walks past.

NOTE: raft versus paxos

Paxos family algorithms reach consensus on single values with no designated leader, at the cost of a protocol most readers cannot hold in their head. Raft trades a stronger leadership assumption for decomposability, election, replication, safety, three subproblems studied separately, which is why this chapter could be written and why the capstone fits in a few hundred tested lines.

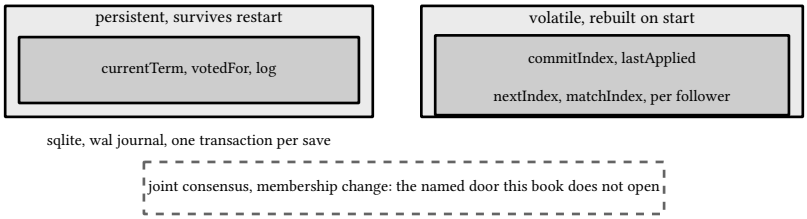


figure 66: hard state persists, volatile state is rebuilt, joint consensus stays closed

11.7. across the eight languages

The build sizes count non-comment source lines, the go column the chapter's one frozen file:

language	build SLOC	dependency	boundary note
c	234	libc	wrapping uint64_t lcg, the log copied by value into an out parameter
rust	356	stdlib	wrapping named in the lcg, an Option<Vec> append, the vote tail read through `map_or`
go	56	stdlib	frozen reference lane, the pcg timeout draw with its own property test
java	269	jdk 27 stdlib	long-wrapped lcg with no limb work, records over immutable lists frozen with List.copyOf

language	build SLOC	dependency	boundary note
c#	87	bcl	shared CorpusLcg helper, append never mutates the input, RemoveRange truncates
javascript	104	node stdlib	lcg state in 32-bit halves through 16-bit limbs, slices rebuild the log
python	180	stdlib only	a hand-masked int for the lcg, the empty log read through a negative index
lua	244	lib.lua harness	native 2^64 wrapping, 1-based indices, the commit file's monotone sweep

sources: Ongaro and Ousterhout, "In Search of an Understandable Consensus Algorithm (Extended Version)", raft.github.io/raft.pdf, sections 5.2, 5.3, 5.4.1, and 5.4.2 and figure 2, accessed 2026-09-08, plus Ongaro's dissertation for the membership change reference. Verified by the eight chapter legs: 4 C programs with 66 embedded checks, the rust runner's 62 checks across 4 programs in `samples-rust/src/Ch11`, `go test` at 8 tests in `patternsbook/ch11`, the java runner's 60 checks across 4 programs in `samples-java/src/Ch11`, 17 xunit facts, node's 7 cases across 5 suites in `test/ch11.test.mjs`, the python runner's 48 checks across 4 modules, and the lua runner's 17 ch11 rows.

12. partitioning and consistent hashing

Partitioning spreads data over machines so capacity grows with the cluster, and the engineering problem is not the spread, it is the reshuffle. `key mod n` places keys perfectly uniformly, and adding one machine moves almost every key, which at cache scale is an outage wearing a maintenance window. This chapter builds the two placement schemes that move the minimum, consistent hashing and rendezvous, plus jump hash, the stateless special case, and one hashing bug worth more than all of them. It is also this book's exact-lane chapter: the raw hash universe and every placement sequence are pinned byte-identical across the seven sibling trees, while the frozen go tests hold the stability and band properties, and the dry runs below say which lane asserts what.

12.1. the naive baseline and its cost

With `n` nodes and `key mod n`, taking the cluster from n to n+1 changes the modulus, so every key whose remainder changes relocates, for typical sizes that is `n/(n+1)` of the key space, near total churn. The goal shape instead: when a node joins, only the keys that move are the ones handed to the new node, roughly `1/(n+1)`, and when a node leaves, only its keys redistribute to survivors.

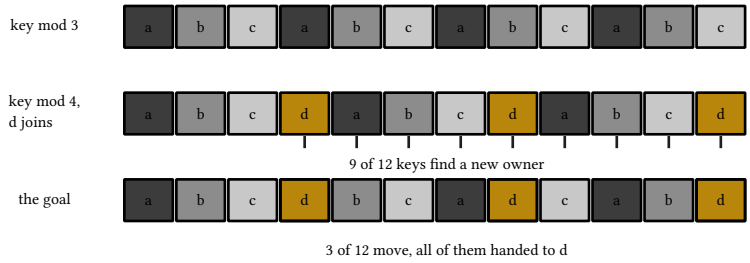


figure 67: key mod n churns almost everything, the goal shape moves 1/(n+1)

12.2. consistent hashing with virtual nodes

Each physical node is hashed onto `vnodes` points around a ring of uint64 values, and a key belongs to the first point at or clockwise after its own hash:

The dry run: the exact-lane contract, split by design.

- the go lane's frozen tests pin stability and bands: ownership never drifts across queries, a 1-of-4 join moves between 1500 and 3500 of 10000 keys with every mover landing on the new node, four-node balance stays inside 550..1450 of 4000, and removal never disturbs a survivor
- the seven sibling trees pin the exact sequences off the same implementation: the join moves exactly 2303 keys, all to n4, with post counts 2870/2267/2560/2303, balance lands 1174/1024/879/923, and removing n2 relocates exactly 705 keys, 321 to n1 and 384 to n3
- ring{alpha,beta} at 64 vnodes owns user:0..9 as a,a,a,b,a,a,a,b,a,b, the ownership strip the seven new trees pin

```c
static void ring_add(ring *r, const char *node) {
  int idx = ring_name_index(r, node);
  if (idx < 0)
    return;
  for (int p = 0; p < r->n_points; p++)
    if (r->points[p].node == idx)
      return; // already a member
  for (int v = 0; v < r->vnodes; v++) {
    char buf[32];
    snprintf(buf, sizeof buf, "%s#%d", node, v);
    r->points[r->n_points].hash = hash_string(buf);
    r->points[r->n_points].node = (short)idx;
    r->n_points++;
  }
  // sort by hash, ties broken by node name for cross-language stability
  for (int i = 1; i < r->n_points; i++)
    for (int j = i; j > 0; j--) {
      ring_point *a = &r->points[j - 1], *b = &r->points[j];
      int c = a->hash != b->hash ? (a->hash < b->hash ? -1 : 1)
                                 : strcmp(r->names[a->node], r->names[b->node]);
      if (c > 0) {
        ring_point tmp = *a;
        *a = *b;
        *b = tmp;
      } else {
        break;
      }
    }
}

static void ring_remove(ring *r, const char *node) {
  int idx = ring_name_index(r, node);
  if (idx < 0)
    return;
  int k = 0;
  for (int i = 0; i < r->n_points; i++)
    if (r->points[i].node != idx)
      r->points[k++] = r->points[i];
  r->n_points = k;
}

// the first point at or clockwise-after the key's hash, wrapping around
static int ring_owner(ring *r, const char *key) {
  if (r->n_points == 0)
    return -1;
  uint64_t h = hash_string(key);
  for (int i = 0; i < r->n_points; i++)
    if (r->points[i].hash >= h)
      return r->points[i].node;
  return r->points[0].node;
}
```

listing 444: C, insertion-sorted points with the node name breaking hash ties, owner scans for the first point at or after the key (source: patterns-concurrency-distributed/samples-c/src/Ch12/ring.c, lines 73-123)

```
    fn add(&mut self, node: &str) {
        let Some(idx) = self.name_index(node) else {
            return;
        };
        if self.points.iter().any(|point| point.node == idx) {
            return;
        }
        for v in 0..self.vnodes {
            let point_name = format!("{node}#{v}");
            self.points.push(RingPoint {
                hash: hash_string(&point_name),
                node: idx,
            });
        }
        self.points.sort_by(|a, b| {
            a.hash
                .cmp(&b.hash)
                .then_with(|| self.names[a.node].cmp(self.names[b.node]))
        });
    }

    fn remove(&mut self, node: &str) {
        let Some(idx) = self.name_index(node) else {
            return;
        };
        self.points.retain(|point| point.node != idx);
    }

    fn owner(&self, key: &str) -> Option<usize> {
        if self.points.is_empty() {
            return None;
        }
        let h = hash_string(key);
        for point in &self.points {
            if point.hash ≥ h {
                return Some(point.node);
            }
        }
        Some(self.points[0].node)
    }
}
```

listing 445: Rust, points sorted by `sort_by` with the name tiebreak chained through `then_with`, owner the first point at or after the key, None on the empty ring (source: patterns-concurrency-distributed/samples-rust/src/Ch12/ring.rs, lines 59-99)

```
type Ring struct {
  vnodes   int
  points   []ringPoint
  nodeOf   map[string]bool
  byString map[string][]uint64
}

type ringPoint struct {
  hash uint64
  node string
}
```

```
func NewRing(vnodes int) *Ring {
  if vnodes < 1 {
    vnodes = 1
  }
  return &Ring{vnodes: vnodes, nodeOf: map[string]bool{}, byString: map[string][]uint64{}}
}

func (r *Ring) Add(node string) {
  if r.nodeOf[node] {
    return
  }
  r.nodeOf[node] = true
  for v := 0; v < r.vnodes; v++ {
    h := hashString(node + "#" + itoa(v))
    r.points = append(r.points, ringPoint{h, node})
    r.byString[node] = append(r.byString[node], h)
  }
  slices.SortFunc(r.points, func(a, b ringPoint) int {
    switch {
    case a.hash < b.hash:
      return -1
    case a.hash > b.hash:
      return 1
    }
    return slices.Compare([]string{a.node}, []string{b.node})
  })
}

func (r *Ring) Remove(node string) {
  if !r.nodeOf[node] {
    return
  }
  delete(r.nodeOf, node)
  delete(r.byString, node)
  kept := r.points[:0]
  for _, p := range r.points {
    if p.node ≠ node {
      kept = append(kept, p)
    }
  }
  r.points = kept
}

// Owner answers which node a key belongs to. The empty ring has no
// answer and says so.
func (r *Ring) Owner(key string) (string, bool) {
  if len(r.points) == 0 {
    return "", false
  }
  h := hashString(key)
  i := sort.Search(len(r.points), func(i int) bool { return r.points[i].hash ≥ h })
  if i == len(r.points) {
    i = 0 // wrap around the circle
  }
  return r.points[i].node, true
}
```

listing 446: Go, ring points sorted by hash, owner is the first point after the key, wrapping (source: patterns-concurrency-distributed/samples/ch12/hashing.go, lines 38-105)

```
    void add(String node) {
      if (members.contains(node)) {
        return;
      }
      int idx = members.size();
      members.add(node);
      for (int v = 0; v < vnodes; v++) {
        points.add(new Point(hashString(node + "#" + v), idx));
      }
      // points order by hash as uint64, ties by node name: the same
      // ordering the c lane's uint64 compare and py's unbounded ints give
      points.sort((a, b) -> {
        int byHash = Long.compareUnsigned(a.hash(), b.hash());
        if (byHash != 0) {
          return byHash;
        }
        return members.get(a.node()).compareTo(members.get(b.node()));
      });
    }

    void remove(String node) {
      int idx = members.indexOf(node);
      if (idx < 0) {
        return;
      }
      points.removeIf(p -> p.node() == idx);
    }

    int size() {
      return points.size();
    }

    // the first point at or clockwise-after the key's hash, wrapping
    // around. -1 when the ring is empty. the circle is uint64, so the
    // comparisons are unsigned
    int owner(String key) {
      if (points.isEmpty()) {
        return -1;
      }
      long h = hashString(key);
      for (Point p : points) {
        if (Long.compareUnsigned(p.hash(), h) >= 0) {
          return p.node();
        }
      }
      return points.get(0).node();
    }
```

listing 447: Java, points sorted by unsigned hash with the node name tiebreak, owner scans for the first point at or after the key (source: patterns-concurrency-distributed/samples-java/src/Ch12/Ring.java, lines 55-101)

```
    public void Add(string node)
    {
        if (!_nodes.Add(node))
        {
            return;
        }
        for (int v = 0; v < vnodes; v++)
        {
            ulong h = Hash.String(node + "#" + v);
            _points.Add(new RingPoint(h, node));
        }
        _points.Sort((a, b) =>
        {
            int byHash = a.Hash.CompareTo(b.Hash);
            return byHash != 0 ? byHash : string.CompareOrdinal(a.Node, b.Node);
        });
    }

    public void Remove(string node)
    {
        if (!_nodes.Remove(node))
        {
            return;
        }
        _points.RemoveAll(p => p.Node == node);
    }
```

listing 448: C#, Add hashes each vnode string and sorts by hash with an ordinal tiebreak, Remove drops the node's points (source: patterns-concurrency-distributed/samples-cs/src/Ch12/Ring.cs, lines 21-46)

```
export class Ring {
  #nodeOf = new Set();

  constructor(vnodes) {
    this.vnodes = vnodes < 1 ? 1 : vnodes;
    this.points = [];
  }

  add(node) {
    if (this.#nodeOf.has(node)) return;
    this.#nodeOf.add(node);
    for (let v = 0; v < this.vnodes; v++) {
      this.points.push({ hash: hashString(`${node}#${v}`), node });
    }
    this.points.sort(cmpPoint);
  }

  remove(node) {
    if (!this.#nodeOf.has(node)) return;
    this.#nodeOf.delete(node);
    this.points = this.points.filter((p) => p.node !== node);
  }

  // first point at or clockwise-after the key's hash, wrapping.
  // null says the ring is empty.
  owner(key) {
    if (this.points.length === 0) return null;
    const h = hashString(key);
    let lo = 0;
    let hi = this.points.length;
    while (lo < hi) {
      const mid = (lo + hi) >> 1;
      if (cmp64(this.points[mid].hash, h) >= 0) hi = mid;
      else lo = mid + 1;
    }
    if (lo === this.points.length) lo = 0;
    return this.points[lo].node;
  }
}
```

listing 449: JavaScript, the ring over {hi, lo} hash pairs, the lower-bound binary search written by hand (source: patterns-concurrency-distributed/samples-js/src/ch12-ring.mjs, lines 14-52)

```
class Ring:
    def __init__(self, vnodes):
        self.vnodes = max(1, vnodes)
        self.nodes = set()
        self.points = []  # parallel: sorted hashes, owners by index

    def add(self, node):
        if node in self.nodes:
            return
        self.nodes.add(node)
        fresh = []
        for v in range(self.vnodes):
            fresh.append((fnv64a(f"{node}#{v}".encode()), node))
        self.points = sorted(self.points + fresh)

    def remove(self, node):
        if node not in self.nodes:
            return
        self.nodes.discard(node)
        self.points = [p for p in self.points if p[1] != node]

    def owner(self, key):
        if not self.points:
            return None
        h = fnv64a(key.encode())
        i = bisect_left(self.points, (h, ""))
        if i == len(self.points):
            i = 0  # wrap around the circle
        return self.points[i][1]
```

listing 450: Python, points sorted as (hash, node) tuples, bisect_left finds the wrap point (source: patterns-concurrency-distributed/samples-py/src/Ch12/ring.py, lines 40-68)

```
function Ring:Add(node)
  if self.nodeOf[node] then return end
  self.nodeOf[node] = true
  for v = 0, self.vnodes - 1 do
    self.points[#self.points + 1] = { h = hashString(node .. "#" .. v), node = node }
  end
  table.sort(self.points, function(a, b)
    if a.h ~= b.h then
      return uless(a.h, b.h)
    end
    return a.node < b.node
  end)
end

function Ring:Remove(node)
  if not self.nodeOf[node] then return end
  self.nodeOf[node] = nil
  local kept = {}
  for _, p in ipairs(self.points) do
    if p.node ~= node then
      kept[#kept + 1] = p
    end
  end
  self.points = kept
end

-- Owner: the first point at or clockwise-after the key's hash. the
-- empty ring has no answer and says so. lower-bound binary search
function Ring:Owner(key)
  if #self.points == 0 then
    return nil, false
  end
  local h = hashString(key)
  local lo, hi = 1, #self.points + 1
  while lo < hi do
    local mid = (lo + hi) // 2
    if uless(h, self.points[mid].h) then
      hi = mid
    else
      lo = mid + 1
    end
  end
  if lo > #self.points then
    lo = 1 -- wrap around the circle
  end
  return self.points[lo].node, true
end
```

listing 451: Lua, the sign bit xored into the comparison key so wrapped negatives sort where they belong (source: patterns-concurrency-distributed/samples-lua/ch12_ring.lua, lines 50-96)

Virtual nodes are the balance mechanism: one hash point per node leaves ownership proportional to random arc lengths, wildly uneven for small clusters, while 100 to 200 vnodes per node averages the arcs until each node's share sits near 1/n. The tiebreak matters as much as the sort: points with equal hashes order by node name in every lane, go through `slices.Compare` on the names, c through `strcmp` inside its insertion sort, rust through `cmp` chained by `then_with` in `sort_by`, c# through `CompareOrdinal`, and the dynamic four through their native string order,

which is one of the two details that keep eight independent implementations on one movement sequence. The other is the hash itself, next section.

The test side is half the fixture, and each tree answers with its own test artifact: the frozen go test file, the xunit and node test files, and the check regions of the C, rust, java, python, and lua modules.

```
  // movement: 10000 keys, 100 vnodes, three nodes then a fourth joins.
  // only the keys whose arc the new node takes may move, and it takes
  // exactly the moved set: moved == gained
  const char *n4names[4] = {"n1", "n2", "n3", "n4"};
  ring big;
  ring_init(&big, n4names, 4, 100);
  ring_add(&big, "n1");
  ring_add(&big, "n2");
  ring_add(&big, "n3");
  static short before[MAX_KEYS];
  for (int i = 0; i < MAX_KEYS; i++) {
    char key[32];
    key_name(key, i);
    before[i] = (short)ring_owner(&big, key);
  }
  ring_add(&big, "n4");
  int moved = 0, gained = 0;
  long post[4] = {0, 0, 0, 0};
  for (int i = 0; i < MAX_KEYS; i++) {
    char key[32];
    key_name(key, i);
    int now = ring_owner(&big, key);
    post[now]++;
    if (now != before[i]) {
      moved++;
      if (now == 3)
        gained++;
    }
  }
  CHECK(moved == 2303);
  CHECK(moved == gained); // non-joining keys never move
  CHECK(post[0] == 2870 && post[1] == 2267 && post[2] == 2560 &&
        post[3] == 2303);
  CHECK(moved ≥ 1500 && moved ≤ 3500); // and the band the go lane pins
```

listing 452: C, the check region: 2303 movers, all gained, post counts exact, the go lane's band stated beside them (source: patterns-concurrency-distributed/samples-c/src/Ch12/ring.c, lines 148-181)

```
    let before: Vec<Option<usize>> = (0..10000)
        .map(|i| big.owner(&format!("user:{i}")))
        .collect();
    big.add("n4");
    let mut moved = 0;
    let mut gained = 0;
    let mut post = [0usize; 4];
    for i in 0..10000 {
        let now = big.owner(&format!("user:{i}")).unwrap();
        post[now] += 1;
        if Some(now) ≠ before[i] {
            moved += 1;
            if now == 3 {
                gained += 1;
            }
        }
    }
    ok(moved == 2303, "exactly 2303 keys moved when n4 joined");
    ok(moved == gained, "non-joining keys never move");
    ok(
        post == [2870, 2267, 2560, 2303],
        "post-join counts pin to 2870 2267 2560 2303",
    );
    ok(
        moved ≥ 1500 && moved ≤ 3500,
        "the movement stays inside the pinned band",
    );
```

listing 453: Rust, the check region: 2303 movers, all captured by n4, post counts exact, the band stated beside (source: patterns-concurrency-distributed/samples-rust/src/Ch12/ring.rs, lines 128-154)

```go
func TestRingMinimalMovement(t *testing.T) {
  const keys = 10000
  r := NewRing(100)
  r.Add("n1")
  r.Add("n2")
  r.Add("n3")

  before := map[string]string{}
  for i := 0; i < keys; i++ {
    k := keyName(i)
    owner, _ := r.Owner(k)
    before[k] = owner
  }

  r.Add("n4") // one node of now four joins

  moved := 0
  n4gained := 0
  for i := 0; i < keys; i++ {
    k := keyName(i)
    now, _ := r.Owner(k)
    if now != before[k] {
      moved++
      if now == "n4" {
        n4gained++
      }
    }
  }
  // expected movement is keys/4 = 2500; allow a generous band for
  // hash structure noise, the point is "fraction", not "all"
  if moved < 1500 || moved > 3500 {
    t.Fatalf("moved %d/%d keys for a 1-of-4 join, want ~2500", moved, keys)
  }
  if moved != n4gained {
    t.Fatalf("moved=%d but new node gained=%d: non-joining keys must never move", moved, n4gained)
  }
}
```

listing 454: Go, a one-of-four join moves a quarter of keys, and only to the new node (source: patterns-concurrency-distributed/samples/ch12/hashing_test.go, lines 26-62)

```
    // movement: 10000 keys, 100 vnodes, three nodes then a fourth joins.
    // only the keys whose arc the new node takes may move, and it takes
    // exactly the moved set: moved == gained
    HashRing big = new HashRing(100);
    big.add("n1");
    big.add("n2");
    big.add("n3");
    int[] before = new int[10000];
    for (int i = 0; i < 10000; i++) {
      before[i] = big.owner(keyName(i));
    }
    big.add("n4");
    int moved = 0;
    int gained = 0;
    long[] post = new long[4];
    for (int i = 0; i < 10000; i++) {
      int now = big.owner(keyName(i));
      post[now]++;
      if (now != before[i]) {
        moved++;
        if (now == 3) {
          gained++;
        }
      }
    }
    ok(moved == 2303, "10000 keys, 3->4 nodes: exactly 2303 move");
    ok(moved == gained, "only keys the new node gained moved");
    ok(post[0] == 2870 && post[1] == 2267 && post[2] == 2560 && post[3] == 2303,
        "post-join counts pin 2870, 2267, 2560, 2303");
    ok(moved >= 1500 && moved <= 3500, "inside the movement band");
```

listing 455: Java, the check region: 2303 movers all gained, post counts 2870, 2267, 2560, 2303, the band beside (source: patterns-concurrency-distributed/samples-java/src/Ch12/Ring.java, lines 128-157)

```
    public void OneNodeJoinMovesOnlyItsOwnShare()
    {
        const int keys = 10000;
        var r = new Ring(100);
        r.Add("n1");
        r.Add("n2");
        r.Add("n3");

        var before = new string[keys];
        for (int i = 0; i < keys; i++)
        {
            before[i] = r.Owner(KeyName(i)).owner;
        }

        r.Add("n4"); // one node of now four joins

        int moved = 0, n4Gained = 0;
        var post = new Dictionary<string, int>
        {
            ["n1"] = 0,
            ["n2"] = 0,
            ["n3"] = 0,
            ["n4"] = 0,
        };
        for (int i = 0; i < keys; i++)
        {
            string now = r.Owner(KeyName(i)).owner;
            post[now]++;
            if (now != before[i])
            {
                moved++;
                if (now == "n4")
                {
                    n4Gained++;
                }
            }
        }
        // witnessed actual 2303; the Go lane's band is 1500..3500 for
        // the expected keys/4 = 2500 fraction
        Assert.True(moved is ≥ 1500 and ≤ 3500,
            $"moved {moved}/{keys} for a 1-of-4 join, want ~2500");
        Assert.Equal(2303, moved);
        Assert.Equal(2303, n4Gained);
        Assert.Equal(2870, post["n1"]);
        Assert.Equal(2267, post["n2"]);
        Assert.Equal(2560, post["n3"]);
        Assert.Equal(2303, post["n4"]);
    }
```

listing 456: C#, xunit pins the witnessed actuals inside the band the go lane asserts (source: patterns-concurrency-distributed/samples-cs/tests/Ch12/RingTests.cs, lines 26-73)

```
  it("10000 keys, 100 vnodes, 3->4 nodes: moved 2303, all to n4", () => {
    const r = new Ring(100);
    r.add("n1");
    r.add("n2");
    r.add("n3");
    const before = Array.from({ length: 10000 }, (_, i) => r.owner(key(i)));
    r.add("n4");

    let moved = 0;
    let gained = 0;
    const post = { n1: 0, n2: 0, n3: 0, n4: 0 };
    for (let i = 0; i < 10000; i++) {
      const now = r.owner(key(i));
      post[now]++;
      if (now !== before[i]) {
        moved++;
        if (now === "n4") gained++;
      }
    }
    assert.equal(moved, 2303, "witnessed movement, band is 1500..3500");
    assert.ok(moved >= 1500 && moved <= 3500, "band 1500..3500");
    assert.equal(gained, moved, "non-joining keys must never move");
    assert.deepEqual(post, { n1: 2870, n2: 2267, n3: 2560, n4: 2303 });
  });
```

listing 457: JavaScript, node:test pins moved 2303, gained equal, post counts exact (source: patterns-concurrency-distributed/samples-js/test/ch12.test.mjs, lines 53-76)

```
    big = Ring(100)
    for node in ("n1", "n2", "n3"):
        big.add(node)
    before = {key_name(i): big.owner(key_name(i)) for i in range(10000)}
    big.add("n4")
    moved = 0
    gained = 0
    counts = {"n1": 0, "n2": 0, "n3": 0, "n4": 0}
    for i in range(10000):
        now = big.owner(key_name(i))
        counts[now] += 1
        if now != before[key_name(i)]:
            moved += 1
            if now == "n4":
                gained += 1
    check(moved == 2303, "10000 keys, 3->4 nodes: exactly 2303 move")
    check(moved == gained, "only keys the new node gained moved")
    check(1500 <= moved <= 3500, "inside the movement band")
    check(
        counts == {"n1": 2870, "n2": 2267, "n3": 2560, "n4": 2303}, "post-join counts"
    )
```

listing 458: Python, the script region: 2303 movers, post counts, the band checked beside (source: patterns-concurrency-distributed/samples-py/src/Ch12/ring.py, lines 89-109)

```
  {name = "movement: 10000 keys, 100 vnodes, a 1-of-4 join moves 2303", fn = function()
    local r = Ring.new(100)
    r:Add("n1"); r:Add("n2"); r:Add("n3")
    local before = {}
    for i = 0, 9999 do
      before[i] = r:Owner(keyName(i))
    end
    r:Add("n4")
    local moved, gained, post = 0, 0, {}
    for i = 0, 9999 do
      local now = r:Owner(keyName(i))
      post[now] = (post[now] or 0) + 1
      if now ~= before[i] then
        moved = moved + 1
        if now == "n4" then gained = gained + 1 end
      end
    end
    T.eq(moved, 2303)
    T.eq(gained, 2303, "non-joining keys must never move")
    T.ok(moved ≥ 1500 and moved ≤ 3500, "inside the shared band")
    T.eq({ post.n1, post.n2, post.n3, post.n4 }, { 2870, 2267, 2560, 2303 })
  end},
```

listing 459: Lua, the movement row: 2303 movers, 2303 gained, counts exact (source: patterns-concurrency-distributed/samples-lua/ch12_ring.lua, lines 123-144)

Adding a fourth node to three moves 10000 keys by roughly 2500, all of them landing on the new node, no key ever shuffling between survivors, and the seven exact lanes say it is 2303 for this generator. Removal symmetrically relocates only the departed node's keys, 705 of them here. That is the whole sales pitch of the ring, and the same test walk checks balance, four nodes within a band around 1000 of 4000 keys each, 1174/1024/879/923 exact, because a scheme that moves nothing but concentrates everything is not a solution.

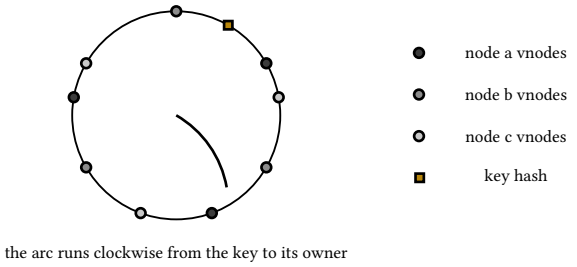


figure 68: the ring, vnode points, and the clockwise owner rule

12.3. the bug the probe caught

The first implementation hashed vnodes with raw FNV-1a and the balance test failed with one node owning 60 percent of 4000 keys. A one-off probe printed the vnode hashes and found the problem in one line: every node's points shared their high bits, the entire vnode population occupied a narrow arc of the ring, and almost every key wrapped around to the first point after that arc. FNV-1a diffuses downstream bytes beautifully into low bits, its high bits barely avalanche, and ring placement lives in the high bits. The bit-level anatomy of hashing, avalanche included, is the dsa hashing chapter's subject, book 10, ch 6 (practical data structures and algorithms, hashing). The fix is a finalizer:

The dry run: the raw hash universe, 13 inputs through fnv64a plus the mix64 finalizer.

- the seven new trees pin the exact decimals: alpha 8596495612706370024, beta 17294041484173018516, n1 17934566338607214163, n2 13970388862098144604, n3 4545224107033280897, n4 12077200234615595018, user:7 977887655448102184, and the routing keys A through F

- the go lane's frozen tests pin no raw hash at all, they pin the behavior the hashes produce, so the raw universe is the seven trees' contract
- javascript keeps src on limb pairs and converts to BigInt only in the test to state the constants exactly, lua parses every pin from a decimal string because a decimal literal past 2^63 lexes as a float, java carries the pins above 2^63 through `Long.parseUnsignedLong` on the identical decimal string because no positive long literal reaches past `Long.MAX_VALUE`, and rust's `u64` literals hold the full unsigned range straight to 2^64 minus 1, no parsing detour anywhere

```c
static uint64_t mix64(uint64_t x) {
  x ^= x >> 30;
  x *= 0xbf58476d1ce4e5b9;
  x ^= x >> 27;
  x *= 0x94d049bb133111eb;
  x ^= x >> 31;
  return x;
}

static uint64_t hash_string(const char *s) {
  const uint64_t prime = 0x100000001b3ULL;
  uint64_t h = 0xcbf29ce484222325ULL;
  for (const unsigned char *p = (const unsigned char *)s; *p; p++) {
    h ^= (uint64_t)*p;
    h *= prime;
  }
  return mix64(h);
}
```

listing 460: C, mix64 after fnv, the outputs are the pinned decimals (source: patterns-concurrency-distributed/samples-c/src/Ch12/hash.c, lines 20-37)

```rust
fn mix64(mut x: u64) -> u64 {
    x ^= x >> 30;
    x = x.wrapping_mul(0xbf58_476d_1ce4_e5b9);
    x ^= x >> 27;
    x = x.wrapping_mul(0x94d0_49bb_1331_11eb);
    x ^= x >> 31;
    x
}

fn hash_string(s: &str) -> u64 {
    const PRIME: u64 = 0x100_0000_01b3;
    let mut h: u64 = 0xcbf2_9ce4_8422_2325;
    for &byte in s.as_bytes() {
        h ^= byte as u64;
        h = h.wrapping_mul(PRIME);
    }
    mix64(h)
}
```

listing 461: Rust, mix64 with every multiply named `wrapping_mul`, the pinned decimals written as plain u64 literals (source: patterns-concurrency-distributed/samples-rust/src/Ch12/hash.rs, lines 15-32)

```
// placement hash goes through a splitmix-style finalizer. Probing
// without it, all four nodes' vnode points landed inside one narrow
// arc and one node owned 60 percent of the keys.
func fnv64a(data []byte) uint64 {
  const prime = 0x100000001b3
  h := uint64(0xcbf29ce484222325)
  for _, b := range data {
    h ^= uint64(b)
    h *= prime
  }
  return mix64(h)
}

func mix64(x uint64) uint64 {
  x ^= x >> 30
  x *= 0xbf58476d1ce4e5b9
  x ^= x >> 27
  x *= 0x94d049bb133111eb
  x ^= x >> 31
  return x
}
```

listing 462: Go, fnv then splitmix-style mix: placement hashes need avalanche in every bit (source: patterns-concurrency-distributed/samples/ch12/hashing.go, lines 11-31)

```
  static final long FNV_OFFSET = 0xcbf29ce484222325L;
  static final long FNV_PRIME = 0x100000001b3L;

  static long mix64(long x) {
    x ^= x >>> 30;
    x *= 0xbf58476d1ce4e5b9L;
    x ^= x >>> 27;
    x *= 0x94d049bb133111ebL;
    x ^= x >>> 31;
    return x;
  }

  static long fnvPlain(String s) {
    long h = FNV_OFFSET;
    for (int i = 0; i < s.length(); i++) {
      h ^= s.charAt(i);
      h *= FNV_PRIME;
    }
    return h;
  }

  static long hashString(String s) {
    return mix64(fnvPlain(s));
  }
```

listing 463: Java, long arithmetic wraps like ulong, mix64 verbatim, the pins above 2^63 carried by parseUnsignedLong (source: patterns-concurrency-distributed/samples-java/src/Ch12/Hash.java, lines 18-41)

```
public static class Hash
{
    public static ulong String(string s)
    {
        const ulong prime = 0x100000001b3;
        ulong h = 0xcbf29ce484222325;
        byte[] data = Encoding.UTF8.GetBytes(s);
        foreach (byte b in data)
        {
            h ^= b;
            h *= prime;
        }
        return Mix64(h);
    }

    public static ulong Mix64(ulong x)
    {
        x ^= x >> 30;
        x *= 0xbf58476d1ce4e5b9;
        x ^= x >> 27;
        x *= 0x94d049bb133111eb;
        x ^= x >> 31;
        return x;
    }
}
```

listing 464: C#, ulong arithmetic, the finalizer verbatim (source: patterns-concurrency-distributed/samples-cs/src/Ch12/Hash.cs, lines 11-35)

```
// fnv-64a then the splitmix finalizer. without the finalizer the
// high bits barely avalanche, ring placement lives in the high
// bits, and one node ends up owning most of the keys.
export function hashString(s) {
  let h = FNV_OFFSET;
  for (let i = 0; i < s.length; i++) {
    h = mul64(xor64(h, { hi: 0, lo: s.charCodeAt(i) }), FNV_PRIME);
  }
  h = xor64(h, shr64(h, 30));
  h = mul64(h, MIX1);
  h = xor64(h, shr64(h, 27));
  h = mul64(h, MIX2);
  h = xor64(h, shr64(h, 31));
  return h;
}
```

listing 465: JavaScript, the finalizer over limb pairs, xor-shift and multiply in 16-bit pieces (source: patterns-concurrency-distributed/samples-js/src/ch12-hash.mjs, lines 82-96)

```
def fnv64a_plain(data):
    h = FNV_OFFSET
    for byte in data:
        h ^= byte
        h = (h * FNV_PRIME) & MASK64
    return h


def mix64(x):
    x ^= x >> 30
    x = (x * 0xBF58476D1CE4E5B9) & MASK64
    x ^= x >> 27
    x = (x * 0x94D049BB133111EB) & MASK64
    x ^= x >> 31
    return x


def fnv64a(data):
    return mix64(fnv64a_plain(data))


def hash_string(text):
    return fnv64a(text.encode("utf-8"))
```

listing 466: Python, the raw fnv and the finalizer side by side, masked to 64 bits (source: patterns-concurrency-distributed/samples-py/src/Ch12/hash.py, lines 21-43)

```
local FNV_BASIS = 0xcbf29ce484222325 -- 14695981039346656037, wrapped
local FNV_PRIME = 0x100000001b3

local function fnv64a(s)
  local h = FNV_BASIS
  for i = 1, #s do
    h = h ~ s:byte(i)
    h = h * FNV_PRIME -- integer multiply wraps mod 2^64
  end
  return h
end

local function mix64(x)
  x = (x ~ (x >> 30)) * 0xbf58476d1ce4e5b9
  x = (x ~ (x >> 27)) * 0x94d049bb133111eb
  return x ~ (x >> 31)
end

local function hashString(s)
  return mix64(fnv64a(s))
end
```

listing 467: Lua, wrapping integer multiplies, the finalizer in three lines (source: patterns-concurrency-distributed/samples-lua/ch12_hash.lua, lines 8-28)

The finalizer is where the carriers separate most. C, c#, go, and lua multiply unsigned 64-bit values that wrap by construction, c and lua because the types wrap, c# because `ulong` overflow wraps silently, go because `uint64` is defined to, rust by naming the wrap, `wrapping_mul` on every multiply, because a plain overflow is a debug-build panic. Java's signed `long` overflow wraps silently the same way, so the recurrence is one line, and the lane meters avalanche as an exact unsigned interval through `Long.compareUnsigned`. Python masks by hand after every multiply,

javascript runs every multiply through 16-bit limbs of 32-bit halves, and the lua lane even meters the avalanche directly, one changed input digit moving 9 output bits through raw fnv and 33 through the finalizer. Same integers, same decimals, very different proofs.

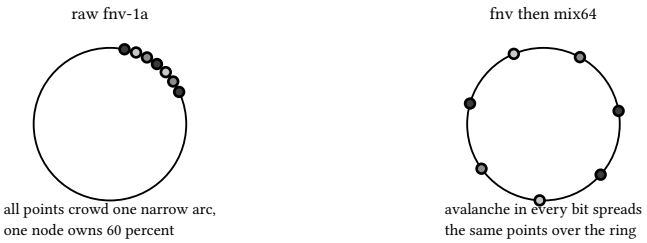


figure 69: raw fnv vnodes crowd one arc, mix64 spreads them over the ring

This is a general lesson wearing a specific costume: check the output distribution of a hash at the bit granularity the consumer uses. A hash that looks fine for checksums, where any bit change detects corruption, can be catastrophically biased for placement, where uniformity per bit region is the product. The same `mix64` now guards the rendezvous scheme's node ranking.

12.4. rendezvous, highest random weight

Rendezvous hashing reaches the same minimal-movement property with no ring and no vnodes: rank every node by `hash(node, key)` and take the maximum:

The dry run: exact winners and exact movement, seven trees against the go lane's bands.

- user:0..5 rank n2,n3,n2,n2,n2,n3 over {n1,n2,n3} and flip to n2,n4,n4,n4,n2,n4 once n4 joins, pinned by the seven new trees
- the join moves exactly 1010 of 4000 keys, every mover captured by n4, exact in the seven new trees
- all eight lanes assert moved equal to gained inside the band 500..1500

```c
// the score of one node for one key: hash(node NUL key)
static uint64_t score(const char *node, const char *key) {
  char buf[40];
  size_t nlen = strlen(node), klen = strlen(key);
  memcpy(buf, node, nlen);
  buf[nlen] = '\0';
  memcpy(buf + nlen + 1, key, klen + 1);
  return hash_bytes(buf, nlen + 1 + klen);
}

// the winner: highest score wins, earlier node in sorted order keeps a tie
static int rd_owner(rendezvous *rd, const char *key) {
  if (rd->n == 0)
    return -1;
  int best = -1;
  uint64_t best_hash = 0;
  for (int i = 0; i < rd->n; i++) {
    uint64_t h = score(rd->nodes[i], key);
    if (h > best_hash) {
      best_hash = h;
      best = i;
    }
  }
  return best;
}
```

listing 468: C, the score is hash over node, a real NUL byte, then key, matching go's join byte for byte (source: patterns-concurrency-distributed/samples-c/src/Ch12/rendezvous.c, lines 58-82)

```rust
    fn score(node: &str, key: &str) -> u64 {
        let mut bytes = Vec::with_capacity(node.len() + 1 + key.len());
        bytes.extend_from_slice(node.as_bytes());
        bytes.push(0);
        bytes.extend_from_slice(key.as_bytes());
        hash_bytes(&bytes)
    }

    fn owner(&self, key: &str) -> Option<usize> {
        let mut best: Option<(usize, u64)> = None;
        for (i, &node) in self.nodes.iter().enumerate() {
            let h = Rendezvous::score(node, key);
            if best.is_none_or(|(_, best_h)| h > best_h) {
                best = Some((i, h));
            }
        }
        best.map(|(i, _)| i)
    }
```

listing 469: Rust, the score pushes a real zero byte onto a Vec, the max tracked as an Option, u64 compares unsigned by construction (source: patterns-concurrency-distributed/samples-rust/src/Ch12/rendezvous.rs, lines 52-69)

```
type Rendezvous struct {
  nodes []string
}

func (rd *Rendezvous) Nodes() []string { return rd.nodes }

func (rd *Rendezvous) Add(node string) {
  rd.nodes = append(rd.nodes, node)
  slices.Sort(rd.nodes)
}

func (rd *Rendezvous) Owner(key string) (string, bool) {
  best := ""
  var bestHash uint64
  for _, n := range rd.nodes {
    h := hashString(n + "\x00" + key)
    if h > bestHash {
      bestHash, best = h, n
    }
  }
  if best == "" {
    return "", false
  }
  return best, true
}

// Jump is Lamping and Marta's jump consistent hash: bucket for a
// key as a function of bucket count, moving only keys that must
```

listing 470: Go, one hash per node per key, winner owns it (source: patterns-concurrency-distributed/samples/ch12/hashing.go, lines 123-150)

```
  // the score of one node for one key: hash(node NUL key)
  static long score(String node, String key) {
    long h = FNV_OFFSET;
    for (int i = 0; i < node.length(); i++) {
      h ^= node.charAt(i);
      h *= FNV_PRIME;
    }
    h ^= 0; // the NUL separator
    h *= FNV_PRIME;
    for (int i = 0; i < key.length(); i++) {
      h ^= key.charAt(i);
      h *= FNV_PRIME;
    }
    return mix64(h);
  }

  static final class Cluster {
    private final List<String> nodes = new ArrayList<>(); // kept sorted

    void add(String node) {
      nodes.add(node);
      nodes.sort(null);
    }

    // the winner: highest score wins, earlier node in sorted order keeps
    // a tie. the scores are uint64, so the comparison is unsigned, the
    // same ordering the c lane's uint64 compare and py's unbounded ints
    // give. null when the cluster is empty
    String owner(String key) {
      if (nodes.isEmpty()) {
        return null;
      }
      String best = null;
      long bestHash = 0;
      for (String node : nodes) {
        long h = score(node, key);
        if (Long.compareUnsigned(h, bestHash) > 0) {
          bestHash = h;
          best = node;
        }
      }
      return best;
    }
```

listing 471: Java, the score hashes node chars, the NUL, then key chars, the max taken by unsigned comparison (source: patterns-concurrency-distributed/samples-java/src/Ch12/Rendezvous.java, lines 34-76)

```
public sealed class Rendezvous
{
    private readonly List<string> _nodes = [];

    public IReadOnlyList<string> Nodes => _nodes;

    public void Add(string node)
    {
        _nodes.Add(node);
        _nodes.Sort(StringComparer.Ordinal);
    }

    public (string owner, bool ok) Owner(string key)
    {
        string best = "";
        ulong bestHash = 0;
        foreach (var n in _nodes)
        {
            ulong h = Hash.String(n + "\x00" + key);
            if (h > bestHash)
            {
                bestHash = h;
                best = n;
            }
        }
        return best == "" ? ("", false) : (best, true);
    }
}
```

listing 472: C#, the whole scheme is a sorted list and one max scan (source: patterns-concurrency-distributed/samples-cs/src/Ch12/Rendezvous.cs, lines 7-34)

```
export class Rendezvous {
  #nodes = [];

  nodes() {
    return [...this.#nodes];
  }

  add(node) {
    this.#nodes.push(node);
    this.#nodes.sort();
  }

  // the node with the highest weight hash, or null for no nodes
  owner(key) {
    let best = "";
    let bestHash = { hi: 0, lo: 0 };
    for (const n of this.#nodes) {
      const h = hashString(n + "\x00" + key);
      if (cmp64(h, bestHash) > 0) {
        bestHash = h;
        best = n;
      }
    }
    return best === "" ? null : best;
  }
}
```

listing 473: JavaScript, cmp64 orders the {hi, lo} weights, the max scan over sorted nodes (source: patterns-concurrency-distributed/samples-js/src/ch12-rendezvous.mjs, lines 7-32)

```
class Rendezvous:
    def __init__(self):
        self.nodes = []

    def add(self, node):
        self.nodes.append(node)
        self.nodes.sort()

    def owner(self, key):
        if not self.nodes:
            return None
        best = None
        best_hash = 0
        for node in self.nodes:
            h = fnv64a(f"{node}\x00{key}".encode())
            if h > best_hash:
                best_hash = h
                best = node
        return best
```

listing 474: Python, the max over hash(node, key) scores (source: patterns-concurrency-distributed/samples-py/src/Ch12/rendezvous.py, lines 37-55)

```lua
local Rendezvous = {}
Rendezvous.__index = Rendezvous

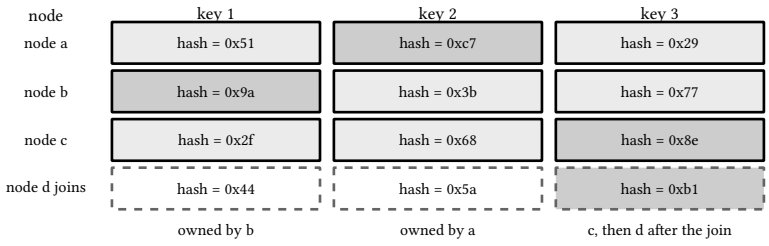
function Rendezvous.new()
  return setmetatable({ nodes = {} }, Rendezvous)
end

function Rendezvous:Add(node)
  self.nodes[#self.nodes + 1] = node
  table.sort(self.nodes)
end

-- Owner ranks every node and takes the highest hash, ties impossible
-- in practice and resolved by sorted order on equal hashes
function Rendezvous:Owner(key)
  local best, bestH
  for _, n in ipairs(self.nodes) do
    local h = hashString(n .. "\0" .. key)
    if best == nil or ukey(h) > ukey(bestH) then
      best, bestH = n, h
    end
  end
  if best == nil then
    return nil, false
  end
  return best, true
end
```

listing 475: Lua, ukey flips the sign bit so the max is unsigned (source: patterns-concurrency-distributed/samples-lua/ch12_rendezvous.lua, lines 34-60)

The join every lane hashes is `node`, one NUL byte, `key`, byte identical in all eight, go and javascript through the `\x00` escape, c by building the buffer with a real zero byte, rust by pushing a real zero byte onto a Vec, c# and python through their own spellings, lua through `\0`, java by an xor against a literal 0 between the two strings. Unsigned comparison is the only trap left, and two lanes pay it openly: lua xors the sign bit into the comparison key because its integers are signed, and java routes every hash ordering through `Long.compareUnsigned`, because >, ≥, and `Long.compare` are signed and would sort the values above 2^63 as negatives. When a node joins it captures exactly the keys where it outranks the incumbent winner, which is again `1/(n+1)` of the space. The trade is lookup cost, o(n) hashes against o(log v) binary searches, against simplicity, no state, no rebuild on membership change, no vnode tuning. Client-side placement, where the cluster list is small and public, favors rendezvous, server-side placement at scale favors the ring.



the winner is the maximum, a joining node captures exactly the keys it outranks the incumbent on

figure 70: highest random weight: per key, every node is scored, the max owns it

12.5. jump consistent hash

Lamping and Veach's jump hash answers "which bucket owns key k when there are n buckets" with a loop and no memory at all, and its stability property is exact: growing from n to n+1 buckets, every key either stays or moves to bucket n, never between old buckets:

The dry run: the table and the two properties.

- rows across bucket counts 1..5 are pinned byte-exact by the seven new trees: k=2 [0,0,0,3,3], k=3 [0,0,2,3,3], k=4 [0,1,1,1,1], k=5 [0,1,1,1,4], k=6 [0,1,2,2,2], k=8 [0,0,0,0,4], k=9 [0,0,2,2,2], and the c, rust, java, python, and lua lanes pin the full table, all-zero rows k=0, k=1, and k=7 included
- growing 3 to 4 buckets over k=1..500 never moves a key anywhere but the new bucket, all eight lanes
- 10000 keys over 5 buckets land 2001/1996/2002/2005/1996, exact in the seven new trees, band 1300..2700 in all eight

```c
static int jump(uint64_t key, int buckets) {
  int b = -1;
  int64_t j = 0;
  while (j < buckets) {
    b = (int)j;
    key = key * 2862933555777941757ULL + 1;
    j = (int64_t)((double)(b + 1) * (2147483648.0 / (double)((key >> 33) + 1)));
  }
  return b;
}
```

listing 476: C, the ladder with j held in int64_t, a windows long is 32 bits and the rung passes 2^31 (source: patterns-concurrency-distributed/samples-c/src/Ch12/jump.c, lines 21-30)

```rust
fn jump(mut key: u64, buckets: i64) -> i64 {
    let mut b: i64 = -1;
    let mut j: i64 = 0;
    while j < buckets {
        b = j;
        key = key.wrapping_mul(2862933555777941757).wrapping_add(1);
        j = ((b + 1) as f64 * (2147483648.0 / (((key >> 33) + 1) as f64))) as i64;
    }
    b
}
```

listing 477: Rust, the rung held in i64, the key multiply named `wrapping_mul`, the division on f64 ieee doubles (source: patterns-concurrency-distributed/samples-rust/src/Ch12/jump.rs, lines 15-24)

```go
func Jump(key uint64, buckets int) int {
  b, j := -1, 0
  for j < buckets {
    b = j
    key = key*2862933555777941757 + 1
    j = int(float64(b+1) * (float64(int64(1)<<31) / float64((key>>33)+1)))
  }
  return b
```

listing 478: Go, the o(log n) ladder, moving only what must move (source: patterns-concurrency-distributed/samples/ch12/hashing.go, lines 153-160)

```
  static int jump(long key, int buckets) {
    int b = -1;
    long j = 0;
    while (j < buckets) {
      b = (int) j;
      key = key * 2862933555777941757L + 1;
      j = (long) ((double) (b + 1) * (2147483648.0 / (double) ((key >>> 33) + 1)));
    }
    return b;
  }
```

listing 479: Java, the ladder verbatim over a long key and ieee doubles, the unsigned shift for the high bits, the rung past 2^31 held in a long (source: patterns-concurrency-distributed/samples-java/src/Ch12/Jump.java, lines 20-29)

```
public static class Jump
{
    public static int Bucket(ulong key, int buckets)
    {
        int b = -1, j = 0;
        while (j < buckets)
        {
            b = j;
            key = unchecked(key * 2862933555777941757UL + 1);
            j = (int)((b + 1) * ((double)(1L << 31) / (double)((key >> 33) + 1)));
        }
        return b;
    }
}
```

listing 480: C#, the float64 ladder over unchecked ulong arithmetic (source: patterns-concurrency-distributed/samples-cs/src/Ch12/Jump.cs, lines 8-21)

```
const JUMP_C = { hi: 0x27bb2ee6, lo: 0x87b0b0fd }; // 2862933555777941757
const ONE = { hi: 0, lo: 1 };

export function jump(key, buckets) {
  let k = { hi: Math.floor(key / 4294967296), lo: key % 4294967296 };
  let b = -1;
  let j = 0;
  while (j < buckets) {
    b = j;
    k = add64(mul64(k, JUMP_C), ONE);
    // key >> 33: the high half shifted right once, a 31-bit value
    const shifted = k.hi >>> 1;
    j = Math.trunc((b + 1) * (2147483648 / (shifted + 1)));
  }
  return b;
}
```

listing 481: JavaScript, the ladder over limb pairs, the 2^31 division on exact doubles (source: patterns-concurrency-distributed/samples-js/src/ch12-jump.mjs, lines 8-23)

```python
def jump(key, buckets):
    b = -1
    j = 0
    while j < buckets:
        b = j
        key = (key * 2862933555777941757 + 1) & MASK64
        j = int(float(b + 1) * (2147483648.0 / float((key >> 33) + 1)))
    return b
```

listing 482: Python, python floats are ieee doubles, exactly what the ladder assumes (source: patterns-concurrency-distributed/samples-py/src/Ch12/jump.py, lines 19-26)

```lua
local function Jump(key, buckets)
  local b, j = -1, 0
  while j < buckets do
    b = j
    key = key * 2862933555777941757 + 1 -- wraps mod 2^64
    j = math.tointeger(math.floor((b + 1) * (2147483648.0 / ((key >> 33) + 1))))
  end
  return b
end
```

listing 483: Lua, the 2147483648.0 division forces the float, tointeger rounds the rung (source: patterns-concurrency-distributed/samples-lua/ch12_jump.lua, lines 11-19)

The ladder's arithmetic is a portability contract of its own: the division must be IEEE double in every lane, c's `double`, rust's `f64`, c#'s `double`, java's ieee `double` with the rung held in a `long` for the same reason c holds `int64_t`, python's float, javascript's Number, lua forcing the fraction through a 2147483648.0 literal so the rung stays float, and go the reference. The c lane's `j` is `int64_t` on purpose, a windows `long` is 32 bits and the rung routinely passes 2^31, the trap its header comment records for the corpus. The restrictions are as real as the elegance: jump hash needs bucket counts to only grow (one at a time, from 1), cannot remove a bucket without renumbering everything, and says nothing about node weighting, which makes it the tool for shard counts inside one deployment, not for membership across machines.

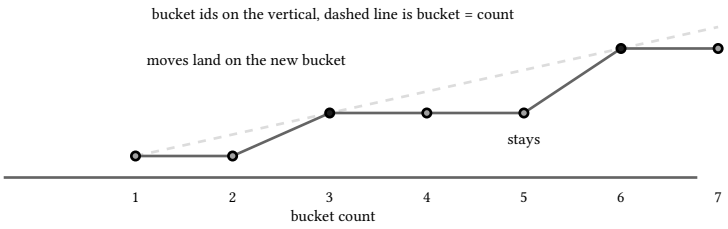


figure 71: one key as buckets grow: stay, or jump to the new bucket, never sideways

NOTE: choosing between them

Fixed shard count, growth by appending shards: jump hash. Dynamic membership, weighted nodes, server side: the vnode ring. Small cluster list known to clients, no server state wanted: rendezvous. All three are placement decisions, none of them replication, which stays quorum work from chapter 10.

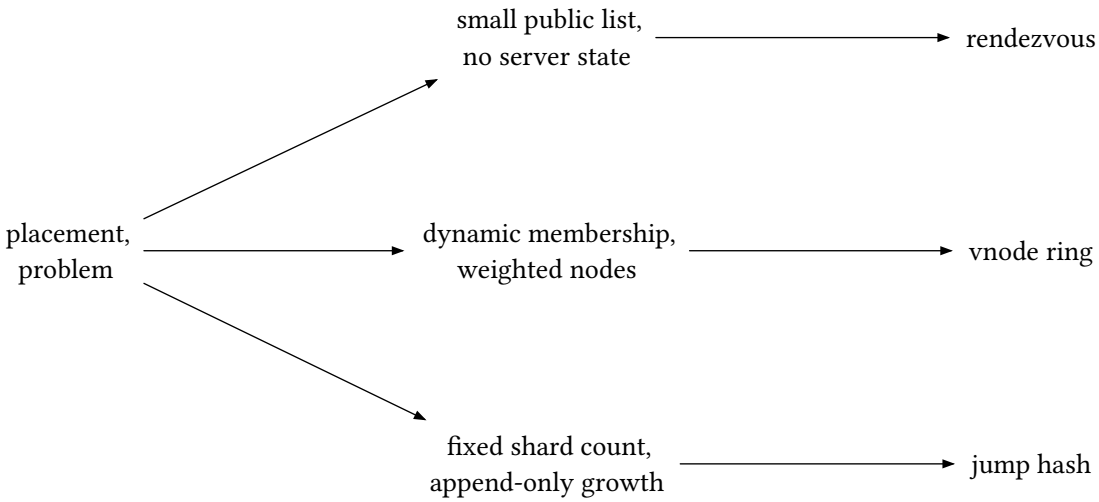


figure 72: choosing a placement scheme, replication is none of them

12.6. across the eight languages

The build sizes count non-comment source lines, the go column the chapter's one frozen file:

language	build SLOC	dependency	boundary note
c	454	libc	strcmp tiebreaks, int64_t rung past 2^31, the join built with a real NUL
rust	471	stdlib	wrapping named on every multiply, u64 literals to 2^64 minus 1, a NUL pushed onto a Vec, the rung in i64
go	126	stdlib	frozen lane pinning movement bands where the siblings pin exact decimals
java	460	jdk 27 stdlib	every hash ordering through Long.compareUnsigned, parseUnsignedLong pins, ieee double ladder
c#	132	bcl	CompareOrdinal tiebreaks, the jump ladder over unchecked ulong
javascript	147	node stdlib, one sibling module	16-bit limb multiplies, the lower bound by hand, BigInt only in tests
python	302	stdlib only	hand-masked multiplies, (hash, node) tuples with bisect_left, float ladder
lua	397	lib.lua harness	sign bit xored into sort keys, decimal-string pins, 2147483648.0 ladder

sources: Karger et al., "Consistent Hashing and Random Trees", STOC 1997, for the ring and its 1/(n+1) bound, Thaler and Ravishankar, "Using Name-Based Mappings to Increase Hit Rates", CSE-TR-356-96, University of Michigan 1996, journal form IEEE/ACM Transactions on Networking 1998, for the HRW formulation, Thatcher, "Rendezvous Hashing Explained", for the walkthrough, Lamping and Veach, "A Fast, Minimal Memory, Consistent Hash Algorithm", arXiv 1406.2294, for jump hash and its growth rule, accessed 2026-09-08. Verified by the eight chapter legs: 4 C programs with 4745 embedded checks, the rust runner's 4745 checks across 4 programs in `samples-rust/src/Ch12`, `go test` at 6 tests in `patternsbook/ch12`, the java runner's 63 checks across 4 programs in `samples-java/src/Ch12`, 13 xunit facts, node's 10 cases across 4 suites in `test/ch12.test.mjs`, the python runner's 55 checks across 4 modules, and the lua runner's 17 ch12 rows.

13. messaging semantics and idempotency

A message system answers three questions badly or well: does the message arrive at most once, at least once, or exactly once, in what order, and what happens when it arrives twice. The first has only one honest answer in practice, at least once, because acknowledgments can be lost after delivery just as deliveries can be lost after sending. Everything else in this chapter is engineering on top of that fact, and the eight trees all learn it the same way: the crash is a parameter, the duplicate is a fixture, and no lane waits on a scheduler to observe either.

13.1. the three semantics

At-most-once is fire and forget, fast and lossy, right for metrics and telemetry. At-least-once is send and retry until acknowledged, lossless but duplicating, because the sender cannot distinguish “lost before processing” from “processed, then the ack drowned”. Exactly-once delivery does not exist across a real network, and what vendors mean by the phrase is exactly-once effect: at-least-once delivery into an idempotent consumer. Every tree builds exactly that, honestly labeled.

The dry run: the dedup contract and which lanes pin each row.

- the duplicator unit: every 2 on id 2 delivers `[2, 2]`, applied exactly once, the echo suppressed
- a full replay of a delivered stream applies nothing new, effects keep first-delivery order, a row the C, java, and python lanes pin, while C# and lua pin the single envelope and partial replays, and java and rust redrive the whole stream twice over, 24 consume calls for 6 effects
- the frozen go test pins `[2, 2]` itself, the seven sibling trees pin the same literal against the shared contract

```c
typedef struct {
  bool applied[MAX_ID + 1];
  char effects[MAX_EFFECTS][24];
  int n_effects;
  int applied_count;
  int duplicates_suppressed;
} consumer;

static void consumer_init(consumer *c) { memset(c, 0, sizeof *c); }

static void consumer_deliver(consumer *c, const envelope *e) {
  if (c->applied[e->id]) {
    c->duplicates_suppressed++;
    return; // duplicate: observed, suppressed
  }
  c->applied[e->id] = true;
  snprintf(c->effects[c->n_effects], 24, "%s:%s", e->key, e->body);
  c->n_effects++;
  c->applied_count++;
}

// deliver twice when id % every == 0
typedef struct {
  int every;
  int log[2 * MAX_EFFECTS];
  int n_log;
} duplicator;

static void duplicator_deliver(duplicator *d, consumer *c, const envelope *e) {
  consumer_deliver(c, e);
  d->log[d->n_log++] = e->id;
  if (d->every > 0 && e->id % d->every == 0) {
    consumer_deliver(c, e); // the duplicate
    d->log[d->n_log++] = e->id;
  }
}
```

listing 484: C, the applied memory is a bitmap over dense ids, deliver is a function the relay calls back (source: patterns-concurrency-distributed/samples-c/src/Ch13/consumer.c, lines 29-64)

```
struct Consumer {
    applied: HashSet<u32>,
    effects: Vec<String>,
    duplicates_suppressed: u32,
}

impl Consumer {
    fn new() -> Consumer {
        Consumer {
            applied: HashSet::new(),
            effects: Vec::new(),
            duplicates_suppressed: 0,
        }
    }

    fn deliver(&mut self, e: &Envelope) {
        if self.applied.contains(&e.id) {
            self.duplicates_suppressed += 1;
            return;
        }
        self.applied.insert(e.id);
        self.effects.push(format!("{}:{}", e.key, e.body));
    }

    fn applied_count(&self) -> usize {
        self.applied.len()
    }
}

struct Duplicator {
    every: u32,
    log: Vec<u32>,
}

impl Duplicator {
    fn new(every: u32) -> Duplicator {
        Duplicator {
            every,
            log: Vec::new(),
        }
    }

    fn deliver(&mut self, c: &mut Consumer, e: &Envelope) {
        c.deliver(e);
        self.log.push(e.id);
        if self.every > 0 && e.id % self.every == 0 {
            c.deliver(e);
            self.log.push(e.id);
        }
    }
}
```

listing 485: Rust, the applied memory a HashSet, the duplicator a struct owning the log that takes the consumer by &mut (source: patterns-concurrency-distributed/samples-rust/src/Ch13/consumer.rs, lines 23-73)

```
type Consumer struct {
  mu      sync.Mutex
  applied map[int]bool
  Effects []string
}

func NewConsumer() *Consumer { return &Consumer{applied: map[int]bool{}} }

func (c *Consumer) Deliver(e Envelope) {
  c.mu.Lock()
  defer c.mu.Unlock()
  if c.applied[e.ID] {
    return // duplicate: observed, suppressed
  }
  c.applied[e.ID] = true
  c.Effects = append(c.Effects, e.Key+":"+e.Body)
}

func (c *Consumer) Applied() int {
  c.mu.Lock()
  defer c.mu.Unlock()
  return len(c.applied)
}

// Duplicator is a lossy transport that redelivers some envelopes,
```

listing 486: Go, the consumer remembers applied ids, duplicates observed and suppressed (source: patterns-concurrency-distributed/samples/ch13/messaging.go, lines 94-118)

```
  // the consumer's memory: applied ids and first-delivery-order effects
  static final class Dedup {
    final HashSet<Integer> applied = new HashSet<>();
    final ArrayList<String> effects = new ArrayList<>();
    int suppressed;

    void deliver(Envelope e) {
      if (!applied.add(e.id())) {
        suppressed++; // duplicate: observed, suppressed
        return;
      }
      effects.add(e.key() + ":" + e.body());
    }
  }

  // the lossy transport: delivers twice when id % every == 0, logging
  // every delivery like C's duplicator log
  static Delivery duplicator(int every, Delivery sink, List<Integer> log) {
    return e -> {
      sink.accept(e);
      log.add(e.id());
      if (every > 0 && e.id() % every == 0) {
        sink.accept(e); // the duplicate
        log.add(e.id());
      }
    };
  }
```

listing 487: Java, the applied memory a HashSet of ids, the duplicator wraps any Delivery and logs every call (source: patterns-concurrency-distributed/samples-java/src/Ch13/Consumer.java, lines 28-54)

```
public sealed class Consumer
{
    private readonly object _gate = new();
    private readonly HashSet<int> _applied = [];

    public List<string> Effects { get; } = [];

    public int Calls { get; private set; }

    public int Applied
    {
        get
        {
            lock (_gate)
            {
                return _applied.Count;
            }
        }
    }

    public void Deliver(Envelope e)
    {
        Calls++;
        lock (_gate)
        {
            if (_applied.Contains(e.Id))
            {
                return; // duplicate: observed, suppressed
            }
            _applied.Add(e.Id);
            Effects.Add(e.Key + ":" + e.Body);
        }
    }
}

// Duplicator is a lossy transport that redelivers some envelopes,
// standing in for real brokers' at-least-once redelivery.
public sealed class Duplicator(int duplicateEvery)
{
    // deliver twice when Id % DuplicateEvery == 0
    public Action<Envelope> Wrap(Action<Envelope> consume) => e =>
    {
        consume(e);
        if (duplicateEvery > 0 && e.Id % duplicateEvery == 0)
        {
            consume(e); // the duplicate
        }
    };
}
```

listing 488: C#, the applied set under a gate, the duplicator wraps any action (source: patterns-concurrency-distributed/samples-cs/src/Ch13/Consumer.cs, lines 6-54)

```
// ch13: the idempotent consumer. paired with at-least-once delivery
// it yields exactly-once effects: duplicates are observed and
// suppressed, gaps are retried by the relay. the duplicator stands
// in for a real broker's redelivery.

export class Consumer {
  #applied = new Set();
  effects = [];

  deliver(e) {
    if (this.#applied.has(e.id)) {
      return; // duplicate: observed, suppressed
    }
    this.#applied.add(e.id);
    this.effects.push(`${e.key}:${e.body}`);
  }

  applied() {
    return this.#applied.size;
  }
}

export class Duplicator {
  // deliver twice when id % duplicateEvery === 0
  constructor(duplicateEvery) {
    this.duplicateEvery = duplicateEvery;
  }

  wrap(consume) {
    return async (e) => {
      await consume(e);
      if (this.duplicateEvery > 0 && e.id % this.duplicateEvery === 0) {
        await consume(e); // the duplicate
      }
    };
  }
}
```

listing 489: JavaScript, a Set holds the memory, the wrapped transport composes (source: patterns-concurrency-distributed/samples-js/src/ch13-consumer.mjs, lines 1-37)

```
class Duplicator:
    def __init__(self, every):
        self.every = every

    def wrap(self, deliver):
        def transport(env):
            deliver(env)
            if self.every > 0 and env.id % self.every == 0:
                deliver(env)

        return transport


class Consumer:
    def __init__(self):
        self.applied = set()
        self.effects = []
        self.suppressed = 0

    def deliver(self, env):
        if env.id in self.applied:
            self.suppressed += 1
            return
        self.applied.add(env.id)
        self.effects.append(f"{env.key}:{env.body}")

    def applied_count(self):
        return len(self.applied)
```

listing 490: Python, a set and two closures, the transport composes the same way (source: patterns-concurrency-distributed/samples-py/src/Ch13/consumer.py, lines 23-50)

```lua
local Consumer = {}
Consumer.__index = Consumer

function Consumer.new()
  return setmetatable({ applied = {}, effects = {} }, Consumer)
end

function Consumer:Deliver(e)
  if self.applied[e.id] then
    return false -- duplicate: observed, suppressed
  end
  self.applied[e.id] = true
  self.effects[#self.effects + 1] = e.key .. ":" .. e.body
  return true
end

function Consumer:Applied()
  local n = 0
  for _ in pairs(self.applied) do n = n + 1 end
  return n
end

-- Duplicator wraps a consume function and redelivers every envelope
-- whose id is a multiple of every
local function duplicator(every, consume)
  return function(e)
    consume(e)
    if every > 0 and e.id % every == 0 then
      consume(e)
    end
  end
end
```

listing 491: Lua, a table keyed by id, Deliver reports false on the duplicate (source: patterns-concurrency-distributed/samples-lua/ch13_consumer.lua, lines 7-38)

`Deliver` consults the applied set before doing anything, so a redelivery costs one membership test and the effect happens once. The shape of the memory is the idiom row: C indexes a `bool` array by the id itself, legal because the fixture's ids are dense and small, C# keeps a `HashSet` behind a lock and counts calls outside it, go a map behind a mutex, java and rust a `HashSet` with no lock because the walk is single-threaded, javascript a `Set`, python a `set`, and lua a plain table keyed by id, with `Deliver` returning `false` on the duplicate so the transport can observe what it carried. The duplicator is a function wrapper in most languages, a function pointer taking the sink in C and a closure over the consume callable in the dynamic trees, while rust's is a struct owning the log that takes the consumer by `&mut`, no callable indirection at all.

The idempotency key is the design decision that decides whether any of this works. It must be stable across retries, which rules out anything derived from the arrival, the wall clock, or a fresh random id, and requires the sender to mint it once and resend it verbatim. The dedup window is the second decision: these samples remember forever, a production system bounds memory with a TTL or a sequence floor below which everything is known applied.

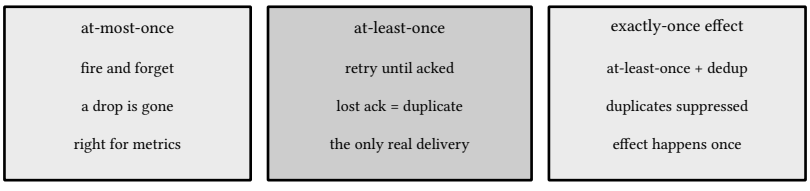


figure 73: three delivery semantics, three failure points, one real option

13.2. the outbox, and a crash in the middle

The transactional outbox pattern solves dual-write: an application that writes to its database and to a broker in two steps can crash between them, losing the message while committing the state. The outbox writes the message into the same transaction as the state, and a relay drains it afterward. The queue itself is the small half, an array or list under a lock, and the relay is where at-least-once becomes visible, so the relay is what the listings show.

The dry run: 10 envelopes, keys cycling k1, k2, k0, bodies b1 through b10, a relay configured to crash after 4 deliveries, a transport that duplicates every third id.

- crash run: 4 delivered, 6 pending, 5 consume calls with id 3 duplicated, 4 applied
- resume run: 6 more deliveries, 13 consume calls in total, ids 6 and 9 duplicated, 10 applied
- the 10 effects land in send order, `k1:b1` through `k1:b10`
- the go lane's frozen test drives the same walk with its own constants, 10 envelopes under one key and one body, and pins the counts, both verified against the same implementation

```c
typedef struct {
  envelope pending[NENV];
  int len;
  int next_id;
} outbox;

static envelope enqueue(outbox *o, const char *key, const char *body) {
  o->next_id++;
  envelope e = {.id = o->next_id, .key = key, .body = body};
  o->pending[o->len++] = e;
  return e;
}

typedef struct {
  outbox *out;
  int delivered;
  int attempts;
  int crash_after; // < 0 means run to completion
} relay;

// true on completion, false on the simulated crash
static bool relay_run(relay *r, void (*deliver)(void *, const envelope *),
                      void *sink) {
  for (;;) {
    if (r->crash_after ≥ 0 && r->delivered ≥ r->crash_after)
      return false; // crashed mid-drain, tail stays pending
    if (r->out->len == 0)
      return true;
    envelope e = r->out->pending[0];
    memmove(r->out->pending, r->out->pending + 1,
            (size_t)(r->out->len - 1) * sizeof(envelope));
    r->out->len--;
    r->attempts++;
    r->delivered++;
    deliver(sink, &e);
  }
}
```

listing 492: C, enqueue into a fixed array, the crash is a false return mid-loop (source: patterns-concurrency-distributed/samples-c/src/Ch13/outbox.c, lines 30-66)

```
    fn enqueue(&mut self, key: &'static str, body: &'static str) -> Envelope {
        self.next_id += 1;
        let e = Envelope {
            id: self.next_id,
            key,
            body,
        };
        self.pending.push_back(e);
        e
    }

    fn len(&self) -> usize {
        self.pending.len()
    }
}

struct Relay {
    delivered: u32,
    attempts: u32,
    crash_after: Option<u32>,
}

impl Relay {
    fn new(crash_after: Option<u32>) -> Relay {
        Relay {
            delivered: 0,
            attempts: 0,
            crash_after,
        }
    }

    fn run<F: FnMut(&Envelope)>(&mut self, out: &mut Outbox, deliver: &mut F) -> bool {
        loop {
            if self.crash_after.is_some_and(|c| self.delivered ≥ c) {
                return false;
            }
            let Some(e) = out.pending.pop_front() else {
                return true;
            };
            self.attempts += 1;
            self.delivered += 1;
            deliver(&e);
        }
    }
}
```

listing 493: Rust, a VecDeque popped at the front, the crash threshold an `Option` read by `is_some_and`, the sink a generic `FnMut` closure (source: patterns-concurrency-distributed/samples-rust/src/Ch13/outbox.rs, lines 36-80)

```
type Outbox struct {
  mu      sync.Mutex
  pending []Envelope
  nextID  int
}

func (o *Outbox) Enqueue(key, body string) Envelope {
  o.mu.Lock()
  defer o.mu.Unlock()
  o.nextID++
  e := Envelope{ID: o.nextID, Key: key, Body: body}
  o.pending = append(o.pending, e)
  return e
}

func (o *Outbox) Len() int {
  o.mu.Lock()
  defer o.mu.Unlock()
  return len(o.pending)
}

// Relay drains the outbox, delivering at-least-once: a crash
```

listing 494: Go, enqueue alongside the state, drain independently (source: patterns-concurrency-distributed/samples/ch13/messaging.go, lines 19-40)

```
type Relay struct {
  mu         sync.Mutex
  outbox     *Outbox
  delivered  int
  attempts   int
  crashAfter int
}

func NewRelay(out *Outbox, crashAfter int) *Relay {
  return &Relay{outbox: out, crashAfter: crashAfter}
}

var ErrCrashed = errors.New("relay crashed mid-drain")

// Run delivers until the outbox is empty or the simulated crash.
func (r *Relay) Run(deliver func(Envelope)) error {
  for {
    r.mu.Lock()
    if r.crashAfter ≥ 0 && r.delivered ≥ r.crashAfter {
      r.mu.Unlock()
      return ErrCrashed
    }
    r.mu.Unlock()

    r.outbox.mu.Lock()
    if len(r.outbox.pending) == 0 {
      r.outbox.mu.Unlock()
      return nil
    }
    e := r.outbox.pending[0]
    r.outbox.pending = r.outbox.pending[1:]
    r.outbox.mu.Unlock()

    r.mu.Lock()
    r.attempts++
    r.delivered++
    r.mu.Unlock()

    deliver(e)
  }
}

func (r *Relay) Attempts() int {
  r.mu.Lock()
  defer r.mu.Unlock()
  return r.attempts
}

// Consumer dedups by idempotency key. Paired with at-least-once
// delivery it yields exactly-once effects: duplicates are observed
```

listing 495: Go, drain, crash mid-flight, resume, redelivery is expected (source: patterns-concurrency-distributed/samples/ch13/messaging.go, lines 43-92)

```
  // crashAfter < 0 means run to completion; run returns false on the
  // simulated crash, true when the outbox drained
  static final class Relay {
    final Box box;
    final int crashAfter;
    int attempts;
    int delivered;

    Relay(Box box, int crashAfter) {
      this.box = box;
      this.crashAfter = crashAfter;
    }

    boolean run(Delivery deliver) {
      while (true) {
        if (crashAfter ≥ 0 && delivered ≥ crashAfter)
          return false; // crashed mid-drain, the tail stays pending
        Envelope e = box.tryDequeue();
        if (e == null) return true;
        attempts++;
        delivered++;
        deliver.accept(e);
      }
    }
  }
```

listing 496: Java, the relay over an ArrayDeque, the crash a false return mid-loop, a clean drain true (source: patterns-concurrency-distributed/samples-java/src/Ch13/Outbox.java, lines 52-76)

```
public sealed class RelayCrashedException : Exception
{
    public RelayCrashedException()
        : base("relay crashed mid-drain")
    {
    }
}

// Relay drains the outbox, delivering at-least-once: a crash
// mid-drain leaves the tail pending, and re-running redelivers.
// crashAfter < 0 means run to completion.
public sealed class Relay(Outbox outbox, int crashAfter)
{
    public int Attempts { get; private set; }

    public int Delivered { get; private set; }

    public RelayCrashedException? Run(Action<Envelope> deliver)
    {
        while (true)
        {
            if (crashAfter ≥ 0 && Delivered ≥ crashAfter)
            {
                return new RelayCrashedException();
            }
            if (!outbox.TryDequeue(out var e))
            {
                return null;
            }
            Attempts++;
            Delivered++;
            deliver(e);
        }
    }
}
```

listing 497: C#, the relay is a primary-constructor class draining a locked queue, the crash a typed exception (source: patterns-concurrency-distributed/samples-cs/src/Ch13/Outbox.cs, lines 50-84)

```
export class Outbox {
  #pending = [];
  #nextID = 0;

  enqueue(key, body) {
    this.#nextID++;
    const e = { id: this.#nextID, key, body };
    this.#pending.push(e);
    return { ...e };
  }

  len() {
    return this.#pending.length;
  }

  // dequeue the head; the relay's reach into the queue, module scope
  take() {
    return this.#pending.shift();
  }
}

export class CrashError extends Error {
  constructor() {
    super("relay crashed mid-drain");
  }
}

export class Relay {
  #outbox;
  #crashAfter;
  attempts = 0;
  delivered = 0;

  constructor(outbox, crashAfter) {
    this.#outbox = outbox;
    this.#crashAfter = crashAfter;
  }

  // deliver until the outbox is empty or the simulated crash
  async run(deliver) {
    for (;;) {
      if (this.#crashAfter ≥ 0 && this.delivered ≥ this.#crashAfter) {
        throw new CrashError();
      }
      const e = this.#outbox.take();
      if (e === undefined) return;
      this.attempts++;
      this.delivered++;
      await deliver(e);
    }
  }
}
```

listing 498: JavaScript, the queue is an array, run is async so the transport can await (source: patterns-concurrency-distributed/samples-js/src/ch13-outbox.mjs, lines 7-58)

```
class Outbox:
    def __init__(self):
        self.pending = []
        self.next_id = 0
        self.mu = threading.Lock()

    def enqueue(self, key, body):
        with self.mu:
            self.next_id += 1
            env = Envelope(self.next_id, key, body)
            self.pending.append(env)
            return env

    def __len__(self):
        with self.mu:
            return len(self.pending)

    def pop_front(self):
        with self.mu:
            if not self.pending:
                return None
            return self.pending.pop(0)


class RelayCrashed(Exception):
    pass


class Relay:
    def __init__(self, outbox, crash_after):
        self.outbox = outbox
        self.crash_after = crash_after
        self.delivered = 0
        self.attempts = 0

    def run(self, deliver):
        while True:
            if self.crash_after ≥ 0 and self.delivered ≥ self.crash_after:
                raise RelayCrashed("relay crashed mid-drain")
            env = self.outbox.pop_front()
            if env is None:
                return
            self.attempts += 1
            self.delivered += 1
            deliver(env)
```

listing 499: Python, the outbox is a list under a lock, the crash raises RelayCrashed (source: patterns-concurrency-distributed/samples-py/src/Ch13/outbox.py, lines 29-73)

```lua
local Outbox = {}
Outbox.__index = Outbox

function Outbox.new()
  return setmetatable({ pending = {}, nextID = 0 }, Outbox)
end

function Outbox:Enqueue(key, body)
  self.nextID = self.nextID + 1
  local e = { id = self.nextID, key = key, body = body }
  self.pending[#self.pending + 1] = e
  return e
end

function Outbox:Len()
  return #self.pending
end

local Relay = {}
Relay.__index = Relay

-- crashAfter ≥ 0 stops the run after that many deliveries, the
-- scripted stand-in for the process dying mid-drain
function Relay.new(outbox, crashAfter)
  return setmetatable({ outbox = outbox, delivered = 0, attempts = 0,
                        crashAfter = crashAfter }, Relay)
end

function Relay:Run(deliver)
  while true do
    if self.crashAfter ≥ 0 and self.delivered ≥ self.crashAfter then
      return false, "relay crashed mid-drain"
    end
    if #self.outbox.pending == 0 then
      return true
    end
    local e = table.remove(self.outbox.pending, 1)
    self.attempts = self.attempts + 1
    self.delivered = self.delivered + 1
    deliver(e)
  end
end
```

listing 500: Lua, table.remove pops the head, the crash returns false with a message (source: patterns-concurrency-distributed/samples-lua/ch13_outbox.lua, lines 8-49)

The crash is a value in every lane, which is what makes the walk scriptable: a false return in C, a boolean false in java too, the crash threshold an `Option` checked by `is_some_and` in rust, a `RelayCrashedException` in C#, `ErrCrashed` in go, a thrown `CrashError` in javascript, a raised `RelayCrashed` in python, and lua's multiple return, `false` plus the message, the standing error mapping for the one language here without exceptions. The resumed relay is a fresh instance over the same queue, so redelivery is structural, and the consumer from the previous section turns the duplicates into single effects. This chapter is an exact-lane chapter: the frozen go tests pin the counts and the crash-resume shape, the seven sibling trees pin the full ordered effect sequence, and the cross-verification that tied the two together rode go overlays against the frozen implementation, re-run at closeout.

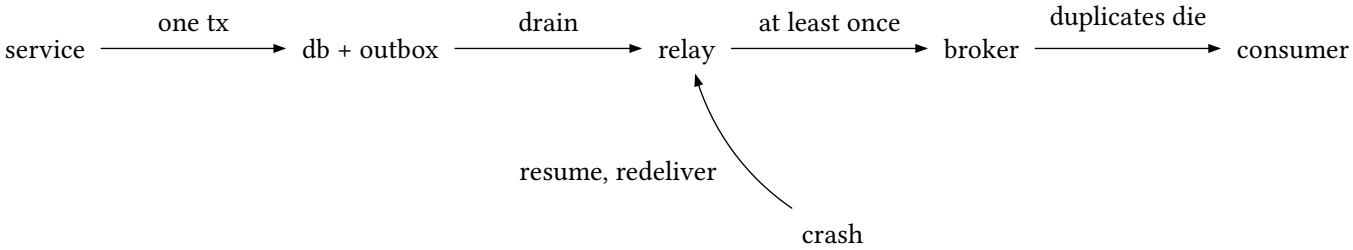


figure 74: the outbox pipeline, the crash point, and where duplicates die

NOTE: ordering across retries

The outbox drain is FIFO over one sender, and that is the entire ordering guarantee. Once multiple senders or multiple relay instances exist, global order is gone and per-key order becomes the achievable target, which is the next pattern's job.

13.3. per-key ordering, lanes

Total order across a cluster is expensive and usually unnecessary. What systems actually promise is per-key order: events about the same entity, same user, same account, apply in sequence, while events about different entities interleave freely. The implementation is partitioning applied to lanes, book 12, ch 12 (design patterns, concurrency, and distributed systems, partitioning and consistent hashing) is the hash this chapter reuses, fnv64a finished with the mix64 finalizer, so the high-bit avalanche lesson carries over unchanged.

The dry run: 3 lanes, 6 keys, 40 rounds.

- the lane map is exact: A and B hash to lane 2, C, D, and E to lane 0, F to lane 1
- 240 events total, every key's processing sequence exactly 0 through 39
- the lane map literals are pinned by the seven new trees, the frozen go test asserts stability and per-key order without pinning the map

```c
// same key, same lane, always: the placement hash decides
static int lane_of(const char *key) { return (int)(hash_string(key) % LANES); }

typedef struct {
  xchan *in;
  envelope order[EVENTS];
  int n;
} lane_arg;

// one writer per lane order slice: no lock, close publishes
static int lane_worker(void *a) {
  lane_arg *l = a;
  envelope e;
  while (xc_recv(l->in, &e))
    l->order[l->n++] = e;
  return 0;
}

typedef struct {
  xchan chans[LANES];
  lane_arg args[LANES];
  thrd_t workers[LANES];
} dispatcher;

static bool disp_open(dispatcher *d) {
  for (int i = 0; i < LANES; i++) {
    xc_init(&d->chans[i]);
    d->args[i].in = &d->chans[i];
    d->args[i].n = 0;
  }
  for (int i = 0; i < LANES; i++)
    if (thrd_create(&d->workers[i], lane_worker, &d->args[i]) != thrd_success)
      return false;
  return true;
}

static void disp_dispatch(dispatcher *d, envelope e) {
  char key[2] = {e.key, 0};
  xc_send(&d->chans[lane_of(key)], e);
}

static void disp_close(dispatcher *d) {
  for (int i = 0; i < LANES; i++)
    xc_close(&d->chans[i]);
  for (int i = 0; i < LANES; i++) {
    int rc;
    thrd_join(d->workers[i], &rc);
  }
}
```

listing 501: C, one rendezvous channel per lane over mtx and cnd, thrd_join publishes the orders (source: patterns-concurrency-distributed/samples-c/src/Ch13/lanes.c, lines 112-160)

```
    let mut orders: Vec<Vec<Envelope>> = vec![Vec::new(); LANES];
    std::thread::scope(|scope| {
        let mut txs = Vec::new();
        let mut handles = Vec::new();
        for order in orders.iter_mut() {
            let (tx, rx) = sync_channel::<Envelope>(0);
            txs.push(tx);
            handles.push(scope.spawn(move || {
                while let Ok(e) = rx.recv() {
                    order.push(e);
                }
            }));
        }
        ok(
            txs.len() == LANES,
            "three rendezvous lanes open with one worker each",
        );
        for round in 0..ROUNDS {
            for k in 0..KEYS {
                let e = Envelope {
                    id: round,
                    key: char::from(b'A' + k as u8),
                };
                let _ = txs[lane_of_key(e.key)].send(e);
            }
        }
        drop(txs);
        for h in handles {
            h.join().unwrap();
        }
    });
```

listing 502: Rust, `sync_channel(0)` is the rendezvous in the stdlib, send blocks until a receiver takes it, dropping the senders closes the lanes (source: patterns-concurrency-distributed/samples-rust/src/Ch13/lanes.rs, lines 65-95)

```go
type KeyedDispatcher struct {
  lanes  []chan Envelope
  orders [][]Envelope
  wg     sync.WaitGroup
}

func NewKeyedDispatcher(laneCount int) *KeyedDispatcher {
  d := &KeyedDispatcher{lanes: make([]chan Envelope, laneCount), orders: make([][]Envelope, laneCount)}
  for i := range d.lanes {
    d.lanes[i] = make(chan Envelope)
    d.wg.Add(1)
    go func(slot int) {
      defer d.wg.Done()
      // one writer per lane slice: no lock, Close publishes
      for e := range d.lanes[slot] {
        d.orders[slot] = append(d.orders[slot], e)
      }
    }(i)
  }
  return d
}

// laneOf maps a routing key to one lane through a placement hash:
// same key, same lane, always.
func (d *KeyedDispatcher) laneOf(key string) int {
  h := fnv64a([]byte(key))
  return int(h % uint64(len(d.lanes)))
}
```

listing 503: Go, one channel and one worker per lane, the placement hash picks the lane (source: patterns-concurrency-distributed/samples/ch13/messaging.go, lines 136-163)

```
func fnv64a(data []byte) uint64 {
  const prime = 0x100000001b3
  h := uint64(0xcbf29ce484222325)
  for _, b := range data {
    h ^= uint64(b)
    h *= prime
  }
  // splitmix finalizer, chapter 12's lesson applied
  h ^= h >> 30
  h *= 0xbf58476d1ce4e5b9
  h ^= h >> 27
  h *= 0x94d049bb133111eb
  h ^= h >> 31
  return h
}

func (d *KeyedDispatcher) Dispatch(e Envelope) {
  d.lanes[d.laneOf(e.Key)] <- e
}

// Close stops the lanes and returns per-lane processing order.
func (d *KeyedDispatcher) Close() [][]Envelope {
  for _, ch := range d.lanes {
    close(ch)
  }
  d.wg.Wait()
  return d.orders
```

listing 504: Go, fnv64a with the splitmix finalizer, close joins the workers and publishes the orders (source: patterns-concurrency-distributed/samples/ch13/messaging.go, lines 165-191)

```
  // the placement mod is unsigned, C's uint64 % and lua's umod: a
  // mixed hash reads negative as a signed long, floorMod would lie
  static int laneOf(String key) {
    return (int) Long.remainderUnsigned(hashString(key), LANES);
  }

  record Envelope(int id, String key, String body) {}

  static final Envelope POISON = new Envelope(-1, "", "");

  // one lane: its queue and the single writer of its order slice
  static final class Lane extends Thread {
    final ArrayBlockingQueue<Envelope> in = new ArrayBlockingQueue<>(1);
    final ArrayList<Envelope> order = new ArrayList<>();

    @Override
    public void run() {
      try {
        while (true) {
          Envelope e = in.take();
          if (e == POISON) return;
          order.add(e);
        }
      } catch (InterruptedException ie) {
        Thread.currentThread().interrupt();
      }
    }
  }
```

listing 505: Java, a lane is an ArrayBlockingQueue of 1, the chapter 8 rendezvous in the stdlib, the mod unsigned because a mixed hash reads negative as a long (source: patterns-concurrency-distributed/samples-java/src/Ch13/Lanes.java, lines 46-73)

```
    public Lanes(int laneCount)
    {
        _channels = new Channel<Envelope>[laneCount];
        _orders = new List<Envelope>[laneCount];
        _drains = new Task[laneCount];
        for (int i = 0; i < laneCount; i++)
        {
            int slot = i;
            _channels[slot] = Channel.CreateUnbounded<Envelope>();
            _orders[slot] = [];
            _drains[slot] = Task.Run(async () =>
            {
                // one writer per lane list: no lock, completion publishes
                await foreach (var e in _channels[slot].Reader.ReadAllAsync())
                {
                    _orders[slot].Add(e);
                }
            });
        }
    }

    public int Count => _channels.Length;

    // laneOf maps a routing key to one lane through a placement
    // hash: same key, same lane, always.
    public int LaneOf(string key)
    {
        ulong h = Hash.String(key);
        return (int)(h % (ulong)_channels.Length);
    }

    public void Dispatch(Envelope e)
    {
        _channels[LaneOf(e.Key)].Writer.TryWrite(e);
    }

    // Close stops the lanes and returns per-lane processing order.
    public async Task<List<Envelope>[]> Close()
    {
        foreach (var ch in _channels)
        {
            ch.Writer.TryComplete();
        }
        await Task.WhenAll(_drains);
        return _orders;
    }
}
```

listing 506: C#, one unbounded Channel per lane, Task.WhenAll publishes (source: patterns-concurrency-distributed/samples-cs/src/Ch13/Lanes.cs, lines 17-63)

```
export class KeyedDispatcher {
  #queues;
  #wake;
  #orders;
  #workers;
  #closed = false;

  constructor(laneCount) {
    this.#queues = Array.from({ length: laneCount }, () => []);
    this.#wake = Array.from({ length: laneCount }, () => []);
    this.#orders = Array.from({ length: laneCount }, () => []);
    this.#workers = Array.from({ length: laneCount }, (_, slot) =>
      this.#laneWorker(slot),
    );
  }

  // laneOf maps a routing key to one lane: same key, same lane
  laneOf(key) {
    return modSmall(hashString(key), this.#queues.length);
  }

  async dispatch(e) {
    if (this.#closed) throw new Error("dispatch after close");
    const slot = this.laneOf(e.key);
    this.#queues[slot].push(e);
    const wake = this.#wake[slot].shift();
    if (wake !== undefined) wake();
  }

  // close stops the lanes and resolves to the per-lane order
  async close() {
    this.#closed = true;
    for (const waiters of this.#wake) {
      for (const wake of waiters) wake();
    }
    await Promise.all(this.#workers);
    return this.#orders;
  }

  async #laneWorker(slot) {
    for (;;) {
      while (this.#queues[slot].length > 0) {
        const e = this.#queues[slot].shift();
        this.#orders[slot].push(e);
      }
      if (this.#closed) return;
      await new Promise((resolve) => this.#wake[slot].push(resolve));
    }
  }
}
```

listing 507: JavaScript, per-lane arrays and wake callbacks, workers park as promises (source: patterns-concurrency-distributed/samples-js/src/ch13-lanes.mjs, lines 8-57)

```
MASK64 = (1 << 64) - 1
FNV_OFFSET = 0xCBF29CE484222325
FNV_PRIME = 0x100000001B3


def fnv64a(data):
    h = FNV_OFFSET
    for byte in data:
        h ^= byte
        h = (h * FNV_PRIME) & MASK64
    h ^= h >> 30
    h = (h * 0xBF58476D1CE4E5B9) & MASK64
    h ^= h >> 27
    h = (h * 0x94D049BB133111EB) & MASK64
    h ^= h >> 31
    return h


class Envelope:
    def __init__(self, env_id, key, body):
        self.id = env_id
        self.key = key
        self.body = body


class KeyedDispatcher:
    def __init__(self, lane_count):
        self.lanes = [[] for _ in range(lane_count)]

    def lane_of(self, key):
        return fnv64a(key.encode()) % len(self.lanes)

    def dispatch(self, env):
        self.lanes[self.lane_of(env.key)].append(env)

    def close(self):
        return self.lanes
```

listing 508: Python, dispatch is a pure function of the key, the lanes are lists (source: patterns-concurrency-distributed/samples-py/src/Ch13/lanes.py, lines 17-53)

```lua
local Dispatcher = {}
Dispatcher.__index = Dispatcher

-- one lane per slot: a FIFO queue plus its own lane, woken on arrival
function Dispatcher.new(S, laneCount)
  local d = { S = S, lanes = {} }
  for i = 1, laneCount do
    d.lanes[i] = { queue = {}, order = {}, parked = false, closed = false }
  end
  return setmetatable(d, Dispatcher)
end

function Dispatcher:laneOf(key)
  return umod(mix64(fnv64a(key)), #self.lanes) + 1 -- 1-based lane slot
end

function Dispatcher:Dispatch(e)
  local lane = self.lanes[self:laneOf(e.key)]
  lane.queue[#lane.queue + 1] = e
  if lane.parked then
    lane.parked = false
    self.S.ready[#self.S.ready + 1] = lane.co
  end
end

-- Start spawns each lane's loop: drain the queue, park when empty,
-- exit when closed and empty. the walk runs the scheduler once after
-- Start so every lane records its coroutine and parks
function Dispatcher:Start()
  for _, lane in ipairs(self.lanes) do
    spawn(self.S, function()
      lane.co = coroutine.running()
      while true do
        while #lane.queue > 0 do
          local e = table.remove(lane.queue, 1)
          lane.order[#lane.order + 1] = e
        end
        if lane.closed then
          return
        end
        lane.parked = true
        coroutine.yield("park")
      end
    end)
  end
end
```

listing 509: Lua, coroutine lanes park on empty queues, the scheduler wakes them (source: patterns-concurrency-distributed/samples-lua/ch13_lanes.lua, lines 64-109)

Each key maps to exactly one lane, each lane has exactly one worker, and FIFO order within the lane preserves the per-key sequence without a global lock. The substrate is where the languages split. Go hands each lane a native channel and publishes the order slices through a `WaitGroup`. C# hands each lane an unbounded `Channel<Envelope>` drained by a task, `Task.WhenAll` the publish edge. C builds the lane out of chapter 8's strict rendezvous shape, a mutex and two condition variables per slot, and `thrd_join` publishes. Java gets the same rendezvous shape from the stdlib, one `ArrayBlockingQueue` of capacity 1 per lane closed by a poison pill, `join` the publish edge, and so does rust, `sync_channel(0)` being a zero-capacity rendezvous channel, the senders dropped to close the lanes, scoped

threads joining inside the scope to publish the borrowed order slices. Javascript keeps a per-lane array plus a wake callback, the worker a promise that parks when its queue is empty. Python's walk is the pure function itself, single-threaded, lane assignment and order deterministic facts with no scheduler in the picture. Lua runs each lane as a coroutine that parks on an empty queue. Two lanes carry the unsigned-modulo trap, lua and java: the hash wraps to a signed 64-bit integer there, floored `%` on a negative disagrees with go's unsigned remainder, so lua's mapping goes through an unsigned-modulo helper and java's through `Long.remainderUnsigned`, because the signed `%` and even `Math.floorMod` both disagree with C's `uint64` remainder. Every threaded lane keeps one writer per order slice, and the join edge is what publishes those slices, chapter 6 applied without ceremony.

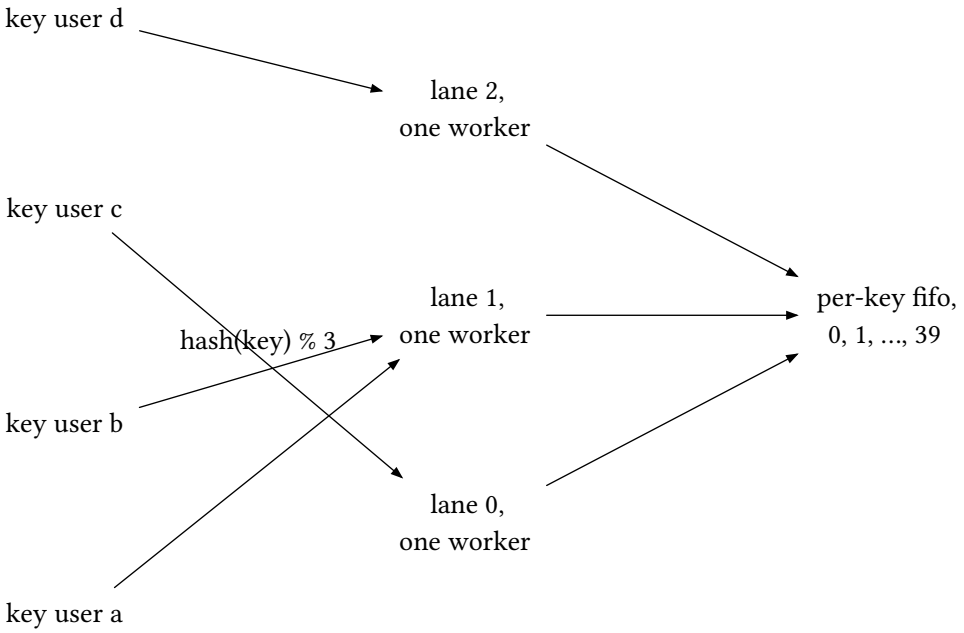


figure 75: one lane per key, keys interleave across lanes, fifo within each lane

13.4. poison messages

A message that consistently fails processing is poison: it fails, retries, fails again, and in a naive relay wedges the lane behind it forever. The standard shape is a retry budget per message, then a dead letter queue, a side queue where the poison lands for human or automated inspection while the lane flows on. Chapter 15 builds the retry budget with backoff and jitter, the same machinery pointed at a different failure, and the tradeoff is always the same: how many retries before the system trades completeness for liveness. The poison walk is the one place this chapter's trees stay silent, the budget itself is chapter 15′s sample, so the lifecycle here is the diagram and the decision, not a fixture.

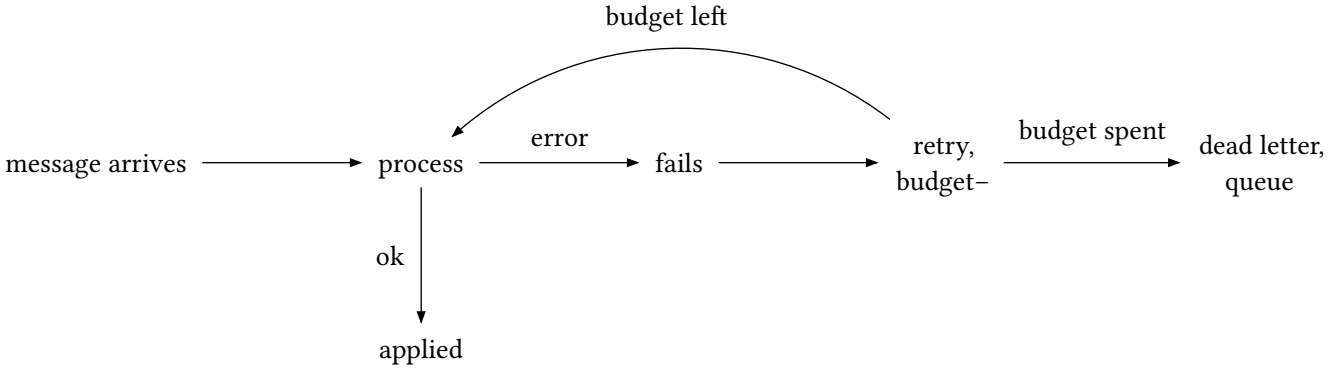


figure 76: the poison message lifecycle, and the lane flows on

13.5. across the eight languages

The build sizes count non-comment source lines, the go column the chapter's one frozen file:

language	build SLOC	dependency	boundary note
c	389	libc plus threads.h	fixed-array outbox, a lane a mutex plus two cnd, thrd_join publishes
rust	438	stdlib	a HashSet consumer, the VecDeque outbox with an Option crash threshold, sync_channel(0) lanes
go	145	stdlib	channel and worker per lane over the frozen hash, ErrCrashed, WaitGroup joins
java	310	jdk 27 stdlib	ArrayBlockingQueue rendezvous lanes closed by a poison pill, the lane mod through Long.remainderUnsigned
c#	156	bcl	unbounded Channel per lane, WhenAll publishes, the crash a typed exception
javascript	114	node stdlib, one sibling module	ch12′s hashString picks the lane, a parked promise wakes per callback
python	228	stdlib only	the lane walk is pure and single-threaded, the outbox a list under a lock
lua	371	lib.lua harness	coroutine lanes park on empty queues, umod fixes floored % on negatives

sources: Heller, "Reliable Pattern: Outbox", for the dual-write problem, Kleppmann, "Designing Data-Intensive Applications", chapter 8 for delivery semantics and the exactly-once-as-effect framing, accessed 2026-09-08. Verified by the eight chapter legs: 3 Ch13 C programs with 548 embedded checks, the rust runner's 548 checks across 3 programs in `samples-rust/src/Ch13`, `go test` at 3 tests in `patternsbook/ch13`, the java runner's 48 checks across 3 programs in `samples-java/src/Ch13`, 6 xunit facts, node's 3 cases in `test/ch13.test.mjs`, 3 python modules with 32 embedded checks, and the lua runner's 12 ch13 rows.

14. observability

Monitoring tells you the system is down, observability lets you ask why. The difference is structure: logs that carry attributes, metrics that carry labels, traces that carry causality, all attached while the code runs, never reconstructed after it fails. The go lane reads its three signals off the stdlib, `log/slog` and `expvar` and the raw material for spans, and the other seven trees hand-roll the same shapes in their own grain, so this chapter builds all three twice over in testable form: a capture handler per language, a labeled counter registry per language, a span tree per language, all asserting the same fixtures.

14.1. structured logging, the capture handler

Slog ships handlers for text and json and, more useful for this book, an interface of four methods a test can implement. The other lanes cut the same shape by hand: a level gate, an in-memory record list, attributes as a map. The gate placement is the lesson that survives translation, the check runs before the record exists, so a dropped line never pays for its arguments.

The dry run: all eight lanes assert the same capture contract.

- info and warn recorded, 2 records, attrs readable as a map
- debug below min dropped before evaluation, the probe's side effect count stays 0
- the frozen go test pins the 2 records and the attr map, the probe literal is pinned by the 7 new trees

```c
// the emit path: serialize one record to one json line
static void capture_handle(capture *c, level l, const char *msg,
                           const attr *attrs, int n_attrs) {
  if (!capture_enabled(c, l))
    return; // below min: dropped, and the caller never evaluated its args
  record *r = &c->records[c->n];
  r->lv = l;
  r->msg = msg;
  r->n_attrs = n_attrs > MAX_ATTRS ? MAX_ATTRS : n_attrs;
  for (int i = 0; i < r->n_attrs; i++)
    r->attrs[i] = attrs[i];
  int off = snprintf(c->lines[c->n], LINE, "{\"level\":\"%s\",\"msg\":\"%s\"",
                     level_name(l), msg);
  for (int i = 0; i < r->n_attrs; i++)
    off += snprintf(c->lines[c->n] + off, (size_t)(LINE - off),
                    ",\"%s\":\"%s\"", r->attrs[i].key,
                    r->attrs[i].secret ? secret_render(r->attrs[i].text)
                                       : r->attrs[i].text);
  snprintf(c->lines[c->n] + off, (size_t)(LINE - off), "}");
  c->n++;
}

// the gate: the message expression is only evaluated when the level passes,
// the laziness slog gets from the variadic signature and C gets from the
// macro. the probe's side effect is the proof
#define LOG(c, lv, msg, attrs, nattrs)                                  \
  do {                                                                  \
    if (capture_enabled((c), (lv)))                                     \
      capture_handle((c), (lv), (msg), (attrs), (nattrs));              \
  } while (0)
```

listing 510: C, the emit path serializes one json line, the LOG macro is the gate the variadic signature buys elsewhere (source: patterns-concurrency-distributed/samples-c/src/Ch14/capture.c, lines 74-103)

```
    fn log(&mut self, lv: Level, msg: impl FnOnce() -> &'static str, attrs: &[Attr]) {
        if !self.enabled(lv) {
            return;
        }
        let msg = msg();
        let mut line = format!("{{\"level\":\"{}\",\"msg\":\"{}\"", level_name(lv), msg);
        for a in attrs {
            let value = if a.secret {
                secret_render(a.text)
            } else {
                a.text.to_string()
            };
            line.push_str(&format!(",\"{}\":\"{}\"", a.key, value));
        }
        line.push('}');
        self.records.push(Record {
            lv,
            msg,
            attrs: attrs.to_vec(),
        });
        self.lines.push(line);
    }
```

listing 511: Rust, the gate runs before the message closure is called, `impl FnOnce` is the laziness, no macro needed (source: patterns-concurrency-distributed/samples-rust/src/Ch14/capture.rs, lines 67-88)

```go
type Capture struct {
  mu      sync.Mutex
  records []Captured
  min     slog.Level
}

type Captured struct {
  Level slog.Level
  Msg   string
  Attrs map[string]any
}

var _ slog.Handler = (*Capture)(nil)

func NewCapture(min slog.Level) *Capture { return &Capture{min: min} }

func (c *Capture) Enabled(_ context.Context, l slog.Level) bool { return l ≥ c.min }

func (c *Capture) Handle(_ context.Context, r slog.Record) error {
  attrs := map[string]any{}
  r.Attrs(func(a slog.Attr) bool {
    // Resolve is the custom handler's obligation: built-in
    // handlers unwrap LogValuer, a naive one leaks the raw value
    attrs[a.Key] = a.Value.Resolve().Any()
    return true
  })
  c.mu.Lock()
  defer c.mu.Unlock()
  c.records = append(c.records, Captured{r.Level, r.Message, attrs})
  return nil
}

func (c *Capture) WithAttrs(attrs []slog.Attr) slog.Handler { return c }

func (c *Capture) WithGroup(name string) slog.Handler { return c }

func (c *Capture) Records() []Captured {
  c.mu.Lock()
  defer c.mu.Unlock()
  return slices.Clone(c.records)
```

listing 512: Go, capture handler: records in memory, level filter, attrs as a map (source: patterns-concurrency-distributed/samples/ch14/observe.go, lines 19-58)

```
  static final class Sink {
    final Level min;
    final ArrayList<Level> levels = new ArrayList<>();
    final ArrayList<String> msgs = new ArrayList<>();
    final ArrayList<List<Attr>> attrs = new ArrayList<>();
    final ArrayList<String> lines = new ArrayList<>();

    Sink(Level min) {
      this.min = min;
    }

    boolean enabled(Level l) {
      return l.ordinal() ≥ min.ordinal();
    }

    // the emit path: serialize one record to one json line
    void handle(Level l, String msg, List<Attr> as) {
      levels.add(l);
      msgs.add(msg);
      attrs.add(as);
      StringBuilder b = new StringBuilder();
      b.append("{\"level\":\"").append(levelName(l)).append("\",\"msg\":\"").append(msg).append('"');
      for (Attr a : as)
        b.append(",\"").append(a.key()).append("\":\"").append(a.render()).append('"');
      b.append('}');
      lines.add(b.toString());
    }
  }

  // the gate: the message supplier runs only when the level passes,
  // the laziness C buys with the macro and java with the supplier
  static void log(Sink c, Level l, Supplier<String> msg, List<Attr> attrs) {
    if (!c.enabled(l)) return; // below min: dropped, never evaluated
    c.handle(l, msg.get(), attrs);
  }
```

listing 513: Java, the message arrives as a Supplier, the gate runs before it, the emit path builds one json line (source: patterns-concurrency-distributed/samples-java/src/Ch14/Capture.java, lines 41-75)

```
// Capture is the in-memory handler: it keeps records instead of
// writing them anywhere, and it resolves ILogValue attributes the
// way built-in handlers unwrap LogValuer (a naive handler leaks the
// raw value).
public sealed class Capture(Level min)
{
    private readonly object _gate = new();
    private readonly List<Captured> _records = [];

    public bool Enabled(Level l) => l ≥ min;

    public void Handle(Level level, string msg, Attr[] attrs)
    {
        var map = new Dictionary<string, object?>();
        foreach (var a in attrs)
        {
            object? v = a.Value;
            if (v is ILogValue lv)
            {
                v = lv.ToLogValue();
            }
            map[a.Key] = v;
        }
        lock (_gate)
        {
            _records.Add(new Captured(level, msg, map));
        }
    }

    public List<Captured> Records()
    {
        lock (_gate)
        {
            return [.. _records];
        }
    }
}
```

listing 514: C#, the in-memory handler resolves ILogValue attributes the way built-ins unwrap LogValuer (source: patterns-concurrency-distributed/samples-cs/src/Ch14/Capture.cs, lines 30-66)

```
export class Capture {
  #records = [];
  #min;

  constructor(min = "info") {
    this.#min = LEVEL[min];
  }

  enabled(level) {
    return LEVEL[level] ≥ this.#min;
  }

  log(level, msg, attrs) {
    if (!this.enabled(level)) return; // dropped before evaluation
    const a = typeof attrs === "function" ? attrs() : (attrs ?? {});
    this.#records.push({ level, msg, attrs: { ...a } });
  }

  debug(msg, attrs) {
    this.log("debug", msg, attrs);
  }

  info(msg, attrs) {
    this.log("info", msg, attrs);
  }

  warn(msg, attrs) {
    this.log("warn", msg, attrs);
  }

  error(msg, attrs) {
    this.log("error", msg, attrs);
  }

  records() {
    return this.#records.map((r) => ({ ...r, attrs: { ...r.attrs } }));
  }

  render() {
    return this.#records.map((r) => JSON.stringify(r)).join("\n");
  }
}
```

listing 515: JavaScript, attrs may be a function, evaluated only after the gate passes (source: patterns-concurrency-distributed/samples-js/src/ch14-capture.mjs, lines 12-53)

```
class Capture:
    def __init__(self, min_level):
        self.min_level = min_level
        self.records = []

    def enabled(self, level):
        return level ≥ self.min_level

    def handle(self, record):
        self.records.append(record)

    def to_json(self):
        return json.dumps(self.records)


class Logger:
    def __init__(self, handler):
        self.handler = handler

    def _emit(self, level, name, msg, attrs):
        if not self.handler.enabled(level):
            return  # dropped: no attr value is ever resolved
        resolved = {}
        for key, value in attrs.items():
            if hasattr(value, "log_value"):
                resolved[key] = value.log_value()
            elif callable(value):
                resolved[key] = value()
            else:
                resolved[key] = value
        self.handler.handle({"level": name, "msg": msg, "attrs": resolved})

    def debug(self, msg, **attrs):
        self._emit(DEBUG, "debug", msg, attrs)

    def info(self, msg, **attrs):
        self._emit(INFO, "info", msg, attrs)

    def warn(self, msg, **attrs):
        self._emit(WARN, "warn", msg, attrs)
```

listing 516: Python, the logger checks enabled before any attr value resolves, callables included (source: patterns-concurrency-distributed/samples-py/src/Ch14/capture.py, lines 31-70)

```lua
local Capture = {}
Capture.__index = Capture

function Capture.new(min)
  return setmetatable({ min = min, records = {} }, Capture)
end

-- log takes the level and a thunk building msg and attrs. the enabled
-- check comes first: a dropped record is never built
function Capture:log(level, build)
  if LEVELS[level] < LEVELS[self.min] then
    return false
  end
  local msg, attrs = build()
  local resolved = {}
  for k, v in pairs(attrs or {}) do
    resolved[k] = redact(v)
  end
  self.records[#self.records + 1] = { level = level, msg = msg, attrs = resolved }
  return true
end

function Capture:Records()
  local out = {}
  for i, r in ipairs(self.records) do
    out[i] = r
  end
  return out
end
```

listing 517: Lua, log takes a thunk, a dropped level never calls it (source: patterns-concurrency-distributed/samples-lua/ch14_capture.lua, lines 29-57)

`Capture` satisfies `slog.Handler`, the compile-time assertion present as always, and every log line a test emits becomes an assertable value: message, level, and attributes. The laziness idiom is where the languages spend differently. Go gets it from the variadic signature, `Enabled` is called early and the arguments of a dropped line are never built. C has no variadic evaluation to lean on, so the LOG macro is the gate, the only mechanism in the eight that hides the evaluation behind a branch. C# gives `Debug` a `Func<Attr[]>` supplier that stays uncalled below the level, java gates the whole message behind a `Supplier<String>`, and rust behind an `impl FnOnce() -> &'static str` parameter, the same trick without the macro. Javascript and python accept attrs as a function and call it only after the gate. Lua takes a whole thunk that builds the message and the attrs, so the dropped record is never constructed at all. This same shape is how production code tests its logging, assertions on what was said, not on log files.

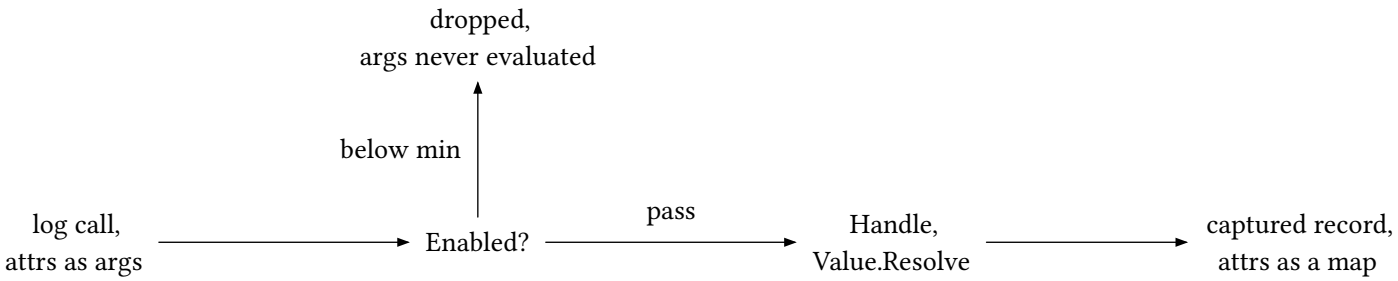


figure 77: one record's path: enabled gates, handle resolves, attrs land as values

Redaction rides on values that know how to sanitize themselves, one wrapper type per language whose render is its length and never its bytes.

The dry run: `Secret("hunter2token")` renders `secret(12 bytes)`, `Secret("api-key-0123456789")` renders `secret(18 bytes)`, and the raw token appears nowhere in the capture. The exact lengths are pinned by the 7 new trees, the frozen go test asserts the `secret(` prefix and the token's absence.

```c
static const char *secret_render(const char *raw) {
  static char buf[32];
  snprintf(buf, sizeof buf, "secret(%d bytes)", (int)strlen(raw));
  return buf;
}
```

listing 518: C, the render the serializer calls, length only (source: patterns-concurrency-distributed/samples-c/src/Ch14/capture.c, lines 68-72)

```rust
fn secret_render(raw: &str) -> String {
    format!("secret({} bytes)", raw.len())
}
```

listing 519: Rust, the render the emit path calls, length only (source: patterns-concurrency-distributed/samples-rust/src/Ch14/capture.rs, lines 38-40)

```go
// the handler resolves it at emit time, so the secret never reaches
// an output stream.
type Secret string

var _ slog.LogValuer = Secret("")

func (s Secret) LogValue() slog.Value {
  return slog.StringValue(fmt.Sprintf("secret(%d bytes)", len(s)))
}
```

listing 520: Go, a secret that logs its length, never its bytes (source: patterns-concurrency-distributed/samples/ch14/observe.go, lines 62-70)

```java
  // an attribute value: plain text, or a secret that redacts itself
  record Attr(String key, String value, boolean secret) {
    String render() {
      return secret ? "secret(" + value.length() + " bytes)" : value;
    }
  }
```

listing 521: Java, the Attr record carries the secret flag, its render is the length, never the bytes (source: patterns-concurrency-distributed/samples-java/src/Ch14/Capture.java, lines 34-39)

```csharp
// ILogValue is the LogValuer analog: the handler resolves it at
// emit time, so a Secret never reaches an output stream raw.
public interface ILogValue
{
    object ToLogValue();
}

// Secret redacts itself in any log line.
public readonly struct Secret(string value) : ILogValue
{
    public object ToLogValue() => $"secret({value.Length} bytes)";
}
```

listing 522: C#, ILogValue is the LogValuer analog, Secret implements it as a struct (source: patterns-concurrency-distributed/samples-cs/src/Ch14/Capture.cs, lines 15-26)

```
export class Secret {
  #raw;

  constructor(raw) {
    this.#raw = raw;
  }

  toJSON() {
    return `secret(${this.#raw.length} bytes)`;
  }
}
```

listing 523: JavaScript, toJSON is the hook, JSON.stringify resolves it at serialization time (source: patterns-concurrency-distributed/samples-js/src/ch14-capture.mjs, lines 55-65)

```
class Secret:
    def __init__(self, raw):
        self.raw = raw

    def log_value(self):
        return f"secret({len(self.raw)} bytes)"
```

listing 524: Python, log_value is the duck-typed hook the emit path probes for (source: patterns-concurrency-distributed/samples-py/src/Ch14/capture.py, lines 23-28)

```
local Secret = {}
Secret.__index = Secret
Secret.__secret = true

local function secret(raw)
  return setmetatable({ raw = raw }, Secret)
end

-- redact renders the shielded form: length only, never the contents
local function redact(v)
  if type(v) == "table" and getmetatable(v) == Secret then
    return string.format("secret(%d bytes)", #v.raw)
  end
  return v
end
```

listing 525: Lua, the metatable identity is the tag, redact is the handler's obligation (source: patterns-concurrency-distributed/samples-lua/ch14_capture.lua, lines 13-27)

And the finding that makes the example worth keeping: the first version of the go handler read `a.Value.Any()` in `Handle` and the token leaked, because resolving `LogValuer` is the handler's job. The built-in handlers call `Value.Resolve()`, and a custom handler that skips it publishes raw secrets while appearing to respect the pattern. Every lane carries the same obligation at the same spot: C's serializer must call `secret_render`, C#'s emit must check `ILogValue`, java's emit must go through the attr's own `render`, rust's emit must check the attr's `secret` flag, the one place the secret flag lives, javascript must go through `JSON.stringify` for `toJSON` to fire, python must probe for `log_value`, lua must run `redact` over the attrs. The fix is one call on the emit path, and the tests now guard it permanently.

14.2. the counter registry, and the one-line mount

`expvar` publishes variables on the default HTTP mux at `/debug/vars` with zero configuration, and `expvar.NewInt` is the whole integration:

```go
func Published() *expvar.Int { return expvar.NewInt("patternsbook_ch14_example") }
```

listing 526: Go, one line and the counter is on the wire (source: patterns-concurrency-distributed/samples/ch14/observe.go, lines 166-166)

One fragility of that one line is recorded, not fixed: `Published` registers a fresh `expvar.Int` on every call, a second publish of the same name panics, so a `go test -count 2` re-run of the chapter dies in the expvar test and verify-go keeps `count=1`. Go is also the only tree whose stdlib ships a metrics mount at all. The siblings own the registry code below and would mount it behind `node:http` in javascript, an `http.server` handler in python, the jdk's own `com.sun.net.httpserver` in java, and nothing stdlib in C, C#, or lua, where the nearest BCL surface is `System.Diagnostics.Metrics` with its `MeterListener`, a row for prose, not for this book's samples.

For labeled counters beyond expvar's flat set, the registry pattern is small enough to own:

The dry run: 20 writers x 100 incs, every 10th inc also errors.

- hits 2000, errors 200, exact under concurrency
- render is `"errors 200\nhits 2000\n"`, sorted by name, the exact string pinned by the 7 new trees while the frozen go test checks both lines
- the writers are real threads in C, rust, C#, go, and python, 20 interleaved coroutines in lua, 20 async writers through the microtask queue in javascript, a cooperative round-robin in 25-inc quanta on one thread in java, all asserting the same totals

```c
// counters by name, kept sorted so the render is sorted
static const char *const counter_names[NAMES] = {"errors", "hits"};

static int name_index(const char *name) {
  for (int i = 0; i < NAMES; i++)
    if (strcmp(counter_names[i], name) == 0)
      return i;
  return -1;
}

static void metrics_inc(metrics *m, const char *name, long long delta) {
  int i = name_index(name);
  if (i < 0)
    return; // unknown counter: not registered, not counted
  mtx_lock(&m->mu);
  m->used[i] = true;
  m->counts[i] += delta;
  mtx_unlock(&m->mu);
}

static long long metrics_get(metrics *m, const char *name) {
  int i = name_index(name);
  if (i < 0)
    return -1;
  mtx_lock(&m->mu);
  long long v = m->counts[i];
  mtx_unlock(&m->mu);
  return v;
}

// sorted counter lines, one per line, the dashboard format
static void metrics_render(metrics *m, char *out, int outsz) {
  mtx_lock(&m->mu);
  int off = 0;
  for (int i = 0; i < NAMES; i++) // walk in name order: sorted output
    if (m->used[i])
      off += snprintf(out + off, (size_t)(outsz - off), "%s %lld\n",
                      counter_names[i], m->counts[i]);
  mtx_unlock(&m->mu);
}
```

listing 527: C, counters by name behind one mutex, the render walks the sorted name array (source: patterns-concurrency-distributed/samples-c/src/Ch14/registry.c, lines 39-78)

```rust
struct Metrics {
    counters: Mutex<BTreeMap<&'static str, i64>>,
}

impl Metrics {
    fn new() -> Metrics {
        Metrics {
            counters: Mutex::new(BTreeMap::new()),
        }
    }

    fn inc(&self, name: &'static str, delta: i64) {
        if !COUNTER_NAMES.contains(&name) {
            return;
        }
        let mut g = self.counters.lock().unwrap();
        *g.entry(name).or_insert(0) += delta;
    }

    fn get(&self, name: &str) -> Option<i64> {
        if !COUNTER_NAMES.contains(&name) {
            return None;
        }
        let g = self.counters.lock().unwrap();
        Some(*g.get(name).unwrap_or(&0))
    }

    fn render(&self) -> String {
        let g = self.counters.lock().unwrap();
        let mut out = String::new();
        for (name, value) in g.iter() {
            out.push_str(&format!("{name} {value}\n"));
        }
        out
    }
}
```

listing 528: Rust, counters in a Mutex<BTreeMap>, the map iterates already sorted, no sort step at render (source: patterns-concurrency-distributed/samples-rust/src/Ch14/registry.rs, lines 21-56)

```
// style: counters by name, a monotonic snapshot function, and
// prometheus-style text rendering for the dashboard.
type Metrics struct {
  mu       sync.Mutex
  counters map[string]*int64
  order    []string
}

func NewMetrics() *Metrics { return &Metrics{counters: map[string]*int64{}} }

func (m *Metrics) Inc(name string, delta int64) {
  m.mu.Lock()
  defer m.mu.Unlock()
  if _, ok := m.counters[name]; !ok {
    c := int64(0)
    m.counters[name] = &c
    m.order = append(m.order, name)
  }
  *m.counters[name] += delta
}

func (m *Metrics) Snapshot() map[string]int64 {
  m.mu.Lock()
  defer m.mu.Unlock()
  out := make(map[string]int64, len(m.counters))
  for k, v := range m.counters {
    out[k] = *v
  }
  return out
}

// Render emits sorted counter lines.
func (m *Metrics) Render() string {
  snap := m.Snapshot()
  names := make([]string, 0, len(snap))
  for k := range snap {
    names = append(names, k)
  }
  sort.Strings(names)
  var b strings.Builder
  for _, k := range names {
    fmt.Fprintf(&b, "%s %d\n", k, snap[k])
  }
  return b.String()
```

listing 529: Go, counters by name, snapshot under lock, sorted text render (source: patterns-concurrency-distributed/samples/ch14/observe.go, lines 73-116)

```
  static final class Metrics {
    final HashMap<String, Long> counters = new HashMap<>();

    void inc(String name, long delta) {
      counters.merge(name, delta, Long::sum);
    }

    // a copy, not a window into the registry
    Map<String, Long> snapshot() {
      return new HashMap<>(counters);
    }

    // sorted counter lines, one per line, the dashboard format
    String render() {
      ArrayList<String> names = new ArrayList<>(counters.keySet());
      Collections.sort(names);
      StringBuilder b = new StringBuilder();
      for (String n : names)
        b.append(n).append(' ').append(counters.get(n)).append('\n');
      return b.toString();
    }
  }
```

listing 530: Java, counters by name in a HashMap, every inc indivisible on the one thread, render sorts and joins (source: patterns-concurrency-distributed/samples-java/src/Ch14/Registry.java, lines 28-49)

```
public sealed class Registry
{
    private readonly object _gate = new();
    private readonly Dictionary<string, long> _counters = [];

    public void Inc(string name, long delta)
    {
        lock (_gate)
        {
            _counters.TryGetValue(name, out long c);
            _counters[name] = c + delta;
        }
    }

    public Dictionary<string, long> Snapshot()
    {
        lock (_gate)
        {
            return new Dictionary<string, long>(_counters);
        }
    }

    // Render emits sorted counter lines.
    public string Render()
    {
        var snap = Snapshot();
        var names = new List<string>(snap.Keys);
        names.Sort(StringComparer.Ordinal);
        var b = new StringBuilder();
        foreach (var k in names)
        {
            b.Append(k).Append(' ').Append(snap[k]).Append('\n');
        }
        return b.ToString();
    }
}
```

listing 531: C#, counters by name under one lock, render sorts and joins (source: patterns-concurrency-distributed/samples-cs/src/Ch14/Registry.cs, lines 9-44)

```
export class Metrics {
  #counters = new Map();

  inc(name, delta = 1) {
    this.#counters.set(name, (this.#counters.get(name) ?? 0) + delta);
  }

  snapshot() {
    return new Map(this.#counters);
  }

  // sorted counter lines, the prometheus text shape
  render() {
    const names = [...this.#counters.keys()].sort();
    return names.map((k) => `${k} ${this.#counters.get(k)}\n`).join("");
  }
}
```

listing 532: JavaScript, a Map of counters, render sorts the keys (source: patterns-concurrency-distributed/samples-js/src/ch14-registry.mjs, lines 6-22)

```
class Metrics:
    def __init__(self):
        self.mu = threading.Lock()
        self.counters = {}

    def inc(self, name, delta=1):
        with self.mu:
            self.counters[name] = self.counters.get(name, 0) + delta

    def snapshot(self):
        with self.mu:
            return dict(self.counters)

    def render(self):
        snap = self.snapshot()
        lines = [f"{name} {snap[name]}" for name in sorted(snap)]
        if not lines:
            return ""
        return "\n".join(lines) + "\n"
```

listing 533: Python, a dict behind a lock, render joins the sorted lines (source: patterns-concurrency-distributed/samples-py/src/Ch14/registry.py, lines 17-35)

```lua
local Metrics = {}
Metrics.__index = Metrics

function Metrics.new()
  return setmetatable({ counters = {} }, Metrics)
end

-- Inc creates on first sight and adds atomically inside one resume:
-- the cooperative scheduler makes each Inc indivisible by itself
function Metrics:Inc(name, delta)
  local c = self.counters[name]
  if not c then
    c = { value = 0 }
    self.counters[name] = c
  end
  c.value = c.value + delta
end

function Metrics:Snapshot()
  local out = {}
  for name, c in pairs(self.counters) do
    out[name] = c.value
  end
  return out
end

-- Render emits sorted counter lines, one name and value per line
function Metrics:Render()
  local names = {}
  for name in pairs(self.counters) do
    names[#names + 1] = name
  end
  table.sort(names)
  local parts = {}
  for _, name in ipairs(names) do
    parts[#parts + 1] = string.format("%s %d\n", name, self.counters[name].value)
  end
  return table.concat(parts)
end
```

listing 534: Lua, counters as tables in a map, one Inc is indivisible inside a resume (source: patterns-concurrency-distributed/samples-lua/ch14_registry.lua, lines 30-68)

The storage idiom splits three ways. C keeps a fixed array of counters walked in name order, so sorted render is the walk order and the registry never allocates. The map languages, go, java, C#, javascript, python, and lua alike, key counters by name and sort at render time, while rust's `BTreeMap` is sorted on arrival, the iteration itself the render order. The serialization boundary is where the chapter's concurrency lesson cashes out: one mutex or lock around the increment is all the exact totals need, chapter 7 earning its keep again, and two lanes need no lock at all, lua because one `Inc` runs inside a single scheduler resume, java because its 20 writers run as one cooperative round-robin and each `inc` is indivisible on the single thread. The render format is deliberately the `name value` exposition style, so a dashboard swap from this registry to a real metrics system is a formatting change, not an architecture change.



figure 78: the registry under one lock, and the one-line stdlib mount beside it

14.3. traces as context propagation

A trace is a tree of spans, and the tree grows through whatever carries context in each language, `context.Context` in go being the reference. `Start` reads the parent out of the context, appends the new span as its child, and returns a context carrying the child, so a function that receives `ctx` and calls `Start` lands in the right place in the tree without knowing its caller. The clock is a parameter, not `time.Now()`, so the test builds a tree with hand-stamped timestamps, asserts the durations and the child order, and never waits.

The dry run: hand-stamped stamps, no clock read anywhere.

- request 3ms, db 1ms, render 1ms, scan 1ms, durations exact
- child order follows creation, db then render then scan
- a span started from a bare context joins no tree
- the go lane's frozen test stamps its own tree with its own names, both walks verified against the same implementation

```c
static span *span_start(tracer *t, const char *name, long now_ms,
                        span *parent) {
  span *s = &t->spans[t->n++];
  s->name = name;
  s->start_ms = now_ms;
  s->end_ms = -1;
  s->n_children = 0;
  if (parent != NULL && parent->n_children < MAX_CHILDREN)
    parent->children[parent->n_children++] = s;
  return s;
}

static void span_end(span *s, long now_ms) { s->end_ms = now_ms; }

static long span_duration(const span *s) {
  if (s->end_ms < 0)
    return 0; // still open: renders as zero, the go lane's zero time
  return s->end_ms - s->start_ms;
}

static int span_write(const span *s, char *out, int outsz, int depth) {
  int off = 0;
  for (int i = 0; i < depth; i++)
    off += snprintf(out + off, (size_t)(outsz - off), "  ");
  off += snprintf(out + off, (size_t)(outsz - off), "%s %ldms\n", s->name,
                  span_duration(s));
  return off;
}

// the tree, deepest last, durations exact
static int span_render(const span *s, char *out, int outsz, int depth) {
  int off = span_write(s, out, outsz, depth);
  for (int i = 0; i < s->n_children; i++)
    off += span_render(s->children[i], out + off, outsz - off, depth + 1);
  return off;
}
```

listing 535: C, the parent is an explicit pointer parameter, the stamps are longs the caller hands in (source: patterns-concurrency-distributed/samples-c/src/Ch14/traces.c, lines 41-76)

```
    fn start(&mut self, name: &'static str, now_ms: i64, parent: Option<usize>) -> usize {
        let idx = self.spans.len();
        self.spans.push(Span {
            name,
            start_ms: now_ms,
            end_ms: None,
            children: Vec::new(),
        });
        if let Some(p) = parent {
            self.spans[p].children.push(idx);
        }
        idx
    }

    fn end(&mut self, idx: usize, now_ms: i64) {
        self.spans[idx].end_ms = Some(now_ms);
    }

    fn duration(&self, idx: usize) -> i64 {
        self.spans[idx]
            .end_ms
            .map_or(0, |e| e - self.spans[idx].start_ms)
    }

    fn children(&self, idx: usize) -> &[usize] {
        &self.spans[idx].children
    }

    fn render_into(&self, idx: usize, out: &mut String, depth: usize) {
        for _ in 0..depth {
            out.push_str("  ");
        }
        out.push_str(&format!(
            "{} {}ms\n",
            self.spans[idx].name,
            self.duration(idx)
        ));
        for &child in &self.spans[idx].children {
            self.render_into(child, out, depth + 1);
        }
    }

    fn render(&self, root: usize) -> String {
        let mut out = String::new();
        self.render_into(root, &mut out, 0);
        out
    }
```

listing 536: Rust, the parent an `Option<usize>` index into one Vec of spans, the open-span sentinel an `Option`, the render recursive (source: patterns-concurrency-distributed/samples-rust/src/Ch14/traces.rs, lines 31-77)

```
type spanKey struct{}

type Span struct {
  Name     string
  start    time.Time
  end      time.Time
  children []*Span
  parent   *Span
}

func Start(ctx context.Context, name string, now time.Time) (context.Context, *Span) {
  s := &Span{Name: name, start: now}
  if parent, ok := ctx.Value(spanKey{}).(*Span); ok {
    parent.children = append(parent.children, s)
    s.parent = parent
  }
  return context.WithValue(ctx, spanKey{}, s), s
}

func (s *Span) End(now time.Time) { s.end = now }

// Render prints the tree with durations, deepest last.
func (s *Span) Render() string {
  var b strings.Builder
  s.write(&b, 0)
  return b.String()
}

func (s *Span) write(b *strings.Builder, depth int) {
  fmt.Fprintf(b, "%s%s %s\n", strings.Repeat("  ", depth), s.Name, s.Duration())
  for _, c := range s.children {
    c.write(b, depth+1)
  }
}

func (s *Span) Duration() time.Duration {
  if s.end.IsZero() {
```

listing 537: Go, span in the context, children attach to their parent, durations from an injectable clock (source: patterns-concurrency-distributed/samples/ch14/observe.go, lines 122-158)

```
  static final class Span {
    final String name;
    final long startMs;
    long endMs = -1; // -1 until ended
    final ArrayList<Span> children = new ArrayList<>();

    Span(String name, long startMs) {
      this.name = name;
      this.startMs = startMs;
    }
  }

  // start stamps in and attaches to the parent handed in, if any
  static Span start(String name, long nowMs, Span parent) {
    Span s = new Span(name, nowMs);
    if (parent != null) parent.children.add(s);
    return s;
  }

  static void end(Span s, long nowMs) {
    s.endMs = nowMs;
  }

  static long duration(Span s) {
    if (s.endMs < 0) return 0; // still open: renders as zero
    return s.endMs - s.startMs;
  }

  // the tree, deepest last, durations exact
  static String render(Span s, int depth) {
    StringBuilder b = new StringBuilder();
    b.append("  ".repeat(depth)).append(s.name).append(' ')
        .append(duration(s)).append("ms\n");
    for (Span c : s.children) b.append(render(c, depth + 1));
    return b.toString();
  }
```

listing 538: Java, the parent a plain parameter, stamps handed in, an open span renders zero (source: patterns-concurrency-distributed/samples-java/src/Ch14/Traces.java, lines 20-55)

```
public sealed class Span : IDisposable
{
    private static readonly AsyncLocal<Span?> Current = new();

    private readonly List<Span> _children = [];
    private DateTimeOffset _end;

    public string Name { get; }

    public DateTimeOffset Started { get; }

    private Span(string name, DateTimeOffset started, Span? parent)
    {
        Name = name;
        Started = started;
        Parent = parent;
    }

    private Span? Parent { get; }

    public IReadOnlyList<Span> Children => _children;

    public static Span Begin(string name, DateTimeOffset now)
    {
        var parent = Current.Value;
        var s = new Span(name, now, parent);
        parent?._children.Add(s);
        Current.Value = s;
        return s;
    }

    public void Dispose() => Current.Value = Parent;

    public void End(DateTimeOffset now) => _end = now;

    public TimeSpan Duration => _end == default ? TimeSpan.Zero : _end - Started;
```

listing 539: C#, AsyncLocal carries the current span, Dispose pops it, Begin attaches to the parent (source: patterns-concurrency-distributed/samples-cs/src/Ch14/Traces.cs, lines 9-44)

```
export function startSpan(parent, name, at) {
  const s = { name, start: at, end: null, children: [], parent };
  if (parent !== null) {
    parent.children.push(s);
  }
  return s;
}

export function endSpan(s, at) {
  s.end = at;
}

export function spanDuration(s) {
  return s.end === null ? 0 : s.end - s.start;
}

// the tree, deepest last, durations in ms
export function renderTree(s) {
  const lines = [];
  const walk = (sp, depth) => {
    lines.push("  ".repeat(depth) + `${sp.name} ${spanDuration(sp)}ms`);
    for (const c of sp.children) walk(c, depth + 1);
  };
  walk(s, 0);
  return lines.join("\n") + "\n";
}
```

listing 540: JavaScript, the parent reference is an explicit first parameter, said plainly (source: patterns-concurrency-distributed/samples-js/src/ch14-traces.mjs, lines 7-32)

```
def start(ctx, name, now):
    span = Span(name, now)
    parent = ctx.get("span")
    if parent is not None:
        parent.children.append(span)
    return {"span": span}, span


def finish(span, now):
    span.end = now


def duration_ms(span):
    if span.end is None:
        return None
    return span.end - span.start


def render(span):
    lines = []

    def walk(node, depth):
        lines.append("  " * depth + f"{node.name} {duration_ms(node)}ms")
        for child in node.children:
            walk(child, depth + 1)

    walk(span, 0)
    return "\n".join(lines) + "\n"
```

listing 541: Python, a ctx dict threads the parent through, start returns the child ctx (source: patterns-concurrency-distributed/samples-py/src/Ch14/traces.py, lines 24-51)

```lua
local function End(s, at)
  s.endMs = at
end

-- Duration is end minus start, zero while open
function Span:Duration()
  if not self.endMs then
    return 0
  end
  return self.endMs - self.startMs
end

local function write(s, depth, parts)
  parts[#parts + 1] = string.format("%s%s %dms\n",
    string.rep("  ", depth), s.name, s:Duration())
  for _, c in ipairs(s.children) do
    write(c, depth + 1, parts)
  end
end

function Span:Render()
  local parts = {}
  write(self, 0, parts)
  return table.concat(parts)
end
```

listing 542: Lua, ctx is a table carrying span, Start attaches and returns a fresh ctx (source: patterns-concurrency-distributed/samples-lua/ch14_traces.lua, lines 20-45)

The context carrier is the idiom row. Go's `context.Value` is the one the chapter's pipeline chapters already threaded, C#'s `AsyncLocal<Span>` flows with the async chain and needs no parameter at all, and the other six are honest explicit threads: a parent pointer in C, an `Option<usize>` index into one Vec of spans in rust, the parent as a plain parameter in java, the first parameter in javascript, a ctx dict in python, a ctx table in lua. The render contract is identical everywhere, indented lines, deepest last, durations from the stamps the caller supplied. The one-millisecond query in the walks is deterministic, and the same property is what makes real span trees testable: durations are values, chapter 9's synctest lesson in a different costume.

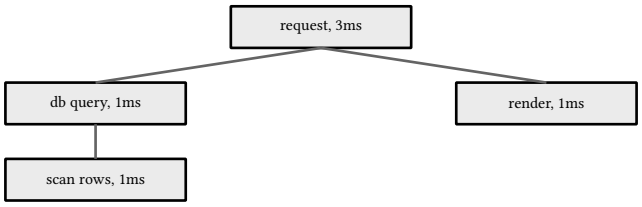


figure 79: the span tree grows through the context, one child per start

NOTE: the three questions

Logs answer "what happened here", metrics answer "how often and how much", traces answer "which path did this request take". A system with only one of the three answers is debugging by archaeology. The raft capstone wires all three: slog with a capture handler in tests, counters for terms and commits, and a span tree around election and replication rounds.

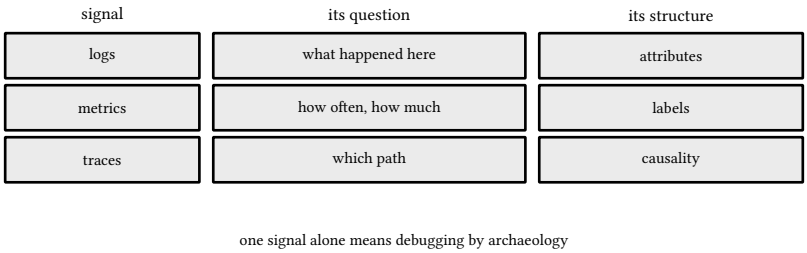


figure 80: three signals, three questions, each carrying its structure

14.4. across the eight languages

The build sizes count non-comment source lines, the go column the chapter's one frozen file:

language	build SLOC	dependency	boundary note
c	336	libc plus threads.h	the registry under one mutex, the span parent an explicit pointer parameter
rust	376	stdlib	an FnOnce message gate, a `Mutex\<BTreeMap\>` registry sorted on arrival, an arena Vec of spans
go	121	stdlib	a slog.Handler over capture, expvar's one-line mount, spans ride context.Value
java	239	jdk 27 stdlib	Supplier gates the message, hand-built json lines, the registry a one-thread round-robin
c#	204	bcl	AsyncLocal carries the current span, secrets redact as ILogValue at emit time
javascript	78	node stdlib	lazy attrs as functions, secrets via toJSON, the parent the first parameter
python	209	stdlib only	a ctx dict threads the parent, the registry behind a lock, render through json
lua	296	lib.lua harness	string.format registry, coroutine lanes prove the totals, ctx a table

sources: go.dev/pkg/log/slog for the Handler contract, `Enabled` timing, `LogValuer`, and `Value.Resolve`, go.dev/pkg/expvar for `NewInt` and the `/debug/vars` mount, go.dev/pkg/runtime/pprof for the profiles available alongside, accessed 2026-09-08. Verified by the eight chapter legs: 3 Ch14 C programs with 82 embedded checks, the rust runner's 82 checks across 3 programs in `samples-rust/src/Ch14`, `go test` at 5 tests in `patternsbook/ch14`, the java runner's 37 checks across 3 programs in `samples-java/src/Ch14`, 10 xunit facts, node's 4 cases in `test/ch14.test.mjs`, 3 python modules with 34 embedded checks, and the lua runner's 14 ch14 rows.

15. resilience

Everything before this chapter assumed failure is an event. In a distributed system failure is weather: timeouts, retries, and degradation are steady-state design, not exception handling. The four tools in this chapter, retry with backoff, the circuit breaker, rate limiting, and the bulkhead, are the standard equipment, and the chapter's specific discipline is making all of them testable through injected clocks and sleeps, no test waits a real second. Every tree runs its timing through a clock it hands in: C advances a vclock struct, rust the same struct with every sleep recorded in a Vec, java advances a vclock long whose sleeps record themselves, C# reads a `TimeProvider` and takes the sleep as a delegate, javascript queues timers on a fake clock whose advance fires them after pumping the microtask queue, python records sleeps on a fake clock and drives the bulkhead through a timer heap under asyncio, lua scripts a scheduler over a time table. The dry runs below assert exact admit, drip, and open-half states, never wall timing.

15.1. retry with exponential backoff and full jitter

Retrying immediately in a tight loop is a denial of service attack on your own dependency, and retrying in synchronized lockstep is one on its recovery. The standard answer is exponential caps plus full jitter, sleep uniform in `[0, cap)` with the cap doubling per attempt, with the sleep and the draw both injected so the schedule is an observation, not a wait.

The dry run: all eight lanes pin the same schedules.

- 4 attempts succeeding on the 3rd, jitter pinned to the cap: sleeps `[10ms, 20ms]`
- 3 attempts, zero jitter, all failing: 2 sleeps, the error names exhaustion and the last cause
- realized full jitter, corpus LCG seed 42 over caps `[10, 20, 40, 80]`: draws `[5, 4, 16, 50]` ms, sum 75
- the frozen go test pins the first two walks plus a draw-under-cap property, the realized vector is pinned by the 7 new trees

```c
// the attempt: NULL return means success, anything else is the cause
typedef const char *(*attempt_fn)(void *ctx);

typedef struct {
  int attempts;
  long base_ms;
  vclock *ck;
  long (*jitter)(lcg *, long cap); // cap in, actual delay out
  lcg rnd;
} retry;

typedef struct {
  bool ok;
  bool exhausted;    // retries exhausted
  const char *cause; // the last failure's identity
  int calls;
} retry_out;

static const char *retry_do(retry *r, attempt_fn f, void *ctx, retry_out *out) {
  out->ok = false;
  out->exhausted = false;
  out->cause = NULL;
  out->calls = 0;
  const char *err = NULL;
  for (int attempt = 0; attempt < r->attempts; attempt++) {
    if (attempt > 0) {
      long cap = r->base_ms << (attempt - 1);
      clock_sleep(r->ck, r->jitter(&r->rnd, cap));
    }
    out->calls++;
    err = f(ctx);
    if (err == NULL) {
      out->ok = true;
      return NULL;
    }
    out->cause = err;
  }
  out->exhausted = true;
  return err;
}
```

listing 543: C, the attempt is a function pointer, the outcome struct carries exhaustion and cause (source: patterns-concurrency-distributed/samples-c/src/Ch15/retry.c, lines 47-86)

```
impl Retry {
    fn run<F: FnMut() -> Option<&'static str>>(
        &mut self,
        ck: &mut VClock,
        mut f: F,
        out: &mut RetryOut,
    ) -> Option<&'static str> {
        out.ok = false;
        out.exhausted = false;
        out.cause = None;
        out.calls = 0;
        let mut err = None;
        for attempt in 0..self.attempts {
            if attempt > 0 {
                let cap = self.base_ms << (attempt - 1);
                ck.sleep(self.jitter.draw(&mut self.rnd, cap));
            }
            out.calls += 1;
            match f() {
                None => {
                    out.ok = true;
                    return None;
                }
                Some(e) => {
                    err = Some(e);
                    out.cause = Some(e);
                }
            }
        }
        out.exhausted = true;
        err
    }
}
```

listing 544: Rust, jitter a `Jitter` enum matched in `draw`, the attempt an `FnMut` closure, sleeps through the injected clock, `RetryOut` mirrors c's outcome struct (source: patterns-concurrency-distributed/samples-rust/src/Ch15/retry.rs, lines 94-126)

```
type Retry struct {
  Attempts int
  Base     time.Duration
  Sleep    func(time.Duration)
  Jitter   func(cap time.Duration) time.Duration
}

func NewRetry(attempts int, base time.Duration) *Retry {
  return &Retry{
    Attempts: attempts,
    Base:     base,
    Sleep:    time.Sleep,
    Jitter: func(cap time.Duration) time.Duration {
      return time.Duration(rand.Int64N(int64(cap)))
    },
  }
}

var ErrRetriesExhausted = errors.New("retries exhausted")

func (r *Retry) Do(f func() error) error {
  var err error
  for attempt := 0; attempt < r.Attempts; attempt++ {
    if attempt > 0 {
      cap := time.Duration(1) << uint(attempt-1) * r.Base
      r.Sleep(r.Jitter(cap))
    }
    if err = f(); err == nil {
      return nil
    }
  }
  if err == nil {
    return ErrRetriesExhausted
  }
  return errors.Join(ErrRetriesExhausted, err)
}

// Breaker is the circuit breaker state machine: closed carries
```

listing 545: Go, caps double, jitter draws under the cap, sleep injected (source: patterns-concurrency-distributed/samples/ch15/resilience.go, lines 15-52)

```
  static final class RetryLoop {
    final int attempts;
    final long baseMs;
    final VClock clock;
    final LongUnaryOperator jitter; // cap in, actual delay out

    RetryLoop(int attempts, long baseMs, VClock clock, LongUnaryOperator jitter) {
      this.attempts = attempts;
      this.baseMs = baseMs;
      this.clock = clock;
      this.jitter = jitter;
    }

    String run(Attempt f, Out out) {
      out.ok = false;
      out.exhausted = false;
      out.cause = null;
      out.calls = 0;
      String err = null;
      for (int attempt = 0; attempt < attempts; attempt++) {
        if (attempt > 0) {
          long cap = baseMs << (attempt - 1);
          clock.sleep(jitter.applyAsLong(cap));
        }
        out.calls++;
        err = f.call();
        if (err == null) {
          out.ok = true;
          return null;
        }
        out.cause = err;
      }
      out.exhausted = true;
      return "retries exhausted; last: " + err;
    }
  }
```

listing 546: Java, caps double, jitter an injected cap-to-delay operator, the vclock records each sleep, an Out carries exhaustion and cause (source: patterns-concurrency-distributed/samples-java/src/Ch15/Retry.java, lines 65-100)

```
    public void Do(Action f)
    {
        Exception? last = null;
        for (int attempt = 0; attempt < attempts; attempt++)
        {
            if (attempt > 0)
            {
                var cap = TimeSpan.FromTicks(baseDelay.Ticks << (attempt - 1));
                sleep(jitter(cap));
            }
            try
            {
                f();
                return;
            }
            catch (Exception e)
            {
                last = e;
            }
        }
        throw new RetriesExhaustedException(last);
    }
}
```

listing 547: C#, caps from base ticks, exhaustion throws with the last error inside (source: patterns-concurrency-distributed/samples-cs/src/Ch15/Retry.cs, lines 14-36)

```
export class Retry {
  constructor(attempts, base, sleep, jitter) {
    this.attempts = attempts;
    this.base = base;
    this.sleep = sleep;
    this.jitter = jitter;
  }

  // run f until it stops throwing or the attempts run out. the
  // thrown error identifies exhaustion and carries the last cause.
  async do(f) {
    let err = null;
    for (let attempt = 0; attempt < this.attempts; attempt++) {
      if (attempt > 0) {
        const cap = 2 ** (attempt - 1) * this.base;
        await this.sleep(this.jitter(cap));
      }
      try {
        return await f();
      } catch (e) {
        err = e;
      }
    }
    throw new RetriesExhaustedError(err);
  }
}
```

listing 548: JavaScript, do is async, the injected sleep queues on the fake clock (source: patterns-concurrency-distributed/samples-js/src/ch15-retry.mjs, lines 52-77)

```
class LCG:
    def __init__(self, seed):
        self.x = seed & MASK64

    def next_u32(self):
        self.x = (LCG_A * self.x + LCG_C) & MASK64
        return self.x >> 32

    def below(self, k):
        return (self.next_u32() * k) >> 32


class Clock:
    def __init__(self):
        self.now = 0
        self.slept = []

    def sleep(self, delay):
        self.slept.append(delay)
        self.now += delay


class Retry:
    def __init__(self, attempts, base, sleep, jitter):
        self.attempts = attempts
        self.base = base
        self.sleep = sleep
        self.jitter = jitter

    def do(self, f):
        err = None
        for attempt in range(self.attempts):
            if attempt > 0:
                cap = (1 << (attempt - 1)) * self.base
                self.sleep(self.jitter(cap))
            err = f()
            if err is None:
                return None
        return f"retries exhausted; last: {err}"
```

listing 549: Python, the LCG beside the clock, the exhausted error is a string naming both (source: patterns-concurrency-distributed/samples-py/src/Ch15/retry.py, lines 22-60)

```lua
local LCG_A = 6364136223846793005
local LCG_C = 1442695040888963407

local function lcg(seed)
  local x = seed
  return function()
    x = LCG_A * x + LCG_C -- wraps mod 2^64
    return x >> 32
  end
end

local function below(u, k)
  return (u * k) >> 32
end

-- Retry runs f with a doubling cap per attempt and an injected jitter
-- draw in [0, cap): the AWS blend, retried herds spread out instead
-- of stampeding in lockstep
local Retry = {}
Retry.__index = Retry

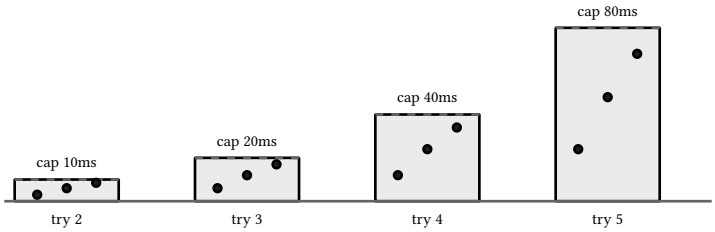
function Retry.new(attempts, base, sleep, jitter)
  return setmetatable({ attempts = attempts, base = base,
                        sleep = sleep, jitter = jitter }, Retry)
end

function Retry:Do(f)
  local err
  for attempt = 0, self.attempts - 1 do
    if attempt > 0 then
      local cap = (1 << (attempt - 1)) * self.base
      self.sleep(self.jitter(cap))
    end
    local ok, e = pcall(f)
    if ok then
      return nil
    end
    err = e
  end
  return "retries exhausted: last error: " .. tostring(err)
end
```

listing 550: Lua, the LCG wraps mod 2^64 natively, failure rides pcall (source: patterns-concurrency-distributed/samples-lua/ch15_retry.lua, lines 8-48)

Sleeps happen between attempts, never after the last, and the exhausted error names both facts in every lane: go joins `ErrRetriesExhausted` with the last real error so `errors.Is` finds either, C fills an outcome struct with the exhausted flag and the cause pointer, rust the same struct with the cause an `Option`, java the same shape, an `Out` with the flag and the cause beside a message naming both, C# throws `RetriesExhaustedException` carrying the last error as its `InnerException`, javascript and python build the exhaustion message around the carried cause, lua concatenates it into the returned message string. The jitter source is the other split: go draws from `rand/v2`, the 7 new lanes draw from chapter 11's corpus LCG so the realized schedule is a pinned vector, and the AWS blend is the same shape in all eight, herds spread across the band instead of stampeding in lockstep.

Retries are only correct for idempotent operations, which is not a formality: retrying a payment creation is how double charges happen. Chapter 13's idempotency keys are the companion tool, the key makes the operation safe to retry, this pattern supplies the patience.



sleep uniform in [0, cap): retried callers land anywhere in the band, never in lockstep

figure 81: caps double per attempt, every sleep drawn under its cap so herds spread

15.2. the circuit breaker

A breaker is a state machine that stops hammering a dependency that is down, so its recovery is not suffocated by its callers:

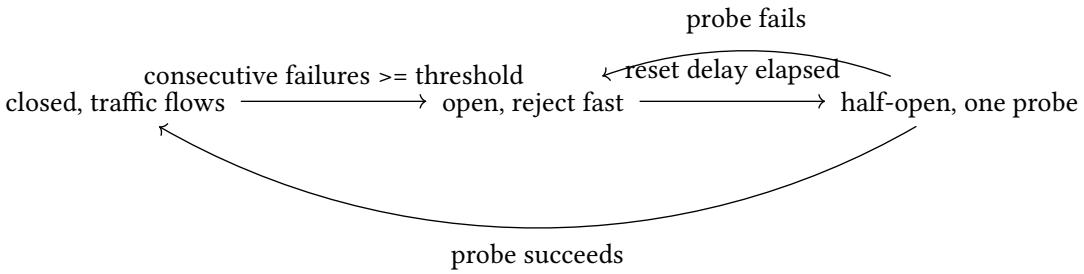


figure 82: breaker states

The dry run: threshold 3, reset delay 5 seconds, the whole walk on the injected clock.

- 2 failures stay closed, a success resets the streak, the 3rd consecutive failure opens
- open rejects fast and the dependency's call count does not move, +2s stays open, the full +5s admits exactly one probe, a second concurrent probe is rejected, java and rust pinning the whole lifecycle's dependency count at exactly 8
- a failed probe reopens with a fresh delay counted from the failure, a successful probe closes
- the go lane's frozen walk advances 2s then 4s more, admitting its probe at 6s cumulative, both walks verified against the same implementation

```c
// may the call proceed? a half-open breaker claims its single probe slot
static bool breaker_allow(breaker *b) {
  switch (b->state) {
  case BS_CLOSED:
    return true;
  case BS_OPEN:
    if (*b->now_ms - b->opened_at >= b->reset_delay_ms) {
      b->state = BS_HALF_OPEN;
      b->probing = true;
      return true;
    }
    return false;
  default: // half-open
    if (b->probing)
      return false; // a probe is already in flight
    b->probing = true;
    return true;
  }
}

// settle the outcome of an allowed call
static void breaker_report(breaker *b, bool failed) {
  if (b->state == BS_HALF_OPEN) {
    if (failed) {
      b->state = BS_OPEN;
      b->opened_at = *b->now_ms; // fresh delay from the failed probe
    } else {
      b->state = BS_CLOSED;
      b->consecutive = 0;
    }
    b->probing = false;
    return;
  }
  if (failed) {
    b->consecutive++;
    if (b->consecutive >= b->fail_threshold) {
      b->state = BS_OPEN;
      b->opened_at = *b->now_ms;
    }
  } else {
    b->consecutive = 0;
  }
}
```

listing 551: C, allow claims the probe slot, report settles it, the clock is a long the test owns (source: patterns-concurrency-distributed/samples-c/src/Ch15/breaker.c, lines 56-98)

```
    fn allow(&mut self, now_ms: i64) -> bool {
        match self.state {
            BState::Closed => true,
            BState::Open => {
                if now_ms - self.opened_at >= self.reset_delay_ms {
                    self.state = BState::HalfOpen;
                    self.probing = true;
                    true
                } else {
                    false
                }
            }
            BState::HalfOpen => {
                if self.probing {
                    false
                } else {
                    self.probing = true;
                    true
                }
            }
        }
    }

    fn report(&mut self, now_ms: i64, failed: bool) {
        if self.state == BState::HalfOpen {
            if failed {
                self.state = BState::Open;
                self.opened_at = now_ms;
            } else {
                self.state = BState::Closed;
                self.consecutive = 0;
            }
            self.probing = false;
            return;
        }
        if failed {
            self.consecutive += 1;
            if self.consecutive >= self.fail_threshold {
                self.state = BState::Open;
                self.opened_at = now_ms;
            }
        } else {
            self.consecutive = 0;
        }
    }
}
```

listing 552: Rust, allow claims the probe slot, report settles it, the clock a `now_ms` value passed into each call, no stored pointer (source: patterns-concurrency-distributed/samples-rust/src/Ch15/breaker.rs, lines 51-96)

```go
type Breaker struct {
  mu            sync.Mutex
  failThreshold int
  resetDelay    time.Duration
  now           func() time.Time

  state       string // "closed", "open", "half-open"
  consecutive int
  openedAt    time.Time
  probing     bool
}

func NewBreaker(failThreshold int, resetDelay time.Duration, now func() time.Time) *Breaker {
  if now == nil {
    now = time.Now
  }
  return &Breaker{
    failThreshold: failThreshold,
    resetDelay:    resetDelay,
    now:           now,
    state:         "closed",
  }
}

var ErrOpen = errors.New("circuit open")

// Allow reports whether the call may proceed, and for a half-open
// breaker claims the single probe slot.
func (b *Breaker) Allow() error {
  b.mu.Lock()
  defer b.mu.Unlock()
  switch b.state {
  case "closed":
    return nil
  case "open":
    if b.now().Sub(b.openedAt) ≥ b.resetDelay {
      b.state = "half-open"
      b.probing = true
      return nil
    }
    return ErrOpen
  default: // half-open
    if b.probing {
      return ErrOpen // probe already in flight
    }
    b.probing = true
    return nil
  }
}
```

listing 553: Go, the breaker fields and constructor, allow admits one half-open probe, the clock injected (source: patterns-concurrency-distributed/samples/ch15/resilience.go, lines 56-104)

```
// Report settles the outcome of an allowed call.
func (b *Breaker) Report(err error) {
  b.mu.Lock()
  defer b.mu.Unlock()
  if b.state == "half-open" {
    if err ≠ nil {
      b.state = "open"
      b.openedAt = b.now()
    } else {
      b.state = "closed"
      b.consecutive = 0
    }
    b.probing = false
    return
  }
  if err ≠ nil {
    b.consecutive++
    if b.consecutive ≥ b.failThreshold {
      b.state = "open"
      b.openedAt = b.now()
    }
  } else {
    b.consecutive = 0
  }
```

listing 554: Go, report settles the probe, a failure reopens with a fresh delay, a success closes (source: patterns-concurrency-distributed/samples/ch15/resilience.go, lines 106-129)

```
    // may the call proceed? a half-open breaker claims its single probe
    boolean allow() {
      switch (state) {
        case CLOSED:
          return true;
        case OPEN:
          if (clock.nowMs - openedAt ≥ resetDelayMs) {
            state = BState.HALF_OPEN;
            probing = true;
            return true;
          }
          return false;
        default: // half-open
          if (probing) return false; // a probe is already in flight
          probing = true;
          return true;
      }
    }

    // settle the outcome of an allowed call
    void report(boolean failed) {
      if (state == BState.HALF_OPEN) {
        if (failed) {
          state = BState.OPEN;
          openedAt = clock.nowMs; // fresh delay from the failed probe
        } else {
          state = BState.CLOSED;
          consecutive = 0;
        }
        probing = false;
        return;
      }
      if (failed) {
        consecutive++;
        if (consecutive ≥ failThreshold) {
          state = BState.OPEN;
          openedAt = clock.nowMs;
        }
      } else {
        consecutive = 0;
      }
    }
```

listing 555: Java, the probing flag claims the single half-open slot, report settles it, a failed probe restamps the open time (source: patterns-concurrency-distributed/samples-java/src/Ch15/Breaker.java, lines 51-92)

```
    // Allow lets the call through, or throws when the circuit is
    // open. For a half-open breaker it claims the single probe slot.
    public void Allow()
    {
        lock (_gate)
        {
            switch (_state)
            {
                case "closed":
                    return;
                case "open":
                    if (_time.GetUtcNow() - _openedAt ≥ _resetDelay)
                    {
                        _state = "half-open";
                        _probing = true;
                        return;
                    }
                    throw new CircuitOpenException();
                default:
                    if (_probing)
                    {
                        throw new CircuitOpenException(); // probe already in flight
                    }
                    _probing = true;
                    return;
            }
        }
    }
```

listing 556: C#, Allow throws when open and claims the single probe slot, Report mirrors go's settle (source: patterns-concurrency-distributed/samples-cs/src/Ch15/Breaker.cs, lines 46-73)

```
  constructor(failThreshold, resetDelay, now) {
    this.#failThreshold = failThreshold;
    this.#resetDelay = resetDelay;
    this.#now = now;
  }

  // allow says the call may proceed; a half-open breaker hands out
  // the single probe slot, a second concurrent caller is rejected
  allow() {
    if (this.#state === "closed") return;
    if (this.#state === "open") {
      if (this.#now() - this.#openedAt ≥ this.#resetDelay) {
        this.#state = "half-open";
        this.#probing = true;
        return;
      }
      throw new CircuitOpenError();
    }
    if (this.#probing) {
      throw new CircuitOpenError(); // probe already in flight
    }
    this.#probing = true;
  }

  // report settles the outcome of an allowed call: err null = ok
  report(err) {
    if (this.#state === "half-open") {
      if (err =/= null) {
        this.#state = "open";
        this.#openedAt = this.#now();
      } else {
        this.#state = "closed";
        this.#consecutive = 0;
      }
      this.#probing = false;
      return;
    }
    if (err =/= null) {
      this.#consecutive++;
      if (this.#consecutive ≥ this.#failThreshold) {
        this.#state = "open";
        this.#openedAt = this.#now();
      }
    } else {
      this.#consecutive = 0;
    }
  }
```

listing 557: JavaScript, the now function is injected, open admission compares against it (source: patterns-concurrency-distributed/samples-js/src/ch15-breaker.mjs, lines 22-68)

```
class Breaker:
    def __init__(self, fail_threshold, reset_delay, clock):
        self.fail_threshold = fail_threshold
        self.reset_delay = reset_delay
        self.clock = clock
        self.state = "closed"
        self.consecutive = 0
        self.opened_at = 0.0
        self.probing = False

    def allow(self):
        if self.state == "closed":
            return
        if self.state == "open":
            if self.clock.now - self.opened_at ≥ self.reset_delay:
                self.state = "half-open"
                self.probing = True
                return
            raise CircuitOpen("circuit open")
        if self.probing:
            raise CircuitOpen("probe already in flight")
        self.probing = True

    def report(self, err):
        if self.state == "half-open":
            if err is not None:
                self.state = "open"
                self.opened_at = self.clock.now
            else:
                self.state = "closed"
                self.consecutive = 0
            self.probing = False
            return
        if err is not None:
            self.consecutive += 1
            if self.consecutive ≥ self.fail_threshold:
                self.state = "open"
                self.opened_at = self.clock.now
        else:
            self.consecutive = 0
```

listing 558: Python, allow raises CircuitOpen, report reopens or closes on the probe outcome (source: patterns-concurrency-distributed/samples-py/src/Ch15/breaker.py, lines 28-68)

```lua
function Breaker:Allow()
  if self.state == "closed" then
    return true
  end
  if self.state == "open" then
    if self.now() - self.openedAt ≥ self.resetDelayMs then
      self.state = "half-open"
      self.probing = true
      return true
    end
    return false, "circuit open"
  end
  -- half-open: exactly one probe in flight
  if self.probing then
    return false, "circuit open" -- the probe already claimed the slot
  end
  self.probing = true
  return true
end

-- Report settles the outcome of an allowed call
function Breaker:Report(err)
  if self.state == "half-open" then
    if err then
      self.state = "open"
      self.openedAt = self.now() -- fresh delay from the failed probe
    else
      self.state = "closed"
      self.consecutive = 0
    end
    self.probing = false
    return
  end
  if err then
    self.consecutive = self.consecutive + 1
    if self.consecutive ≥ self.failThreshold then
      self.state = "open"
      self.openedAt = self.now()
    end
  else
    self.consecutive = 0
  end
end
```

listing 559: Lua, allow returns false with a message, the walk is scripted instants (source: patterns-concurrency-distributed/samples-lua/ch15_breaker.lua, lines 26-68)

Every transition in that walk is an assertion, and none of them took real time. The state itself is a string in go and lua, an enum in C, rust, and java, and the machine's edge conditions are identical: the delay comparison is ≥, the half-open slot is claimed before the call, and a failed probe stamps a fresh `openedAt`. The threading row is quiet here, a mutex in go and C# around fields that the scripted walks in C, rust, java, javascript, python, and lua touch one step at a time.

The design decision a breaker forces: failure threshold and reset delay are SLA statements. A threshold of 3 with a 5 second delay says "3 failures in a row is a dependency outage, I will check back every 5 seconds", and tuning those numbers to the dependency's reality is the engineering.

15.3. rate limiting with a token bucket

The token bucket holds `burst` tokens and refills at `rate` per second, allowing short bursts over the average rate, which is what makes it the default limiter shape. The refill is lazy, computed from elapsed time on each take, which is the property that keeps the limiter correct through idle periods with zero background cost, no ticker anywhere in any of the 8 trees.

The dry run: rate 10 per second, burst 4, the clock advanced by hand.

- takes 1 through 4 succeed, the 5th is refused
- +100ms refills exactly 1 token, another 100ms one more, a long idle caps at burst
- WaitTake from empty at rate 2 per second: exactly one 500ms wait
- the go lane's frozen test runs the same walk at burst 3, both verified against the same implementation

```c
static void tb_init(token_bucket *tb, double rate, double burst, long *now_ms) {
  tb->rate = rate;
  tb->burst = burst;
  tb->now_ms = now_ms;
  tb->tokens = burst;
  tb->last_fill = *now_ms;
}

static void tb_refill(token_bucket *tb) {
  long elapsed = *tb->now_ms - tb->last_fill;
  tb->tokens += (double)elapsed * tb->rate / 1000.0;
  if (tb->tokens > tb->burst)
    tb->tokens = tb->burst; // refill caps at burst
  tb->last_fill = *tb->now_ms;
}

// draw one token if available
static bool tb_try_take(token_bucket *tb) {
  tb_refill(tb);
  if (tb->tokens ≥ 1.0) {
    tb->tokens -= 1.0;
    return true;
  }
  return false;
}

// block until a token is available; the injected sleep advances the clock
static bool tb_wait_take(token_bucket *tb, vclock *ck) {
  for (;;) {
    tb_refill(tb);
    if (tb->tokens ≥ 1.0) {
      tb->tokens -= 1.0;
      return true;
    }
    double deficit = 1.0 - tb->tokens;
    long wait = (long)(deficit / tb->rate * 1000.0);
    ck->waits[ck->n_waits++] = wait;
    clock_advance(ck, wait); // the sleep resolves through the clock
  }
}
```

listing 560: C, lazy refill on read in doubles, the wait derives from the deficit (source: patterns-concurrency-distributed/samples-c/src/Ch15/bucket.c, lines 35-74)

```
    fn new(rate: f64, burst: f64, now_ms: i64) -> TokenBucket {
        TokenBucket {
            tokens: burst,
            rate,
            burst,
            last_fill: now_ms,
        }
    }

    fn refill(&mut self, now_ms: i64) {
        let elapsed = now_ms - self.last_fill;
        self.tokens += elapsed as f64 * self.rate / 1000.0;
        if self.tokens > self.burst {
            self.tokens = self.burst;
        }
        self.last_fill = now_ms;
    }

    fn try_take(&mut self, now_ms: i64) -> bool {
        self.refill(now_ms);
        if self.tokens ≥ 1.0 {
            self.tokens -= 1.0;
            return true;
        }
        false
    }

    fn wait_take(&mut self, ck: &mut VClock) -> bool {
        loop {
            self.refill(ck.now_ms);
            if self.tokens ≥ 1.0 {
                self.tokens -= 1.0;
                return true;
            }
            let deficit = 1.0 - self.tokens;
            let wait = (deficit / self.rate * 1000.0) as i64;
            ck.waits.push(wait);
            ck.advance(wait);
        }
    }
}
```

listing 561: Rust, lazy refill on read in f64 capped at burst, `now_ms` handed in by value, the wait derived from the deficit (source: patterns-concurrency-distributed/samples-rust/src/Ch15/bucket.rs, lines 41-81)

```
type TokenBucket struct {
  mu       sync.Mutex
  tokens   float64
  rate     float64 // tokens per second
  burst    float64
  now      func() time.Time
  lastFill time.Time
}

func NewTokenBucket(rate, burst float64, now func() time.Time) *TokenBucket {
  if now == nil {
    now = time.Now
  }
  return &TokenBucket{tokens: burst, rate: rate, burst: burst, now: now, lastFill: now()}
}

func (tb *TokenBucket) refill() {
  now := tb.now()
  elapsed := now.Sub(tb.lastFill).Seconds()
  tb.tokens += elapsed * tb.rate
  if tb.tokens > tb.burst {
    tb.tokens = tb.burst
  }
  tb.lastFill = now
}

// TryTake draws one token if available.
func (tb *TokenBucket) TryTake() bool {
  tb.mu.Lock()
  defer tb.mu.Unlock()
  tb.refill()
  if tb.tokens ≥ 1 {
    tb.tokens--
    return true
  }
  return false
}

// WaitTake blocks until a token is available; sleep is injected for
// the deterministic tests.
func (tb *TokenBucket) WaitTake(sleep func(time.Duration)) bool {
  for {
    tb.mu.Lock()
    tb.refill()
    if tb.tokens ≥ 1 {
      tb.tokens--
      tb.mu.Unlock()
      return true
    }
    deficit := 1 - tb.tokens
    wait := time.Duration(deficit / tb.rate * float64(time.Second))
    tb.mu.Unlock()
    sleep(wait)
  }
}
```

listing 562: Go, lazy refill on read, burst cap, wait-take computes the deficit delay (source: patterns-concurrency-distributed/samples/ch15/resilience.go, lines 141-195)

```
  static final class TokenBucket {
    double tokens;
    final double rate;  // tokens per second
    final double burst; // capacity
    final VClock clock;
    long lastFill;

    TokenBucket(double rate, double burst, VClock clock) {
      this.rate = rate;
      this.burst = burst;
      this.clock = clock;
      this.tokens = burst;
      this.lastFill = clock.nowMs;
    }

    void refill() {
      long elapsed = clock.nowMs - lastFill;
      tokens += (double) elapsed * rate / 1000.0;
      if (tokens > burst) tokens = burst; // refill caps at burst
      lastFill = clock.nowMs;
    }

    // draw one token if available
    boolean tryTake() {
      refill();
      if (tokens ≥ 1.0) {
        tokens -= 1.0;
        return true;
      }
      return false;
    }

    // block until a token is available; the injected sleep records the
    // computed wait and advances the clock, the sleep resolving through
    // the clock like C's injected clock_advance
    boolean waitTake() {
      while (true) {
        refill();
        if (tokens ≥ 1.0) {
          tokens -= 1.0;
          return true;
        }
        double deficit = 1.0 - tokens;
        long wait = (long) (deficit / rate * 1000.0);
        clock.waits.add(wait);
        clock.advance(wait);
      }
    }
  }
```

listing 563: Java, lazy refill in doubles capped at burst, waitTake computes the deficit delay and pushes the clock through it (source: patterns-concurrency-distributed/samples-java/src/Ch15/Bucket.java, lines 29-77)

```
    private void RefillLocked()
    {
        var now = _time.GetUtcNow();
        double elapsed = (now - _lastFill).TotalSeconds;
        _tokens += elapsed * _rate;
        if (_tokens > _burst)
        {
            _tokens = _burst;
        }
        _lastFill = now;
    }

    public bool TryTake()
    {
        lock (_gate)
        {
            RefillLocked();
            if (_tokens ≥ 1)
            {
                _tokens--;
                return true;
            }
            return false;
        }
    }

    public bool WaitTake(Action<TimeSpan> sleep)
    {
        while (true)
        {
            TimeSpan wait;
            lock (_gate)
            {
                RefillLocked();
                if (_tokens ≥ 1)
                {
                    _tokens--;
                    return true;
                }
                double deficit = 1 - _tokens;
                wait = TimeSpan.FromSeconds(deficit / _rate);
            }
            sleep(wait);
        }
    }
}
```

listing 564: C#, refill from TimeProvider elapsed, WaitTake sleeps the computed wait through the delegate (source: patterns-concurrency-distributed/samples-cs/src/Ch15/Bucket.cs, lines 25-70)

```
export class TokenBucket {
  #tokens;
  #rate;
  #burst;
  #now;
  #lastFill;

  constructor(rate, burst, now) {
    this.#rate = rate;
    this.#burst = burst;
    this.#now = now;
    this.#lastFill = now();
    this.#tokens = burst;
  }

  #refill() {
    const t = this.#now();
    const elapsed = (t - this.#lastFill) / 1000;
    this.#tokens = Math.min(this.#burst, this.#tokens + elapsed * this.#rate);
    this.#lastFill = t;
  }

  // draw one token if available
  tryTake() {
    this.#refill();
    if (this.#tokens ≥ 1) {
      this.#tokens--;
      return true;
    }
    return false;
  }

  // wait until a token is available; sleep is injected
  async waitTake(sleep) {
    for (;;) {
      this.#refill();
      if (this.#tokens ≥ 1) {
        this.#tokens--;
        return true;
      }
      const deficit = 1 - this.#tokens;
      const wait = Math.round((deficit / this.#rate) * 1000);
      await sleep(wait);
    }
  }
}
```

listing 565: JavaScript, elapsed seconds feed the refill, Math.min caps at burst (source: patterns-concurrency-distributed/samples-js/src/ch15-bucket.mjs, lines 6-51)

```
class TokenBucket:
    def __init__(self, rate, burst, clock):
        self.rate = rate
        self.burst = float(burst)
        self.clock = clock
        self.tokens = float(burst)
        self.last_fill = clock.now

    def _refill(self):
        elapsed = self.clock.now - self.last_fill
        self.tokens += elapsed * self.rate
        if self.tokens > self.burst:
            self.tokens = self.burst
        self.last_fill = self.clock.now

    def try_take(self):
        self._refill()
        if self.tokens ≥ 1:
            self.tokens -= 1
            return True
        return False

    def wait_take(self, sleep):
        while True:
            self._refill()
            if self.tokens ≥ 1:
                self.tokens -= 1
                return True
            deficit = 1 - self.tokens
            sleep(deficit / self.rate)
```

listing 566: Python, the clock object advances, the sleep callback advances it further (source: patterns-concurrency-distributed/samples-py/src/Ch15/bucket.py, lines 23-52)

```lua
local TokenBucket = {}
TokenBucket.__index = TokenBucket

function TokenBucket.new(rate, burst, now)
  return setmetatable({
    tokens = burst, rate = rate, burst = burst,
    now = now, lastFillMs = now(),
  }, TokenBucket)
end

function TokenBucket:refill()
  local nowMs = self.now()
  local elapsed = (nowMs - self.lastFillMs) / 1000.0
  self.tokens = self.tokens + elapsed * self.rate
  if self.tokens > self.burst then
    self.tokens = self.burst
  end
  self.lastFillMs = nowMs
end

-- TryTake draws one token if a full one is available
function TokenBucket:TryTake()
  self:refill()
  if self.tokens ≥ 1 then
    self.tokens = self.tokens - 1
    return true
  end
  return false
end

-- WaitTake blocks until a token is available. the sleep is injected
-- and the test advances the clock inside it, so the wait is the walk
function TokenBucket:WaitTake(sleep)
  while true do
    self:refill()
    if self.tokens ≥ 1 then
      self.tokens = self.tokens - 1
      return true
    end
    local deficit = 1 - self.tokens
    sleep(deficit / self.rate * 1000.0)
  end
end
```

listing 567: Lua, floats appear exactly where the arithmetic needs them, elapsed seconds times rate (source: patterns-concurrency-distributed/samples-lua/ch15_bucket.lua, lines 10-52)

The arithmetic is float in all eight lanes and lands on the same numbers because it is the same expression, elapsed seconds times rate, capped at burst, with the wait derived as deficit over rate. `WaitTake` loops: refill, draw if a full token is there, otherwise sleep exactly the deficit delay and come back. The injected sleep is what turns the blocking path into part of the walk, the fake clock advances inside the sleep so the next refill sees the time the caller just waited.

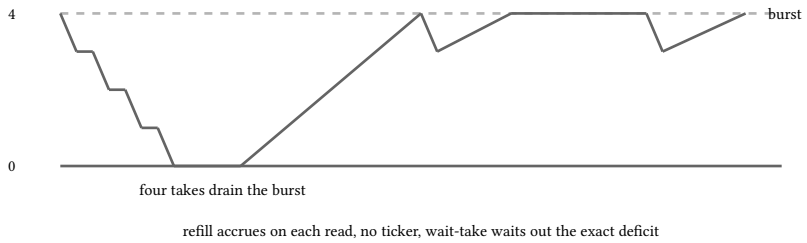


figure 83: token level over time: burst drain, lazy refill on read, cap at burst

15.4. bulkhead and fallback

A bulkhead caps concurrency to a dependency so saturation there cannot consume every worker here. The shape is chapter 6′s counting semaphore with a deadline: entry either finds a slot or expires into a full error, `Leave` frees one, and the expiry timer runs on the chapter's injected clock, so a full bulkhead fails in zero real time.

The dry run: capacity 2, the wait a virtual 20ms.

- entries 1 and 2 take the slots, the third expires into the full error with the clock at 20ms, `Leave` frees a slot and the next entry succeeds without burning any time
- a slot freed mid-wait admits the waiter early, pinned in the java, javascript, python, and lua lanes
- fallback: a broken dependency answers with the degraded string, a healthy one passes the primary through

```c
static void bh_init(bulkhead *b, int capacity, vclock *ck) {
  b->capacity = capacity;
  b->used = 0;
  b->ck = ck;
}

// take a slot, or let the wait expire through the clock and report full
static bool bh_try_enter(bulkhead *b, long wait_ms) {
  if (b->used < b->capacity) {
    b->used++;
    return true;
  }
  b->ck->now_ms += wait_ms; // the timer burns down, then gives up
  return false;
}

static void bh_leave(bulkhead *b) { b->used--; }

static int bh_in_use(const bulkhead *b) { return b->used; }

// the fetch the bulkhead guards: it only runs when a slot was taken
typedef struct {
  bulkhead *b;
  bool broken;
  int calls;
} service;

static bool service_call(service *s, const char **out) {
  if (!bh_try_enter(s->b, 20)) {
    *out = NULL;
    return false; // full: the dependency was never touched
  }
  s->calls++;
  bh_leave(s->b);
  if (s->broken) {
    *out = NULL;
    return false;
  }
  *out = "primary";
  return true;
}

// fallback wraps a call with a degraded answer for when it fails
static const char *fallback(service *s, const char *degraded) {
  const char *v = NULL;
  if (service_call(s, &v))
    return v;
  return degraded;
}
```

listing 568: C, a counter and a clock burn, the guarded service only runs on a taken slot (source: patterns-concurrency-distributed/samples-c/src/Ch15/bulkhead.c, lines 31-79)

```rust
impl Bulkhead {
    fn new(capacity: usize) -> Bulkhead {
        Bulkhead { capacity, used: 0 }
    }

    fn try_enter(&mut self, ck: &mut VClock, wait_ms: i64) -> bool {
        if self.used < self.capacity {
            self.used += 1;
            return true;
        }
        ck.now_ms += wait_ms;
        false
    }

    fn leave(&mut self) {
        self.used -= 1;
    }

    fn in_use(&self) -> usize {
        self.used
    }
}

struct Service<'a> {
    bulkhead: &'a mut Bulkhead,
    broken: bool,
    calls: u32,
}

impl Service<'_> {
    fn call(&mut self, ck: &mut VClock) -> Option<&'static str> {
        if !self.bulkhead.try_enter(ck, 20) {
            return None;
        }
        self.calls += 1;
        self.bulkhead.leave();
        if self.broken {
            return None;
        }
        Some("primary")
    }

    fn fallback(&mut self, ck: &mut VClock, degraded: &'static str) -> &'static str {
        self.call(ck).unwrap_or(degraded)
    }
}
```

listing 569: Rust, a counter and a clock burn, the service borrows the bulkhead by `&mut`, fallback is `unwrap_or` on the Option (source: patterns-concurrency-distributed/samples-rust/src/Ch15/bulkhead.rs, lines 30-75)

```
type Bulkhead struct {
  slots chan struct{}
}

func NewBulkhead(capacity int) *Bulkhead {
  return &Bulkhead{slots: make(chan struct{}, capacity)}
}

var ErrFull = errors.New("bulkhead full")

func (b *Bulkhead) TryEnter(wait time.Duration) error {
  timer := time.NewTimer(wait)
  defer timer.Stop()
  select {
  case b.slots <- struct{}{}:
    return nil
  case <-timer.C:
    return ErrFull
  }
}

func (b *Bulkhead) Leave() { <-b.slots }

func (b *Bulkhead) InUse() int { return len(b.slots) }
```

listing 570: Go, buffered channel as a counting semaphore, timer-bounded entry (source: patterns-concurrency-distributed/samples/ch15/resilience.go, lines 199-222)

```
  static final class Head {
    final int capacity;
    int used;
    final Clock clock; // the expiry timer rides the clock
    final ArrayDeque<Waiter> waiters = new ArrayDeque<>();

    Head(int capacity, Clock clock) {
      this.capacity = capacity;
      this.clock = clock;
    }

    // take a slot, or let the wait expire through the clock and report
    // full: nothing frees it in this script, the whole wait burns down
    boolean tryEnter(long waitMs) {
      if (used < capacity) {
        used++;
        return true;
      }
      clock.advance(waitMs); // the timer burns down, then gives up
      return false;
    }

    // park with a wait budget: a leave() before the deadline admits it
    Waiter park(long waitMs) {
      Waiter w = new Waiter(clock.nowMs + waitMs);
      waiters.add(w);
      return w;
    }

    void leave() {
      used--;
      while (!waiters.isEmpty()) {
        Waiter w = waiters.poll();
        if (w.deadline > clock.nowMs) { // not expired: hand it the slot
          w.admitted = true;
          used++;
          break;
        }
      }
    }

    int inUse() {
      return used;
    }
  }
```

listing 571: Java, a count and a waiter deque, the expiry burns the clock, leave hands the slot to an unexpired parked waiter (source: patterns-concurrency-distributed/samples-java/src/Ch15/Bulkhead.java, lines 40-84)

```
// Bulkhead bounds in-flight work. Enter takes a slot or gives up
// after the wait, which is a Task.Delay on the injected clock: the
// expiry is virtual time, Leave releases.
public sealed class Bulkhead
{
    private readonly SemaphoreSlim _slots;
    private readonly int _capacity;

    public Bulkhead(int capacity)
    {
        _capacity = capacity;
        _slots = new SemaphoreSlim(capacity, capacity);
    }

    public int InUse => _capacity - _slots.CurrentCount;

    public async Task Enter(TimeProvider time, TimeSpan wait)
    {
        if (_slots.Wait(0))
        {
            return;
        }
        await Task.Delay(wait, time);
        if (_slots.Wait(0))
        {
            return;
        }
        throw new BulkheadFullException();
    }

    public void Leave() => _slots.Release();
}
```

listing 572: C#, SemaphoreSlim counts the slots, Task.Delay on the TimeProvider expires the wait (source: patterns-concurrency-distributed/samples-cs/src/Ch15/Bulkhead.cs, lines 11-42)

```
export class Bulkhead {
  #capacity;
  #inUse = 0;
  #waiters = [];

  constructor(capacity) {
    this.#capacity = capacity;
  }

  inUse() {
    return this.#inUse;
  }

  // true = slot taken, false = bulkhead full after waitMs of
  // virtual time
  async tryEnter(clock, waitMs) {
    const deadline = clock.now + waitMs;
    for (;;) {
      if (this.#inUse < this.#capacity) {
        this.#inUse++;
        return true;
      }
      const remaining = deadline - clock.now;
      if (remaining ≤ 0) return false;
      const waiter = { live: true, resolve: () => {} };
      this.#waiters.push(waiter);
      const slotFree = new Promise((resolve) => {
        waiter.resolve = resolve;
      });
      const expired = clock.sleep(remaining).then(() => {
        waiter.live = false;
        return false;
      });
      await Promise.race([slotFree.then(() => true), expired]);
      // loop: re-check the count and the deadline
    }
  }

  leave() {
    this.#inUse--;
    while (this.#waiters.length > 0) {
      const w = this.#waiters.shift();
      if (w.live) {
        w.resolve();
        break;
      }
    }
  }
}
```

listing 573: JavaScript, the wait races slot against timer, a woken waiter re-checks the count (source: patterns-concurrency-distributed/samples-js/src/ch15-bulkhead.mjs, lines 7-55)

```
class Bulkhead:
    def __init__(self, capacity):
        self.capacity = capacity
        self.used = 0
        self.waiters = []

    def in_use(self):
        return self.used

    async def try_enter(self, clock, wait_ms):
        if self.used < self.capacity:
            self.used += 1
            return True
        slot = asyncio.Event()
        self.waiters.append(slot)
        expiry = clock.timer(wait_ms)
        # index 0 is the slot, index 1 the deadline
        won = await _first_between(slot.wait(), expiry.wait())
        self.waiters.remove(slot)
        if won == 0:
            self.used += 1
            return True
        return False

    def leave(self):
        self.used -= 1
        if self.waiters:
            self.waiters[0].set()


async def _first_between(*waits):
    tasks = [asyncio.ensure_future(w) for w in waits]
    done, pending = await asyncio.wait(tasks, return_when=asyncio.FIRST_COMPLETED)
    for t in pending:
        t.cancel()
    for i, t in enumerate(tasks):
        if t in done:
            return i
    return None


def fallback(fn, degraded):
    def call():
        value, err = fn()
        if err is not None:
            return degraded, None
        return value, None

    return call
```

listing 574: Python, an Event race through the timer heap, fallback wraps the pair (source: patterns-concurrency-distributed/samples-py/src/Ch15/bulkhead.py, lines 58-107)

```lua
local Bulkhead = {}
Bulkhead.__index = Bulkhead

function Bulkhead.new(C, capacity)
  return setmetatable({ C = C, cap = capacity, inUse = 0 }, Bulkhead)
end

-- TryEnter takes a slot, or waits once for waitMs of virtual time and
-- retries: a slot freed during the wait is taken, otherwise full
function Bulkhead:TryEnter(waitMs)
  while self.inUse == self.cap do
    vsleep(self.C, waitMs)
    if self.inUse == self.cap then
      return false, "bulkhead full"
    end
  end
  self.inUse = self.inUse + 1
  return true
end

function Bulkhead:Leave()
  self.inUse = self.inUse - 1
end

function Bulkhead:InUse()
  return self.inUse
end

-- Fallback wraps f with a degraded answer for when f fails
local function Fallback(f, degraded)
  return function()
    local ok, v = pcall(f)
    if not ok then
      return degraded
    end
    return v
  end
end
```

listing 575: Lua, one virtual sleep then a re-check, fallback degrades on error (source: patterns-concurrency-distributed/samples-lua/ch15_bulkhead.lua, lines 57-94)

The slot counter is the easy half, a buffered channel in go, a `SemaphoreSlim` in C#, a plain count in the scripted lanes. The wait is the interesting half: go races the channel send against a timer through `select`, C# awaits `Task.Delay` on the injected provider, javascript races a slot promise against the fake clock's sleep promise and re-checks the count after the wake so a stolen slot cannot overfill the hold, python races an `asyncio.Event` against the timer heap, lua sleeps once virtually and re-checks, C and rust run single-threaded and just burn the clock, and java scripts the rescue without threads at all, a parked waiter deque where `leave` hands the slot straight to a waiter whose deadline has not passed, the admit pinned at the release instant. Fallback completes the set as the smallest tool in the chapter, a degraded answer when the real one fails, stale cache over error page, and its test is 2 lines because the pattern is 2 lines.

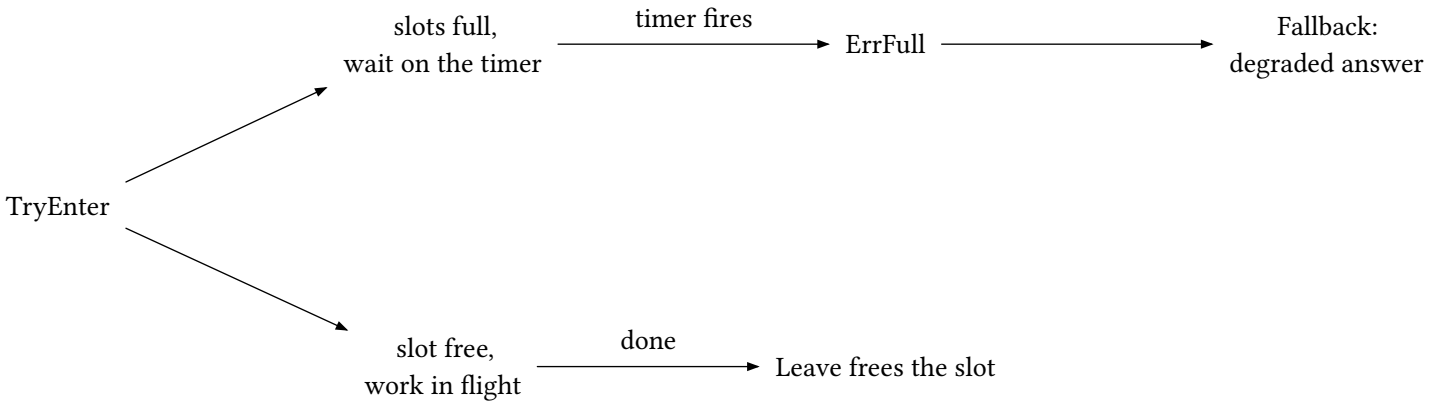


figure 84: a full bulkhead fails fast, the fallback answers degraded

PITFALL: retry storms compose

Every layer adding retries multiplies the load multiplier: 3 retries at the client times 3 at the proxy times 3 at the service is up to 27 attempts per user action, aimed at whatever is already struggling. The discipline: one layer owns aggressive retries, the others fail fast and let the breaker decide. Timeout budgets keep the same rule, the sum of nested timeouts must shrink, never grow, toward the leaf.

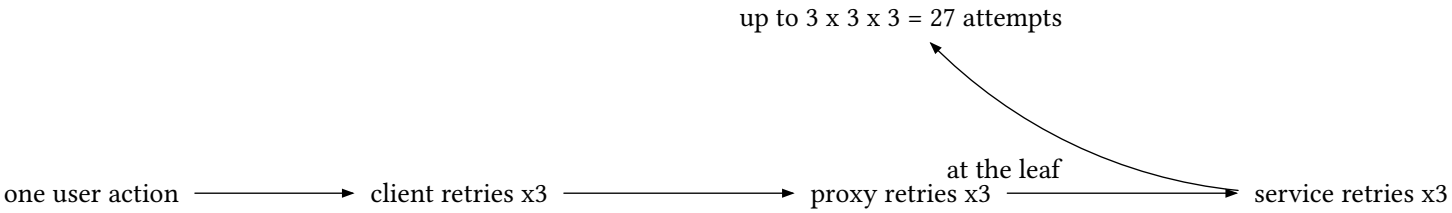


figure 85: retries multiply down the stack, one layer owns aggression

15.5. across the eight languages

The build sizes count non-comment source lines, the go column the chapter's one frozen file:

language	build SLOC	dependency	boundary note
c	459	libc	single-threaded, timers through a virtual clock, the bulkhead just burns it
rust	557	stdlib	an enum jitter strategy, the clock passed by value into every breaker and bucket call, sleeps recorded in a Vec
go	188	stdlib	channel semaphore raced by select, rand.Int64N the jitter, timer-bounded entry
java	462	jdk 27 stdlib	every timer through a hand-in vclock, the bulkhead rescue scripted without threads
c#	232	bcl	SemaphoreSlim counts the slots, Task.Delay expires on the injected TimeProvider
javascript	207	node stdlib	slot promise raced against the fake clock's sleep, a woken waiter re-checks
python	389	stdlib only	asyncio.Event raced against the timer heap, heapq drives every deadline
lua	458	lib.lua harness	one virtual sleep then a re-check, fallback degrades on the error return

sources: AWS Architecture Blog, "Exponential Backoff and Jitter", for full jitter against synchronized retries, Nygard, "Release It!", for circuit breakers and bulkheads, go.dev/pkg/time for the timer semantics used by the

bulkhead wait, accessed 2026-09-08. Verified by the eight chapter legs: 4 Ch15 C programs with 90 embedded checks, the rust runner's 90 checks across 4 programs in `samples-rust/src/Ch15`, `go test` at 8 tests in `patternsbook/ch15`, the java runner's 90 checks across 4 programs in `samples-java/src/Ch15`, 9 xunit facts, node's 9 cases in `test/ch15.test.mjs`, 4 python modules with 60 embedded checks, and the lua runner's 17 ch15 rows.

16. capstone: raft replicated configuration service

The capstone is raft itself, small enough to hold, complete enough to be real: elections with randomized timeouts, log replication with the consistency check, commitment with the prior-term restriction, persistence across restarts, and a configuration service on top that hides leadership changes behind retries. It lives at `patterns-concurrency-distributed/capstone/`, one module, two packages, nineteen tests, zero skipped, every file under 370 lines.

16.1. the shape

Every node is chapter 8's pattern in miniature: one goroutine owning all state, consuming one inbox channel, no locks anywhere in the consensus code. The transport is a hub of inbox channels with a drop predicate standing in for the network. And the clock is explicit: elections fire on tick counts, never on wall time, so a test advances the world one round at a time and every interleaving is a choice the test made.

```
type Node struct {
  id   int
  hub  *Hub
  dir  string
  done chan struct{}

  currentTerm uint64
  votedFor    int // -1 = none this term
  log         []Entry
  commitIndex int
  lastApplied int

  role     Role
  elapsed  int // ticks since last leader contact
  timeout  int // randomized tick budget
  leaderID int // -1 unknown

  votes       map[int]bool
  nextIndex   []int
  matchIndex  []int
  sentThrough []int // leader: highest index sent per peer, for match bookkeeping
  pending     map[int]chan ClientReply

  applied map[string]string
}
```

listing 576: the whole node: persistent triple, volatile state, leader bookkeeping (source: patterns-concurrency-distributed/capstone/raft/node.go, lines 8-32)

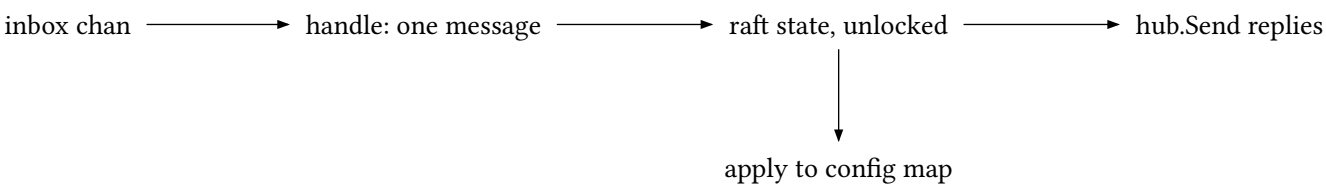


figure 86: one node's event loop

The hub's one subtlety earned its comment the honest way, by panicking during development: delivery is serialized against teardown under the hub's mutex, so a send can never land on a closed channel when a node restarts. And the partition predicate applies only to the four raft rpcs, ticks are a node's own clock, probes are the harness looking in, client requests arrive at the node they address. The first version dropped ticks addressed through a partitioned link and the isolated node's election timer froze, invisible until a test asserted its term never grew.

```
// Send delivers unless the partition predicate eats the message or
// the target is torn down. Ticks, probes, and client requests are
// local or harness traffic and bypass the partition, a tick is a
// node's own clock, a probe is the test harness looking in, a client
// request arrives at the node it was sent to. Only the four raft
// rpcs cross the network and feel partitions. The mutex serializes
// delivery against teardown, so a send can never land on a closed
// channel.
func (h *Hub) Send(m Msg) {
  h.mu.Lock()
  defer h.mu.Unlock()
  if h.closed[m.To] {
    return
  }
  local := m.Tick ≠ nil || m.Probe ≠ nil || m.Client ≠ nil
  if !local && h.drop ≠ nil && h.drop(m.From, m.To) {
    return
  }
  h.inboxes[m.To] ← m
```

listing 577: drop only what crosses the network (source: patterns-concurrency-distributed/capstone/raft/hub.go, lines 29-48)

16.2. elections

Chapter 11's rules, wired to state:

```
func (n *Node) onTick() {
  if n.role == Leader {
    n.broadcastAppend()
    return
  }
  n.elapsed++
  if n.elapsed ≥ n.timeout {
    n.startElection()
  }
}

func (n *Node) startElection() {
  n.role = Candidate
  n.currentTerm++
  n.votedFor = n.id
  n.leaderID = -1
  n.elapsed = 0
  n.persist()
  n.votes = map[int]bool{n.id: true}
  if len(n.votes) ≥ n.majority() {
    n.becomeLeader()
    return
  }
  for peer := 0; peer < n.hub.Size(); peer++ {
    if peer == n.id {
      continue
    }
    n.hub.Send(n.voteRequestMsg(peer))
  }
```

```
}

func (n *Node) voteRequestMsg(peer int) Msg {
  lastTerm := uint64(0)
  if len(n.log) > 0 {
    lastTerm = n.log[len(n.log)-1].Term
  }
  return Msg{To: peer, From: n.id, Vote: &VoteRequest{
    Term: n.currentTerm, CandidateID: n.id,
    LastLogIndex: len(n.log), LastLogTerm: lastTerm,
  }}
}

// onVote is the receiver side including the 5.4.1 restriction.
func (n *Node) onVote(from int, req *VoteRequest) {
  if req.Term > n.currentTerm {
    n.stepDown(req.Term)
  }
  grant := false
  if req.Term == n.currentTerm && (n.votedFor == -1 || n.votedFor == req.CandidateID) {
    if n.logUpToDate(req.LastLogTerm, req.LastLogIndex) {
      grant = true
      n.votedFor = req.CandidateID
      n.elapsed = 0
      n.persist()
    }
  }
  n.hub.Send(Msg{To: from, From: n.id, VoteReply: &VoteReply{
    Term: n.currentTerm, VoteGranted: grant,
  }})
}

// logUpToDate is ch11's UpToDate rule against my own log.
func (n *Node) logUpToDate(candLastTerm uint64, candLastIndex int) bool {
  myLastTerm := uint64(0)
  if len(n.log) > 0 {
    myLastTerm = n.log[len(n.log)-1].Term
  }
  if candLastTerm ≠ myLastTerm {
    return candLastTerm > myLastTerm
  }
  return candLastIndex ≥ len(n.log)
}

func (n *Node) onVoteReply(from int, reply *VoteReply) {
  if reply.Term > n.currentTerm {
    n.stepDown(reply.Term)
    return
  }
  if n.role ≠ Candidate || reply.Term ≠ n.currentTerm || !reply.VoteGranted {
    return
  }
  n.votes[from] = true
  if len(n.votes) ≥ n.majority() {
    n.becomeLeader()
  }
}
```

listing 578: tick counting, candidacy, the 5.4.1 restriction on granting votes (source: patterns-concurrency-distributed/capstone/raft/node.go, lines 136-221)

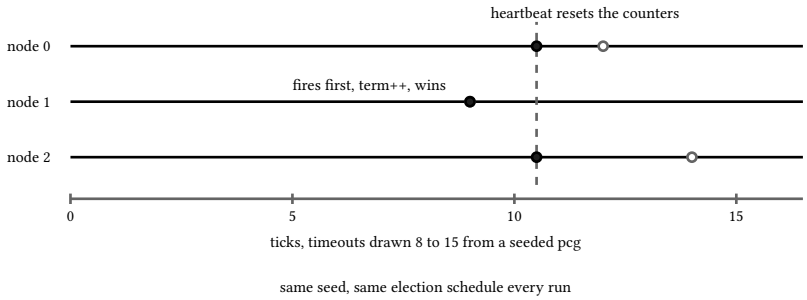


figure 87: three tick counters, randomized timeouts, someone fires first and resets everyone

Each node's timeout is 8 to 15 ticks drawn from a seeded PCG, so split votes resolve because someone's timer fires first, and the same seed means the same schedule every run. `onVote` is the paper's figure 2 condition written down: step down on a higher term, then grant only if the vote is unused this term and the candidate's log is at least as up-to-date, larger last term wins, equal terms go to length.

16.3. replication and commitment

The follower path is ch11′s pure `Append` with the persistence and commit clamps attached:

```
func (n *Node) onAppend(from int, req *AppendRequest) {
  if req.Term < n.currentTerm {
    n.hub.Send(Msg{To: from, From: n.id, AppendReply: &AppendReply{
      Term: n.currentTerm, Success: false, ConflictIndex: len(n.log) + 1,
    }})
    return
  }
  if req.Term > n.currentTerm {
    n.currentTerm = req.Term
    n.votedFor = -1
  }
  n.role = Follower
  n.leaderID = req.LeaderID
  n.elapsed = 0
  n.persist()

  // consistency check on prev
  if req.PrevLogIndex > len(n.log) ||
    (req.PrevLogIndex > 0 && n.log[req.PrevLogIndex-1].Term ≠ req.PrevLogTerm) {
    n.hub.Send(Msg{To: from, From: n.id, AppendReply: &AppendReply{
      Term: n.currentTerm, Success: false, ConflictIndex: len(n.log) + 1,
    }})
    return
  }

  // merge entries: skip matching, truncate conflicts, extend
  out := n.log
  for i, e := range req.Entries {
    idx := req.PrevLogIndex + i + 1
    if idx <= len(out) {
      if out[idx-1].Term ≠ e.Term {
        out = append(out[:idx-1], req.Entries[i:]...)
        break
      }
      continue
    }
    out = append(out, req.Entries[i:]...)
    break
  }
  n.log = out
  n.persist()

  if req.LeaderCommit > n.commitIndex {
    upto := req.LeaderCommit
    if upto > len(n.log) {
      upto = len(n.log)
    }
    n.commitIndex = upto
    n.applyCommitted()
  }

  n.hub.Send(Msg{To: from, From: n.id, AppendReply: &AppendReply{
    Term: n.currentTerm, Success: true,
  }})
}
```

listing 579: prev check, conflict truncation, commit clamped to my log (source: patterns-concurrency-distributed/capstone/raft/node.go, lines 253-307)

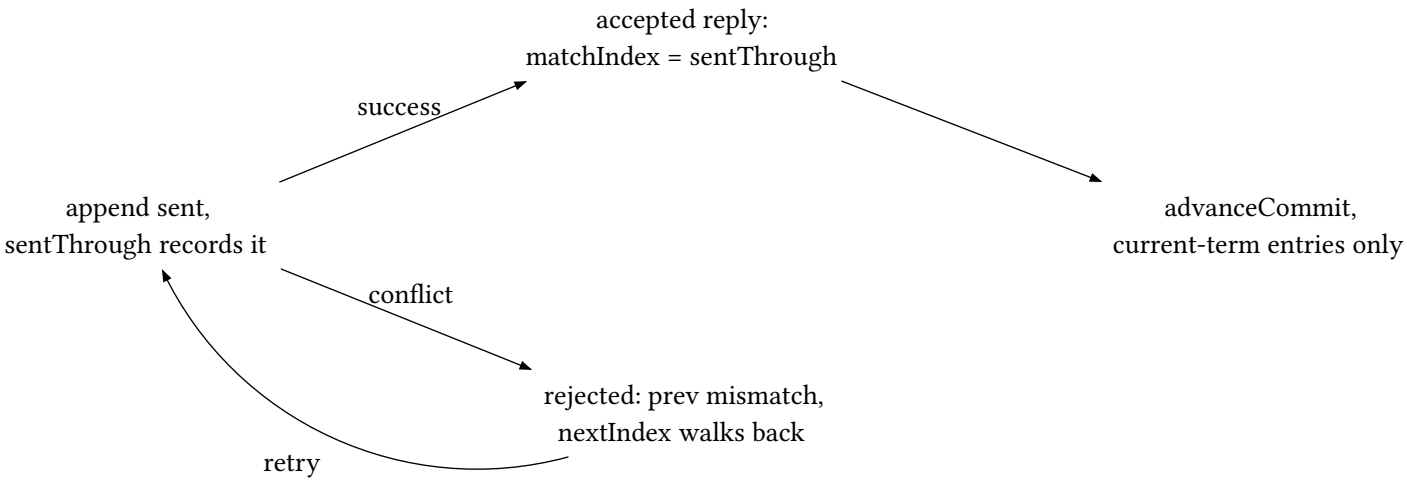


figure 88: leader bookkeeping: replies set matchindex to what was sent

The leader path tracks `nextIndex` and `matchIndex` per peer, with one addition beyond the paper's minimum, `sentThrough`, the highest index actually put on the wire per peer, because a success reply must set `matchIndex` to what was sent, not what the leader wishes was sent. Commitment counts replicas only for current-term entries and, on becoming leader, the node appends a no-op entry from its own term so prior-term entries ride its commitment, section 5.4.2 in two lines:

```
func (n *Node) advanceCommit() {
  for N := len(n.log); N > n.commitIndex; N-- {
    if n.log[N-1].Term ≠ n.currentTerm {
      continue
    }
    replicas := 1 // self
    for peer := 0; peer < len(n.matchIndex); peer++ {
      if peer ≠ n.id && n.matchIndex[peer] ≥ N {
        replicas++
      }
    }
    if replicas ≥ n.majority() {
      n.commitIndex = N
      n.applyCommitted()
      n.broadcastAppend() // spread leaderCommit promptly
      return
    }
  }
}
```

listing 580: the counting restriction, wired (source: patterns-concurrency-distributed/capstone/raft/leader.go, lines 70-88)

```
func (n *Node) becomeLeader() {
  n.role = Leader
  n.leaderID = n.id
  n.elapsed = 0
  size := n.hub.Size()
  n.nextIndex = make([]int, size)
  n.matchIndex = make([]int, size)
  n.sentThrough = make([]int, size)
  for i := range n.nextIndex {
    n.nextIndex[i] = len(n.log) + 1
  }
  // the immediate heartbeat claims leadership and resets every
  // other node's election timer
  n.broadcastAppend()
  // commit a no-op entry from the current term so prior-term
  // entries commit indirectly, section 5.4.2
  n.appendLocal(Entry{Term: n.currentTerm, Kind: "noop"})
}
```

listing 581: leadership claimed with a heartbeat and a no-op (source: patterns-concurrency-distributed/capstone/raft/node.go, lines 223-240)

Persistence is the paper's hard state, `currentTerm`, `votedFor`, the log, written to SQLite as one transaction per save over a wal journal at synchronous full, so a crash leaves the old state or the new, never half:

```
func saveHardState(dir string, id int, s hardState) error {
  if dir == "" {
    return nil
  }
  db, err := openStoreDB(dbPath(dir, id))
  if err ≠ nil {
    return err
  }
  defer db.Close()
  tx, err := db.Begin()
  if err ≠ nil {
    return err
  }
  defer tx.Rollback()
  // the log is rewritten whole: the in-memory log can truncate and
  // overwrite suffixes, so a snapshot in one transaction is the
  // honest mapping, never a divergent tail
  if _, err := tx.Exec(upsertKV, "current_term", s.CurrentTerm); err ≠ nil {
    return err
  }
  if _, err := tx.Exec(upsertKV, "voted_for", s.VotedFor); err ≠ nil {
    return err
  }
  if _, err := tx.Exec("delete from log"); err ≠ nil {
    return err
  }
  for i, e := range s.Log {
    if _, err := tx.Exec(
      "insert into log(idx, term, kind, key, val) values(?, ?, ?, ?, ?)",
      i+1, e.Term, e.Kind, e.Key, e.Val,
    ); err ≠ nil {
      return err
    }
  }
  return tx.Commit()
}
```

listing 582: one transaction per save: term, vote, and the whole log rewritten atomically (source: patterns-concurrency-distributed/capstone/raft/sqlite_store.go, lines 75-110)

The json store from the first draft survives beside it as the reference implementation, and a parity test cross-checks the two on every shape the log takes:

```
func saveState(dir string, id int, s hardState) error {
  if dir == "" {
    return nil
  }
  b, err := json.Marshal(s)
  if err ≠ nil {
    return err
  }
  final := statePath(dir, id)
  tmp := final + ".tmp"
  if err := os.WriteFile(tmp, b, 0o644); err ≠ nil {
    return err
  }
  return os.Rename(tmp, final)
}
```

listing 583: write temp, rename over: the reference store the parity test cross-checks (source: patterns-concurrency-distributed/capstone/raft/persist.go, lines 24-38)

16.4. the service

The configuration service wraps the cluster and routes writes to whoever leads. Its interesting logic is failure handling: an isolated stale leader neither refuses nor commits, it stalls, so a timeout is treated as a rejection, the clock advances to give the majority time to elect, and the write rides the new leader:

```
func (s *ConfigService) write(kind, key, val string) error {
  var lastErr error
  for round := 0; round < 2; round++ {
    for id := 0; id < s.size; id++ {
      var r raft.ClientReply
      if kind == "set" {
        r = s.cluster.Set(id, key, val)
      } else {
        r = s.cluster.Delete(id, key)
      }
      switch {
      case r.Err == nil:
        return nil
      case errors.Is(r.Err, raft.ErrNotLeader):
        lastErr = r.Err
        continue
      default: // timeout from a stalled leader
        lastErr = r.Err
      }
    }
    // nobody answered authoritatively: tick the world so a
    // majority can hold a fresh election: 16 exceeds the
    // 8..15 election timeout recipe, any pending timer fires
    s.cluster.TickN(16)
  }
  return errors.Join(raft.ErrNoLeader, lastErr)
}

// Get reads the committed configuration. Reads go to the leader
// first and fall back to followers, which serve their applied
```

listing 584: round the nodes, treat stalls as rejections, tick between rounds (source: patterns-concurrency-distributed/capstone/service/service.go, lines 35-64)

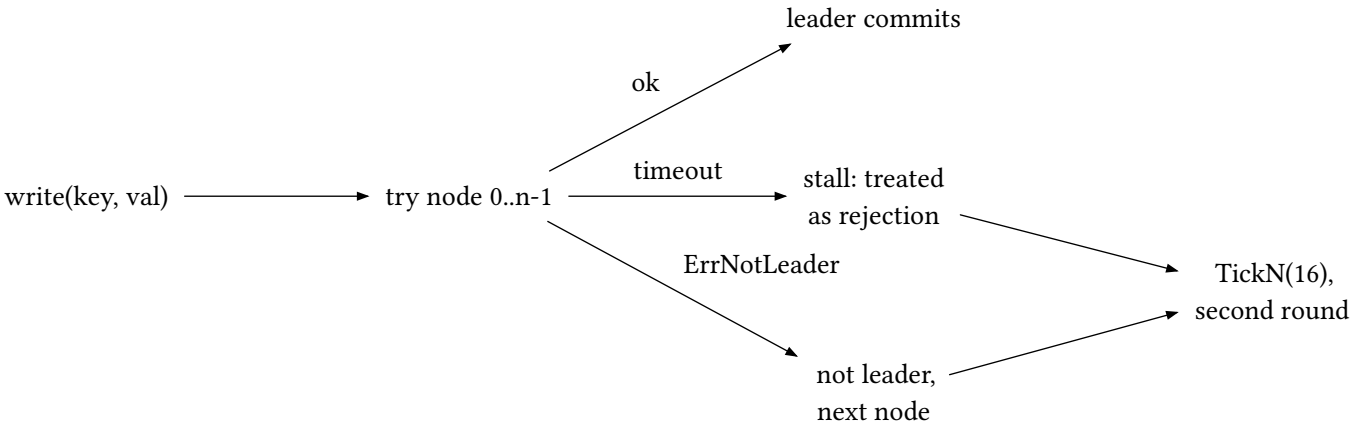


figure 89: stalls are rejections, the clock advances so a majority can elect

That loop is chapter 15's retry discipline applied to consensus, and the test for it is the capstone's centerpiece: elect a leader, commit a value, partition the leader away, write through the service anyway, and assert the write landed and the old leader resynchronized after healing. A follower rejecting a client write with `ErrNotLeader` is also pinned, the error the service loops on.

16.5. what the tests prove

The suite runs green through `make verify-go`, repeatedly, and under `go test -race`:

```
  // cut the leader away: nodes 0,1,2, leader isolated
  c.Partition(func(from, to int) bool {
    return from == lead || to == lead
  })

  // the majority side elects a new leader and commits
  newLead := c.TickUntilOtherLeader(lead, 300)
  if newLead < 0 {
    t.Fatal("majority side failed to elect")
  }
  if r := c.Set(newLead, "during", "2"); r.Err ≠ nil {
    t.Fatalf("set during partition: %v", r.Err)
  }

  // the isolated old leader cannot commit: no quorum
  old := c.Set(lead, "ghost", "x")
  if old.Err == nil {
    t.Fatal("isolated leader committed without quorum")
  }

  // heal: the old leader steps down for the higher term, catches
  // up, and the ghost entry it appended locally is overwritten
  c.Heal()
  newLead = c.TickUntilLeader(1, 300)
  if newLead < 0 {
    t.Fatal("no leader after heal")
  }
  c.Drain()

  for i := 0; i < 3; i++ {
    applied := c.Applied(i)
    if applied["during"] ≠ "2" {
      t.Fatalf("node %d applied %v, partition write missing", i, applied)
    }
    if _, ghost := applied["ghost"]; ghost {
      t.Fatalf("node %d applied the uncommitted ghost entry: %v", i, applied)
    }
  }
}
```

listing 585: the partition test: majority commits, minority stalls, ghost never applies (source: patterns-concurrency-distributed/capstone/raft/raft_test.go, lines 85-124)

the test	the property it pins
single node	elects and commits without peers
three node replication	same commit index, same applied map
follower write	rejected with ErrNotLeader
majority partition	ghost entry overwritten, never applied
lagging follower	catches up by heartbeat alone
restart	term, vote, log intact, no double vote
election restriction	stale log loses at a higher term
quiescent drain	everything settles in bounded time

figure 90: eight raft tests, eight pinned properties

Single-node election and commit. Three-node replication with every node reaching the same commit index and the same applied map. Follower write rejection. The majority-partition scenario, the minority's uncommitted "ghost"

entry overwritten rather than applied after healing, which is the log truncation path doing its job. A lagging follower catching up through heartbeats alone. Restart with term, vote, and log intact, the recovered node not voting twice in one term. The election restriction holding when a stale-log node arrives with a pumped-up term. And quiescent drain in bounded time.

The honest boundaries, stated in the code and here: membership is fixed at construction, the paper's joint consensus is a named door this book does not open. Snapshots do not exist, the log replays in full on restart, right at teaching sizes, wrong at millions of entries. Reads serve the leader's applied state first and fall back to followers, eventual but committed, and a linearizable read would need the read-index round trip the paper describes. The transport is in-memory, so the capstone proves the protocol, not the serialization. Each of those lines is where production raft, etcd and its kin, spends its next ten thousand lines.

sources: Ongaro and Ousterhout, raft.github.io/raft.pdf, figure 2 and sections 5.2 through 5.4.2, implemented and verified against go 1.27, accessed 2026-09-08. Verified by `make verify-go` plus `go test -race`, 15 raft-package tests (8 protocol, 7 store) and 4 service tests, zero skipped.

17. appendices: topic matrix and sources

The topic matrix is this book's audit trail: every pattern, primitive, and protocol the chapters build, the cost it carries, and the go standard library counterpart to reach for in production code. The six-voice rebuild gave chapters 1 to 15 five sibling trees, the java wave added the sixth, and the rust wave the seventh, every featured listing now exists once per language, c, rust, go, java, c#, javascript, python, and lua, built behind the same fixtures, while the raft capstone of chapter 16 and this appendix stay go-only by design. The build enforces the rest, every chapter listing reads a real file that its own pinned toolchain compiles and tests. The totals: c 64 files and 6755 checks, rust 64 files and 5952 checks, go 89 sample tests and 19 capstone tests, java 64 files and 703 checks, c# 178 tests, javascript 121 tests, python 64 files and 582 asserts, lua 271 checks, zero skipped anywhere, with the go concurrency chapters also held under `go test -race`.

17.1. topic coverage

Every topic maps to a chapter: 0 unmapped. The counterpart column carries the book's honest boundary, many rows say `none in bcl` because the go standard library genuinely ships no raft, no consistent hash ring, no circuit breaker, and knowing which problems it leaves to you is the practical knowledge.

topic	chapter	cost	go stdlib counterpart
structural interface satisfaction	ch 1	compile time, no runtime dispatch table	language feature
compile-time satisfaction proof	ch 1	one var, zero allocations	var _ Iface = (*T)(nil)
embedding as forwarding	ch 1	receiver stays inner, no override	language feature
null object	ch 1	a type with zero behavior	io.Discard, http.NoBody
functional options	ch 2	closure per option	grpc dial options, zap options
builder with joined errors	ch 2	issues collected to Build	errors.Join since 1.20
lazy singleton	ch 2	one evaluation	sync.OnceValue, OnceValues
prototype as clone	ch 2	reference fields re-copied by hand	none in bcl, slices.Clone for slices
flyweight interning	ch 2	interned forever	unique.Make since 1.24
adapter	ch 3	one wrapper, conversion owned	http.HandlerFunc
decorator as middleware	ch 3	ordering is behavior	net/http middleware chain
facade	ch 3	steps hidden, errors wrapped	stdlib api surfaces
composite	ch 3	recursive satisfaction	fs.FS trees, html Node
proxy with gate and memo	ch 3	policy caller cannot bypass	database/sql conn pool
strategy	ch 4	named function type	sort.Interface precedent
observer with channels	ch 4	drop on full buffer, by design	chan broadcast idioms
command with undo	ch 4	closure plus inverse	none in bcl
iterator, push style	ch 4	fuses instead of allocating	iter.Seq since 1.23
state transition table	ch 4	whole machine inspectable	none in bcl
visitor as type switch	ch 4	open on visitors, closed on variants	type switch
map, filter, reduce	ch 5	intermediate slices	slices package since 1.21

topic	chapter	cost	go stdlib counterpart
composition, pipe	ch 5	left to right	none in bcl
per-iteration loop variable	ch 5	per-module go directive	language, go 1.22
memoization	ch 5	memory for time, pure fns only	none in bcl, hand rolled
data race definition	ch 6	broken program, not timing	go test -race
happens-before edges	ch 6	the only currency	go.dev/ref/mem
counting semaphore channel	ch 6	capacity C bounds C workers	buffered chan idiom
atomic word ops	ch 6	single words only	sync/atomic types
mutex discipline	ch 7	composes invariants	sync.Mutex, RWMutex
once family	ch 7	one call, edge to all callers	OnceFunc, OnceValue, OnceValues
cond predicates	ch 7	recheck loop mandatory	sync.Cond
fused map ops	ch 7	read-mostly niches only	sync.Map LoadOrStore
cas loop	ch 7	livelock risk on contention	atomic.CompareAndSwap
waitgroup.Go	ch 7	add folded into go	WaitGroup.Go since 1.25
pool reuse	ch 7	cache, never a store	sync.Pool
generator and stages	ch 8	one goroutine per stage	go blog pipelines
fan-in merge	ch 8	forwarder per source	sync.WaitGroup close idiom
worker pool	ch 8	fixed concurrency	none in bcl, chan queue
bounded parallel ordered map	ch 8	slot per input	errgroup pattern
hedged first wins	ch 8	loser must tolerate no reader	select with timeout
goroutine leak detection	ch 8	poll and count	goroutineleak profile, 1.27
deadlock cycle rule	ch 9	ordering breaks cycles	runtime total-deadlock detector
rwmutex writer fairness	ch 9	no recursive read lock	sync.RWMutex doc guarantee
timeout guard in tests	ch 9	deadline per wait	context.WithTimeout kin
mutex contention profile	ch 9	sampled, near-free off	SetMutexProfileFraction
fake-clock bubble	ch 9	self-contained tests only	testing/synctest, 1.25
lamport clocks	ch 10	o(1), one-directional	none in bcl
vector clocks	ch 10	o(n) per message	none in bcl
quorum intersection	ch 10	minority must stall	none in bcl
cap choice per operation	ch 10	cp or ap, per op	none in bcl
raft leader election	ch 11	randomized timeouts, one vote per term	none in bcl
election restriction 5.4.1	ch 11	last term then length	none in bcl
appendentries consistency check	ch 11	prev pair, truncation on conflict	none in bcl
commit restriction 5.4.2	ch 11	current term only by count	none in bcl
consistent hash ring	ch 12	vnodes tune balance	none in bcl
rendezvous hashing	ch 12	o(n) hashes per lookup	none in bcl
jump consistent hash	ch 12	append-only buckets	none in bcl
placement hash finalizer	ch 12	fnv high bits do not avalanche	mix64 finalizer
at-least-once plus dedup	ch 13	exactly-once effect	none in bcl
transactional outbox	ch 13	relay and crash resume	none in bcl
per-key ordering lanes	ch 13	one lane per key	partitioned consumer

topic	chapter	cost	go stdlib counterpart
structured logging	ch 14	attrs at emit time	log/slog since 1.21
logvaluer redaction	ch 14	handler must Resolve	slog.LogValuer
expvar counters	ch 14	one line to /debug/vars	expvar.NewInt
context-propagated spans	ch 14	tree through ctx	none in bcl, otel
retry with full jitter	ch 15	attempts times spread	none in bcl, x/time
circuit breaker	ch 15	threshold plus reset delay	none in bcl
token bucket	ch 15	burst over average	golang.org/x/time/rate
bulkhead	ch 15	bounded in-flight	buffered chan semaphore
raft, complete	ch 16	elections, replication, commit, persistence	none in bcl
leader-transparent service	ch 16	stall treated as rejection	none in bcl

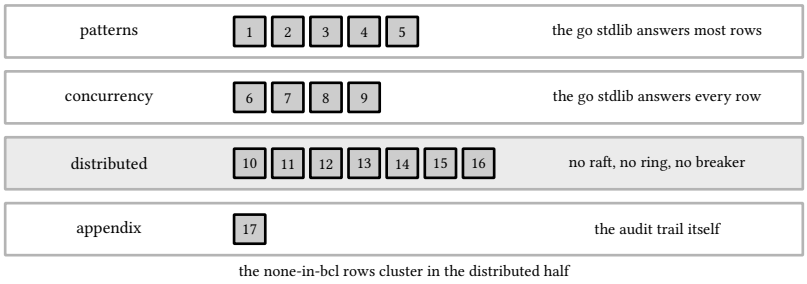


figure 91: seventeen chapters in clusters, and where the none-in-bcl boundary sits

17.2. pinned sources

Every chapter ends with its own sources line naming the pages consulted while writing it. The book level pins are:

topic	url	accessed
effective go, interfaces and embedding	https://go.dev/doc/effective_go	2026-09-08
go 1.27 release notes, generic methods, leak profile	https://go.dev/doc/go1.27	2026-09-08
sync package, Once family, Cond, Map, WaitGroup.Go	https://pkg.go.dev/sync	2026-09-08
unique package, handle interning	https://pkg.go.dev/unique	2026-09-08
the go memory model	https://go.dev/ref/mem	2026-09-08
iter package, Seq and Pull	https://pkg.go.dev/iter	2026-09-08
go 1.22 release notes, loop variable scoping	https://go.dev/doc/go1.22	2026-09-08
testing/synctest bubble	https://pkg.go.dev/testing/synctest	2026-09-08
race detector mechanics	https://go.dev/doc/articles/race_detector	2026-09-08
raft paper and site	https://raft.github.io/	2026-09-08
log/slog handler contract and LogValuer	https://pkg.go.dev/log/slog	2026-09-08
expvar package	https://pkg.go.dev/expvar	2026-09-08
runtime and runtime/pprof profiles	https://pkg.go.dev/runtime/pprof	2026-09-08
local runtime probes on go 1.27.0 windows/amd64	go doc verification of WaitGroup.Go, synctest, context.Cause, RWMutex fairness, and runtime probes	2026-09-08
net/http, HandlerFunc and middleware conventions	https://pkg.go.dev/net/http	2026-09-23
errors, Join and sentinel wrapping with Is	https://pkg.go.dev/errors	2026-09-23

topic	url	accessed
language specification, range over function, function literals	https://go.dev/ref/spec	2026-09-23
slices, SortStableFunc stability	https://pkg.go.dev/slices	2026-09-23
an introduction to generics, type parameter syntax	https://go.dev/blog/intro-generics	2026-09-23
go concurrency patterns, pipelines and cancellation	https://go.dev/blog/pipelines	2026-09-23
context, cancellation trees, Cause and AfterFunc	https://pkg.go.dev/context	2026-09-23
runtime, SetMutexProfileFraction	https://pkg.go.dev/runtime	2026-09-23
time, timer semantics for the bulkhead wait	https://pkg.go.dev/time	2026-09-23
lamport, logical clocks and one-directional ordering	lamport, time, clocks, and the ordering of events in a distributed system, cacm 21, 1978	2026-09-23
the fallacies of distributed computing	deutsch and others at sun, the eight fallacies of distributed computing	2026-09-23
latency numbers and the interactive latency guide	norvig's latency numbers and the berkeley interactive latency guide	2026-09-23
gilbert and lynch, brewer's conjecture and feasibility	gilbert and lynch, brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services, 2002	2026-09-23
ongaro's dissertation, the membership change reference	ongaro, consensus: bridging theory and practice, stanford phd dissertation, 2014	2026-09-23
karger et al., consistent hashing and random trees	karger et al., consistent hashing and random trees: strategies for relieving hot spots on the world wide web, stoc 1997	2026-09-23
thaler and ravishankar, name-based mapping, the hrw formulation, cse-tr-356-96, journal form ieee/acm transactions on networking 1998	https://www.microsoft.com/en-us/research/wp-content/uploads/2017/02/HRW98.pdf	2026-09-23
thatcher, rendezvous hashing explained	thatcher, rendezvous hashing explained, the walk-through	2026-09-23
lamping and veach, jump consistent hash	https://arxiv.org/abs/1406.2294	2026-09-23
heller, reliable pattern: outbox	heller, reliable pattern: outbox, the dual-write problem	2026-09-23
kleppmann, delivery semantics and exactly-once as effect	kleppmann, designing data-intensive applications, o'reilly 2017, ch 8	2026-09-23
aws architecture blog, exponential backoff and jitter	https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/	2026-09-23
nygard, circuit breakers and bulkheads	nygard, release it! design and deploy production-ready software, pragmatic bookshelf 2e, 2018	2026-09-23
c23 threads.h, thrd, mtx, cnd, call_once	https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3220.pdf	2026-09-23
stdatomic memory_order, the ordering ladder	https://en.cppreference.com/w/c/atomic/memory_order	2026-09-23
system.threading.channels	https://learn.microsoft.com/en-us/dotnet/api/system.threading.channels	2026-09-23

topic	url	accessed
timeprovider, injected time in tests	https://learn.microsoft.com/en-us/dotnet/api/system.timeprovider	2026-09-23
node worker_threads over sharedarraybuffer	https://nodejs.org/api/worker_threads.html	2026-09-23
node:test runner	https://nodejs.org/api/test.html	2026-09-23
python threading, locks and condition variables	https://docs.python.org/3.14/library/threading.html	2026-09-23
asyncio tasks, cancellation, and queues	https://docs.python.org/3/library/asyncio-task.html	2026-09-23
lua 5.5 manual, coroutines, section 2.6	https://www.lua.org/manual/5.5/manual.html#coroutines	2026-09-23

NOTE: version drift rule

The 2026-09-08 pins were verified against go 1.27.0 on windows/amd64, including local `go doc` probes of `WaitGroup.Go`, `testing/synctest`, `context.Cause`, and the RWMutex fairness wording. The 2026-09-23 pins rode the six-voice rebuild, c23 `threads.h` and `stdatomic` ordering, `System.Threading.Channels` and `TimeProvider`, node `worker_threads` and the `node:test` runner, python `threading` and `asyncio`, the lua 5.5 coroutine section, and the go stdlib pages the sibling listings consulted. Findings that contradicted intuition, the fnv high-bit clustering in chapter 12, the slog handler's obligation to call `Resolve` in chapter 14, the harness-vs-network message split the raft hub needed in chapter 16, are recorded in their chapters as verified behavior, and this table is the changelog if a future release moves any of them.

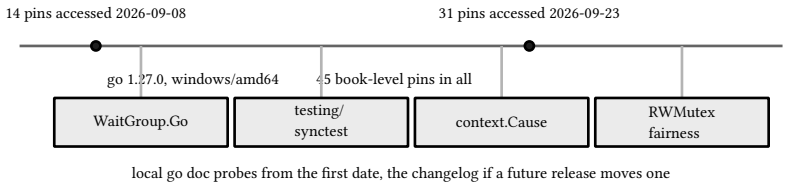


figure 92: the pin set on its access timeline, two dates, the go doc probes marked